

Numerical Solutions of Differential Equations

Waleed A A Ali

Table of contents

Introduction	6
Material	7
Unit Outline	8
Aims & Objectives	9
Intended Learning Outcomes:	9
Questions	10
1 Internal Workings of MATLAB	11
1.1 Floating-Point Arithmetic	11
1.2 Computational Complexity	13
 I Linear Algebra	 18
2 Solving Linear Systems of Equations	20
2.1 Computational Stability of Linear Systems	21
2.2 Direct Methods	23
2.2.1 Direct Substitution	23
2.2.2 Forward/Backward Substitution	24
2.2.3 TDMA Algorithm	26
2.2.4 Cramer's Rule	28
2.2.5 Gaussian Elimination & LU Factorisation	29
2.2.6 Other Direct Methods	31
2.3 Iterative Methods	31
2.3.1 Constructing an Iterative Method	32
2.3.2 Computational Cost & Stopping Criteria	34
2.4 In-Built MATLAB Procedures	35
2.5 Exercises	39
 II Solving Initial Value Problems	 47
3 The Euler Method	49
3.1 Steps of the Euler Method	50
3.2 Accuracy	53

3.3	Set of IVPs	55
3.4	Higher Order IVPs	58
3.4.1	Sets of Higher Order IVPs	61
3.4.2	Stability of a Set of ODEs	62
3.5	Limitations of the Euler Method	63
3.5.1	Bounds on the Step size	65
3.5.2	Estimated Bound	67
3.6	MATLAB Code	72
4	The Modified Euler Method	76
4.1	Steps of the Modified Euler Method	76
4.2	Accuracy of the Modified Euler Method	78
4.3	MATLAB Code	80
5	Fourth Order Runge-Kutta Method	84
5.1	MATLAB Code	86
6	MATLAB's In-Built Procedures	90
7	Implicit IVP Solvers	94
7.1	Backwards Euler Method	94
7.2	Stability of the Backwards Euler Method	95
7.3	Order of Accuracy	96
7.4	MATLAB Code	97
7.5	Stiff Differential Equations	99
III	Solving Boundary Value Problems	102
8	Boundary Value Problems	104
8.1	Example of Boundary Value Problems	104
8.2	Finite Difference Method for Boundary Value Problems	107
8.2.1	Discretisation of the Differential Equation	110
8.3	MATLAB Code	115
8.4	Comparison Between Forward, Backward & Centred Difference Approximations	118
8.5	MATLAB's In-Built Procedures	119
9	Mixed Value Problems	123
9.1	Finite Difference Method for MVPs	123
10	Symmetric Boundary Conditions	128
10.1	Finite Difference Method for Symmetric Boundary Value Problems	128

IV Solving Partial Differential Equations	132
11 Heat Equation	134
11.1 The Method of Lines for the Heat Equation	134
11.2 Linear Advection Equation	139
11.3 Convection-Diffusion Equation	142
11.4 Asymptotic Stability	143
11.4.1 Stability of the Euler Method for the Advection Equation	145
11.4.2 Stability of the Euler Method for the Heat Equation	148
11.4.3 Stability of the Convection-Diffusion Equation	150
11.5 Convection-Diffusion Equation with Centred Differencing	153
12 Wave Equation	155
12.1 The Method of Lines for the Wave Equation	155
12.2 Stability	157
Appendices	159
A MATLAB Basics	159
A.1 Command Window	159
A.2 Executing Commands in the Command Window	160
A.3 Defining Variables	162
A.4 Naming Variables	163
A.5 Scripts & Functions	164
A.6 Exercises	166
B Arrays in MATLAB	169
B.1 Vectors	169
B.2 Matrices	172
B.3 Referencing Terms in Arrays	175
B.4 Matrix Operations	177
B.5 Substitution & Concatenation	180
B.6 Finding Terms	182
B.7 Exercises	185
C Loops	188
C.1 <code>if</code> Loops	188
C.2 <code>while</code> Loops	191
C.3 Multiple Conditions for <code>if</code> & <code>while</code> Loops	196
C.4 <code>for</code> Loops	196
C.5 Exercises	198
D Plotting in MATLAB	201
D.1 Forming Lists for Plotting	201
D.2 Line Properties	203

D.3	Multiple Plots	204
D.3.1	Legends	204
D.4	Figure Properties	206
D.5	Subplots	206
D.6	Aesthetics	207
D.7	Discrete Plots	210
D.8	Plot Cheat Sheet	214
E	Reading & Writing Data	216
E.1	Writing Into Data Files	216
E.1.1	Output Formats	218
E.1.2	Alignment	219
E.2	Reading From Data Files	220
E.3	Reading & Writing Data with Excel	222
F	Gaussian Elimination Method	224
G	Matrix Decompositions	230
G.1	Orthogonality & QR Factorisation	230
G.1.1	QR Decomposition Using Reflections	234
G.1.2	QR Decomposition Using Rotations	237
G.1.3	QR Decomposition in MATLAB	240
G.2	Eigenvalue Decomposition	241
G.2.1	Eigendecomposition	242
G.3	Singular Value Decomposition (SVD)	243
H	Data Fitting	252
H.1	Linear Regression	252
H.2	Lines of Best Fit Using <code>polyfit</code>	255
I	Eigenvalue Problems	257
I.1	Calculating Eigenvalues Using the Power Method	264
J	Numerical Solutions of Non-Linear Equations	268
J.1	One-Dimensional Root-Finding Algorithm	268
J.2	Bisection Method	269
J.3	Secant Method	269
J.4	Newton-Raphson Method (NR)	270
J.5	Solving Systems of Non-Linear Equations Numerically	272
J.6	Minimisation Problems	273
J.7	Method of Steepest Descent	274
J.8	Variants of the Newton-Raphson Method	277
J.9	Applications of Minimisation Methods	278

Introduction

This unit will cover some of the numerical techniques used for solving differential equations and using MATLAB to implement these numerical methods.

Material

All the material will be posted on the [Microsoft Teams Page](#) for the unit. Note that is document is regularly being updated so if you find any mistakes or parts missing then do let me know.

Unit Outline

All lectures will be held online on **Fridays, 10.00 - 12.00** on Microsoft Teams, to which you should have received a link. The details for the link are as follows:

Meeting ID: 358 213 646 365 0

Passcode: Us2N6hB2

Lecture	Date	Topic
1	17/10	• Introduction to NSDE unit for TCC • Aims & Objectives of the unit • Floating point arithmetic • Computational complexity • Code timing and profiling • Applications for solving linear systems • Computational stability • Solving linear systems using direct methods – Direct substitution – Forward/Backward substitution
2	24/10	• Solving linear systems using direct methods – Tridiagonal matrix algorithm – Cramer's Rule
3	31/10	• Solving linear systems using iterative methods • Euler method for IVPs
4	07/11	• Modified Euler method • Runge-Kutta method • Backwards Euler method • solving stiff IVPs
5	14/11	• Solving BVPs using the finite difference method
6	21/11	• Solving MVPs and symmetric BVPs
7	28/11	• Method of lines • Apply MoL for diffusion and/or convection
8	05/12	Stability of the method of lines

Aims & Objectives

The aim for this unit is to be able to understand and derive different numerical techniques for solving differential equations and being able to implement them on MATLAB.

Intended Learning Outcomes:

- Understand the internal working mechanisms of MATLAB,
- Solve linear systems using direct and iterative methods,
- Use different differencing schemes to assess their ability to solve ODEs and PDEs,
- Assess the stability of different numerical methods.

Questions

For any questions, queries or issues that you see in the material, do not hesitate to contact me on w.a.a.ali@bath.ac.uk.

1 Internal Workings of MATLAB

1.1 Floating-Point Arithmetic

Since computers have limited resources, only a finite strict subset $\mathcal{F}$ of the real numbers can be represented. This set of possible stored values is known as ***Floating-Point Numbers*** and these are characterised by properties that are different from those in $\mathbb{R}$, since any real number x is – in principle – truncated by the computer, giving rise to a new number denoted by $fl(x)$, which does not necessarily coincide with the original number x .

A computer represents a real number x as a floating-point number in $\mathcal{F}$ as

$$x = (-1)^s \times (a_1 a_2 \dots a_t) \times \beta^E \quad (1.1)$$

where:

- $s \in \{0, 1\}$ determines the sign of the number;
- $\beta \geq 2$ is the base;
- $E \in \mathbb{Z}$ is the exponent.
- $a_1 a_2 \dots a_t$ is the mantissa (or significand). The mantissa has length t which is the maximum number of digits that can be stored. Each term in the mantissa must satisfy $0 \leq a_i \leq \beta - 1$ for all $i = 1, 2, \dots, t$ and $a_1 \neq 0$ (to ensure that the same number cannot have different representations). The digits $a_1 a_2 \dots a_p$ (with $p \leq t$) are often called the p first significant digits of x .

The set $\mathcal{F}$ is therefore fully characterised by the basis β , the number of significant digits t and the range of values that E can take.

A computer typically uses binary representation, meaning that the base is $\beta = 2$ with the available digits $\{0, 1\}$ (also known as bits) and each digit is the coefficient of a power of 2. Available platforms (like MATLAB and Python) typically use the IEEE754 double precision format for $\mathcal{F}$, which uses 64-bits as follows:

- 1 bit for s (either 0 or 1) to determine the sign;
- 11 bits for E (which can be $0, 1, 2, \dots, 10$);
- 52 bits for $a_2 a_3 \dots a_{53}$ (since $a_1 \neq 0$, it has to be equal to 1).

For 32-bit storage, the exponent is at most 7 and the mantissa has 23 digits. Note that 0 does not belong to $\mathcal{F}$ since it cannot be represented in the form shown in Equation 1.1 and it is therefore handled separately.

The smallest and the largest positive real numbers that can be written in floating points can be found by using the `realmin` and `realmax` commands. A positive number smaller

than $x_{\min}$ yields underflow and a positive number greater than $x_{\max}$ yields overflow. The elements in $\mathcal{F}$ are more dense near $x_{\min}$, and less dense while approaching $x_{\max}$. However, the relative distance is small in both cases. Note that any number bigger than `realmax` or smaller than `-realmax` will be assigned the values ∞ and $-\infty$ respectively.

```

1 >> realmin
2 ans =
3     2.2251e-308
4 >> realmax
5 ans =
6     1.7977e308

```

If a non-zero real number x is replaced by its floating-point representation $fl(x) \in \mathcal{F}$, then there will inevitably be a round-off error, especially if the number is either too large or too small relative to the other numbers involved. For a floating point number x , there is a distance ε_x where any value in the interval $(x - \varepsilon_x, x + \varepsilon_x)$ cannot be written as a floating point and will therefore be assigned the value x . This interval width is called the **Machine Epsilon** and can be found for any floating point number x by using the command `eps(x)`.

```

1 >> ep1=eps(1)
2 ep1 =
3     2.2204e-16
4 >> 1-(1+ep1/2)
5 ans =
6     0

```

The larger the floating number is, the larger the machine epsilon will be, meaning that larger numbers will have much greater tolerances of error. The smaller the number is, the larger the relative size will be, rendering the numbers insignificant overall.

```

1 >> eps(2^100)
2 ans =
3     2.8147e+14
4 >> eps(2^-50)
5 ans =
6     1.9722e-31

```

Since $\mathcal{F}$ is a strict subset of $\mathbb{R}$, elementary algebraic operations on floating-point numbers do not inherit all the properties of analogous operations on $\mathbb{R}$. Precisely, commutativity still holds for addition and multiplication, i.e. $fl(x + y) = fl(y + x)$ and $fl(xy) = fl(yx)$. Associativity is violated whenever a situation of overflow or underflow occurs or, similarly, whenever two numbers with opposite signs but similar absolute values are added, the result may be quite inexact and the situation is referred to as loss of significant digits.

Properly handling floating point computations can be tricky sometimes and, if not correctly done, may have serious consequences. There are many webpages (and books) collecting

examples of different disasters caused by a poor handling of computer arithmetic or a bad algorithmic implementation. See, for instance, [Software Bugs](#) and the [Patriot Missile Fail](#) among others.

1.2 Computational Complexity

The **Computational Complexity** of an algorithm can be defined as the relationship between the size of the input and the difficulty of running the algorithm to completion. The size (or at least, an attribute of the size) of the input is usually denoted n , for instance, for a 1-D array, n can be its length.

The difficulty of a problem can be measured in several ways. One suitable way to describe the difficulty of the problem is to count the number of **Floating-Point Operations**, such as additions, subtractions, multiplications, divisions and assignments. Floating-point operations, also called **flops**, usually measures the speed of a computer, measured as the maximum number of floating-point operations which the computer can execute in one second. Although each basic operation takes a different amount of time, the number of basic operations needed to complete a function is sufficiently related to the running time to be useful, and it is usually easy to count and less dependent on the specific machine (hardware) that is used to perform the computations.

A common notation for complexity is the **Big-O** notation (denoted $\mathcal{O}$), which establishes the relationship in the growth of the number of basic operations with respect to the size of the input as the input size becomes very large. In general, the basic operations grow in direct response to the increase in the size n of the input and, as n gets large, the highest power dominates. Therefore, only the highest power term is included in Big-O notation; moreover, coefficients are not required to characterise growth and are usually dropped (although this will also depend on the precision of the estimates).

Formally, a function f behaves as $f(x) \sim \mathcal{O}(p(x))$ as x tends to infinity if

$$\lim_{x \rightarrow \infty} \frac{f(x)}{p(x)} = \text{constant}.$$

For example, the polynomial $f(x) = x^4 + 2x^2 + x + 5$ behaves like x^4 as x tends to infinity since this term will be the fastest to grow. This can be written as $f(x) \sim \mathcal{O}(x^4)$ as $x \rightarrow \infty$.

Counting flops

Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be given by

$$f(n) = \left(\sum_{j=1}^n j \right)^2$$

This function f can be coded as **fun** in MATLAB as follows:

```

1 function [out]=fun(n)
2
3 out = 0;
4
5 for i=1:1:n
6
7     for j=1:1:n
8
9         out = out + i*j;
10
11     end
12
13 end
14
15 end

```

For example, $f(3)$ should perform the overall calculation

$$(1 \times 1) + (1 \times 2) + (1 \times 3) + (2 \times 1) + (2 \times 2) + (2 \times 3) + (3 \times 1) + (3 \times 2) + (3 \times 3),$$

so `fun(3)` should output `out=36`.

This code requires the following operations:

- $1 + n + 2n^2$ assignments:
 - 1: `out=0`;
 - n : `i=1:1:n`;
 - n^2 : for every `i`, `j=1:1:n`;
 - n^2 : for every `i`, `out=out+i*j`;
- n^2 multiplications: `i*j`;
- n^2 additions: `out=out+i*j`.

Therefore, for any n , this code will need $4n^2 + n + 1$ flops, meaning that the computational complexity is $\mathcal{O}(n^2)$, i.e. the code runs in polynomial time. It is not uncommon to find algorithms that run in exponential time $\mathcal{O}(c^n)$, like some recursive algorithms, or in logarithmic time $\mathcal{O}(\log n)$.

For more complicated codes, it is important to see where most of the time is spent in a code and how execution can be improved. A rudimentary way of timing can be done by the `tic` `toc`:

```

1 >> tic;
2 >> Run code or code block
3 >> toc;

```

This will produce a simple time in seconds that MATLAB took from `tic` until `toc`, so if

`toc` has not been types, then the timer will continue.

For more advanced analysis, MATLAB uses a **Code Profiler** to analyse code which includes run times for each iteration, times a code has been called and a lot more.

Iterative vs Recursive

Suppose that a code needs to be written that finds the N^{th} Fibonacci number starting the sequence with (1,1). This can be done in two ways:

- **Iteratively** by having a self-contained code that generates all the terms of the sequence up to N and displays the last term.

```
1 function [F]=Fib_Iter(N)
2
3 S=ones(1,N);
4
5 for n=3:1:N
6
7     S(n)=S(n-1)+S(n-2);
8
9 end
10
11 F=S(end);
12
13 end
```

- **Recursively** by have a self-referential code that keeps referring back to itself to generate the last term in the sequence from the previous terms.

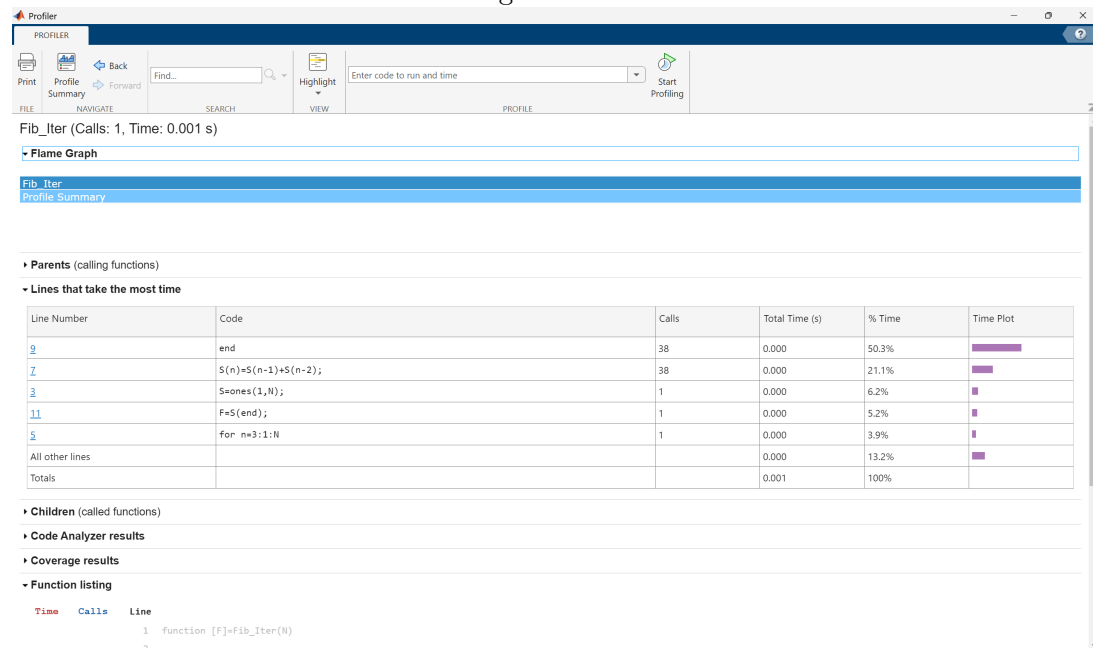
```
1 function [F]=Fib_Rec(N)
2
3 if N<3
4
5     F=1;
6
7 else
8
9     F=Fib_Rec(N-1)+Fib_Rec(N-2);
10
11 end
12
13 end
```

When running these codes for an input of $N = 10$, the times are very short, of the order of 10^{-5} seconds but as N gets larger, the recursive code starts to take much

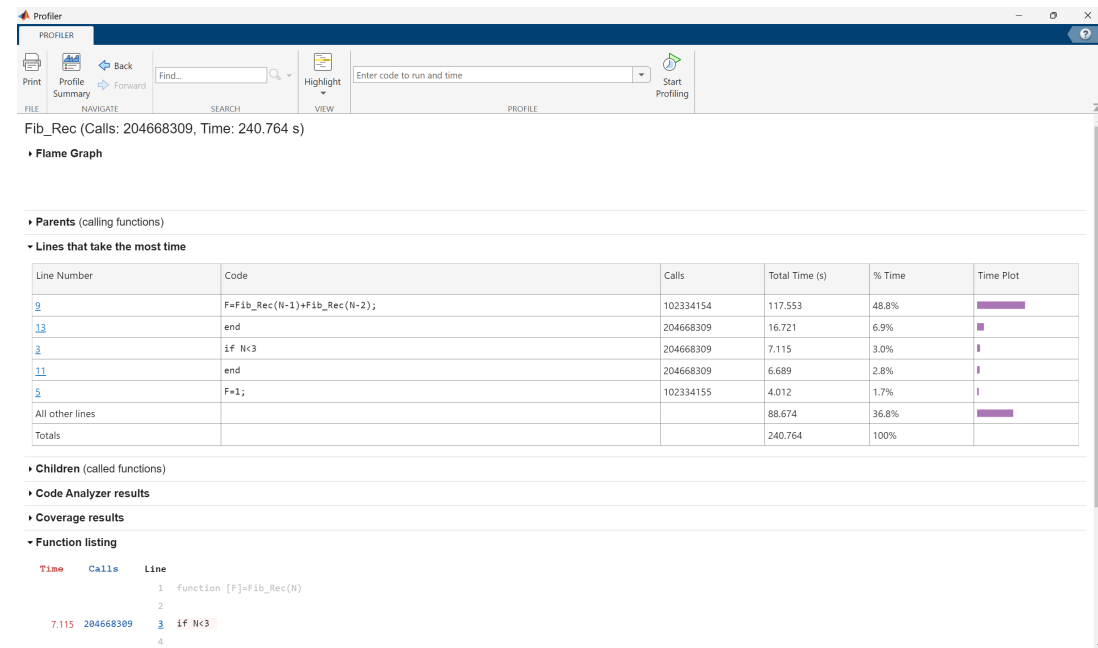
longer. Suppose the code efficiency is to be analysed for the input $N = 40$, this can be done using the profiler as follows:

```
1 >> profile on
2 >> Fib_Iter(40);
3 >> profile off
4 >> profile viewer
```

This will give a full breakdown of how many times every line was run and how much time it took. For `Fib_Iter(40)`, a total of 38 operations were performed, each taking such a short amount of time that it registers as “0 seconds”.



However, performing the profiler for `Fib_Rec(40)` gives a *dramatically* different answer with the code taking nearly 247 seconds and having to call itself more than 102 million times.



This is why it is important to profile longer codes to see which parts take the longest time and which loops are the most time consuming.

i Good Practice

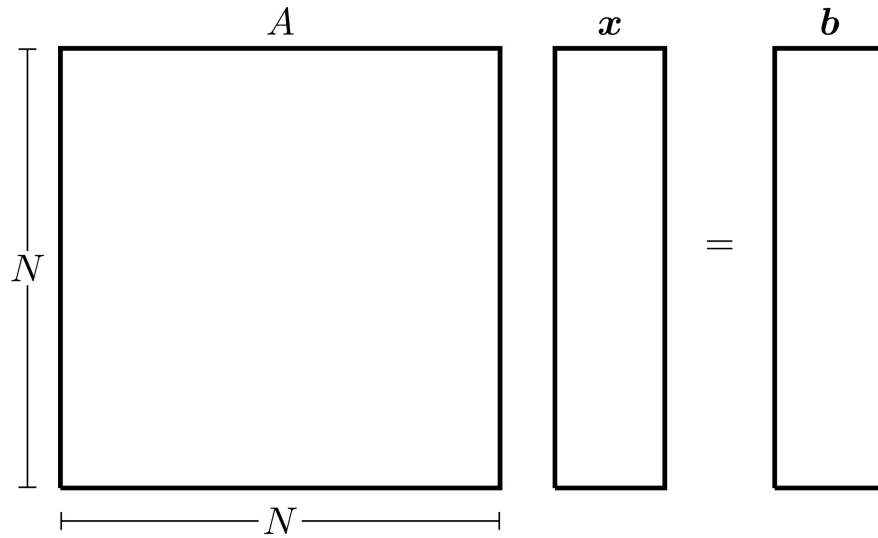
To reduce computational time in general, avoid self-referential codes because these tend to grow in usage exponentially. Another important practice is to use in-built MATLAB syntax, like using `sum` to add elements in a vector rather than manually hard coding it. This is where being familiar with a lot of the MATLAB syntax is important; MATLAB has a lot of built-in codes and syntaxes which can save a lot of time.

Part I

Linear Algebra

This section will cover some of the main methods that can be used to solve sets of linear equations of the form

$$A\mathbf{x} = \mathbf{b} \quad \text{where} \quad A \in \mathbb{R}^{N \times N}, \quad \mathbf{x} \in \mathbb{R}^N, \quad \mathbf{b} \in \mathbb{R}^N.$$



This can be done by using a *Direct Method* if the solution of the system can be obtained in a finite number of steps or an *Iterative Method* if the solution, in principle, requires an infinite number of steps.

The choice between direct and iterative methods may depend on several factors, primarily the predicted theoretical efficiency of the scheme, but also the particular type of matrix (such as systems that are sparse, diagonally dominant, tridiagonal and so forth) and the memory storage requirements.

2 Solving Linear Systems of Equations

...

Before embarking on the main purpose of the course, which is solving differential equations, first solving linear systems will be necessary. The linear systems will take the form

$$A\mathbf{x} = \mathbf{b} \quad \text{where} \quad A \in \mathbb{C}^{N \times N}, \mathbf{x} \in \mathbb{C}^N \quad \text{and} \quad \mathbf{b} \in \mathbb{C}^N.$$

This is a situation when the LHS forms a system of equations with a vector of unknowns $\mathbf{x}$ and the RHS is known.

Simple Example of a Linear System

Let a, b and c be integers such that:

- their sum is equal to 20
- a is twice as large as b
- b is bigger than c by 10.

These three relationships can be written in equation form as:

$$a + b + c = 20$$

$$a = 2b$$

$$b - c = 10$$

This can be written in matrix form as:

$$\underbrace{\begin{pmatrix} 1 & 1 & 1 \\ 1 & -2 & 0 \\ 0 & 1 & -1 \end{pmatrix}}_A \underbrace{\begin{pmatrix} a \\ b \\ c \end{pmatrix}}_{\mathbf{x}} = \underbrace{\begin{pmatrix} 20 \\ 0 \\ 10 \end{pmatrix}}_{\mathbf{b}}.$$

There are two main ways in which this can be done, depending on the form of the matrix:

- Direct Methods:
 - Direct substitution for diagonal systems;
 - Forward substitution for lower triangular systems;
 - Backward substitution for upper triangular systems;
 - TDMA for tridiagonal systems;
 - Cramer's Rule and Gaussian Elimination for more general matrix systems.

- Iterative Methods
 - Jacobi;
 - Gauss-Seidel.
- In-built Methods:
 - Backslash operator.

2.1 Computational Stability of Linear Systems

Before tackling any linear algebra techniques, it is important to understand *Computational Stability*.

Consider the linear system

$$A\mathbf{x} = \mathbf{b} \quad \text{where} \quad A \in \mathbb{C}^{N \times N}, \mathbf{x} \in \mathbb{C}^N \quad \text{and} \quad \mathbf{b} \in \mathbb{C}^N.$$

In real-life applications, the matrix A is usually fully known and often invertible while the vector $\mathbf{b}$ may not be known exactly and its measurement may often include rounding errors. Suppose that the vector $\mathbf{b}$ has a small error $\delta\mathbf{b}$, then the solution $\mathbf{x}$ will also have a small error $\delta\mathbf{x}$, meaning that the system will in fact be

$$A(\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b}. \quad (2.1)$$

Subtracting $A\mathbf{x} = \mathbf{b}$ from Equation 2.1 gives $A\delta\mathbf{x} = \delta\mathbf{b}$, therefore $\delta\mathbf{x} = A^{-1}\delta\mathbf{b}$.

For $p \in \mathbb{N}$, consider the ratio between the p -norm of the error $\|\delta\mathbf{x}\|_p$ and the p -norm of the exact solution $\|\mathbf{x}\|_p$:

$$\begin{aligned} \frac{\|\delta\mathbf{x}\|_p}{\|\mathbf{x}\|_p} &= \frac{\|A^{-1}\delta\mathbf{b}\|_p}{\|\mathbf{x}\|_p} && \text{since } \delta\mathbf{x} = A^{-1}\delta\mathbf{b} \\ &\leq \frac{\|A^{-1}\|_p \|\delta\mathbf{b}\|_p}{\|\mathbf{x}\|_p} && \text{by the Submultiplicative Property} \\ &= \frac{\|A^{-1}\|_p \|\delta\mathbf{b}\|_p}{\|\mathbf{x}\|_p} \times \frac{\|A\|_p}{\|A\|_p} && \text{multiplying by } 1 = \frac{\|A\|_p}{\|A\|_p} \\ &= \|A\|_p \|A^{-1}\|_p \frac{\|\delta\mathbf{b}\|_p}{\|A\|_p \|\mathbf{x}\|_p} && \text{rearranging} \\ &\leq \|A\|_p \|A^{-1}\|_p \frac{\|\delta\mathbf{b}\|_p}{\|\mathbf{b}\|_p} && \text{since } \mathbf{b} = A\mathbf{x} \text{ then } \|\mathbf{b}\|_p \leq \|A\|_p \|\mathbf{x}\|_p \\ &&& \text{by the Submultiplicative Property,} \\ &&& \text{meaning that } \frac{1}{\|\mathbf{b}\|_p} \geq \frac{1}{\|A\|_p \|\mathbf{x}\|_p} \end{aligned}$$

i Note 1: Submultiplicative Property of Matrix Norms

For matrices A and B and a vector $\mathbf{x}$,

$$\|A\mathbf{x}\| \leq \|A\|\|\mathbf{x}\|,$$

$$\|AB\| \leq \|A\|\|B\|.$$

In both cases, the equality holds when either A or B are orthogonal.

Let $\kappa_p(A) = \|A^{-1}\|_p \|A\|_p$, then

$$\frac{\|\delta\mathbf{x}\|_p}{\|\mathbf{x}\|_p} \leq \kappa_p(A) \frac{\|\delta\mathbf{b}\|_p}{\|\mathbf{b}\|_p}$$

The quantity $\kappa_p(A)$ is called the **Condition Number**¹ and it can be regarded as a measure of how sensitive a matrix is to perturbations, in other words, it gives an indication as to the stability of the matrix system. A problem is **Well-Conditioned** if the condition number is small, and is **Ill-Conditioned** if the condition number is large (the terms “small” and “large” are somewhat subjective here and will depend on the context). Bear in mind that in practice, calculating the condition number may be computationally expensive since it requires inverting the matrix A .

The condition number derived above follows the assumption that the error only occurs in $\mathbf{b}$ which then results in an error in $\mathbf{x}$. If an error δA is also committed in A , then for sufficiently small δA , the error bound for the ratio is

$$\frac{\|\delta\mathbf{x}\|_p}{\|\mathbf{x}\|_p} \leq \frac{\kappa_p(A)}{1 - \kappa_p(A) \frac{\|\delta A\|_p}{\|A\|_p}} \left(\frac{\|\delta\mathbf{b}\|_p}{\|\mathbf{b}\|_p} + \frac{\|\delta A\|_p}{\|A\|_p} \right).$$

An example for which A is large is a discretisation matrix of a PDE, in this case, the condition number of A can be very large and increases rapidly as the number of mesh points increases. For example, for a PDE with N mesh points in 2-dimensions, the condition number $\kappa_2(A)$ is of order $\mathcal{O}(N)$ and it is not uncommon to have N between 10^6 and 10^8 . In this case, errors in $\mathbf{b}$ may be amplified enormously in the solution process. Thus, if $\kappa_p(A)$ is large, there may be difficulties in solving the system reliably, a problem which plagues calculations with partial differential equations.

Moreover, if A is large, then the system $A\mathbf{x} = \mathbf{b}$ may be solved using an *iterative method* which generate a sequence of approximations $\mathbf{x}_n$ to $\mathbf{x}$ while ensuring that each iteration is easy to perform and that $\mathbf{x}_n$ rapidly tends to $\mathbf{x}$, within a certain tolerance, as n tends to infinity. If $\kappa_p(A)$ is large, then the number of iterations to reach this tolerance increases rapidly as the size of A increases, often being proportional to $\kappa_p(A)$ or even to $\kappa_p(A)^2$. Thus

¹Note that A^{-1} exists only if A is non-singular, meaning that the condition number number only exists if A is non-singular.

not only do errors in $\mathbf{x}$ accumulate for large $\kappa_p(A)$, but the number of computation required to find $\mathbf{x}$ increases as well.

In MATLAB, the condition number can be calculated using the `cond(A,p)` command where A is the square matrix in question and p is the chosen norm which can only be equal to 1, 2, `inf` or `'Fro'` (when using the Frobenius norm). Also note that `cond(A)` without the second argument p produces the condition number with the 2-norm by default.

Properties of the Condition Number

Let A and B be invertible matrices, $p \in \mathbb{N}$ and $\lambda \in \mathbb{R}$. The condition number κ_p has the following properties:

- $\kappa_p(A) \geq 1$;
- $\kappa_p(A) = 1$ if and only if A is an orthogonal matrix, i.e. $A^{-1} = A^T$;
- $\kappa_p(A^T) = \kappa_p(A^{-1}) = \kappa_p(A)$;
- $\kappa_p(\lambda A) = \kappa_p(A)$;
- $\kappa_p(AB) \leq \kappa_p(A)\kappa_p(B)$.

2.2 Direct Methods

Direct methods can be used to solve matrix systems in a finite number of steps, although these steps could possibly be computationally expensive.

2.2.1 Direct Substitution

Direct substitution is the simplest direct method and requires the matrix A to be a diagonal with none of the diagonal terms being 0 (otherwise the matrix will not be invertible).

Consider the matrix system $A\mathbf{x} = \mathbf{b}$ where

$$A = \begin{pmatrix} a_1 & & & \\ & a_2 & & \\ & & \ddots & \\ & & & a_{N-1} & \\ & & & & a_N \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{N-1} \\ x_N \end{pmatrix} \quad \text{and} \quad \mathbf{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_{N-1} \\ b_N \end{pmatrix}$$

and $a_1, a_2, \dots, a_N \neq 0$. Direct substitution involves simple multiplication and division:

$$A\mathbf{x} = \mathbf{b} \quad \Rightarrow \quad \begin{pmatrix} a_1 & & & \\ & a_2 & & \\ & & \ddots & \\ & & & a_{N-1} & \\ & & & & a_N \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{N-1} \\ x_N \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_{N-1} \\ b_N \end{pmatrix}$$

$$\begin{array}{ccc}
a_1 x_1 = b_1 & & x_1 = \frac{b_1}{a_1} \\
a_2 x_2 = b_2 & & x_2 = \frac{b_2}{a_2} \\
\vdots & & \vdots \\
a_{N-1} x_{N-1} = b_{N-1} & \Rightarrow & x_{N-1} = \frac{b_{N-1}}{a_{N-1}} \\
a_N x_N = b_N & & x_N = \frac{b_N}{a_N}.
\end{array}$$

The solution can be written explicitly as $x_n = \frac{b_n}{a_n}$ for all $n = 1, 2, \dots, N$. Every step can be done independently, meaning that direct substitution lends itself well to parallel computing. In total, direct substitution requires exactly N computations (all being division).

Example of Direct Substitution

Consider the system $A\mathbf{x} = \mathbf{b}$ where

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & -1 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \quad \text{and} \quad \mathbf{b} = \begin{pmatrix} 4 \\ 2 \\ 4 \end{pmatrix}.$$

Solving the system using direct substitution:

$$\begin{aligned}
A\mathbf{x} = \mathbf{b} & \Rightarrow \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \\ 4 \end{pmatrix} \\
& \Rightarrow \begin{pmatrix} x_1 \\ 2x_2 \\ -x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \\ 4 \end{pmatrix} \Rightarrow \begin{aligned} x_1 &= 4 \\ x_2 &= 1 \\ x_3 &= -4. \end{aligned}
\end{aligned}$$

2.2.2 Forward/Backward Substitution

Forward/backward substitution require that the matrix A be lower/upper triangular.

Consider the matrix system $A\mathbf{x} = \mathbf{b}$ where

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1,N-1} & a_{1N} \\ & a_{22} & \dots & a_{2,N-1} & a_{2N} \\ & & \ddots & \vdots & \\ & & & a_{N-1,N-1} & a_{N-1,N} \\ & & & & a_{NN} \end{pmatrix},$$

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{N-1} \\ x_N \end{pmatrix} \quad \text{and} \quad \mathbf{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_{N-1} \\ b_N \end{pmatrix}$$

and $a_{11}, a_{22}, \dots, a_{NN} \neq 0$ (so that the determinant is non-zero). The matrix A is upper triangular in this case and will require backwards substitution:

$$\begin{aligned}
 A\mathbf{x} = \mathbf{b} \quad \Rightarrow \quad & \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1,N-1} & a_{1N} \\ & a_{22} & \cdots & a_{2,N-1} & a_{2N} \\ & & \ddots & \vdots & \\ & & & a_{N-1,N-1} & a_{N-1,N} \\ & & & & a_{NN} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{N-1} \\ x_N \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_{N-1} \\ b_N \end{pmatrix} \\
 \Rightarrow \quad & \begin{aligned} a_{11}x_1 &+ a_{12}x_2 + \cdots + a_{1,N-1}x_{N-1} + a_{1N}x_N &= b_1 \\ & a_{22}x_2 + \cdots + a_{2,N-1}x_{N-1} + a_{2N}x_N &= b_2 \\ & \vdots \\ & a_{N-1,N-1}x_{N-1} + a_{N-1,N}x_N &= b_{N-1} \\ & & a_{NN}x_N &= b_N \end{aligned}
 \end{aligned}$$

Backward substitution involves using the solutions from the later equations to solve the earlier ones, this gives:

$$\begin{aligned}
 x_N &= \frac{b_N}{a_{NN}} \\
 x_{N-1} &= \frac{b_{N-1} - a_{N-1,N}x_N}{a_{N-1,N-1}} \\
 &\vdots \\
 x_2 &= \frac{b_2 - a_{2N}x_N - a_{2,N-1}x_{N-1} - \cdots - a_{23}x_3}{a_{22}} \\
 x_1 &= \frac{b_1 - a_{1N}x_N - a_{1,N-1}x_{N-1} - \cdots - a_{12}x_2}{a_{11}}.
 \end{aligned}$$

This can be written more explicitly as:

$$x_n = \begin{cases} \frac{b_N}{a_{NN}} & \text{for } n = N \\ \frac{1}{a_{nn}} \left(b_n - \sum_{i=n+1}^N a_{ni}x_i \right) & \text{for } n = N-1, \dots, 2, 1. \end{cases}$$

A similar version can be obtained for the forward substitution for lower triangular matrices as follows:

$$x_n = \begin{cases} \frac{b_1}{a_{11}} & \text{for } n = 1 \\ \frac{1}{a_{nn}} \left(b_n - \sum_{i=1}^{n-1} a_{ni}x_i \right) & \text{for } n = 2, 3, \dots, N-1. \end{cases}$$

For any $n = 1, 2, \dots, N-1$, calculating it requires 1 division, $N-n$ multiplications and $N-n$ subtractions. Therefore cumulatively, $x_1, x_2, \dots, x_{N-1}$ require N divisions, $\frac{1}{2}(N^2 - N)$ multiplications and $\frac{1}{2}(N^2 - N)$ additions with one more division required for x_N , meaning that in total, backward (and forward) substitution requires $N^2 + 1$ computations.

Example of Backward Substitution

Consider the system $A\mathbf{x} = \mathbf{b}$ where

$$A = \begin{pmatrix} 1 & 2 & 1 \\ 0 & -1 & 4 \\ 0 & 0 & -1 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \quad \text{and} \quad \mathbf{b} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}.$$

This problem can be solved by using backward substitution:

$$\begin{aligned} A\mathbf{x} = \mathbf{b} &\implies \begin{pmatrix} 1 & 2 & 1 \\ 0 & -1 & 4 \\ 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \implies \begin{aligned} x_1 + 2x_2 + x_3 &= 1 \\ -x_2 + 4x_3 &= 0 \\ -x_3 &= 1 \end{aligned} \\ -x_2 + 4x_3 &= 0 \xRightarrow{x_3=-1} -x_2 - 4 = 0 \implies x_2 = -4 \\ x_1 + 2x_2 + x_3 &= 1 \xRightarrow{x_2=-4, x_3=-1} x_1 - 8 - 1 = 1 \implies x_1 = 10. \end{aligned}$$

2.2.3 TDMA Algorithm

The *TriDiagonal Matrix Algorithm*, abbreviated as **TDMA** (also called the *Thomas Algorithm*) was developed by Llewellyn Thomas which solves tridiagonal matrix systems.

Consider the matrix system $A\mathbf{x} = \mathbf{b}$ where

$$A = \begin{pmatrix} m_1 & r_1 & & & \\ l_2 & m_2 & r_2 & & \\ & \ddots & \ddots & \ddots & \\ & & l_{N-1} & m_{N-1} & r_{N-1} \\ & & & l_N & m_N \end{pmatrix},$$

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{N-1} \\ x_N \end{pmatrix} \quad \text{and} \quad \mathbf{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_{N-1} \\ b_N \end{pmatrix}.$$

The m terms denote the diagonal elements, l denote subdiagonal elements (left of the diagonal terms) and r denote the superdiagonal elements (right of the diagonal terms). The TDMA algorithm works in two steps: first, TDMA performs a forward sweep to eliminate all the subdiagonal terms and rescale the matrix to have 1 as the diagonal (the same can

also be done to eliminate the superdiagonal instead). This give the matrix system

$$\begin{pmatrix} 1 & R_1 & & & \\ & 1 & R_2 & & \\ & & \ddots & \ddots & \\ & & & 1 & R_{N-1} \\ & & & & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{N-1} \\ x_N \end{pmatrix} = \begin{pmatrix} B_1 \\ B_2 \\ \vdots \\ B_{N-1} \\ B_N \end{pmatrix}$$

where

$$R_n = \begin{cases} \frac{r_1}{m_1} & n = 1 \\ \frac{r_n}{m_n - l_n R_{n-1}} & n = 2, 3, \dots, N-1 \end{cases}$$

$$B_n = \begin{cases} \frac{b_1}{m_1} & n = 1 \\ \frac{b_n - l_n B_{n-1}}{m_n - l_n R_{n-1}} & n = 2, 3, \dots, N. \end{cases}$$

This can now be solved with backward substitution:

$$x_n = \begin{cases} B_N & n = N \\ B_n - R_n x_{n+1} & n = N-1, N-2, \dots, 2, 1. \end{cases}$$

The computational complexity can be calculated as follows:

Term	$\times$	$+$	$\div$
R_1	0	0	1
R_2	1	1	1
$\vdots$	$\vdots$	$\vdots$	$\vdots$
R_{N-1}	1	1	1
B_1	0	0	1
B_2	2	2	1
$\vdots$	$\vdots$	$\vdots$	$\vdots$
B_{N-1}	2	2	1
B_N	2	2	1
x_1	1	1	0
x_2	1	1	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$
x_{N-1}	1	1	0

This gives a total of $3N - 5$ computations for R , $5N - 4$ computations for B and $2N - 2$ computations for x giving a total of $10N - 11$ computations.

There are similar ways of performing eliminations that be done for pentadiagonal systems as well as tridiagonal systems with a full first row.

2.2.4 Cramer's Rule

Cramer's Rule is a method that can be used to solve *any* system $A\mathbf{x} = \mathbf{b}$ (of course provided that A is non-singular).

Cramer's rule states that the elements of the vector $\mathbf{x}$ are given by

$$x_n = \frac{\det(A_n)}{\det(A)} \quad \text{for all } n = 1, 2, \dots, N$$

where A_n is the matrix obtained from A by replacing the n^{th} column by $\mathbf{b}$. This method seems very simple to execute thanks to its very simple formula, but in practice, it can be very computationally expensive.

Example of Cramer's Rule

Consider the system $A\mathbf{x} = \mathbf{b}$ where

$$A = \begin{pmatrix} 0 & 4 & 7 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad \text{and} \quad \mathbf{b} = \begin{pmatrix} 14 \\ 1 \\ 7 \end{pmatrix}.$$

The determinant of A is equal to 7. Using Cramer's rule, the solution $\mathbf{x} = (x_1, x_2, x_3)^T$ can be calculated as:

$$x_1 = \frac{\det(A_1)}{\det(A)} = \frac{\det \begin{pmatrix} 14 & 4 & 7 \\ 1 & 0 & 1 \\ 7 & 1 & 0 \end{pmatrix}}{7} = \frac{21}{7} = 3.$$

$$x_2 = \frac{\det(A_2)}{\det(A)} = \frac{\det \begin{pmatrix} 0 & 14 & 7 \\ 1 & 1 & 1 \\ 0 & 7 & 0 \end{pmatrix}}{7} = \frac{49}{7} = 7.$$

$$x_3 = \frac{\det(A_3)}{\det(A)} = \frac{\det \begin{pmatrix} 0 & 4 & 14 \\ 1 & 0 & 1 \\ 0 & 1 & 7 \end{pmatrix}}{7} = \frac{-14}{7} = -2.$$

Generally, for a matrix of size $N \times N$, the determinant will require $\mathcal{O}(N!)$ computations (other matrix forms or methods may require fewer, of $\mathcal{O}(N^3)$ at least). Cramer's rule requires calculating the determinants of $N + 1$ matrices each is size $N \times N$ and performing N divisions, therefore the computational complexity of Cramer's rule is $\mathcal{O}(N + (N + 1) \times N!) = \mathcal{O}(N + (N + 1)!)$. This means that if a machine runs at 1 Gigafllops per second (10^9 fllops), then a matrix system of size 20×20 will require 1620 years to compute.

2.2.5 Gaussian Elimination & LU Factorisation

Consider the linear system

$$A\mathbf{x} = \mathbf{b} \quad \text{where} \quad A \in \mathbb{R}^{N \times N}, \quad \mathbf{x} \in \mathbb{R}^N, \quad \mathbf{b} \in \mathbb{R}^N.$$

In real-life situations, the matrix A does not always have a form that lends itself to being easily solvable (like a diagonal, triangular, sparse, etc.). However, there are ways in which a matrix can be broken down into several matrices, each of which can be dealt with separately and reducing computational time.

One way of doing this is by **Gaussian Elimination** which is a series of steps that reduces a matrix A into an upper triangular matrix. The steps of the Gaussian elimination are presented in details in Appendix F.

In order to convert the linear system $A\mathbf{x} = \mathbf{b}$ into the upper triangular matrix system $U\mathbf{x} = \mathbf{g}$ by Gaussian elimination, a series of transformations have to be done, each represented by a matrix $M^{(n)}$ giving the form

$$U = MA \quad \text{where} \quad M = M^{(N-1)}M^{(N-2)} \dots M^{(1)}.$$

For every $n = 1, 2, \dots, N - 1$, the matrix $M^{(n)}$ is lower triangular (because the process is intended to eliminate the lower triangular elements) which means that the product of all these matrices M must also be lower triangular. Note that since $M^{(n)}$ and M are both non-singular and lower triangular, then their inverses must also be lower triangular. This means that the matrix $L = M^{-1}$ will be lower triangular and invertible. Additionally, since Gaussian elimination tends to keep the diagonal terms unchanged, then the diagonal terms of M , and consequently L , will all be 1.

This means that the matrix A can be written as $A = LU$ where $L = M^{-1}$ is a lower triangular matrix and U is an upper triangular matrix. This is called the **LU Decomposition** of A .

In the cases when there might be pivoting issues (which is when the pivot points might be equal to 0 during the Gaussian Elimination), the LU decomposition will more precisely be the **PLU Decomposition** (or the **LU Decomposition with Partial Pivoting**) where the method will produce an additional permutation matrix P where $PA = LU$. This matrix P will swap rows when needed in order to have non-zero pivot points and is in fact orthogonal (i.e. $P^{-1} = P^T$).

The LU decomposition can be used to solve the linear system $A\mathbf{x} = \mathbf{b}$ by splitting the matrix A into two matrices with more manageable forms. Indeed, since $A = LU$, then the system becomes $LU\mathbf{x} = \mathbf{b}$, this can be solved as follows:

- Solve the lower triangular system $L\mathbf{y} = \mathbf{b}$ for $\mathbf{y}$ using forward substitution;
- Solve the upper triangular $U\mathbf{x} = \mathbf{y}$ for $\mathbf{x}$ using backwards substitution.

This is a much better way of solving the system since both equations involve a triangular matrix and this requires $\mathcal{O}(N^2)$ computations (forward and backward substitutions).

The advantage of using the LU decomposition is that if problems of the form $A\mathbf{x}_i = \mathbf{b}_i$ need to be solved with many different right hand sides $\mathbf{b}_i$ and a fixed A , then only one LU decomposition is needed, and the cost for solving the individual systems is only the repeated forward and back substitutions. Note that there are other strategies optimised for specific cases (i.e. symmetric positive definite matrices, banded matrices, tridiagonal matrices).

In MATLAB, the LU decomposition can be done by a simple `lu` command:

```

1  >> A=[5,0,1;1,2,1;2,1,1];
2  >> [L,U]=lu(A)
3  L =
4      1.0000      0      0
5      0.2000      1.0000      0
6      0.4000      0.5000      1.0000
7  U =
8
9      5.0000      0      1.0000
10      0      2.0000      0.8000
11      0      0      0.2000
12 >> L*U-A      % Verify if LU is equal to A
13 ans =
14      0      0      0
15      0      0      0
16      0      0      0

```

Note that if the output for L is *not* lower triangular, that means there are some pivoting issues that had to be overcome and L had to change to accommodate for that to maintain the fact that $A = LU$. In this case, the PLU decomposition would be better suited to avoid that, this is done by adding one extra output to the `lu` command, in this case, A will actually be the product $A = P^T LU$.

```

1  >>> A=[1,0,1;1,0,1;2,1,1];
2  >> [L,U]=lu(A)
3  L =
4      0.5000      1.0000      1.0000
5      0.5000      1.0000      0
6      1.0000      0      0
7  U =
8
9      2.0000      1.0000      1.0000
10      0     -0.5000      0.5000
11      0      0      0
12 >> L*U-A      % Verify if LU is equal to A even though
13                % L is not lower triangular
14 ans =
15      0      0      0

```

```

16      0  0  0
17      0  0  0
18  >> [L,U,P]=lu(A)
19  L =
20      1.0000      0      0
21      0.5000      1.0000      0
22      0.5000      1.0000      1.0000
23  U =
24
25      2.0000      1.0000      1.0000
26      0      -0.5000      0.5000
27      0      0      0
28  P =
29      0  0  1
30      0  1  0
31      1  0  0
32  >> L*U-A      % Verify if P'LU is equal to A
33  ans =
34      0  0  0
35      0  0  0
36      0  0  0

```

2.2.6 Other Direct Methods

There are many other direct methods with more involved calculations like *QR decomposition* and *Singular Value Decomposition* amongst others. All these methods will be placed in Appendix G.

2.3 Iterative Methods

For a large matrix A , solving the system $A\mathbf{x} = \mathbf{b}$ directly can be computationally restrictive as seen in the different methods shown in Section 2.2. An alternative would be to use *iterative* methods which generate a sequence of approximations $\mathbf{x}^{(k)}$ to the exact solution $\mathbf{x}$. The hope is that the iterative method converges to the exact solution, i.e.

$$\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x}.$$

A possible strategy to realise this process is to consider the following recursive definition

$$\mathbf{x}^{(k)} = B\mathbf{x}^{(k-1)} + \mathbf{g} \quad \text{for } k \geq 1,$$

where B is a suitable matrix called the **Iteration Matrix** (which would generally depend on A) and $\mathbf{g}$ is a suitable vector (depending on A and $\mathbf{b}$). Since the iterations $\mathbf{x}^{(k)}$ must

tend to $\mathbf{x}$ as k tends to infinity, then

$$\mathbf{x}^{(k)} = B\mathbf{x}^{(k-1)} + \mathbf{g} \quad (2.2)$$

$$\xRightarrow[k \rightarrow \infty]{} \mathbf{x} = B\mathbf{x} + \mathbf{g}. \quad (2.3)$$

Next, a sufficient condition needs to be derived; define $\mathbf{e}^{(k)}$ as the error incurred from iteration k , i.e. $\mathbf{e}^{(k)} := \mathbf{x} - \mathbf{x}^{(k)}$ and consider the linear systems

$$\mathbf{x} = B\mathbf{x} + \mathbf{g} \quad \text{and} \quad \mathbf{x}^{(k)} = B\mathbf{x}^{(k-1)} + \mathbf{g}.$$

Subtracting these gives

$$\begin{aligned} \mathbf{x} - \mathbf{x}^{(k)} &= (B\mathbf{x} + \mathbf{g}) - (B\mathbf{x}^{(k-1)} + \mathbf{g}) \\ \implies \mathbf{x} - \mathbf{x}^{(k)} &= B(\mathbf{x} - \mathbf{x}^{(k-1)}) \\ \implies \mathbf{e}^{(k)} &= B\mathbf{e}^{(k-1)}. \end{aligned}$$

In order to find a bound for the error, take the 2-norm of the error equation

$$\mathbf{e}^{(k)} = B\mathbf{e}^{(k-1)} \quad \xRightarrow{\|\cdot\|_2} \quad \|\mathbf{e}^{(k)}\|_2 = \|B\mathbf{e}^{(k-1)}\|_2.$$

By the submultiplicative property of matrix norms given in Note 1, the error $\|\mathbf{e}^{(k)}\|$ can be bounded above as

$$\|\mathbf{e}^{(k)}\|_2 = \|B\mathbf{e}^{(k-1)}\|_2 \leq \|B\|_2 \|\mathbf{e}^{(k-1)}\|_2.$$

This can be iterated backwards, so for $k \geq 1$,

$$\|\mathbf{e}^{(k)}\|_2 \leq \|B\|_2 \|\mathbf{e}^{(k-1)}\|_2 \leq \|B\|_2^2 \|\mathbf{e}^{(k-2)}\|_2 \leq \dots \leq \|B\|_2^k \|\mathbf{e}^{(0)}\|_2.$$

Generally, this means that the error at any iteration k can be bounded above by the error at the initial iteration $\mathbf{e}^{(0)}$. Therefore, since $\mathbf{e}^{(0)}$ is arbitrary, if $\|B\|_2 < 1$ then the set of vectors $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$ generated by the iterative scheme $\mathbf{x}^{(k)} = B\mathbf{x}^{(k-1)} + \mathbf{g}$ will converge to the exact solution $\mathbf{x}$ which solves $A\mathbf{x} = \mathbf{b}$, hence giving a sufficient condition for convergence.

2.3.1 Constructing an Iterative Method

A general technique to devise an iterative method to solve $A\mathbf{x} = \mathbf{b}$ is based on a “splitting” of the matrix A . First, write the matrix A as $A = P - (P - A)$ where P is a suitable non-singular matrix (somehow linked to A and “easy” to invert). Then

$$\begin{aligned} P\mathbf{x} &= [A + (P - A)]\mathbf{x} && \text{since } P = A + P - A \\ &= (P - A)\mathbf{x} + A\mathbf{x} && \text{expanding} \\ &= (P - A)\mathbf{x} + \mathbf{b} && \text{since } A\mathbf{x} = \mathbf{b} \end{aligned}$$

Therefore, the vector $\mathbf{x}$ can be written implicitly as

$$\mathbf{x} = P^{-1}(P - A)\mathbf{x} + P^{-1}\mathbf{b}$$

which is of the form given in Equation 2.3 where $B = P^{-1}(P - A) = I - P^{-1}A$ and $\mathbf{g} = P^{-1}\mathbf{b}$. It would then stand to reason that if the iterative procedure was of the form

$$\mathbf{x}^{(k)} = P^{-1}(P - A)\mathbf{x}^{(k-1)} + P^{-1}\mathbf{b}$$

(as in Equation 2.2), then the method should converge to the exact solution (provided a suitable choice for P). Of course, for the iterative procedure, the iteration needs an initial vector to start which will be

$$\mathbf{x}^{(0)} = \begin{pmatrix} x_1^{(0)} \\ x_2^{(0)} \\ \vdots \\ x_N^{(0)} \end{pmatrix}.$$

The choice of the matrix P should depend on A in some way. So suppose that the matrix A is broken down into three parts, $A = D + L + U$ where D is the matrix of the diagonal entries of A , L is the strictly lower triangular part of A (i.e. not including the diagonal) and U is the strictly upper triangular part of A .

i Note

For example

$$\underbrace{\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}}_A = \underbrace{\begin{pmatrix} a & 0 & 0 \\ 0 & e & 0 \\ 0 & 0 & i \end{pmatrix}}_D + \underbrace{\begin{pmatrix} 0 & 0 & 0 \\ d & 0 & 0 \\ g & h & 0 \end{pmatrix}}_L + \underbrace{\begin{pmatrix} 0 & b & c \\ 0 & 0 & f \\ 0 & 0 & 0 \end{pmatrix}}_U.$$

• **Jacobi Method:** $P = D$

The matrix P is chosen to be equal to the diagonal part of A , then the splitting procedure gives the iteration matrix $B = I - D^{-1}A$ and the iteration itself is $\mathbf{x}^{(k)} = B\mathbf{x}^{(k-1)} + D^{-1}\mathbf{b}$ for $k \geq 0$, which can be written component-wise as

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^N a_{ij}x_j^{(k-1)} \right) \quad \text{for all } i = 1, \dots, N. \quad (2.4)$$

If A is strictly diagonally dominant by rows², then the Jacobi method converges. Note that each component $x_i^{(k)}$ of the new vector $\mathbf{x}^{(k)}$ is computed independently of the others, meaning that the update is simultaneous which makes this method suitable for parallel programming.

²A matrix $A \in \mathbb{R}^{N \times N}$ is **Diagonally Dominant** if every diagonal entry is larger in absolute value than the

- **Gauss-Seidel Method:** $P = D + L$

The matrix P is chosen to be equal to the lower triangular part of A , therefore the iteration matrix is given by $B = (D + L)^{-1}(D + L - A)$ and the iteration itself is $\mathbf{x}^{(k+1)} = B\mathbf{x}^{(k)} + (D + L)^{-1}\mathbf{b}$ which can be written component-wise as

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^N a_{ij}x_j^{(k)} \right) \quad \text{for all } i = 1, \dots, N. \quad (2.5)$$

Contrary to Jacobi method, Gauss-Seidel method updates the components in sequential mode.

There are many other methods that use splitting like:

- Damped Jacobi method: $P = \frac{1}{\omega}D$ for some $\omega \neq 0$
- Successive over-relaxation method: $P = \frac{1}{\omega}D + L$ for some $\omega \neq 0$
- Symmetric successive over-relaxation method: $P = \frac{1}{\omega(2-\omega)}(D + \omega L)D^{-1}(D + \omega U)$ for some $\omega \neq 0, 2$.

2.3.2 Computational Cost & Stopping Criteria

There are essentially two factors contributing to the effectiveness of an iterative method for $A\mathbf{x} = \mathbf{b}$: the computational cost per iteration and the number of performed iterations. The computational cost per iteration depends on the structure and sparsity of the original matrix A and on the choice of the splitting. For both Jacobi and Gauss-Seidel methods, without further assumptions on A , the computational cost per iteration is $\mathcal{O}(N^2)$. Iterations should be stopped when one or more stopping criteria are satisfied, as will be discussed below. For both Jacobi and Gauss-Seidel methods, the cost of performing k iterations is $\mathcal{O}(kN^2)$; so as long as $k \ll N$, these methods are much cheaper than Gaussian elimination.

In theory, iterative methods require an infinite number of iterations to converge to the exact solution of a linear system but in practice, aiming for the exact solution is neither reasonable nor necessary. Indeed, what is actually needed is an approximation $\mathbf{x}^{(k)}$ for which the error is guaranteed to be lower than a desired tolerance $\tau > 0$. On the other hand, since the error is itself unknown (as it depends on the exact solution), a suitable *a posteriori* error estimator is needed which predicts the error starting from quantities that have already been computed. There are two natural estimators one may consider:

sum of the absolute value of all the other terms in that row. More formally

$$|a_{ii}| \geq \sum_{\substack{j=1 \\ j \neq i}}^N |a_{ij}| \quad \text{for all } i = 1, \dots, N.$$

The matrix is **Strictly Diagonally Dominant** if the inequality is strict.

- **Residual:** The residual error at the k^{th} iteration, denoted $\mathbf{r}^{(k)}$ is given by the error between $A\mathbf{x}^{(k)}$ and $\mathbf{b}$, namely $\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}$. An iterative method can be stopped at the first iteration step $k = k_{\min}$ for which

$$\|\mathbf{r}^{(k)}\| \leq \tau \|\mathbf{b}\|.$$

- **Increment:** The incremental error at the k^{th} iteration, denoted $\delta^{(k)}$ is the error between consecutive approximations, namely $\delta^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}$. An iterative method can be stopped after the first iteration step $k = k_{\min}$ for which

$$\|\delta^{(k)}\| \leq \tau.$$

Of course, another way to stop the iteration is by imposing a maximum number of allowable iterations K , this is usually a good starting point since it is not possible to know beforehand if the method does indeed converge. Enforcing a maximum number of iterations will determine if the initial guess is suitable, if the method is suitable or indeed if there is any convergence.

2.4 In-Built MATLAB Procedures

Given that MATLAB is well-suited to dealing with matrices, it has a very powerful method of solving linear systems and it is using the ***Backslash Operator***. This is a powerful in-built method that can solve any square linear system regardless of its form. MATLAB does this by first determining the general form of the matrix (sparse, triangular, Hermitian, etc.) before applying the appropriate optimised method.

For the linear system

$$A\mathbf{x} = \mathbf{b} \quad \text{where} \quad A \in \mathbb{R}^{N \times N}, \quad \mathbf{x} \in \mathbb{R}^N, \quad \mathbf{b} \in \mathbb{R}^N$$

MATLAB can solve this using the syntax $\mathbf{x} = \mathbf{A} \backslash \mathbf{b}$.

Starting Example

Returning to the example in the beginning of this section, the matrix system was

$$\underbrace{\begin{pmatrix} 1 & 1 & 1 \\ 1 & -2 & 0 \\ 0 & 1 & -1 \end{pmatrix}}_A \underbrace{\begin{pmatrix} a \\ b \\ c \end{pmatrix}}_{\mathbf{x}} = \underbrace{\begin{pmatrix} 20 \\ 0 \\ 10 \end{pmatrix}}_{\mathbf{b}}.$$

This can be solved as follows:

```
1 >> A=[1,1,1;1,-2,0;0,1,-1];
2 >> b=[20;0;10];
3 >> A\b
4 ans =
5     15.0000
6     7.5000
7    -2.5000
```

i Note

The [MATLAB website](#) shows the following flowcharts for how $A \setminus b$ classifies the problem before solving it.

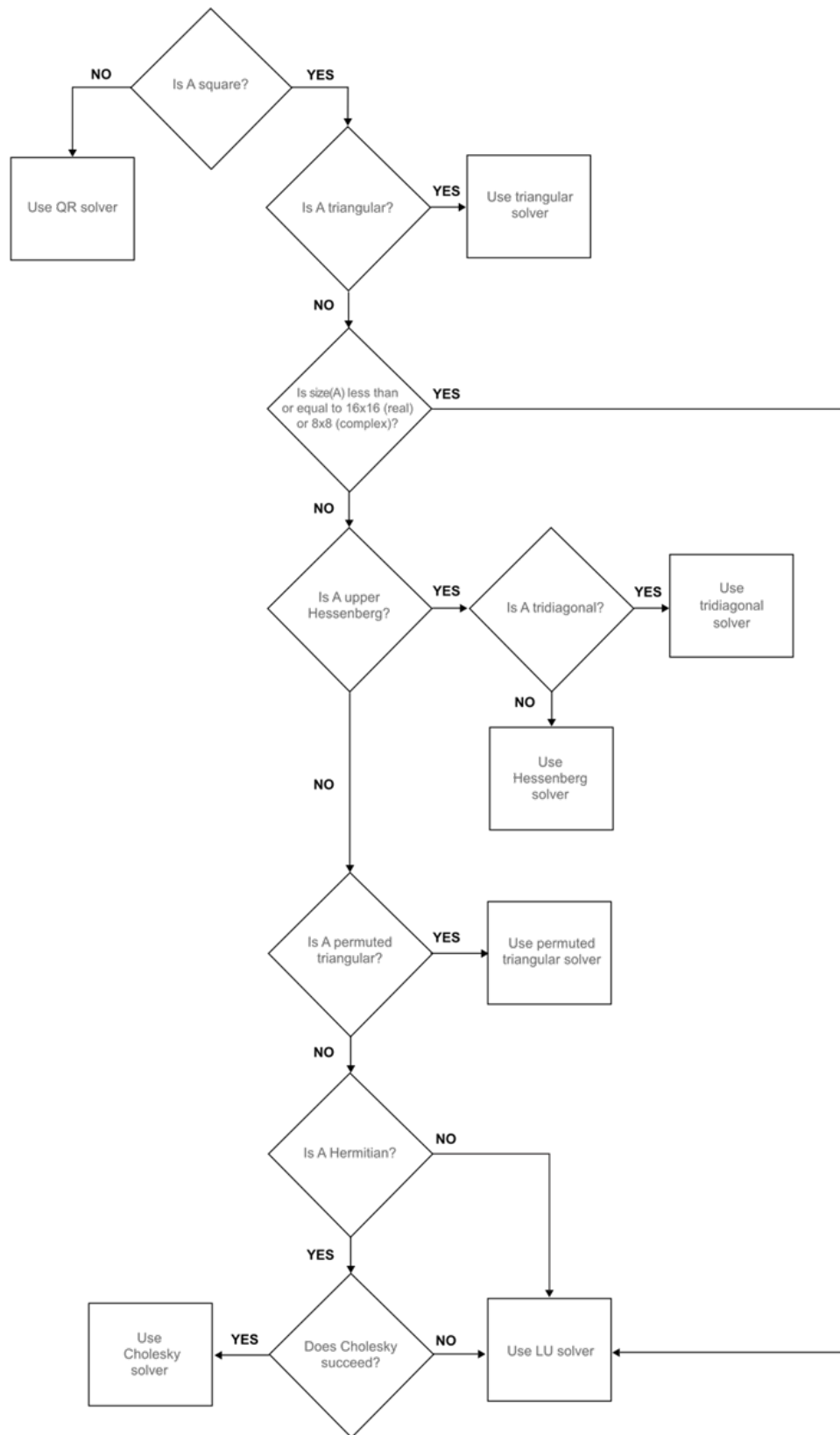


Figure 2.1: If the matrix A is full.

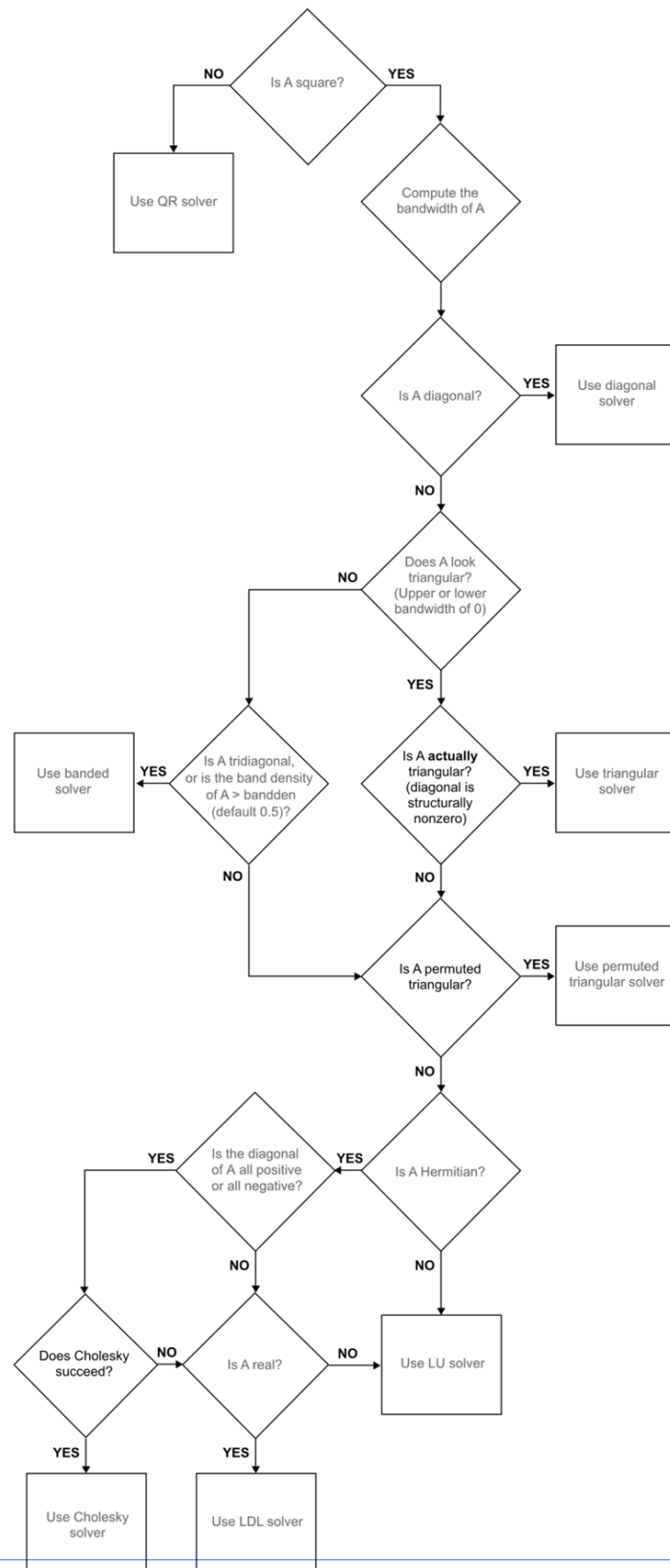


Figure 2.2: If the matrix A is sparse.

2.5 Exercises

Exercise 2.1

Let A and B be invertible matrices, $p \in \mathbb{N}$ and $\lambda \in \mathbb{R}$. The condition number κ_p has the following properties:

- i. $\kappa_p(A) \geq 1$;
- ii. If A is an orthogonal matrix ($A^{-1} = A^T$), then $\kappa_2(A) = 1$;
- iii. $\kappa_2(A^T) = \kappa_2(A^{-1}) = \kappa_2(A)$;
- iv. $\kappa_p(\lambda A) = \kappa_p(A)$;
- v. $\kappa_p(AB) \leq \kappa_p(A)\kappa_p(B)$.

You may use the fact that $\|A\|_2 = \|A^T\|_2$.

Solution 2.1

$$\kappa_p(A) = \|A\|_p \|A^{-1}\|_p$$

- i. Using the submultiplicative property of matrices

$$\kappa_p(A) = \|A\|_p \|A^{-1}\|_p \geq \|AA^{-1}\|_p = \|\mathcal{I}\|_p = 1.$$

- ii. If A is orthogonal ($A^{-1} = A^T$), then the submultiplicative property of matrices becomes an equality, so

$$\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 = \|AA^{-1}\|_2 = \|\mathcal{I}\|_2 = 1.$$

- iii.

$$\begin{aligned} \bullet \kappa_2(A^T) &= \|A^T\|_2 \|(A^{-1})^T\|_2 = \|A^T\|_2 \|A^{-1T}\|_2 \\ &= \|A\|_2 \|A^{-1}\|_2 = \kappa_2(A). \end{aligned} \quad (2.6)$$

$$\bullet \kappa_2(A^{-1}) = \|A^{-1}\|_2 = \|(A^{-1})^{-1}\|_2 = \|A\|_2 \|A^{-1}\|_2 = \kappa_2(A).$$

- iv.

$$\begin{aligned} \kappa_p(\lambda A) &= \|\lambda A\|_p \|(\lambda A)^{-1}\|_p = \|\lambda A\|_p \left\| \frac{1}{\lambda} A^{-1} \right\|_p = \\ &= |\lambda| \|A\|_p \frac{1}{|\lambda|} \|A^{-1}\|_p = \|A\|_p \|A^{-1}\|_p = \kappa_p(A). \end{aligned} \quad (2.7)$$

v.

$$\begin{aligned}\kappa_p(AB) &= \|AB\|_p \|(AB)^{-1}\|_p = \|AB\|_p \|A^{-1}B^{-1}\|_p \\ &\leq \|A\|_p \|B\|_p \|B^{-1}\|_p \|A^{-1}\|_p = \kappa_p(A)\kappa_p(B). \quad (2.8)\end{aligned}$$

Exercise 2.2

Solve the following linear systems of the form $A\mathbf{x} = \mathbf{b}$ using the following direct methods:

i. Direct substitution

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 5 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 3 \\ 9 \\ 0 \end{pmatrix}$$

ii. Backward substitution

$$A = \begin{pmatrix} 4 & 2 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & 10 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

iii. Forwards substitution

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 4 & 5 & 0 \\ 9 & 1 & 2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

iv. TDMA

$$A = \begin{pmatrix} 2 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \end{pmatrix}$$

v. Cramer's Rule

$$A = \begin{pmatrix} 1 & 0 & 2 \\ 2 & 1 & 2 \\ -1 & 0 & 0 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 12 \\ 0 \\ 6 \end{pmatrix}$$

Solutions 2.2

$$\text{i. } \mathbf{x} = \begin{pmatrix} 3 \\ -4.5 \\ 0 \end{pmatrix}$$

$$\text{ii. } \mathbf{x} = \begin{pmatrix} 0 \\ 0.45 \\ 0.1 \end{pmatrix}$$

$$\text{iii. } \mathbf{x} = \begin{pmatrix} 0 \\ 0 \\ 0.5 \end{pmatrix}$$

$$\text{iv. } \mathbf{x} = \begin{pmatrix} 1.5 \\ 2 \\ 2.5 \\ 2 \\ 1.5 \end{pmatrix}$$

$$\text{v. } \mathbf{x} = \begin{pmatrix} 0 \\ -12 \\ 6 \end{pmatrix}$$

Exercise 2.3

Using the formulas derived, write MATLAB codes that can perform:

- Direct substitution
- Backward substitution
- Forward substitution
- TDMA
- Cramer's Rule

Use the examples in Exercise 2.2 as test cases.

Solutions 2.3

Direct Substitution

```

1 function D_Sub
2
3 A=diag([1,-2,5]);
4 b=[3;9;0];
5
6 if isdiag(A)~=1
7     error('A must be a diagonal matrix')
8 end
9
10 if any(diag(A)==0)
11     error('All diagonal terms must be non-zero')
12 end
13
14 N=length(b);
15
16 x=zeros(1,N);
17
18 for n=1:N
19     x(n)=b(n)/A(n,n);
20 end
21
22 disp('x=')
23 disp(x)
24
25 end

```

Backward Substitution

```

1 function B_Sub
2
3 A=[4,2,1;0,2,1;0,0,10];
4 b=[1;1;1];
5
6 if istriu(A)~=1
7     error('A must be an upper triangluar matrix')
8 end
9
10 if any(diag(A)==0)
11     error('All diagonal terms must be non-zero')
12 end
13
14 N=length(b);
15
16 x=zeros(1,N);
17
18 x(N)=b(N)/A(N,N);
19
20 for n=N-1:-1:1
21     S=0;
22     for i=n+1:1:N
23         S=S+A(n,i)*x(i);
24     end
25     x(n)=(b(n)-S)/A(n,n);
26 end
27
28 disp('x=')
29 disp(x)
30
31 end

```

Forward Substitution

```

1 function F_Sub
2
3 A=[1,0,0;4,5,0;9,1,2];
4 b=[0;0;1];
5
6 if istril(A)~=1
7     error('A must be a lower triangular matrix')
8 end
9
10 if any(diag(A)==0)
11     error('All diagonal terms must be non-zero')
12 end
13
14 N=length(b);
15
16 x=zeros(1,N);
17
18 x(1)=b(1)/A(1,1);
19
20 for n=2:1:N
21     S=0;
22     for i=1:1:n-1
23         S=S+A(n,i)*x(i);
24     end
25     x(n)=(b(n)-S)/A(n,n);
26 end
27
28 disp('x=')
29 disp(x)
30
31 end

```

TDMA

```

1  function TDMA
2
3  m=[2,2,2,2,2];
4  l=[-1,-1,-1,-1];
5  r=[-1,-1,-1,-1];
6  A=diag(m)+diag(l,-1)+diag(r,1);
7
8  b=[1;0;1;0;1];
9
10 N=length(b);
11
12 R=zeros(1,N);
13 B=zeros(1,N);
14 x=zeros(1,N);
15
16 l=[0,l];
17 r=[r,0];
18
19 R(1)=r(1)/m(1);
20 for n=2:1:N-1
21     R(n)=r(n)/(m(n)-l(n)*R(n-1));
22 end
23
24 B(1)=b(1)/m(1);
25 for n=2:1:N
26     B(n)=(b(n)-l(n)*B(n-1))/(m(n)-l(n)*R(n-1));
27 end
28
29 x(N)=B(N);
30 for n=N-1:-1:1
31     x(n)=B(n)-R(n)*x(n+1);
32 end
33
34 disp('x=')
35 disp(x)
36
37 end

```

Cramer's Rule

```

1  function Cramer
2
3  A=[1,0,2;2,1,2;-1,0,1];
4  b=[12;0;6];
5
6  N=length(b);
7
8  d=det(A);
9
10 x=zeros(N,1);
11
12 for n=1:1:N
13     AA=A;
14     AA(:,n)=b;
15     x(n)=det(AA)/d;
16 end
17
18 disp('x=')
19 disp(x)
20
21 end

```

Part II

Solving Initial Value Problems

This section will develop the techniques used to solve initial value problems.

3 The Euler Method

Consider the first order ordinary differential equation (ODE)

$$\frac{dy}{dt} = f(t, y), \quad t \in [t_0, t_f]$$

where f is a known function, t_0 is an initial time and t_f is the final time. An *initial condition* can be prescribed to this differential equation which will assign a “starting value” for the unknown function y at the starting time as $y(t_0) = y_0$. The combination of the first order ODE and the initial value gives the ***Initial Value Problem*** (or ***IVP***)

$$\frac{dy}{dt} = f(t, y) \quad \text{with} \quad y(t_0) = y_0, \quad t \in [t_0, t_f].$$

There are many analytic methods for solving first order ordinary differential equations, but they all hold restrictions, like linearity or homogeneity. This chapter will develop the simplest numerical technique for solving any first order ordinary differential equation, this method is called the ***Euler Method***.

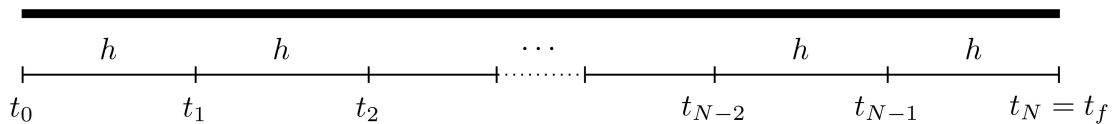
Consider the following first order IVP

$$\frac{dy}{dt} = f(t, y), \quad \text{with} \quad y(t_0) = y_0 \quad t \in [t_0, t_f].$$

The function f is known and in most cases, is assumed to be “well-behaved” (does not have discontinuities or sharp corners). The term y_0 is known as the ***Initial Value*** of the function y at the starting time t_0 . Solving this initial value problem is essentially finding an unknown curve $y(t)$ that starts at the point (t_0, y_0) and ends at time t_f .

The first step in the Euler method (as is the case in most numerical techniques) is to discretise the domain. This changes the domain from the continuous interval $[t_0, t_f]$ to N subintervals, each with constant¹ width h (sometimes also denoted δt), which is known as the ***Stepsize***. The discretised interval will be the set of points

$$\{t_0, t_0 + h, t_0 + 2h, \dots, t_0 + Nh\}.$$



¹In most cases, the interval width h is constant but more advanced numerical techniques have different subinterval widths.

The aim of the numerical procedure is to start from the starting point (t_0, y_0) and progressively find consequent points until the final time t_f is reached.

The Euler method uses the gradient, namely $\frac{dy}{dt}$, at the starting point (t_0, y_0) in order to find the value of y at the subsequent point which will be labelled (t_1, y_1) . This will, in turn, determine the new gradient at (t_1, y_1) and this process is then continued until the final time is reached. The smaller the value of h is, the more points there will be between t_0 and t_f resulting in a more accurate final solution to the initial value problem.

The accuracy of the Euler method is usually characterised by how small h is or how large N is. Since the stepsize may not always give an appropriate subdivision (like dividing the interval $[0, 1]$ into subintervals of width 0.4), then the number of subdivisions N can be used to find an appropriate h by using

$$h = \frac{t_f - t_0}{N}.$$

3.1 Steps of the Euler Method

Consider the IVP

$$\frac{dy}{dt} = f(t, y), \quad \text{with} \quad y(t_0) = y_0 \quad t \in [t_0, t_f].$$

Parallel Example

The steps of the Euler method will be explained theoretically and applied to this IVP in parallel to demonstrate the steps:

$$\frac{dy}{dt} = 6 - 2y \quad \text{with} \quad y(0) = 0, \quad t \in [0, 2].$$

In this case, the function on the RHS is $f(t, y) = 6 - 2y$. Note that this IVP has the exact solution

$$y(t) = 3 - 3e^{-2t}.$$

1. Discretise the interval $[t_0, t_f]$ with stepsize h to form the set of points

$$\{t_0, t_0 + h, t_0 + 2h, \dots, t_0 + Nh\}.$$

Interval Discretisation

Suppose that the interval $[0, 2]$ is to be split into 5 subintervals, then $N = 5$ and

$$h = \frac{t_f - t_0}{N} = \frac{2 - 0}{5} = 0.4.$$

Therefore the discretised points are

$$\{0.0, 0.4, 0.8, 1.2, 1.6, 2.0\}.$$

Note that N denotes the number of subintervals and **not** the number of points, that would be $N + 1$ points since the starting point is 0.

2. At the starting point (t_0, y_0) , the gradient is known since

$$y'(t_0) = f(t_0, y_0).$$

Gradient at (t_0, y_0)

At the initial point,

$$y'(t_0) = f(t_0, y_0) \implies y'(0) = f(0, 0) = 6 - 2(0) = 6.$$

So the starting gradient is 6.

3. The next step is to find the value of y at the subsequent time $t_1 = t_0 + h$. For this purpose, consider the Taylor series expansion of y at $t = t_1$,

$$y(t_1) = y(t_0 + h) = y(t_0) + hy'(t_0) + \frac{h^2}{2!}y''(t_0) + \mathcal{O}(h^3).$$

i Note

The term $\mathcal{O}(h^3)$ simply means that the terms after this point have a common factor of h^3 and these terms are regarded as *higher order terms* and can be neglected since they are far smaller than the first terms provided h is small.

Since h is assumed to be sufficiently small, then all terms higher order terms, in this case h^2 or higher, can be neglected (i.e. $h^n \approx 0$ for $n \geq 2$). Therefore

$$y(t_1) \approx y(t_0) + hy'(t_0).$$

Let Y_1 denote the *approximated value* of the solution at the point t_1 , i.e. $Y_1 \approx y(t_1)$, so in this case,

$$Y_1 = y_0 + hy'(t_0). \quad (3.1)$$

This determines the value of Y_1 which is an approximation to $y(t_1)$.

Calculating Y_1

The point Y_1 can be calculated as follows:

$$Y_1 = y_0 + hy'(t_0) = 0 + (0.4)(6) = 2.4.$$

This means that the next point is $(t_1, Y_1) = (0.4, 2.4)$.

4. This iteration can be continued to find Y_{n+1} (which is the approximate value of $y(t_{n+1})$) for all $n = 1, 2, \dots, N - 1$

$$Y_{n+1} = Y_n + hy'(t_n) \quad \text{where} \quad y'(t_n) = f(t_n, Y_n).$$

Calculating Y_n

The values of Y_2, Y_3, Y_4 and Y_5 can be calculated as follows:

$$Y_2 : \quad y'(t_1) = f(t_1, Y_1) \quad \implies \quad y'(0.4) = f(0.4, 2.4) = 6 - 2(2.4) = 1.2$$

$$\implies \quad Y_2 = Y_1 + hy'(t_1) = 2.4 + (0.4)(1.2) = 2.88$$

$$Y_3 : \quad y'(t_2) = f(t_2, Y_2) \quad \implies \quad y'(0.8) = f(0.8, 2.88) = 6 - 2(2.88) = 0.24$$

$$\implies \quad Y_3 = Y_2 + hy'(t_2) = 2.88 + (0.4)(0.24) = 2.976$$

$$Y_4 : \quad y'(t_3) = f(t_3, Y_3) \quad \implies \quad y'(1.2) = f(1.2, 2.976) = 6 - 2(2.976) = 0.048$$

$$\implies \quad Y_4 = Y_3 + hy'(t_3) = 2.976 + (0.4)(0.048) = 2.9952$$

$$Y_5 : \quad y'(t_4) = f(t_4, Y_4) \quad \implies \quad y'(1.6) = f(1.6, 2.9952) = 6 - 2(2.9952) = 0.0096$$

$$\implies \quad Y_5 = Y_4 + hy'(t_4) = 2.9952 + (0.4)(0.0096) = 2.99904$$

5. The solution to the IVP can now be approximated by the function that passes through the points

$$(t_0, Y_0), \quad (t_1, Y_1), \quad \dots \quad (t_N, Y_N).$$

Solution to the IVP

The approximate solution to the IVP

$$\frac{dy}{dt} = 6 - 2y \quad \text{with} \quad y(0) = 0, \quad t \in [0, 2]$$

is the function that passes through the points:

$$(0, 0), \quad (0.4, 2.4), \quad (0.8, 2.88), \quad (1.2, 2.976), \quad (1.6, 2.9952), \quad (2, 2.99904).$$

This is a good approximation since the *exact* locations, as per the exact solution are, (to 4 decimal places):

$$(0, 0), \quad (0.4, 1.6520), \quad (0.8, 2.3943), \quad (1.2, 2.7278), \quad (1.6, 2.8777), \quad (2, 2.9451)$$

which is not bad for such a coarse interval breakdown.

The Euler method needs N steps to complete and every step $n \in \{1, 2, \dots, N\}$ requires finding $y'(t_{n-1}) = f(t_{n-1}, y_{n-1})$ and $Y_n = Y_{n-1} + hy'(t_{n-1})$. Of course, the larger N is, the smaller h becomes, meaning that more steps will be required but the solution will be closer to the exact solution

Notice that the terms on the right hand side of Equation 3.1 are all known and for this reason, the Euler method is known as an **Explicit Method**.

3.2 Accuracy

Consider the Taylor series expansion for the function y at the point $t_1 = t_0 + h$

$$y(t_1) = y(t_0 + h) = y(t_0) + hy'(t_0) + \frac{h^2}{2!}y''(t_0) + \mathcal{O}(h^3).$$

Using **Taylor's Theorem**², this can be written as

$$y(t_1) = y(t_0 + h) = y(t_0) + hy'(t_0) + \frac{h^2}{2!}y''(\tau_1)$$

for some point τ_1 between t_0 and t_1 . The Euler method determines the approximation Y_1 to the function y at the point t_1 , particularly,

$$Y_1 = y(t_0) + hy'(t_0) \approx y(t_1).$$

The **Local Truncation Error** at the first step, denoted e_1 , is defined as the absolute difference between the exact and approximated values at the first step, and this is given by

$$e_1 = |y(t_1) - Y_1| = \frac{h^2}{2!} |y''(\tau_1)|.$$

This can be done for all the locations to give a list of local truncation errors $e_1, e_2, e_3, \dots, e_N$. Note that *technically*, these errors are hypothetical since the *exact solution* y , and thus $y(t_n)$, are not known but these are put as placeholders to establish the full accuracy of the method. In this case, the local truncation error e is said to be of *second order* since $e = \mathcal{O}(h^2)$.

As the iteration progresses, the errors will accumulate to result in a **Global Integration Error** denoted E . In this case, the global integration error is

$$E = |y(t_f) - Y_N|.$$

The global integration error has to be at most the accumulation of all the local truncation errors, namely

$$E = |y(t_f) - Y_N| \leq \underbrace{\sum_{n=1}^N e_n}_{\text{sum of all local truncation errors}} = \sum_{n=1}^N \frac{h^2}{2!} |y''(\tau_n)| = h^2 \sum_{n=1}^N \frac{1}{2} |y''(\tau_n)|.$$

²**Taylor's Theorem** states that for a function f that is at least $N + 1$ times differentiable in the open interval (x, x_0) (or (x_0, x)), then

$$\begin{aligned} f(x) = & f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2!}f''(x_0)(x - x_0)^2 + \frac{1}{3!}f'''(x_0)(x - x_0) \\ & + \dots + \frac{1}{N!}f^{(N)}(x_0)(x - x_0)^N + \frac{1}{(N+1)!}f^{(N+1)}(\xi)(x - x_0)^{N+1} \end{aligned}$$

for some point ξ between x and x_0 .

$$\implies E \leq h^2 \sum_{n=1}^N \frac{1}{2} |y''(\tau_n)| \quad (3.2)$$

A bound for the sum needs to be found in order bound the global integration error. To this end, consider the set of the second derivatives in the sum above, i.e.

$$\left\{ \frac{1}{2} |y''(\tau_1)|, \frac{1}{2} |y''(\tau_2)|, \dots, \frac{1}{2} |y''(\tau_n)| \right\}.$$

Since all these terms take a finite value, then at least one of these terms must be larger than all the rest, this is denoted M and can be written as

$$M = \max \left\{ \frac{1}{2} |y''(\tau_1)|, \frac{1}{2} |y''(\tau_2)|, \dots, \frac{1}{2} |y''(\tau_n)| \right\}.$$

This can also be expressed differently as

$$M = \max_{\tau \in [t_0, t_f]} \left\{ \frac{1}{2} |y''(\tau)| \right\}.$$

Therefore, since

$$\frac{1}{2} |y''(\tau_n)| \leq M \quad \text{for all } n = 1, 2, \dots, N$$

then

$$\sum_{n=1}^N \frac{1}{2} |y''(\tau_n)| \leq \sum_{n=1}^N M = NM.$$

Thus, returning back to the expression for E in Equation 3.2

$$E \leq h^2 \sum_{n=1}^N \frac{1}{2} |y''(\tau_n)| \leq NMh^2 = Mh \cdot (Nh) = Mh(t_f - t_0) = \mathcal{O}(h).$$

Hence, the global integration error $E = \mathcal{O}(h)$, this means that the Euler method is a **First Order Method**. This means that both h and the global integration error behave linearly to one another, so if h is halved, then the global integration error is halved as well.

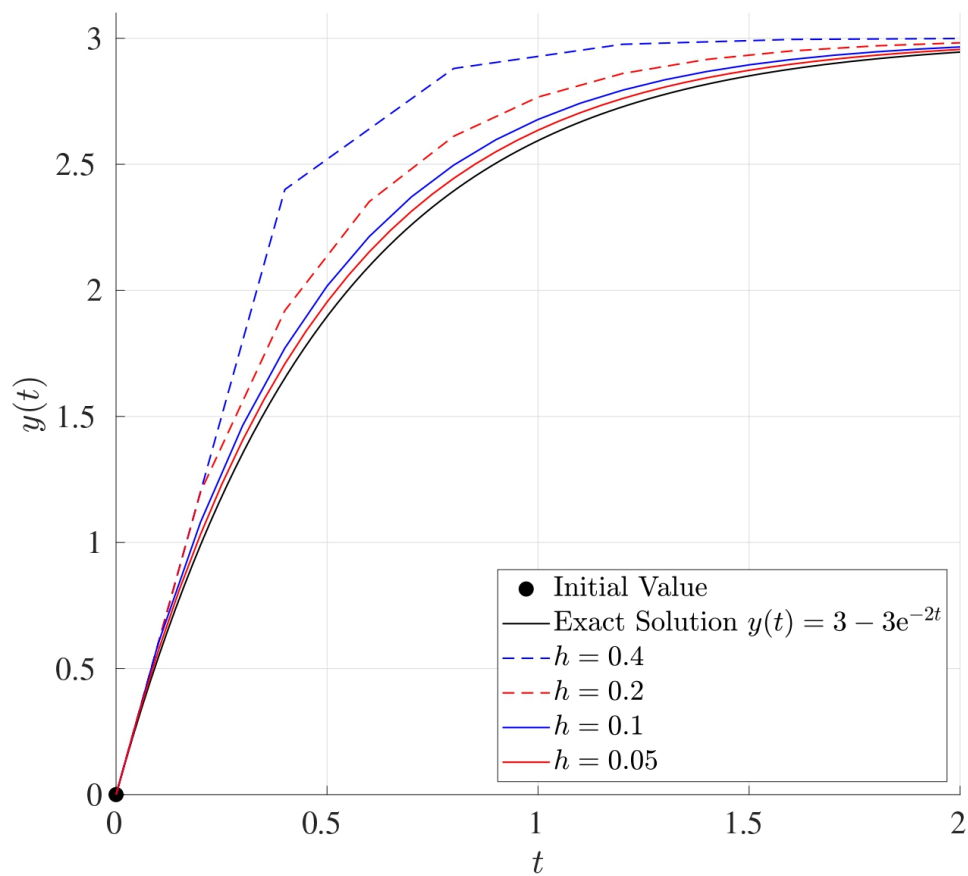
In conclusion, the local truncation error of the Euler method is $e = \mathcal{O}(h^2)$ while the global integration error $E = \mathcal{O}(h)$ when h is small.

Different Stepsizes

Returning to the IVP

$$\frac{dy}{dt} = 6 - 2y \quad \text{with} \quad y(0) = 0, \quad t \in [0, 2].$$

The Euler method can be repeated for different values of h and these can be seen in the figure below.



The table below shows the global integration error for the different values of h :

h	E
0.4	0.05399
0.2	0.03681
0.1	0.02036
0.05	0.01060

When the value of h is halved, the global integration error is approximately halved as well.

3.3 Set of IVPs

So far, the Euler Method has been used to solve a single IVP, however this can be extended to solving a set of linear IVPs.

Consider the set of K linear IVPs defined on the interval $[t_0, t_f]$:

$$\begin{aligned}\frac{dy_1}{dt} &= a_{11}y_1 + a_{12}y_2 + \cdots + a_{1K}y_K + b_1, & y_1(t_0) &= \tilde{y}_1 \\ \frac{dy_2}{dt} &= a_{21}y_1 + a_{22}y_2 + \cdots + a_{2K}y_K + b_2, & y_2(t_0) &= \tilde{y}_2 \\ &\vdots & & \\ \frac{dy_K}{dt} &= a_{K1}y_1 + a_{K2}y_2 + \cdots + a_{KK}y_K + b_K, & y_K(t_0) &= \tilde{y}_K\end{aligned}$$

where, for $i, j = 1, 2, \dots, K$, the functions $y_i = y_i(t)$ are unknown, a_{ij} are known constant coefficients and b_i are all known (these can generally depend on t).

This set of initial value problems need to be written in matrix form as

$$\begin{aligned}\frac{d\mathbf{y}}{dt} &= A\mathbf{y} + \mathbf{b} \quad \text{with} \quad \mathbf{y}(t_0) = \mathbf{y}_0, \quad t \in [t_0, t_f] \\ \text{where } \mathbf{y}(t) &= \begin{pmatrix} y_1(t) \\ y_2(t) \\ \vdots \\ y_K(t) \end{pmatrix}, \quad A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1K} \\ a_{21} & a_{22} & \cdots & a_{2K} \\ \vdots & \vdots & \ddots & \vdots \\ a_{K1} & a_{K2} & \cdots & a_{KK} \end{pmatrix}, \\ \mathbf{b} &= \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_K \end{pmatrix}, \quad \mathbf{y}_0 = \begin{pmatrix} \tilde{y}_1 \\ \tilde{y}_2 \\ \vdots \\ \tilde{y}_K \end{pmatrix}.\end{aligned}$$

In this case, $\mathbf{y}(t)$ is the unknown solution vector, A is a matrix of constants, $\mathbf{y}_0$ is the vector of initial values and $\mathbf{b}$ is a vector of known terms (possibly depending on t) and is referred to as the **Inhomogeneity** or **Forcing Term**.

The Euler iteration would be performed in a similar way as before. First, the interval $[t_0, t_f]$ needs to be discretised into N equally spaced subintervals, each of width h to give the set of discrete times $(t_0, t_1, \dots, t_N)$ where $t_n = t_0 + nh$ for $n = 0, 1, \dots, N$. Let $\mathbf{Y}_n$ be the approximation to the function vector $\mathbf{y}$ at the time $t = t_n$, then

$$\mathbf{Y}_{n+1} = \mathbf{Y}_n + h\mathbf{y}'(t_n) \quad \text{where} \quad \mathbf{y}'(t_n) = A\mathbf{Y}_n + \mathbf{b}_n \quad \text{for} \quad n = 0, 1, 2, \dots, N-1$$

subject to the initial values $\mathbf{Y}_0 = \mathbf{y}_0$. (Note that if the vector $\mathbf{b}$ depends on t , then $\mathbf{b}_n = \mathbf{b}(t_n)$.)

Sets of IVPs

Consider the two coupled IVPs on the interval $[0, 1]$:

$$\begin{aligned}\frac{dy}{dt} &= y + 2z, & y(0) &= 1 \\ \frac{dz}{dt} &= \frac{3}{2}y - z, & z(0) &= 0\end{aligned}$$

Before attempting to solve this set of IVPs, it needs to be written in matrix form as

$$\frac{d\mathbf{y}}{dt} = A\mathbf{y} + \mathbf{b} \quad \text{with} \quad \mathbf{y}(0) = \mathbf{y}_0.$$

In this case,

$$\mathbf{y}(t) = \begin{pmatrix} y(t) \\ z(t) \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 2 \\ \frac{3}{2} & -1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \mathbf{y}_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Let $N = 5$, so

$$h = \frac{t_f - t_0}{N} = \frac{1 - 0}{5} = 0.2.$$

The Euler iteration will be

$$\mathbf{Y}_{n+1} = \mathbf{Y}_n + h\mathbf{y}'(t_n) \quad \text{where} \quad \mathbf{y}'(t_n) = A\mathbf{Y}_n + \mathbf{b}_n \quad \text{for} \quad n = 0, 1, 2, 3, 4.$$

This can be written as

$$\mathbf{Y}_{n+1} = \mathbf{Y}_n + h[A\mathbf{Y}_n + \mathbf{b}_n] \quad \text{for} \quad n = 0, 1, 2, 3, 4$$

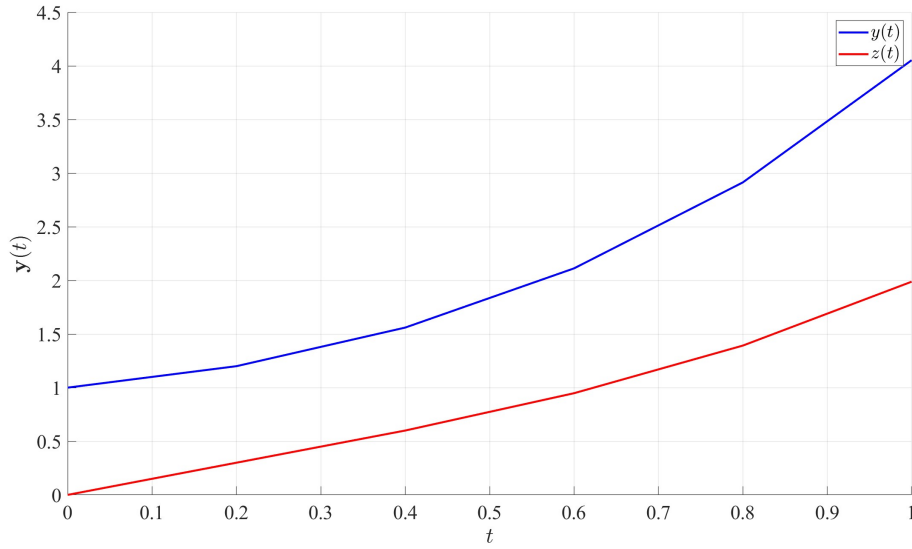
keeping in mind that $t_n = hn = 0.2n$ the vector $\mathbf{b}_n = \mathbf{b}(t_n) = \mathbf{0}$ and $\mathbf{Y}_0 = \mathbf{y}_0$:

$$\begin{aligned}\mathbf{Y}_1 &= \mathbf{Y}_0 + 0.2[A\mathbf{Y}_0 + \mathbf{b}_0] = \begin{pmatrix} 1 \\ 0 \end{pmatrix} + 0.2 \left[\begin{pmatrix} 1 & 2 \\ \frac{3}{2} & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} \right] \\ &= \begin{pmatrix} 1.2 \\ 0.3 \end{pmatrix} \\ \mathbf{Y}_2 &= \mathbf{Y}_1 + 0.2[A\mathbf{Y}_1 + \mathbf{b}_1] = \begin{pmatrix} 1.2 \\ 0.3 \end{pmatrix} + 0.2 \left[\begin{pmatrix} 1 & 2 \\ \frac{3}{2} & -1 \end{pmatrix} \begin{pmatrix} 1.2 \\ 0.3 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} \right] \\ &= \begin{pmatrix} 1.56 \\ 0.6 \end{pmatrix} \\ \mathbf{Y}_3 &= \mathbf{Y}_2 + 0.2[A\mathbf{Y}_2 + \mathbf{b}_2] = \begin{pmatrix} 1.56 \\ 0.6 \end{pmatrix} + 0.2 \left[\begin{pmatrix} 1 & 2 \\ \frac{3}{2} & -1 \end{pmatrix} \begin{pmatrix} 1.56 \\ 0.6 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} \right] \\ &= \begin{pmatrix} 2.112 \\ 0.948 \end{pmatrix}\end{aligned}$$

$$\begin{aligned} \mathbf{Y}_4 &= \mathbf{Y}_3 + 0.2[AY_3 + \mathbf{b}_3] = \begin{pmatrix} 2.112 \\ 0.948 \end{pmatrix} + 0.2 \left[\begin{pmatrix} 1 & 2 \\ \frac{3}{2} & -1 \end{pmatrix} \begin{pmatrix} 2.112 \\ 0.948 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} \right] \\ &= \begin{pmatrix} 2.9136 \\ 1.3920 \end{pmatrix} \end{aligned}$$

$$\begin{aligned} \mathbf{Y}_5 &= \mathbf{Y}_4 + 0.2[AY_4 + \mathbf{b}_4] = \begin{pmatrix} 2.9136 \\ 1.3920 \end{pmatrix} + 0.2 \left[\begin{pmatrix} 1 & 2 \\ \frac{3}{2} & -1 \end{pmatrix} \begin{pmatrix} 2.9136 \\ 1.3920 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} \right] \\ &= \begin{pmatrix} 4.0531 \\ 1.9877 \end{pmatrix} \end{aligned}$$

therefore $y(1) = 4.0531$, $z(1) = 1.9877$.



3.4 Higher Order IVPs

The previous sections solved one first order IVP and a set of first order IVPs. What happens if a higher order IVP is to be solved? Or a set of higher order IVPs? The difference will be minimal, subject to a few manipulations first.

Consider the K^{th} order linear IVP on the interval $[t_0, t_f]$

$$\frac{d^K y}{dt^K} + a_{K-1} \frac{d^{K-1} y}{dt^{K-1}} + \cdots + a_2 \frac{d^2 y}{dt^2} + a_1 \frac{dy}{dt} + a_0 y = f(t) \quad (3.3)$$

where $a_k \in \mathbb{R}$ and f is a known function. This IVP is to be solved subject to the initial conditions

$$y(t_0) = \eta_0, \quad \frac{dy}{dt}(t_0) = \eta_1 \quad \cdots \quad \frac{d^{K-1} y}{dt^{K-1}}(t_0) = \eta_{K-1}.$$

This K^{th} order IVP can be written as a set of K first order IVPs. Indeed, let the functions y_k be given by

$$\begin{aligned} y_1(t) &= \frac{dy}{dt} \\ y_2(t) &= y_1'(t) = \frac{d^2y}{dt^2} \\ y_3(t) &= y_2'(t) = \frac{d^3y}{dt^3} \\ &\vdots \\ y_{K-3}(t) &= y_{K-4}'(t) = \frac{d^{K-3}y}{dt^{K-3}} \\ y_{K-2}(t) &= y_{K-3}'(t) = \frac{d^{K-2}y}{dt^{K-2}} \\ y_{K-1}(t) &= y_{K-2}'(t) = \frac{d^{K-1}y}{dt^{K-1}} \end{aligned}$$

Notice that

$$\begin{aligned} \frac{dy_{K-1}}{dt} &= \frac{d^K y}{dt^K} = -a_{K-1} \frac{d^{K-1}y}{dt^{K-1}} - \cdots - a_2 \frac{d^2y}{dt^2} - a_1 \frac{dy}{dt} - a_0 y + f(t) \\ &= -a_{K-1} y_{K-1} - \cdots - a_2 y_2 - a_1 y_1 - a_0 y + f(t) \end{aligned}$$

Let $\mathbf{y}$ be the vector of the unknown functions $y, y_1, y_2, \dots, y_{K-1}$. This means that the IVP in Equation 3.3 can be written in matrix form $\mathbf{y}' = A\mathbf{y} + \mathbf{b}$ as follows:

$$\begin{aligned} \frac{d\mathbf{y}}{dt} &= \frac{d}{dt} \begin{pmatrix} y \\ y_1 \\ y_2 \\ \vdots \\ y_{K-3} \\ y_{K-2} \\ y_{K-1} \end{pmatrix} = \begin{pmatrix} y' \\ y_1' \\ y_2' \\ \vdots \\ y_{K-3}' \\ y_{K-2}' \\ y_{K-1}' \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{K-2} \\ y_{K-1} \\ \frac{d^K y}{dt^K} \end{pmatrix} \\ &= \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{K-2} \\ y_{K-1} \\ -a_{K-1}y_{K-1} - \cdots - a_2y_2 - a_1y_1 - a_0y + f(t) \end{pmatrix} \end{aligned}$$

$$= \underbrace{\begin{pmatrix} 0 & 1 & 0 & \dots & 0 & 0 & 0 \\ 0 & 0 & 1 & \dots & 0 & 0 & 0 \\ 0 & 0 & 0 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 0 & 1 & 0 \\ 0 & 0 & 0 & \dots & 0 & 0 & 1 \\ -a_0 & -a_1 & -a_2 & \dots & -a_{K-3} & -a_{K-2} & -a_{K-1} \end{pmatrix}}_A \underbrace{\begin{pmatrix} y \\ y_1 \\ y_2 \\ \vdots \\ y_{K-3} \\ y_{K-2} \\ y_{K-1} \end{pmatrix}}_y + \underbrace{\begin{pmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \\ f(t) \end{pmatrix}}_b = A\mathbf{y} + \mathbf{b}.$$

The initial condition vector will be

$$\mathbf{y}_0 = \begin{pmatrix} y(0) \\ y_1(0) \\ y_2(0) \\ \vdots \\ y_{K-3}(0) \\ y_{K-2}(0) \\ y_{K-1}(0) \end{pmatrix} = \begin{pmatrix} y(0) \\ \frac{dy}{dt}(0) \\ \frac{d^2y}{dt^2}(0) \\ \vdots \\ \frac{d^{K-3}y}{dt^{K-3}}(0) \\ \frac{d^{K-2}y}{dt^{K-2}}(0) \\ \frac{d^{K-1}y}{dt^{K-1}}(0) \end{pmatrix} = \begin{pmatrix} \eta_0 \\ \eta_1 \\ \eta_2 \\ \vdots \\ \eta_{K-3} \\ \eta_{K-2} \\ \eta_{K-1} \end{pmatrix}.$$

The matrix A is called the **Companion Matrix** and is a matrix with 1 on the super diagonal and the last row is the minus of the coefficients in the higher order IVP, and zeros otherwise. Now that the K^{th} order IVP has been converted into a set of K linear IVPs, it can be solved just as in Section 3.3. Note that *any* linear K^{th} order IVP can always be converted into a set of K first order IVPs but the converse is not always possible.

Higher Order IVPs

Consider the following higher order IVP

$$\frac{d^4y}{dt^4} - 8\frac{d^3y}{dt^3} + 7\frac{d^2y}{dt^2} - \frac{dy}{dt} + 2y = \cos(t) \quad \text{for } t \in \mathbb{R}_{\geq 0}$$

$$\text{with } y(0) = 4, \quad \frac{dy}{dt}(0) = 1, \quad \frac{d^2y}{dt^2}(0) = 3, \quad \frac{d^3y}{dt^3}(0) = 0.$$

Let $u = \frac{dy}{dt}$, $v = u' = \frac{d^2y}{dt^2}$ and $w = v' = \frac{d^3y}{dt^3}$. The derivatives of u, v and w are:

$$u' = v$$

$$v' = w$$

$$w' = \frac{d^4y}{dt^4} = 8\frac{d^3y}{dt^3} - 7\frac{d^2y}{dt^2} + \frac{dy}{dt} - 2y + \cos(t) = 8w - 7v + u + 2y + \cos(t)$$

Define the vector $\mathbf{y} = (y, u, v, w)^T$

$$\frac{d\mathbf{y}}{dt} = \frac{d}{dt} \begin{pmatrix} y \\ u \\ v \\ w \end{pmatrix} = \begin{pmatrix} u \\ v \\ w \\ \cos(t) + 8w - 7v + u - 2y \end{pmatrix}$$

$$= \underbrace{\begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -2 & 1 & -7 & 8 \end{pmatrix}}_A \underbrace{\begin{pmatrix} y \\ u \\ v \\ w \end{pmatrix}}_{\mathbf{y}} + \underbrace{\begin{pmatrix} 0 \\ 0 \\ 0 \\ \cos(t) \end{pmatrix}}_{\mathbf{b}(t)} = A\mathbf{y} + \mathbf{b}(t).$$

The initial condition vector will be

$$\mathbf{y}_0 = \begin{pmatrix} y(0) \\ u(0) \\ v(0) \\ w(0) \end{pmatrix} = \begin{pmatrix} y(0) \\ \frac{dy}{dt}(0) \\ \frac{d^2y}{dt^2}(0) \\ \frac{d^3y}{dt^3}(0) \end{pmatrix} = \begin{pmatrix} 4 \\ 1 \\ 3 \\ 0 \end{pmatrix}.$$

Now the IVP can be solved using the Euler method as before but only the first function is the most relevant, all others have been used as placeholders.

3.4.1 Sets of Higher Order IVPs

The method above can be extended into a set of higher order IVPs.

Set of Higher Order IVPs

Consider the following coupled system of higher order IVPs

$$y'' + 6y' + y = \sin(t), \quad z''' - 8z'' = 5y - 2y' + e^{2t}$$

$$\text{with } y(0) = 1, \quad \frac{dy}{dt}(0) = 2, \quad z(0) = 4, \quad \frac{dz}{dt}(0) = 1, \quad \frac{d^2z}{dt^2}(0) = 2$$

In the case of a coupled system, the vector function $\mathbf{y}$ should consist of all the unknown functions and their derivatives up to but not including their highest order derivative. In other words,

$$\begin{aligned} \frac{d\mathbf{y}}{dt} &= \frac{d}{dt} \begin{pmatrix} y \\ y' \\ z \\ z' \\ z'' \end{pmatrix} = \begin{pmatrix} y' \\ y'' \\ z' \\ z'' \\ z''' \end{pmatrix} = \begin{pmatrix} y' \\ -y - 6y' + \sin(t) \\ z' \\ z'' \\ 5y - 2y' + 8z'' + e^{2t} \end{pmatrix} \\ &= \underbrace{\begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ -1 & -6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 5 & -2 & 0 & 0 & 8 \end{pmatrix}}_A \underbrace{\begin{pmatrix} y \\ y' \\ z \\ z' \\ z'' \end{pmatrix}}_{\mathbf{y}} + \underbrace{\begin{pmatrix} 0 \\ \sin(t) \\ 0 \\ 0 \\ e^{2t} \end{pmatrix}}_{\mathbf{b}}. \end{aligned}$$

The vector of initial values would be

$$\mathbf{y}(0) = \begin{pmatrix} y(0) \\ y'(0) \\ z(0) \\ z'(0) \\ z''(0) \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 4 \\ 1 \\ 2 \end{pmatrix}.$$

Now this can be solved just as before with the most relevant terms being the first and third (since those are y and z).

3.4.2 Stability of a Set of ODEs

Consider the set of K homogeneous ODEs

$$\frac{d\mathbf{y}}{dt} = A\mathbf{y}.$$

Let $\lambda_1, \lambda_2, \dots, \lambda_K$ be the eigenvalues of the matrix A and $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_K$ be their *distinct* corresponding eigenvectors (distinct for the sake argument). Analytically, the set of differential equations $\mathbf{y}' = A\mathbf{y}$ has the general solution

$$\mathbf{y}(t) = C_1\mathbf{v}_1e^{\lambda_1 t} + C_2\mathbf{v}_2e^{\lambda_2 t} + \dots + C_K\mathbf{v}_Ke^{\lambda_K t}$$

where $C_1, C_2, \dots, C_n$ are constants that can be determined from the initial values.

Definition 3.1. The initial value problem

$$\frac{d\mathbf{y}}{dt} = A\mathbf{y} + \mathbf{b} \quad \text{with} \quad \mathbf{y}(0) = \mathbf{y}_0$$

is said to be **Asymptotically Stable** if $\mathbf{y} \rightarrow \mathbf{0}$ as $t \rightarrow \infty$, in other words, all functions in $\mathbf{y}$ tend to 0 as t tends to infinity.

This definition will be important when looking at the long term behaviour of solutions from the eigenvalues to then determine stepsize bounds.

Theorem 3.1. *The initial value problem*

$$\frac{d\mathbf{y}}{dt} = A\mathbf{y} + \mathbf{b}$$

*is asymptotically stable if **all** the eigenvalues of the matrix A have negative real parts. If A has at least one eigenvalue with a non-negative real part, then the system is not asymptotically stable.*

Notice that the stability of a set of ODEs does *not* depend on the forcing term $\mathbf{b}$ nor does it depend on the initial condition $\mathbf{y}(0)$.

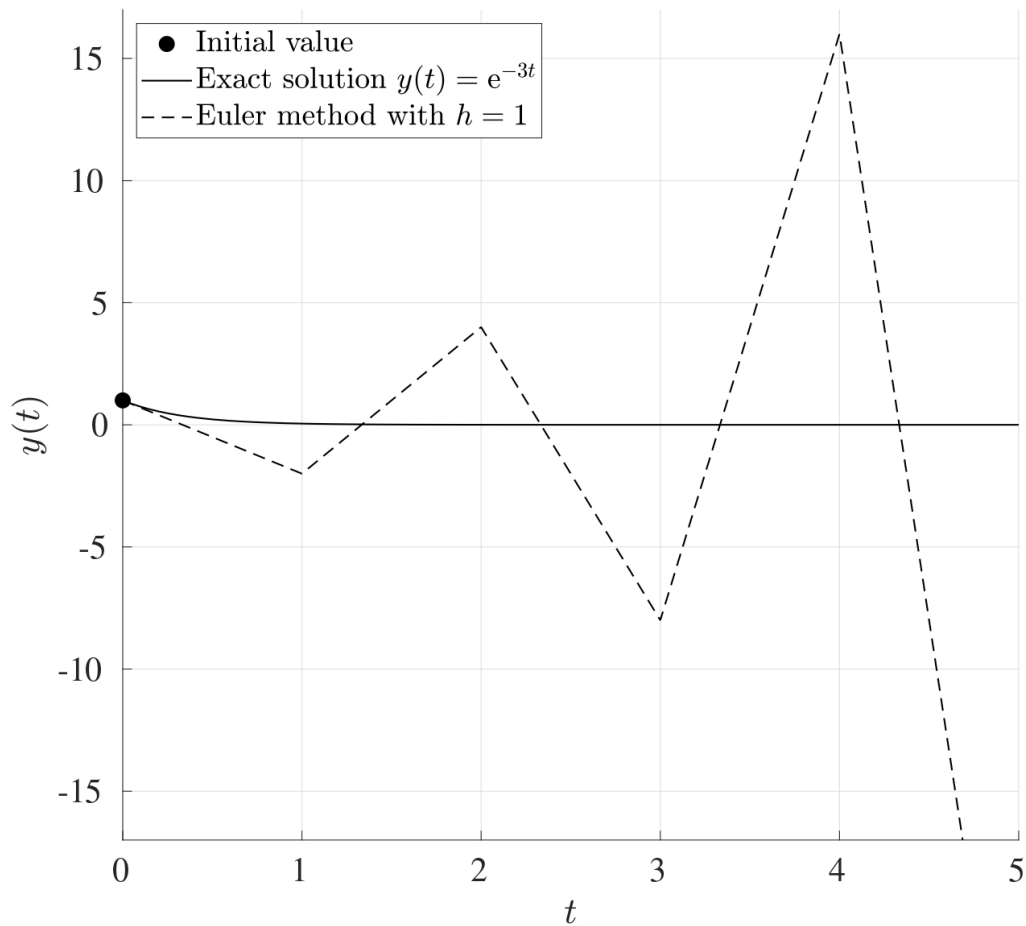
3.5 Limitations of the Euler Method

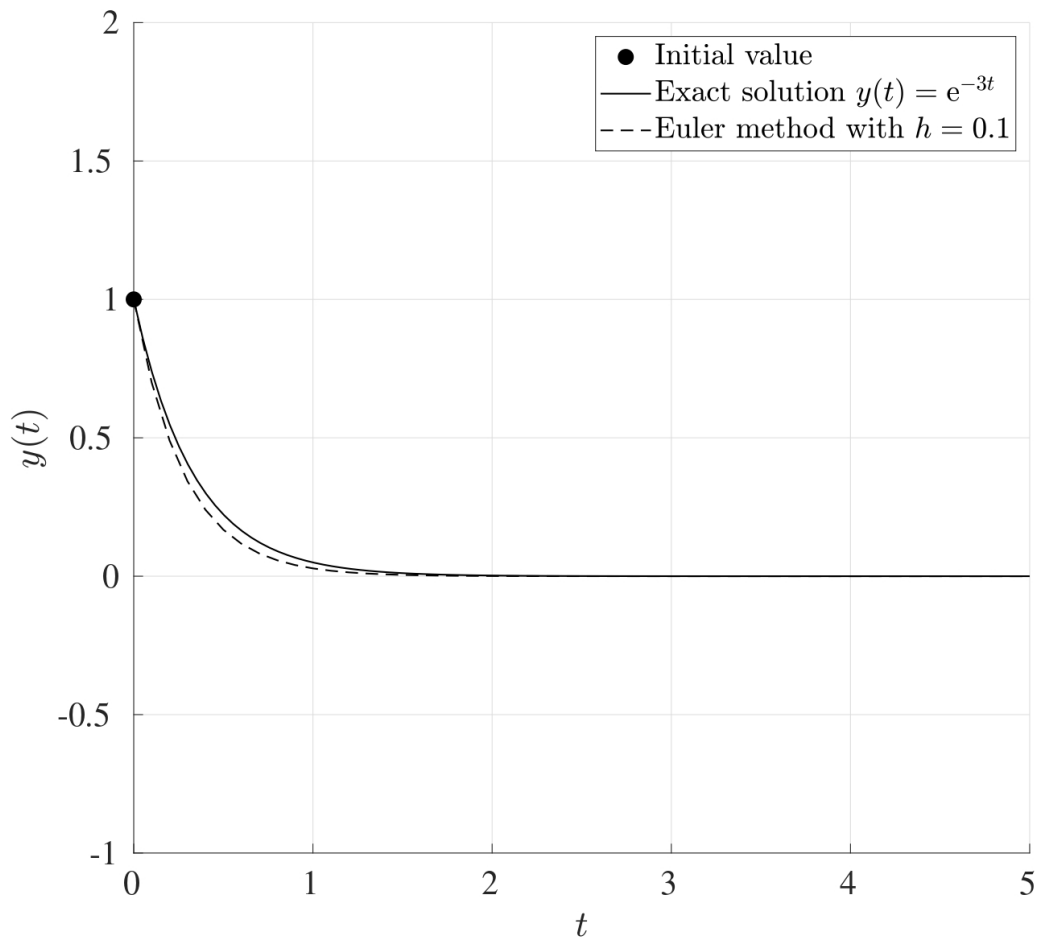
In some cases, if the stepsize h is taken to be too large, then the Euler method can give misleading results.

For example, consider the initial value problem:

$$\frac{dy}{dt} = -3y \quad \text{with} \quad y(0) = 1, \quad t \in [0, 5].$$

Choosing a large stepsize h can render the method ineffective. Case in point, when $h = 1$, the approximate solution oscillates and grows quite rapidly, however choosing a smaller value of h , say $h = 0.1$, gives a very good approximation to the exact solution. These are illustrated in the figures below.





Another situation when the Euler method fails is when the IVP does not have a unique solution. For example, consider the IVP:

$$\frac{dy}{dt} = y^{\frac{1}{3}} \quad \text{with} \quad y(0) = 0, \quad t \in [0, 2].$$

This has the exact solution $y(t) = \left(\frac{2}{3}t\right)^{\frac{3}{2}}$ however this is not unique since $y(t) = 0$ is also a perfectly valid solution. The Euler method in this case will not be able to capture the first non-trivial solution but will only capture the second trivial solution giving a straight line at 0³.

³In general, according to the **Picard-Lindelöf Theorem**, an IVP of the form $y' = f(t, y)$ with $y(0) = y_0$ has a *unique solution* if the function f is continuous in t and uniformly Lipschitz continuous in y . In this example shown above, the function $f(t, y) = y^{\frac{1}{3}}$ does not satisfy the aforementioned conditions and therefore the initial value problem does not have a unique solution. These concepts of continuity are far beyond the realms of this course and no further mention of them will be made.

3.5.1 Bounds on the Stepsize

Consider the initial value problem

$$\frac{dy}{dt} = Ay + b \quad \text{with} \quad y(0) = y_0.$$

If A is asymptotically stable, then a maximum bound h_0 for the stepsize can be found to ensure that the iterations converge. (This means that asymptotic stability of A is a necessary and sufficient condition for the existence of an upper bound h_0 such that if $h < h_0$, then the Euler iteration converges.)

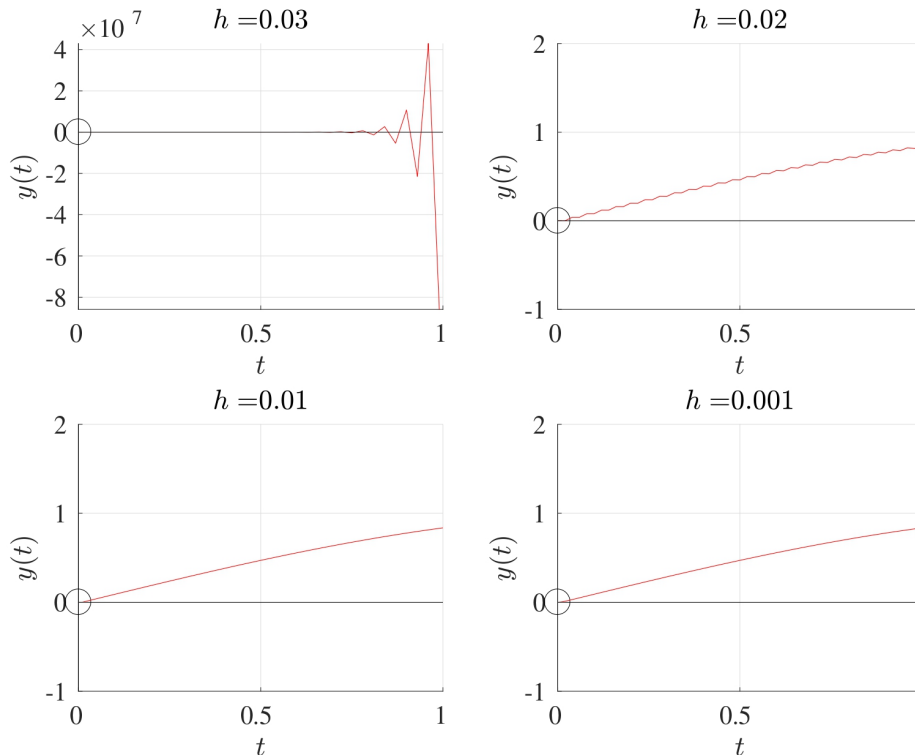
If the stepsize is too large, then the method may not converge but on the other hand if it is too low, then the iteration will take a considerable amount of time to perform. Therefore an “optimal” stepsize is needed to obtain sufficiently accurate solutions.

Different Stepsizes

Consider the following initial value problem

$$\frac{dy}{dt} = 100(\sin(t) - y) \quad \text{with} \quad y(0) = 0.$$

The figure below shows the Euler method being used to solve the initial value problem in the interval $[0, 1]$ for the stepsizes $h = 0.03, 0.02, 0.01, 0.001$.



When $h = 0.03$, the Euler method does not converge. At $h = 0.02$, the Euler method converges but there clearly is a distinct artefact in the solution that shows a slight

oscillation. For h less than 0.02, this oscillation is no longer observed and the Euler method is convergent.

Consider the IVP

$$\frac{d\mathbf{y}}{dt} = A\mathbf{y} + \mathbf{b} \quad \text{with} \quad \mathbf{y}(0) = \mathbf{y}_0.$$

Let $\lambda_1, \lambda_2, \dots, \lambda_K$ be the eigenvalues of A . Suppose that the matrix A is asymptotically stable (i.e. $\Re(\lambda_k) < 0$ for all $k = 1, 2, \dots, K$). In order for the Euler iterations to converge, the stepsize h needs be less than the threshold stepsize h_0 where

$$h_0 = 2 \min_{k=1,2,\dots,K} \left\{ \frac{|\Re(\lambda_k)|}{|\lambda_k|^2} \right\} \quad (3.4)$$

$$\text{or } h_0 = 2 \min_{k=1,2,\dots,K} \left\{ \frac{1}{|\lambda_k|} \right\} \quad \text{if all the eigenvalues are real.}$$

In other words, if the initial value problem is asymptotically stable, then the Euler method is stable if and only if $h < h_0$. This means that the convergence of the Euler is characterised by the eigenvalue that is furthest away from the origin, also called the **Dominant Eigenvalue**.

Euler Upper Bound

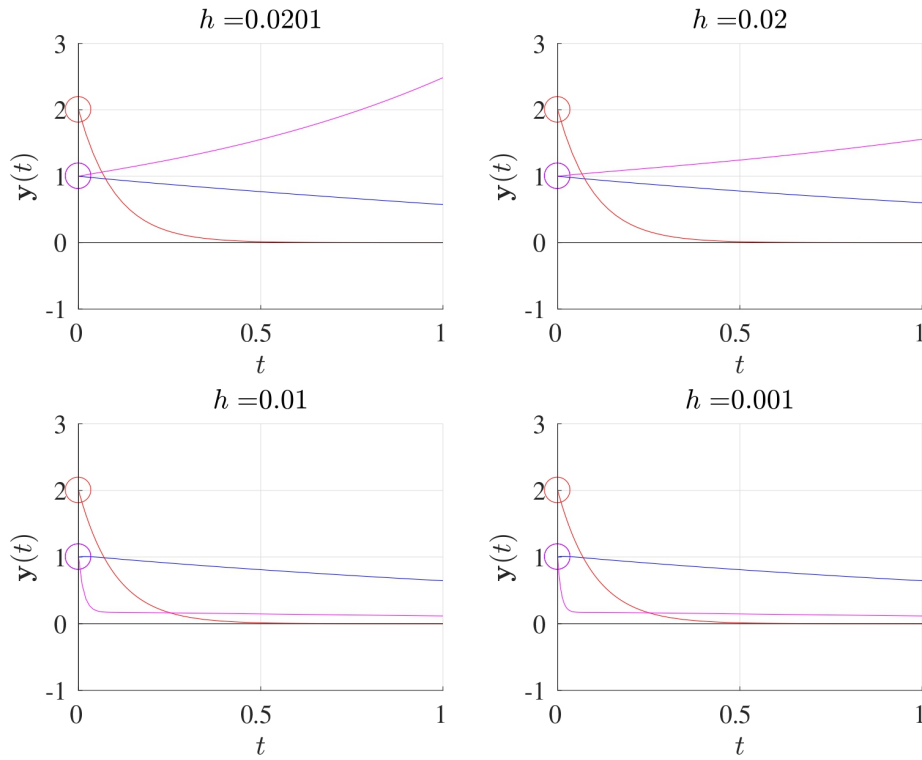
Consider the system of differential equations $\mathbf{y}' = A\mathbf{y}$ with $\mathbf{y}(0) = \mathbf{y}_0$ where

$$A = \begin{pmatrix} -1 & 0 & 3 \\ 0 & -10 & 0 \\ 18 & -1 & -100 \end{pmatrix}.$$

The eigenvalues of the matrix A are $-0.4575, -100.5425, -10$. Since all the eigenvalues are negative, this system is asymptotically stable. Since all the eigenvalues are real, then the threshold stepsize for a convergent Euler method is

$$\begin{aligned} h_0 &= 2 \min \left\{ \frac{1}{|\lambda_k|} \right\} = 2 \min \left\{ \frac{1}{|-0.4575|}, \frac{1}{|-100.5425|}, \frac{1}{|-10|} \right\} \\ &= 2 \min \{2.0858, 0.0099, 0.1\} = 2 \times 0.0099 = 0.0199. \end{aligned}$$

Solutions for different stepsizes are as shown below with the initial values $y_1(0) = 1$ (blue), $y_2(0) = 2$ (red) and $y_3(0) = 1$ (magenta). It can be seen that if $h \geq h_0$, then at least one solution will diverge but if $h < h_0$, then all solutions converge to 0.



3.5.2 Estimated Bound

One drawback in attempting to determine the value of h_0 using Equation 3.4 is that *all* the eigenvalues of the matrix A have to be determined before h_0 can be found. This can be computationally expensive for especially for very large matrices.

An estimate for the threshold stepsize h_0 can be found with far fewer computations using the *sup-norm* $\|\cdot\|_\infty$ (also known as the *infinity norm* or the *Chebyshev norm*). Recall that for a vector $\mathbf{x} = (x_1, x_2, \dots, x_n)$, the **sup-norm** of $\mathbf{x}$ is the maximum absolute value in the vector, i.e.

$$\|\mathbf{x}\|_\infty = \max |x_n|.$$

Whereas for a matrix A , the **sup-norm** of A is the maximal absolute row sum. In other words, for a given matrix A , take the absolute value of all the terms, take the sum of each row and the sup-norm will be the largest out of these.

Sup-Norm of Vectors & Matrices

Consider the vector $\mathbf{x}$ and matrix M given by

$$\mathbf{x} = \begin{pmatrix} 1 \\ -4 \\ -9 \\ 7 \end{pmatrix}, \quad M = \begin{pmatrix} 5 & 2 & 4 & 1 \\ -9 & 5 & 3 & -7 \\ 6 & 0 & -1 & 4 \\ 9 & 5 & -2 & 4 \end{pmatrix}.$$

The sup-norm of $\mathbf{x}$ is simply the largest absolute element which is 9, therefore $\|\mathbf{x}\|_{\infty} = 9$. As for M , to find the sup-norm, first take the absolute value of all the terms, then add the rows. The sup-norm is the maximum element that results:

$$\begin{pmatrix} 5 & 2 & 4 & 1 \\ -9 & 5 & 3 & -7 \\ 6 & 0 & -1 & 4 \\ 9 & 5 & -2 & 4 \end{pmatrix} \xrightarrow{|\cdot|} \begin{pmatrix} 5 & 2 & 4 & 1 \\ 9 & 5 & 3 & 7 \\ 6 & 0 & 1 & 4 \\ 9 & 5 & 2 & 4 \end{pmatrix} \begin{matrix} \rightarrow 12 \\ \rightarrow 24 \\ \rightarrow 11 \\ \rightarrow 20 \end{matrix} \left. \vphantom{\begin{pmatrix} 5 & 2 & 4 & 1 \\ 9 & 5 & 3 & 7 \\ 6 & 0 & 1 & 4 \\ 9 & 5 & 2 & 4 \end{pmatrix}} \right\} \text{maximum is 24.}$$

Therefore $\|M\|_{\infty} = 24$.

Both of these can be found in MATLAB using `norm(x,Inf)` and `norm(M,Inf)`.

Theorem 3.2. Consider the set of linear IVPs

$$\frac{d\mathbf{y}}{dt} = A\mathbf{y} + \mathbf{b} \quad \text{with} \quad \mathbf{y}(0) = \mathbf{y}_0$$

where A is asymptotically stable. Then the Euler method is numerically convergent for any choice of h which satisfies

$$\|\mathcal{I} + hA\|_{\infty} \leq 1.$$

Computing all the eigenvalues of the matrix A can be computationally expensive but obtaining the sup-norm is takes far fewer computations, however as a drawback, the resulting value of h_0 would be an estimate.

Stepsize Bound Estimate 1 (Tridiagonal)

Consider the differential equation $\mathbf{y}' = A\mathbf{y}$ where

$$A = \begin{pmatrix} -2 & 1 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 1 & -2 \end{pmatrix}.$$

To find the upper bound for the stepsize for which the Euler method converges, first

evaluate $\mathcal{I} + hA$:

$$\mathcal{I} + hA = \begin{pmatrix} 1-2h & h & 0 & 0 & 0 \\ h & 1-2h & h & 0 & 0 \\ 0 & h & 1-2h & h & 0 \\ 0 & 0 & h & 1-2h & h \\ 0 & 0 & 0 & h & 1-2h \end{pmatrix}$$

To find the sup-norm, take the absolute value of all the terms and find the maximal row sum:

$$\xrightarrow{|\bullet|} \begin{pmatrix} |1-2h| & h & 0 & 0 & 0 \\ h & |1-2h| & h & 0 & 0 \\ 0 & h & |1-2h| & h & 0 \\ 0 & 0 & h & |1-2h| & h \\ 0 & 0 & 0 & h & |1-2h| \end{pmatrix} \begin{matrix} \rightarrow |1-2h| + h \\ \rightarrow |1-2h| + 2h \\ \rightarrow |1-2h| + 2h \\ \rightarrow |1-2h| + 2h \\ \rightarrow |1-2h| + h. \end{matrix}$$

Let $a = |1-2h| + 2h$ and $b = |1-2h| + h$. Since $h > 0$, then $a > b$, therefore

$$\|\mathcal{I} + hA\|_{\infty} = |1-2h| + 2h.$$

In order to satisfy the inequality $\|\mathcal{I} + hA\|_{\infty} \leq 1$, consider the cases when $1-2h \geq 0$ and $1-2h < 0$ separately:

1. If $1-2h \geq 0$, then $h \leq \frac{1}{2}$:

$$\|\mathcal{I} + hA\|_{\infty} = |1-2h| + 2h = 1-2h + 2h = 1.$$

Therefore $\|\mathcal{I} + hA\|_{\infty} = 1 \leq 1$ is indeed true.

2. If $1-2h < 0$, then $h > \frac{1}{2}$:

$$\|\mathcal{I} + hA\|_{\infty} = |1-2h| + 2h = 2h-1 + 2h = 4h-1.$$

If $\|\mathcal{I} + hA\|_{\infty} \leq 1$, then $4h-1 \leq 1$. Simplifying this would result in $h \leq \frac{1}{2}$ which contradicts with the assumption that $h > \frac{1}{2}$.

From these two cases, it is clear that $h \not> \frac{1}{2}$ (since that case leads to a contradiction), therefore $h \leq \frac{1}{2}$. Thus for a convergent Euler method, the stepsize h must be less than the threshold stepsize $h_0 = \frac{1}{2}$.

This can be compared to the exact bound; the eigenvalues of the matrix A are

$$-3.7321, \quad -3, \quad -2, \quad -1, \quad -0.2679.$$

Therefore

$$h_0 = 2 \min \left\{ \frac{1}{|\lambda_k|} \right\} = 0.5359$$

which is a larger bound compared to the one obtained using the sup-norm method. Observe that if the size of the matrix was larger but followed the same theme (i.e. 2 on the main diagonal and -1 and the sub and super diagonals), then no further calculations are required for the sup-norm method, the outcome will still be $h_0 = \frac{1}{2}$. As for the eigenvalue method, all the eigenvalues have to be recalculated again.

Stepsize Bound Estimate 2 (Bidiagonal)

Consider the differential equation $\mathbf{y}' = A\mathbf{y}$ where

$$A = \begin{pmatrix} -1 & 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 1 & -1 \end{pmatrix}.$$

To find the upper bound for the stepsize for which the Euler method converges, first evaluate $\mathcal{I} + hA$:

$$\mathcal{I} + hA = \begin{pmatrix} 1-h & 0 & 0 & 0 & 0 \\ h & 1-h & 0 & 0 & 0 \\ 0 & h & 1-h & 0 & 0 \\ 0 & 0 & h & 1-h & 0 \\ 0 & 0 & 0 & h & 1-h \end{pmatrix}$$

To find the sup-norm, take the absolute value of all the terms and find the maximal row sum:

$$\xrightarrow{|\cdot|} \begin{pmatrix} |1-h| & 0 & 0 & 0 & 0 \\ h & |1-h| & 0 & 0 & 0 \\ 0 & h & |1-h| & 0 & 0 \\ 0 & 0 & h & |1-h| & 0 \\ 0 & 0 & 0 & h & |1-h| \end{pmatrix} \begin{matrix} \rightarrow |1-h| \\ \rightarrow |1-h| + h \\ \rightarrow |1-h| + h \\ \rightarrow |1-h| + h \\ \rightarrow |1-h| + h \end{matrix}$$

Let $a = |1-h| + h$ and $b = |1-h|$. Clearly $a > b$ since $h > 0$, therefore

$$\|\mathcal{I} + hA\|_{\infty} = |1-h| + h.$$

In order to satisfy the inequality, $\|\mathcal{I} + hA\|_{\infty} \leq 1$, consider the cases when $1-h \geq 0$ and $1-h < 0$:

1. If $1-h \geq 0$, then $h \leq 1$:

$$\|\mathcal{I} + hA\|_{\infty} = |1-h| + h = 1-h+h=1,$$

therefore $\|\mathcal{I} + hA\|_{\infty} \leq 1$ is indeed true.

2. If $1-h < 0$, then $h > 1$:

$$\|\mathcal{I} + hA\|_{\infty} = |1-h| + h = h-1+h=2h-1.$$

If $\|\mathcal{I} + hA\|_{\infty} \leq 1$, then $2h-1 \leq 1$, meaning that $h \leq 1$ which contradicts with the assumption that $h > 1$.

This means that for a convergent Euler method, the stepsize h must be less than $h_0 = 1$. This can be compared to the exact upper bound. The eigenvalues of the matrix A are just -1 five times, therefore

$$h_0 = 2 \min \left\{ \frac{1}{|\lambda_k|} \right\} = 2,$$

this shows that the sup-norm method gives a tighter than using eigenvalues.

The sup-norm method works well when the matrix in question has a diagonal, bidiagonal or tridiagonal structure where the diagonal terms are the same. In general, the sup-norm method might *not* be suitable for any matrix.

Stepsize Bound Estimate 3 (General)

Consider the differential equation $\mathbf{y}' = A\mathbf{y}$ where

$$A = \begin{pmatrix} -1 & -2 \\ 4 & -3 \end{pmatrix}.$$

Find the sup-norm:

$$\mathcal{I} + hA = \begin{pmatrix} 1-h & -2h \\ 4h & 1-3h \end{pmatrix} \xrightarrow{|\bullet|} \begin{pmatrix} |1-h| & 2h \\ 4h & |1-3h| \end{pmatrix} \rightarrow \begin{matrix} |1-h|+2h \\ |1-3h|+4h \end{matrix}$$

Let $a = |1-h|+2h$ and $b = |1-3h|+4h$. Here, it is not obvious which is larger, a or b . Therefore, consider the three cases $0 < h < \frac{1}{3}$, $\frac{1}{3} < h < 1$ and $h > 1$.

1. $0 < h < \frac{1}{3}$: In this case, $1-h > 0$ and $1-3h > 0$, therefore $a = |1-h|+2h = 1+h$ and $b = |1-3h|+4h = 1+h$, hence $\|\mathcal{I} + hA\|_{\infty} = 1+h$. In order to satisfy $\|\mathcal{I} + hA\|_{\infty} \leq 1$, this would mean that $h < 0$ which contradicts with the fact that $h > 0$. Therefore $h \notin (0, \frac{1}{3})$.
2. $\frac{1}{3} < h < 1$: In this case, $1-h > 0$ and $1-3h < 0$, therefore $a = |1-h|+2h = 1+h$ and $b = |1-3h|+4h = 7h-1$. This should now be split into two subcases to check which one will lead to a contradiction:

- i. Suppose that $a > b$, then

$$1+h > 7h-1 \implies h < \frac{1}{3}$$

which contradicts with $h > \frac{1}{3}$

- ii. Suppose that $a < b$, then

$$1+h < 7h-1 \implies h > \frac{1}{3}$$

not leading to any contradiction. therefore since $b > a$, then $\|\mathcal{I} + hA\|_{\infty} = b = 7h-1$.

In order to satisfy $\|\mathcal{I} + hA\|_{\infty} \leq 1$ then $h < \frac{2}{7}$ which contradicts with the fact that $\frac{1}{3} < h$. Therefore $h \notin (\frac{1}{3}, 1)$.

3. $h > 1$: In this case, $1-h < 0$ and $1-3h < 0$, therefore $a = |1-h|+2h = 3h-1$ and $b = |1-3h|+4h = 7h-1$. Clearly $b > a$ since $h > 0$, so $\|\mathcal{I} + hA\|_{\infty} = 7h-1$. In order to satisfy $\|\mathcal{I} + hA\|_{\infty} \leq 1$ then $h < \frac{2}{7}$ which contradicts with the fact that $h > 1$. This means that $h \not> 1$.

So in every possible case, there will be a contradiction when using the sup-norm method. This *does not* mean that the system is asymptotically unstable, in fact, the eigenvalues of the matrix A are $-2 \pm 2.65i$ meaning that the system is asymptotically stable and the threshold stepsize is in fact $h_0 = 0.0992$.

This example shows that the sup-norm method cannot be used for any matrix system, but if a matrix has a banded structure, then it would be appropriate and would require fewer computations compared to finding all the eigenvalues.

3.6 MATLAB Code

The following MATLAB code performs the Euler iteration for the following set of IVPs on the interval $[0, 1]$:

$$\begin{aligned}\frac{du}{dt} &= 2u + v + w + \cos(t), & u(0) &= 0 \\ \frac{dv}{dt} &= \sin(u) + e^{-v+w}, & v(0) &= 1 \\ \frac{dw}{dt} &= uv - w, & w(0) &= 0.\end{aligned}$$

i Linearity

Note that this code is built for a general case that *does not* have to be linear even though the entire derivation process was built on the fact that the system is linear.

```
1 function IVP_Euler
2
3 %% Solve a set of first order IVPs using Euler
4
5 % This code solves a set of IVP when written explicitly
6 % on the interval [t0,tf] subject to the initial conditions
7 % y(0)=y0. The output will be the graph of the solution(s)
8 % and the vector value at the final point tf. Note that the
9 % IVPs do not need to be linear or homogeneous.
10
11 %% Lines to change:
12
13 % Line 28 : t0 - Start time
14 % Line 31 : tf - End time
15 % Line 34 : N - Number of subdivisions
16 % Line 37 : y0 - Vector of initial values
17 % Line 106+ : Which functions to plot, remembering to assign
18 % a colour, texture and legend label
```

```

19 % Line 120+ : Set of differential equations written
20 %           explicitly. These can also be non-linear and
21 %           include forcing terms. These equations can
22 %           also be written in matrix form if the
23 %           equations are linear.
24
25 %% Set up input values
26
27 % Start time
28 t0=0;
29
30 % End time
31 tf=1;
32
33 % Number of subdivisions
34 N=5000;
35
36 % Column vector initial values y0=y(t0)
37 y0=[0;1;0];
38
39 %% Set up IVP solver parameters
40
41 % T = Vector of times t0,t1,...,tN.
42 % This is generated using linspace which splits the
43 % interval [t0,tf] into N+1 points (or N subintervals)
44 T=linspace(t0,tf,N+1);
45
46 % Stepsize
47 h=(tf-t0)/N;
48
49 % Number of differential equations
50 K=length(y0);
51
52 %% Perform the Euler iteration
53
54 % Y = Solution matrix
55 % The matrix Y will contain K rows and N+1 columns. Every
56 % row corresponds to a different IVP and every column
57 % corresponds to a different time. So the matrix Y will
58 % take the following form:
59 % y_1(t_0) y_1(t_1) y_1(t_2) ... y_1(t_N)
60 % y_2(t_0) y_2(t_1) y_2(t_2) ... y_2(t_N)
61 % ...
62 % y_K(t_0) y_K(t_1) y_K(t_2) ... y_K(t_N)
63 Y=zeros(K,N+1);

```

```

64
65 % The first column of the vector Y is the initial vector y0
66 Y(:,1)=y0;
67
68 % Set the current time t to be the starting time t0 and the
69 % current value of the vector y to be the strtaing values y0
70 t=t0;
71 y=y0;
72
73 for n=2:1:N+1
74
75     dydt=DYDT(t,y,K); % Find gradient at the current step
76
77     y=y+h*dydt;        % Find y at the current step
78
79     t=T(n);            % Update the new time
80
81     Y(:,n)=y;          % Replace row n in Y with y
82
83 end
84
85 %% Setting plot parameters
86
87 % Clear figure
88 clf
89
90 % Hold so more than one line can be drawn
91 hold on
92
93 % Turn on grid
94 grid on
95
96 % Setting font size and style
97 set(gca,'FontSize',20,'FontName','Times')
98 set(legend,'Interpreter','Latex')
99
100 % Label the axes
101 xlabel('$t$', 'Interpreter','Latex')
102 ylabel('$\mathbf{y}(t)$', 'Interpreter','Latex')
103
104 % Plot the desried solutions. If all the solutions are
105 % needed, then consider using a for loop in that case
106 plot(T,Y(1,:), '-b', 'LineWidth',2, 'DisplayName', '$y_1(t)$')
107 plot(T,Y(2,:), '-r', 'LineWidth',2, 'DisplayName', '$y_2(t)$')
108 plot(T,Y(3,:), '-k', 'LineWidth',2, 'DisplayName', '$y_3(t)$')

```

```

109
110 % Display the values of the vector y at tf
111 disp(strcat('The vector y at t=',num2str(tf),' is:'))
112 disp(Y(:,end))
113
114 end
115
116 function [dydt]=DYDT(t,y,K)
117
118 % When the equation are written in explicit form
119
120 dydt=zeros(K,1);
121
122 dydt(1)=2*y(1)+y(2)+y(3)+cos(t);
123
124 dydt(2)=sin(y(1))+exp(-y(2)+y(3));
125
126 dydt(3)=y(1)*y(2)-y(3);
127
128 % If the set of equations is linear, then these can be
129 % written in matrix form as dydt=A*y+b(t). For example, if
130 % the set of equations is:
131 % dudt = 7u - 2v + w + exp(t)
132 % dvdt = 2u + 3v - 9w + cos(t)
133 % dwdt = 2v + 5w + 2
134 % Then:
135 % A=[7,-2,1;2,3,-9;0,2,5];
136 % b=@(t) [exp(t);cos(t);2];
137 % dydt=A*y+b(t)
138
139 end

```

4 The Modified Euler Method

The Euler method can be effective when it comes to solving differential equations numerically but on occasions, the global error of $\mathcal{O}(h)$ is rather poor. The Euler method can be modified and improved to give **Modified** or **Improved Euler Method** (also known as the *Heun Method*, named after Karl Heun).

4.1 Steps of the Modified Euler Method

The Modified Euler Method utilises the **Fundamental Theorem of Calculus** which states that for a differentiable function y defined on the interval $[t_0, t_1]$ (where $t_1 = t_0 + h$ for some stepsize h),

$$y(t_1) - y(t_0) = \int_{t_0}^{t_1} y'(t) \, dt.$$

In the interval $[t_0, t_1]$, the derivative $y'(t)$ may be approximated by the derivative at the leftmost point $y'(t_0)$, this approximation forms the basis of the standard Euler method;

$$\begin{aligned} y(t_1) - y(t_0) &= \int_{t_0}^{t_1} y'(t) \, dt \\ &= \int_{t_0}^{t_1} y'(t_0) \, dt \\ &= hy'(t_0) \\ \implies y(t_1) &= y(t_0) + hy'(t_0). \end{aligned}$$

However, if $y'(t)$ varies substantially then this approximation can lead to some poor predictions. This can be modified so rather than approximating $y'(t)$ by $y'(t_0)$ only, it can be approximated by taking an average between $y'(t_0)$ and $y'(t_1)$, namely

$$y'(t) \approx \frac{1}{2} (y'(t_0) + y'(t_1)).$$

Thus

$$\begin{aligned}
 y(t_1) - y(t_0) &= \int_{t_0}^{t_1} y'(t) \, dt \\
 &= \int_{t_0}^{t_1} \frac{1}{2} (y'(t_0) + y'(t_1)) \, dt \\
 &= \frac{h}{2} (y'(t_0) + y'(t_1)) \\
 \implies y(t_1) &= y(t_0) + \frac{h}{2} (y'(t_0) + y'(t_1)) .
 \end{aligned}$$

Initially, one might suspect that the derivative $y'(t_1)$ can be found from the differential equation itself, namely, $y'(t_1) = f(t_1, y(t_1))$ but to do that, a **Prediction-Correction** procedure needs to be employed where the Euler method can be used to predict a value of $y(t_1)$ and this is then corrected afterwards. This is done as follows:

- Predictor: $\tilde{Y}_{n+1} = Y_n + hf(t_n, Y_n)$
- Corrector: $Y_{n+1} = Y_n + \frac{h}{2} [f(t_n, Y_n) + f(t_{n+1}, \tilde{Y}_{n+1})]$.

Modified Euler Method

Consider the differential equation

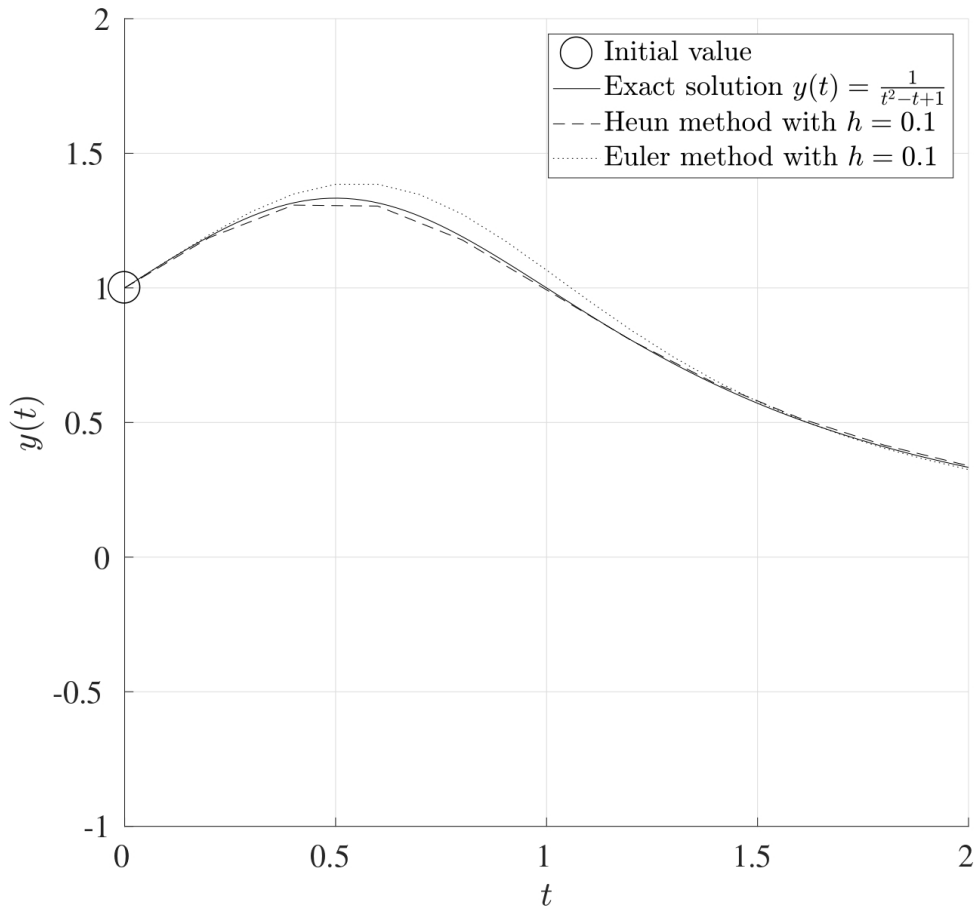
$$\frac{dy}{dt} = (1 - 2t)y^2 \quad \text{with} \quad y(0) = 1, \quad t \in [0, 2].$$

This differential equation is non-linear but has a known particular solution which is

$$y(t) = \frac{1}{t^2 - t + 1}$$

and this will be compared to the approximate solutions obtained from the standard and Modified Euler methods.

The figure below shows how the standard and modified Euler methods compare to the exact solution for the same stepsize $h = 0.1$. This suggests that the Modified Euler method has improved accuracy compared to the Euler method for the same stepsize, however as a consequence, the function f on the right hand side of the differential equation has to be calculated twice for every step; once in the prediction stage and once for the correction. However even with this in mind, doubling the number of calculations to improve accuracy can also warrant for a coarser choice of the stepsize to allow for a more efficient use of computational time.



4.2 Accuracy of the Modified Euler Method

In order to assess the accuracy of the Modified Euler method, consider the Taylor series expansion of y at the points t_0 and t_1 about $t_{0.5} = t_0 + \frac{1}{2}h$:

$$y(t_1) = y\left(t_{0.5} + \frac{h}{2}\right) = y(t_{0.5}) + \frac{h}{2}y'(t_{0.5}) + \left(\frac{h}{2}\right)^2 \frac{1}{2!}y''(t_{0.5}) + \mathcal{O}(h^3),$$

$$y(t_0) = y\left(t_{0.5} - \frac{h}{2}\right) = y(t_{0.5}) - \frac{h}{2}y'(t_{0.5}) + \left(\frac{h}{2}\right)^2 \frac{1}{2!}y''(t_{0.5}) + \mathcal{O}(h^3).$$

Subtracting $y(t_0)$ from $y(t_1)$ gives

$$y(t_1) - y(t_0) = hy'(t_{0.5}) + \mathcal{O}(h^3). \quad (4.1)$$

The Taylor series expansion can also be done for the derivative of y at the points t_0 and t_1 about $t_{0.5} = t_0 + \frac{1}{2}h$ in a similar way as above, i.e.

$$\begin{aligned} y'(t_1) &= y' \left(t_{0.5} + \frac{h}{2} \right) = y'(t_{0.5}) + \frac{h}{2} y''(t_{0.5}) + \mathcal{O}(h^2), \\ y'(t_0) &= y' \left(t_{0.5} - \frac{h}{2} \right) = y'(t_{0.5}) - \frac{h}{2} y''(t_{0.5}) + \mathcal{O}(h^2). \end{aligned}$$

Adding $y'(t_0)$ to $y'(t_1)$ gives

$$y'(t_1) + y'(t_0) = 2y'(t_{0.5}) + \mathcal{O}(h^2),$$

thus multiplying by $\frac{h}{2}$ and using equation Equation 4.1 yields

$$\frac{h}{2} [y'(t_1) + y'(t_0)] = y(t_1) - y(t_0) + \mathcal{O}(h^3). \quad (4.2)$$

The first step of the Modified Euler method is to predict the value of $y'(t_1)$ using the Euler iteration;

$$\tilde{Y}_1 = \underbrace{y(t_0) + hy'(t_0)}_{\approx y(t_1)} + \mathcal{O}(h^2).$$

Hence

$$y'(t_1) = f(t_1, y(t_1)) \approx f(t_1, \tilde{Y}_1) + \mathcal{O}(h^2).$$

All this information can now be used to obtain the improved update Y_1 which is the corrected form of $\tilde{Y}_1$. Thus from equation Equation 4.2,

$$\begin{aligned} \underbrace{y(t_1)}_{\approx Y_1} &= \underbrace{y(t_0)}_{=Y_0} + \frac{h}{2} \left[\underbrace{y'(t_1)}_{=f(t_1, \tilde{Y}_1)} + \underbrace{y'(t_0)}_{=f(t_0, Y_0)} \right] + \mathcal{O}(h^3) \\ \implies Y_1 &= Y_0 + \frac{h}{2} [f(t_1, \tilde{Y}_1) + f(t_0, Y_0)]. \end{aligned} \quad (4.3)$$

Equations Equation 4.3 and Equation 4.2 can be used to find the local truncation error for the Modified Euler method at the first time step which is

$$e = |y(t_1) - Y_1| = \left| y(t_1) - \left[y(t_0) + \frac{h}{2} (y'(t_1) + y'(t_0)) \right] \right| + \mathcal{O}(h^3) = \mathcal{O}(h^3).$$

Therefore the *local truncation error* $e = \mathcal{O}(h^3)$ meaning that the Modified Euler method is third order accurate which is an improvement over the Euler method.

The global integration error can be obtained just as before to show that the global integration error of the Modified Euler method is $E = \mathcal{O}(h^2)$ meaning that this is a second order method. In particular, if the stepsize h is halved, the global integration error will be reduced by a factor of four while the local truncation error will reduce by a factor of eight.

4.3 MATLAB Code

The following MATLAB code performs the Modified Euler iteration for the following set of IVPs on the interval $[0, 1]$:

$$\begin{aligned}\frac{du}{dt} &= 2u + v + w + \cos(t), & u(0) &= 0 \\ \frac{dv}{dt} &= \sin(u) + e^{-v+w}, & v(0) &= 1 \\ \frac{dw}{dt} &= uv - w, & w(0) &= 0.\end{aligned}$$

i Linearity

Note that this code is built for a general case that *does not* have to be linear even though the entire derivation process was built on the fact that the system is linear.

```
1 function IVP_Mod_Euler
2
3 %% Solve a set of first order IVPs using Modified Euler
4
5 % This code solves a set of IVP when written explicitly
6 % on the interval [t0,tf] subject to the initial conditions
7 % y(0)=y0. The output will be the graph of the solution(s)
8 % and the vector value at the final point tf. Note that the
9 % IVPs do not need to be linear or homogeneous.
10
11 %% Lines to change:
12
13 % Line 28 : t0 - Start time
14 % Line 31 : tf - End time
15 % Line 34 : N - Number of subdivisions
16 % Line 37 : y0 - Vector of initial values
17 % Line 116+ : Which functions to plot, remembering to assign
18 %             a colour, texture and legend label
19 % Line 130+ : Set of differential equations written
20 %             explicitly. These can also be non-linear and
21 %             include forcing terms. These equations can
22 %             also be written in matrix form if the
23 %             equations are linear.
24
25 %% Set up input values
26
27 % Start time
28 t0=0;
```

```

29
30 % End time
31 tf=1;
32
33 % Number of subdivisions
34 N=5000;
35
36 % Column vector initial values y0=y(t0)
37 y0=[0;1;0];
38
39 %% Set up IVP solver parameters
40
41 % T = Vector of times t0,t1,...,tN.
42 % This is generated using linspace which splits the
43 % interval [t0,tf] into N+1 points (or N subintervals)
44 T=linspace(t0,tf,N+1);
45
46 % Stepsize
47 h=(tf-t0)/N;
48
49 % Number of differential equations
50 K=length(y0);
51
52 %% Perform the Modified Euler iteration
53
54 % Y = Solution matrix
55 % The matrix Y will contain K rows and N+1 columns. Every
56 % row corresponds to a different IVP and every column
57 % corresponds to a different time. So the matrix Y will
58 % take the following form:
59 % y_1(t_0) y_1(t_1) y_1(t_2) ... y_1(t_N)
60 % y_2(t_0) y_2(t_1) y_2(t_2) ... y_2(t_N)
61 % ...
62 % y_K(t_0) y_K(t_1) y_K(t_2) ... y_K(t_N)
63 Y=zeros(K,N+1);
64
65 % The first column of the vector Y is the initial vector y0
66 Y(:,1)=y0;
67
68 % Set the current time t to be the starting time t0 and the
69 % current value of the vector y to be the strtaing values y0
70 t=t0;
71 y=y0;
72
73 for n=2:1:N+1

```

```

74
75 % Prediction Step:
76 % Use the Euler iteration to obtain an approximation for
77 % the derivatives at the current time step
78
79 dydt=DYDT(t,y,K); % Find gradient at the current step
80 y_pred=y+h*dydt; % Predict y at current time step
81
82 % Corrector Step:
83 % Use the Modified Euler to correct y_pred
84
85 dydt_pred=DYDT(t,y_pred,K); % Predict the gradient
86 % from the predicted y
87 y=y+0.5*h*(dydt+dydt_pred); % Find y at the current step
88
89 t=T(n); % Update the new time
90
91 Y(:,n)=y; % Replace row n in Y with y
92
93 end
94
95 %% Setting plot parameters
96
97 % Clear figure
98 clf
99
100 % Hold so more than one line can be drawn
101 hold on
102
103 % Turn on grid
104 grid on
105
106 % Setting font size and style
107 set(gca,'FontSize',20,'FontName','Times')
108 set(legend,'Interpreter','Latex')
109
110 % Label the axes
111 xlabel('$t$', 'Interpreter','Latex')
112 ylabel('$\mathbf{y}(t)$', 'Interpreter','Latex')
113
114 % Plot the desired solutions. If all the solutions are
115 % needed, then consider using a for loop in that case
116 plot(T,Y(1,:), '-b', 'LineWidth', 2, 'DisplayName', '$y_1(t)$')
117 plot(T,Y(2,:), '-r', 'LineWidth', 2, 'DisplayName', '$y_1(t)$')
118 plot(T,Y(3,:), '-k', 'LineWidth', 2, 'DisplayName', '$y_1(t)$')

```

```

119
120 % Display the values of the vector y at tf
121 disp(strcat('The vector y at t=',num2str(tf),' is:'))
122 disp(Y(:,end))
123
124 end
125
126 function [dydt]=DYDT(t,y,K)
127
128 % When the equation are written in explicit form
129
130 dydt=zeros(K,1);
131
132 dydt(1)=2*y(1)+y(2)+y(3)+cos(t);
133
134 dydt(2)=sin(y(1))+exp(-y(2)+y(3));
135
136 dydt(3)=y(1)*y(2)-y(3);
137
138 % If the set of equations is linear, then these can be
139 % written in matrix form as dydt=A*y+b(t). For example, if
140 % the set of equations is:
141 % dudt = 7u - 2v + w + exp(t)
142 % dvdt = 2u + 3v - 9w + cos(t)
143 % dwdt = 2v + 5w + 2
144 % Then:
145 % A=[7,-2,1;2,3,-9;0,2,5];
146 % b=@(t) [exp(t);cos(t);2];
147 % dydt=A*y+b(t)
148
149 end

```

5 Fourth Order Runge-Kutta Method

The Modified Euler method extended the Euler method to a two-stage procedure with a global integration error of $\mathcal{O}(h^2)$. This can be extended further to a **Multi-Stage Method**, also called a **Runge-Kutta Method** with p stages and a global error integration error of $\mathcal{O}(h^p)$ for any arbitrarily large p (in this case, the Modified Euler method is known as a second order Runge-Kutta method since it has two stages). For instance, the *fourth order Runge-Kutta method* requires four calculations for every step and has a global integration error of $\mathcal{O}(h^4)$, this is formulated as follows:

$$\begin{aligned}K_1 &= f(t_n, Y_n), \\K_2 &= f\left(t_n + \frac{h}{2}, Y_n + \frac{h}{2}K_1\right), \\K_3 &= f\left(t_n + \frac{h}{2}, Y_n + \frac{h}{2}K_2\right), \\K_4 &= f(t_{n+1}, Y_n + hK_3) \\Y_{n+1} &= Y_n + \frac{h}{6} [K_1 + 2K_2 + 2K_3 + K_4].\end{aligned}$$

Runge-Kutta methods like this are quite versatile and are generally the most used methods for their accuracy since the stepsize h does not need to be too small to achieve good results. Even though every step requires four calculations, the value of h can be made larger in order to reduce the cost but retain considerable accuracy.

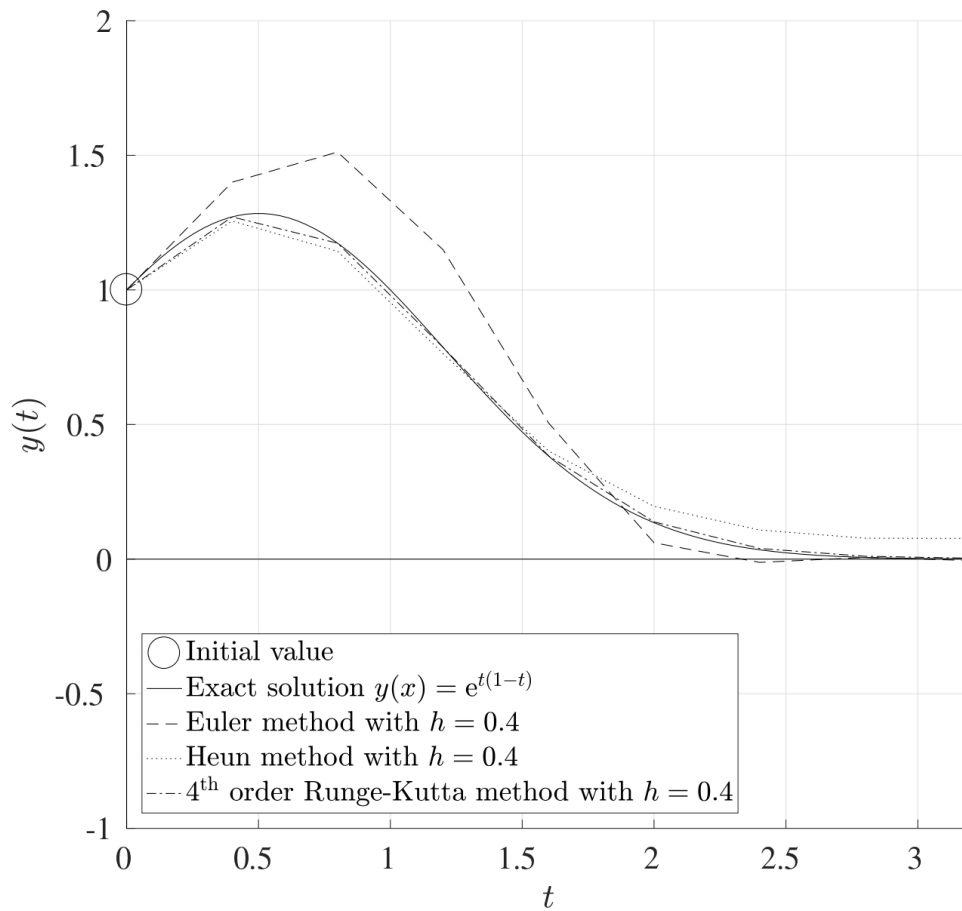
Runge-Kutta Method

Consider the differential equation

$$\frac{dy}{dt} = y(1 - 2t) \quad \text{where} \quad y(0) = 1, \quad t \in [0, 3.2].$$

The exact solution to this differential equation is known to be

$$y(t) = e^{t(1-t)}.$$



The figure above shows the exact solution to the differential equation (solid line) with the three different methods used to approximate the solution all at the same resolution of $h = 0.4$. The stepsize h is quite coarse but this is merely for the purposes of demonstration. The Euler method is the least accurate for this coarse grid, the Heun method improves the accuracy while the fourth order Runge-Kutta method is the most accurate out of the three even for the same stepsize.

5.1 MATLAB Code

The following MATLAB code performs the fourth order Runge-Kutta iteration for the following set of IVPs on the interval $[0, 1]$:

$$\begin{aligned}\frac{du}{dt} &= 2u + v + w + \cos(t), & u(0) &= 0 \\ \frac{dv}{dt} &= \sin(u) + e^{-v+w}, & v(0) &= 1 \\ \frac{dw}{dt} &= uv - w, & w(0) &= 0.\end{aligned}$$

i Linearity

Note that this code is built for a general case that *does not* have to be linear even though the entire derivation process was built on the fact that the system is linear.

```
1 function IVP_RK4
2
3 %% Solve a set of first order IVPs using RK4
4
5 % This code solves a set of IVP when written explicitly
6 % on the interval [t0,tf] subject to the initial conditions
7 % y(0)=y0. The output will be the graph of the solution(s)
8 % and the vector value at the final point tf. Note that the
9 % IVPs do not need to be linear or homogeneous.
10
11 %% Lines to change:
12
13 % Line 28 : t0 - Start time
14 % Line 31 : tf - End time
15 % Line 34 : N - Number of subdivisions
16 % Line 37 : y0 - Vector of initial values
17 % Line 110+ : Which functions to plot, remembering to assign
18 %             a colour, texture and legend label
19 % Line 124+ : Set of differential equations written
20 %             explicitly. These can also be non-linear and
21 %             include forcing terms. These equations can
22 %             also be written in matrix form if the
23 %             equations are linear.
24
25 %% Set up input values
26
27 % Start time
28 t0=0;
```

```

29
30 % End time
31 tf=1;
32
33 % Number of subdivisions
34 N=50;
35
36 % Column vector initial values y0=y(t0)
37 y0=[0;1;0];
38
39 %% Set up IVP solver parameters
40
41 % T = Vector of times t0,t1,...,tN.
42 % This is generated using linspace which splits the
43 % interval [t0,tf] into N+1 points (or N subintervals)
44 T=linspace(t0,tf,N+1);
45
46 % Stepsize
47 h=(tf-t0)/N;
48
49 % Number of differential equations
50 K=length(y0);
51
52 %% Perform the RK4 iteration
53
54 % Y = Solution matrix
55 % The matrix Y will contain K rows and N+1 columns. Every
56 % row corresponds to a different IVP and every column
57 % corresponds to a different time. So the matrix Y will
58 % take the following form:
59 % y_1(t_0) y_1(t_1) y_1(t_2) ... y_1(t_N)
60 % y_2(t_0) y_2(t_1) y_2(t_2) ... y_2(t_N)
61 % ...
62 % y_K(t_0) y_K(t_1) y_K(t_2) ... y_K(t_N)
63 Y=zeros(K,N+1);
64
65 % The first column of the vector Y is the initial vector y0
66 Y(:,1)=y0;
67
68 % Set the current time t to be the starting time t0 and the
69 % current value of the vector y to be the strtaing values y0
70 t=t0;
71 y=y0;
72
73 for n=2:1:N+1

```

```

74
75     % Determine the coefficients of RK4
76
77     K1=DYDT(t,y,K);
78     K2=DYDT(t+h/2,y+h*K1/2,K);
79     K3=DYDT(t+h/2,y+h*K2/2,K);
80     K4=DYDT(t+h,y+h*K3,K);
81     y=y+(h/6)*(K1+2*K2+2*K3+K4);
82
83     t=T(n);           % Update the new time
84
85     Y(:,n)=y;         % Replace row n in Y with y
86
87 end
88
89 %% Setting plot parameters
90
91 % Clear figure
92 clf
93
94 % Hold so more than one line can be drawn
95 hold on
96
97 % Turn on grid
98 grid on
99
100 % Setting font size and style
101 set(gca,'FontSize',20,'FontName','Times')
102 set(legend,'Interpreter','Latex')
103
104 % Label the axes
105 xlabel('$t$', 'Interpreter','Latex')
106 ylabel('$\mathbf{y}(t)$', 'Interpreter','Latex')
107
108 % Plot the desired solutions. If all the solutions are
109 % needed, then consider using a for loop in that case
110 plot(T,Y(1,:), '-b', 'LineWidth', 2, 'DisplayName', '$y_1(t)$')
111 plot(T,Y(2,:), '-r', 'LineWidth', 2, 'DisplayName', '$y_2(t)$')
112 plot(T,Y(3,:), '-k', 'LineWidth', 2, 'DisplayName', '$y_3(t)$')
113
114 % Display the values of the vector y at tf
115 disp(strcat('The vector y at t=', num2str(tf), ' is:'))
116 disp(Y(:,end))
117
118 end

```

```

119
120 function [dydt]=DYDT(t,y,K)
121
122 % When the equation are written in explicit form
123
124 dydt=zeros(K,1);
125
126 dydt(1)=2*y(1)+y(2)+y(3)+cos(t);
127
128 dydt(2)=sin(y(1))+exp(-y(2)+y(3));
129
130 dydt(3)=y(1)*y(2)-y(3);
131
132 % If the set of equations is linear, then these can be
133 % written in matrix form as dydt=A*y+b(t). For example, if
134 % the set of equations is:
135 % dudt = 7u - 2v + w + exp(t)
136 % dvdt = 2u + 3v - 9w + cos(t)
137 % dwdt =      2v + 5w + 2
138 % Then:
139 % A=[7,-2,1;2,3,-9;0,2,5];
140 % b=@(t) [exp(t);cos(t);2];
141 % dydt=A*y+b(t)
142
143 end

```

6 MATLAB's In-Built Procedures

So far, the three main iterative methods have been developed that solve IVPs numerically. MATLAB, however, has its own built-in procedures that can solve IVPs with a combination of several methods. The two main ones are `ode23` (which uses a combination of a second and third order RK methods) and `ode45` (which uses a combination of a fourth and fifth order RK methods).

Both `ode45` and `ode23` are hybrid methods and use adaptive meshing, this means that the time span grid is not necessarily uniform, but it changes depending on the gradients; if the gradient is large at some point, then the stepsize will be small to capture these drastic changes.

The following MATLAB code solves the following set of IVPs on the interval $[0, 1]$ using `ode45`:

$$\begin{aligned}\frac{du}{dt} &= 2u + v + w + \cos(t), & u(0) &= 0 \\ \frac{dv}{dt} &= \sin(u) + e^{-v+w}, & v(0) &= 1 \\ \frac{dw}{dt} &= uv - w, & w(0) &= 0.\end{aligned}$$

```
1 function IVP_InBuilt
2
3 %% Solve a set of first order IVPs using In-Built codes
4
5 % This code solves a set of IVP when written explicitly
6 % on the interval [t0,tf] subject to the initial conditions
7 % y(0)=y0. The output will be the graph of the solution(s)
8 % and the vector value at the final point tf. Note that the
9 % IVPs do not need to be linear or homogeneous.
10
11 %% Lines to change:
12
13 % Line 28 : t0 - Start time
14 % Line 31 : tf - End time
15 % Line 43 : T_Span - Time span for evaluation
16 % Line 46 : y0 - Vector of initial values
17 % Line 86+ : Which functions to plot, remembering to assign
18 %           a colour, texture and legend label
19 % Line 100+ : Set of differential equations written
```

```

20 %           explicitly. These can also be non-linear and
21 %           include forcing terms. These equations can
22 %           also be written in matrix form if the
23 %           equations are linear.
24
25 %% Set up input values
26
27 % Start time
28 t0=0;
29
30 % End time
31 tf=1;
32
33 % Time span
34 % In-built methods tend to use adaptive meshing; decreasing
35 % the stepsize near locations with drastic derivative
36 % changes and increasing near small derivative changes.
37 % Sometimes this is not desired but a uniform meshing is
38 % required from the start time t0 to the end time tf being
39 % split into N equal sub intervals. This can be changed
40 % here:
41 % Adaptive meshing: T_Span=[t0 tf]
42 % Specific meshing: T_Span=linspace(t0,tf,N)
43 T_Span=[t0 tf];
44
45 % Column vector initial values y0=y(t0)
46 y0=[0;1;0];
47
48 %% Set up IVP solver parameters
49
50 % Number of differential equations
51 K=length(y0);
52
53 %% Use solver
54
55 % Set the solver tolerance
56 tol=odeset('RelTol',1e-6);
57
58 % Solve the IVP using ode45 or ode23
59 [T,Y]=ode45(@(t,y) DYDT(t,y,K),T_Span,y0,tol);
60
61 % Convert T and Y to columns for consistency
62 T=T';
63 Y=Y';
64

```

```

65 %% Setting plot parameters
66
67 % Clear figure
68 clf
69
70 % Hold so more than one line can be drawn
71 hold on
72
73 % Turn on grid
74 grid on
75
76 % Setting font size and style
77 set(gca,'FontSize',20,'FontName','Times')
78 set(legend,'Interpreter','Latex')
79
80 % Label the axes
81 xlabel('$t$', 'Interpreter','Latex')
82 ylabel('$\mathbf{y}(t)$', 'Interpreter','Latex')
83
84 % Plot the desired solutions. If all the solutions are
85 % needed, then consider using a for loop in that case
86 plot(T,Y(1,:), '-b', 'LineWidth',2, 'DisplayName', '$y_1(t)$')
87 plot(T,Y(2,:), '-r', 'LineWidth',2, 'DisplayName', '$y_2(t)$')
88 plot(T,Y(3,:), '-k', 'LineWidth',2, 'DisplayName', '$y_3(t)$')
89
90 % Display the values of the vector y at tf
91 disp(strcat('The vector y at t=', num2str(tf), ' is:'))
92 disp(Y(:,end))
93
94 end
95
96 function [dydt]=DYDT(t,y,K)
97
98 % When the equation are written in explicit form
99
100 dydt=zeros(K,1);
101
102 dydt(1)=2*y(1)+y(2)+y(3)+cos(t);
103
104 dydt(2)=sin(y(1))+exp(-y(2)+y(3));
105
106 dydt(3)=y(1)*y(2)-y(3);
107
108 % If the set of equations is linear, then these can be
109 % written in matrix form as dydt=A*y+b(t). For example, if

```

```

110 % the set of equations is:
111 % dudt = 7u - 2v + w + exp(t)
112 % dvdt = 2u + 3v - 9w + cos(t)
113 % dwdt =      2v + 5w + 2
114 % Then:
115 % A=[7,-2,1;2,3,-9;0,2,5];
116 % b=@(t) [exp(t);cos(t);2];
117 % dydt=A*y+b(t)
118
119 end

```

7 Implicit IVP Solvers

In some cases, IVPs can be difficult to solve because of the non-linearity of its terms, this is where *Implicit Methods* should be used to accommodate for these issues.

7.1 Backwards Euler Method

Consider the Euler method at the starting time $t = t_0$. The value of the function y at $t_1 = t_0 + h$ is approximated by

$$y(t_1) \approx Y_1 = Y_0 + hy'(t_0)$$

and this gives an upper bound for a stable stepsize of

$$h_0 = 2 \min \left(\frac{|\Re(\lambda_k)|}{|\lambda_k|^2} \right)$$

in order to ensure that the Euler method is computationally stable. However, suppose that this modified slightly by using the gradient at $y(t_1)$ rather than at $y(t_0)$, in other words, suppose that the value of y at t_1 is approximated by

$$y(t_1) \approx Y_1 = Y_0 + \underline{hy'(t_1)}.$$

This approach is known as the **Backwards Euler Method** and is an implicit procedure since the value of $y'(t_1)$ is not known to begin with.

The general formulation is as follows: Consider the system of differential equations

$$\mathbf{y}' = A\mathbf{y} + \mathbf{b}(t) \quad \text{with} \quad \mathbf{y}(t_0) = \mathbf{y}_0, \quad x \in [t_0, t_f].$$

Discretise the interval $[t_0, t_f]$ into N equal subintervals, each with width $h = \frac{t_f - t_0}{N}$. At the time step $t = t_n = t_0 + nh$, the backwards Euler method is

$$\mathbf{Y}_{n+1} = \mathbf{Y}_n + h\mathbf{y}'(t_{n+1}) = \mathbf{Y}_n + h[A\mathbf{Y}_{n+1} + \mathbf{b}(t_{n+1})].$$

This can be rearranged to give

$$(I - hA)\mathbf{Y}_{n+1} = \mathbf{Y}_n + h\mathbf{b}(t_{n+1}).$$

Rearranging further gives the basis for the Backwards Euler iteration which is

$$\mathbf{Y}_{n+1} = (I - hA)^{-1} [\mathbf{Y}_n + h\mathbf{b}(t_{n+1})]$$

whereas the standard Euler method in matrix form is

$$\mathbf{Y}_{n+1} = (I + hA)^{-1} \mathbf{Y}_n + h\mathbf{g}(t_{n+1}).$$

The Euler method requires *explicit calculations* using matrix multiplications but the backwards Euler method requires matrix inversion instead.

7.2 Stability of the Backwards Euler Method

Consider the initial value problem in its scalar form

$$\frac{dy}{dt} = \lambda y + b(t) \quad \text{with} \quad y(0) = y_0.$$

The backwards Euler method at the time $t = t_{n+1} = t_0 + (n+1)h$ gives

$$Y_{n+1} = (1 - h\lambda)^{-1} [Y_n + hg(t_{n+1})].$$

This initial condition can be perturbed by adding a small parameter $\varepsilon \neq 0$ to give the perturbed differential equation

$$\frac{dz}{dt} = \lambda z + g(t) \quad \text{with} \quad z(0) = y_0 + \varepsilon.$$

The backwards Euler then yields

$$Z_{n+1} = (1 - h\lambda)^{-1} [Z_n + hg(t_{n+1})]$$

The differential equations in Y and Z can be subtracted to give a perturbation term E where

$$E_{n+1} = Z_{n+1} - Y_{n+1} = (1 - h\lambda)^{-1} [Z_n - Y_n] = (1 - h\lambda)^{-1} E_n.$$

Notice that once again, the forcing function $g(t)$ has been eliminated and therefore does not affect the stability of the backwards Euler method. The differential equation for E will have the initial condition $E_0 = Z_0 - Y_0 = \varepsilon$. This expression can be used to represent E_n in terms of ε recursively as:

$$\begin{aligned} E_n &= (1 - h\lambda)^{-1} E_{n-1} = (1 - h\lambda)^{-2} E_{n-2} \\ &= \dots = (1 - h\lambda)^{-(n-1)} E_1 = (1 - h\lambda)^{-n} E_0 = (1 - h\lambda)^{-n} \varepsilon. \\ \implies E_n &= (1 - h\lambda)^{-n} \varepsilon. \end{aligned}$$

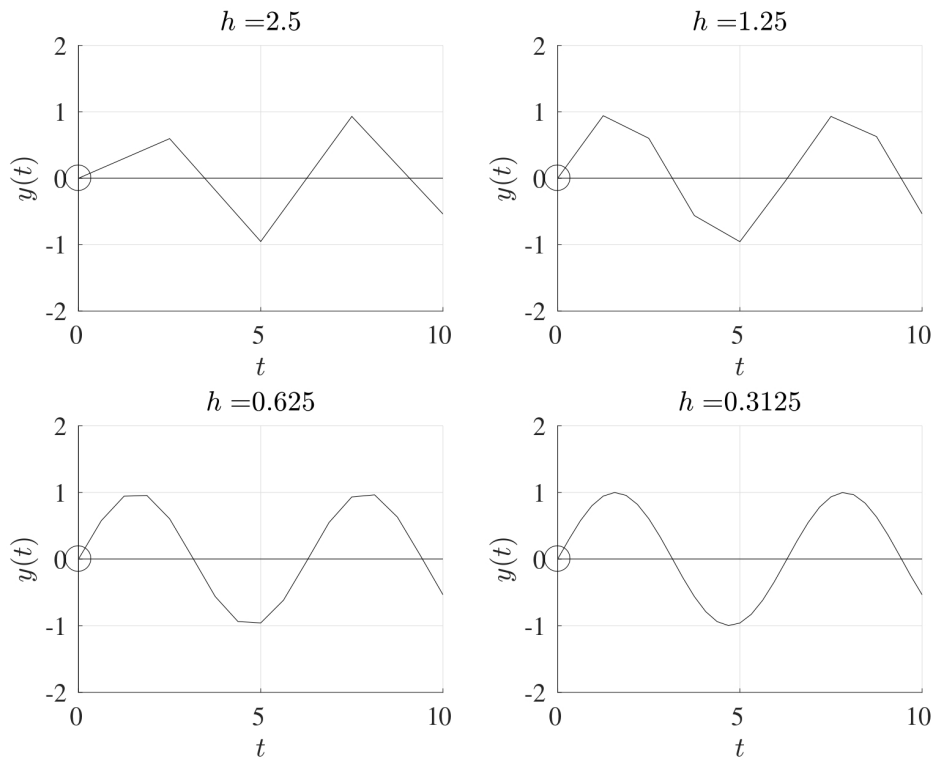
This means that the method is stable for stepsizes h that satisfy $|1 - h\lambda| > 1$ and since $\lambda < 0$ for an asymptotically stable system, then this inequality is *always* satisfied. This means that the backwards Euler method is stable *for all* stepsizes $h > 0$, no matter how large.

Backwards Euler Method

Consider the differential equation

$$y' = -100y + 100 \sin(t) \quad \text{with} \quad y(0) = 1.$$

In this case, $\lambda < 0$ meaning that this differential equation is asymptotic stable. The maximum allowable stepsize for the Euler method is $h_0 = \frac{2}{|-100|} = 0.02$. However, the backwards Euler method is stable for any stepsize h as seen below (very large stepsizes will still converge but they will not give any useful information).



The formulation presented above also holds for sets of differential equations in the same way with one difference. Instead of having $(1 - h\lambda)^{-1} = \frac{1}{1-h\lambda}$, the procedure for systems will require the matrix inverse $(1 - \lambda A)^{-1}$ or the MATLAB backslash operator can be used instead.

7.3 Order of Accuracy

The backwards Euler method is numerically stable for all values the stepsize h and has the same order of accuracy as the Euler method, i.e. the local truncation error is of $\mathcal{O}(h^2)$ while the global integration error is of $\mathcal{O}(h)$. However, this increased stability comes at a cost, the backwards Euler methods requires double the computational cost compared to the Euler method.

7.4 MATLAB Code

The following MATLAB code performs the Backwards Euler iteration for the system $\mathbf{y}' = \mathbf{A}\mathbf{y} + \mathbf{b}(t)$ subject to $\mathbf{y}(0) = \mathbf{y}_0$ where

$$\mathbf{A} = \begin{pmatrix} -7 & -2 & 1 \\ 2 & -1 & -9 \\ 0 & 0 & -5 \end{pmatrix}, \quad \mathbf{b}(t) = \begin{pmatrix} \sin(t) \\ 0 \\ 2 \end{pmatrix}, \quad \mathbf{y}_0 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}.$$

```
1 function IVP_Back_Euler
2
3 %% Solve a set of first order IVPs using Backwards Euler
4
5 % This code solves a set of IVP when written in the form
6 % dydt=A*y+b(t) on the interval [t0,tf] subject to the
7 % initial conditions y(0)=y0. The output will be the graph
8 % of the solution(s) and the vector value at the final
9 % point tf.
10
11 %% Lines to change:
12
13 % Line 25 : t0 - Start time
14 % Line 28 : tf - End time
15 % Line 31 : N - Number of subdivisions
16 % Line 34 : A - Matrix A
17 % Line 37 : b - Forcing vector b(t)
18 % Line 40 : y0 - Vector of initial values
19 % Line 106+ : Which functions to plot, remembering to assign
20 %             a colour, texture and legend label
21
22 %% Set up input values
23
24 % Start time
25 t0=0;
26
27 % End time
28 tf=1;
29
30 % Number of subdivisions
31 N=5000;
32
33 % Matrix A
34 A=[-7,-2,1;2,-1,-9;0,0,-5];
35
36 % Vector b, which can be a function of t in general
37 b=@(t) [sin(t);0;2];
```

```

38
39 % Column vector initial values y0=y(t0)
40 y0=[0;1;0];
41
42 %% Set up IVP solver parameters
43
44 % T = Vector of times t0,t1,...,tN.
45 % This is generated using linspace which splits the
46 % interval [t0,tf] into N+1 points (or N subintervals)
47 T=linspace(t0,tf,N+1);
48
49 % Stepsize
50 h=(tf-t0)/N;
51
52 % Number of differential equations
53 K=length(y0);
54
55 %% Perform the Euler iteration
56
57 % Y = Solution matrix
58 % The matrix Y will contain K rows and N+1 columns. Every
59 % row corresponds to a different IVP and every column
60 % corresponds to a different time. So the matrix Y will
61 % take the following form:
62 % y_1(t_0) y_1(t_1) y_1(t_2) ... y_1(t_N)
63 % y_2(t_0) y_2(t_1) y_2(t_2) ... y_2(t_N)
64 % ...
65 % y_K(t_0) y_K(t_1) y_K(t_2) ... y_K(t_N)
66 Y=zeros(K,N+1);
67
68 % The first column of the vector Y is the initial vector y0
69 Y(:,1)=y0;
70
71 % Set the current time t to be the starting time t0 and the
72 % current value of the vector y to be the strtaing values y0
73 y=y0;
74
75 for n=2:1:N+1
76
77     t=T(n); % Update the new time
78
79     y=(eye(K,K)-h*A)\(y+h*b(t)); % Find y at the current step
80
81     Y(:,n)=y; % Replace row n in Y with y
82

```

```

83 end
84
85 %% Setting plot parameters
86
87 % Clear figure
88 clf
89
90 % Hold so more than one line can be drawn
91 hold on
92
93 % Turn on grid
94 grid on
95
96 % Setting font size and style
97 set(gca,'FontSize',20,'FontName','Times')
98 set(legend,'Interpreter','Latex')
99
100 % Label the axes
101 xlabel('$t$', 'Interpreter','Latex')
102 ylabel('$\mathbf{y}(t)$', 'Interpreter','Latex')
103
104 % Plot the desired solutions. If all the solutions are
105 % needed, then consider using a for loop in that case
106 plot(T,Y(1,:), '-b', 'LineWidth', 2, 'DisplayName', '$y_1(t)$')
107 plot(T,Y(2,:), '-r', 'LineWidth', 2, 'DisplayName', '$y_1(t)$')
108 plot(T,Y(3,:), '-k', 'LineWidth', 2, 'DisplayName', '$y_1(t)$')
109
110 end

```

7.5 Stiff Differential Equations

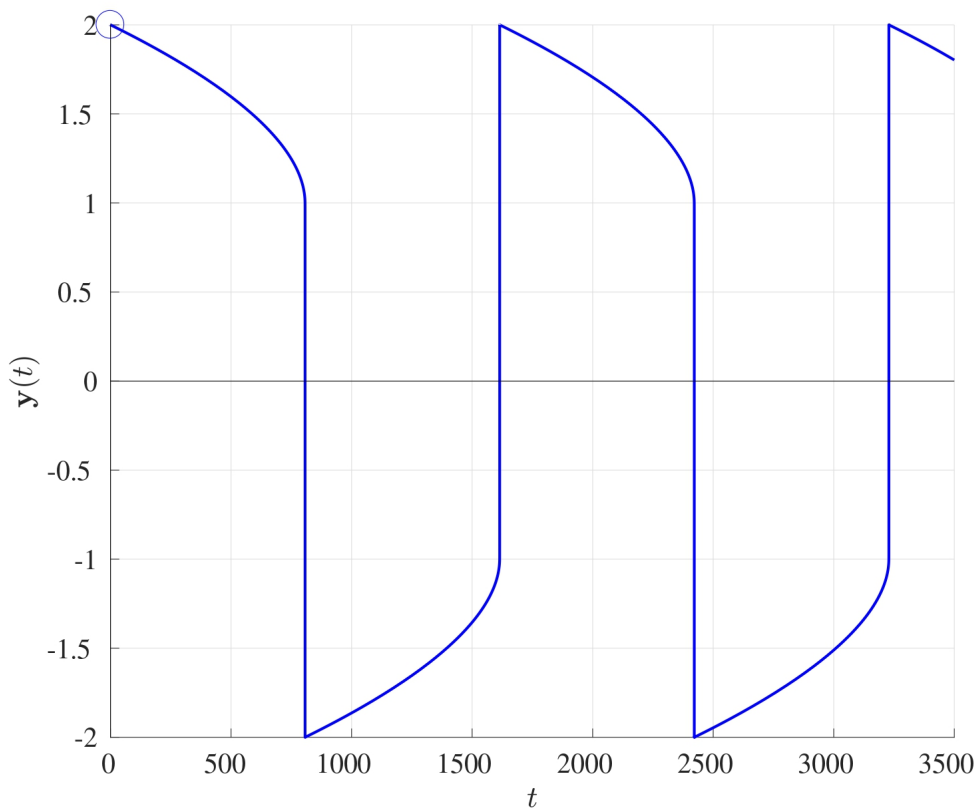
Stiff sets of differential equations with a large value of the total computational cost N_0 can be very difficult to solve numerically using explicit methods but implicit methods can work very well. MATLAB has its very own built-in stiff differential equation solver under the command `ode15s` and this can be implemented exactly as `ode45`. This solves sets of differential equations implicitly using numerical differentiation of orders 1 to 5.

Stiff IVPs

Consider the set of differential equations on the interval $[0, 3500]$

$$\begin{aligned}\frac{dy_1}{dt} &= y_2 & y_1(0) &= 2 \\ \frac{dy_2}{dt} &= 1000(1 - y_1^2)y_2 - y_1 & y_2(0) &= 0.\end{aligned}$$

This is a very stiff set of differential equations, solving this using `ode45` takes upwards of 92 seconds while solving using the stiff solver `ode15s` requires a mere 0.233 seconds (depending on you machine). The result of solving this differential equation is shown below for $y_1(t)$ only since $y_2(t)$ takes very large values and this distorts the graphical interpretation.



Using the stiff solver optimises the stepsizes for stiff regions. Particularly, if a region is deemed to be considerably “stiff”, the `ode15s` will use smaller stepsizes to solve the problem but if there is a region where the differential is not “stiff”, then larger stepsizes will be used. Therefore, `ode15s` usually requires fewer grid points overall, for instance to solve the above set of differential equations, `ode15s` only requires 1,836 grid points while `ode45` requires 7,820,485 grid points, that is over 4,200 times more grid points than `ode15s`. This just goes to show that stiff differential need implicit methods, even though the cost for every step is greater than that of an explicit method, fewer steps

are required in total.

An alternative stiff differential equation solver is `ode23s` which achieves that same outcome as `ode15s` but with a lower accuracy and more grid points using only second and third order methods.

Part III

Solving Boundary Value Problems

This section will cover ways of solving Boundary Value Problems (BVPs) using the different differencing schemes.

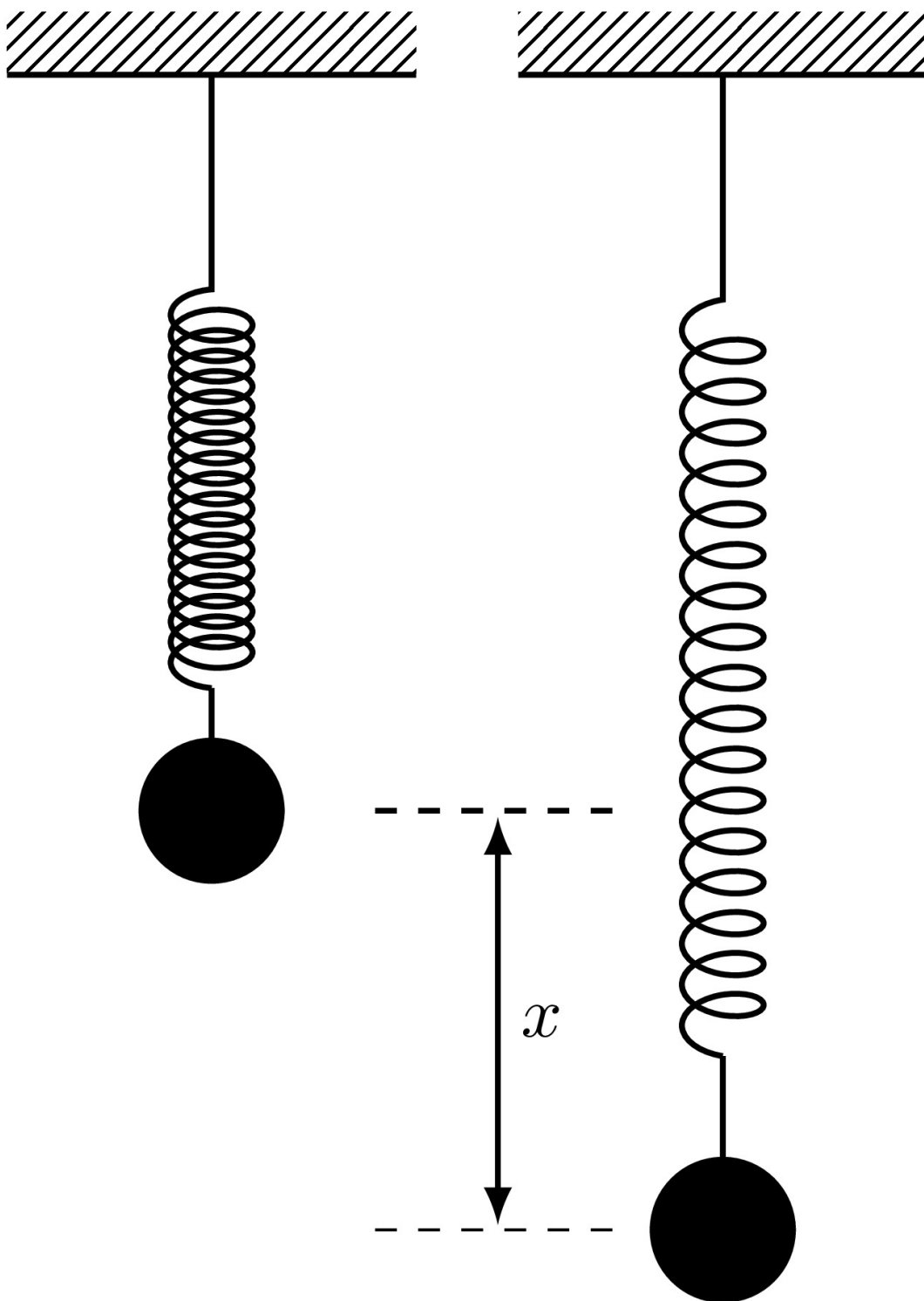
8 Boundary Value Problems

Boundary Value Problems (BVPs) are similar in many ways to initial value problems in the sense that a set of differential equations are given that are to be solved subject to certain conditions. In initial value problems, these conditions are imposed at the starting time but in boundary value problems, they are imposed at particular locations.

One of the most important differences when it comes to solving BVPs versus IVPs is the existence of solutions. Solutions to initial value problems always *exist* and are *unique* (subject to certain restriction on the right hand side), this is as a consequence of the *Picard-Lindelöf* theorem. The same cannot be said for boundary value problems; the solution to BVPs could exist and be unique, exist and not be unique or not exist at all.

8.1 Example of Boundary Value Problems

Consider a mass m hanging from a spring with spring constant K . Suppose that the spring is extended (by pulling the mass) by a distance x as seen below.



Then by *Hooke's Law*, the force pulling the mass back to its equilibrium position is given

by

$$F = -Kx.$$

As the mass is released, it will accelerate upwards with an acceleration a and the force responsible for this acceleration is given by *Newton's Second Law of Motion*

$$F = ma.$$

The acceleration a is the second derivative of the displacement x with respect to time and since it acts in a direction opposite to the extension, then

$$a = -\frac{d^2x}{dt^2} \implies F = -m\frac{d^2x}{dt^2}$$

Equating the two expressions for the force from Newton's Second Law and Hooke's Law will give

$$-Kx = -m\frac{d^2x}{dt^2} \implies \frac{d^2x}{dt^2} + \omega^2x = 0 \quad \text{where} \quad \omega = \sqrt{\frac{K}{m}}.$$

This differential equation represents the simple harmonic motion of a mass hanging on a frictionless massless spring which oscillates with a frequency ω . Since this is a second order differential equation, two conditions need to be imposed:

- *Initial conditions* can be imposed at the starting time, specifically $x(0)$ and $x'(0)$ which prescribe the initial position and initial speed,
- *Boundary conditions* can be imposed at different times, say $x(0)$ and $x(10)$ which prescribe the location at time $t = 0$ and time $t = 10$.

Consider the differential equation for the undamped simple harmonic oscillator with frequency 1, namely

$$\frac{d^2u}{dt^2} = -u.$$

This differential equation has the general analytic solution

$$u(t) = C_1 \cos(t) + C_2 \sin(t)$$

where C_1 and C_2 are constants of integration which will be determined from the boundary conditions.

Three qualitatively different sets of boundary conditions will be investigated:

- $u(0) = 1$ and $u(\frac{5\pi}{2}) = -1$: The constants C_1 and C_2 can be found as:

$$1 = u(0) = C_1 \cos(0) + C_2 \sin(0) = C_1 \implies C_1 = 1$$

$$-1 = u\left(\frac{5\pi}{2}\right) = C_1 \cos\left(\frac{5\pi}{2}\right) + C_2 \sin\left(\frac{5\pi}{2}\right) = C_2 \implies C_2 = -1.$$

Therefore the analytic solution to the boundary value problem subject to these conditions is

$$u(t) = \cos(t) - \sin(t)$$

and this is captured by the finite difference approximation. In this case, the solution to the boundary value problem *exists* and is *unique*.

- $u(0) = 0$ and $u(2\pi) = 0$: The constants C_1 and C_2 can be found as:

$$0 = u(0) = C_1 \cos(0) + C_2 \sin(0) = C_1 \implies C_1 = 0$$

$$0 = u(2\pi) = C_1 \cos(2\pi) + C_2 \sin(2\pi) = C_1 \implies C_1 = 0.$$

These two conditions provide an expressions for the constant C_1 only and not C_2 , therefore the particular solution will be

$$u(t) = C_2 \sin(t)$$

which is valid for *any* value of C_2 . Therefore in this case, the solution *exists* but is *not unique*.

- $u(0) = 1$ and $u(2\pi) = -1$: The constants C_1 and C_2 can be found as:

$$1 = u(0) = C_1 \cos(0) + C_2 \sin(0) = C_1 \implies C_1 = 1$$

$$-1 = u(\pi) = C_1 \cos(2\pi) + C_2 \sin(2\pi) = C_1 \implies C_1 = -1.$$

In this case, the boundary values have resulted in a contradiction and therefore the solution *does not exist* when subject to these boundary conditions.

This final case is when the solution to a boundary value problem does *not exist*.

8.2 Finite Difference Method for Boundary Value Problems

Consider the general second order boundary value problem

$$a(x)\frac{d^2u}{dx^2} + b(x)\frac{du}{dx} + c(x)u = f(x) \quad \text{with} \quad 0 < x < L$$

$$\text{and} \quad u(0) = u_l, \quad u(L) = u_r$$

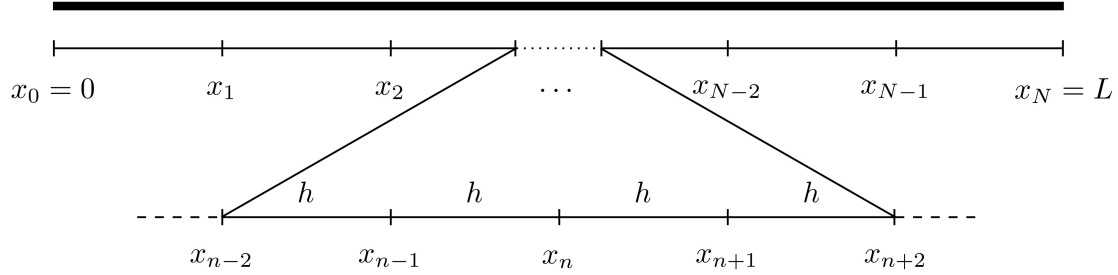
where the functions a, b, c and f are known functions of x . Boundary value problems like this are solved using an incredibly versatile method known as the ***Finite Difference Method***. This procedure essentially changes a differential equation into a set of difference equations by using approximations to the derivatives.

The term *finite difference approximation* refers to how derivatives can be approximated using linear expressions of the function at neighbouring points. For instance, the derivative of some function u at a given point X can be approximated as the gradient of u between two points around X , for example

$$\frac{df}{dx}(X) \approx \frac{f(X+h) - f(X-h)}{2h}.$$

There are many other ways in which these approximations can be made depending on the way in which the grid has been set up or on the context of the problem.

Consider a general unknown function $u(x)$ defined on $[0, L]$ where $u(0)$ and $u(L)$ are given (as boundary conditions). First, split the interval into N equally sized sections, each of width h , and label the points as $x_0, x_1, \dots, x_N$ where $x_n = nh$.



For first and second derivatives, there are three main approximations that are most widely used:

- **Forward Difference:**

$$\frac{du}{dx}(x_n) \approx \frac{u(x_{n+1}) - u(x_n)}{h}$$

$$\frac{d^2u}{dx^2}(x_n) \approx \frac{u(x_{n+2}) - 2u(x_{n+1}) + u(x_n)}{h^2}$$

- **Backward Difference:**

$$\frac{du}{dx}(x_n) \approx \frac{u(x_n) - u(x_{n-1})}{h}$$

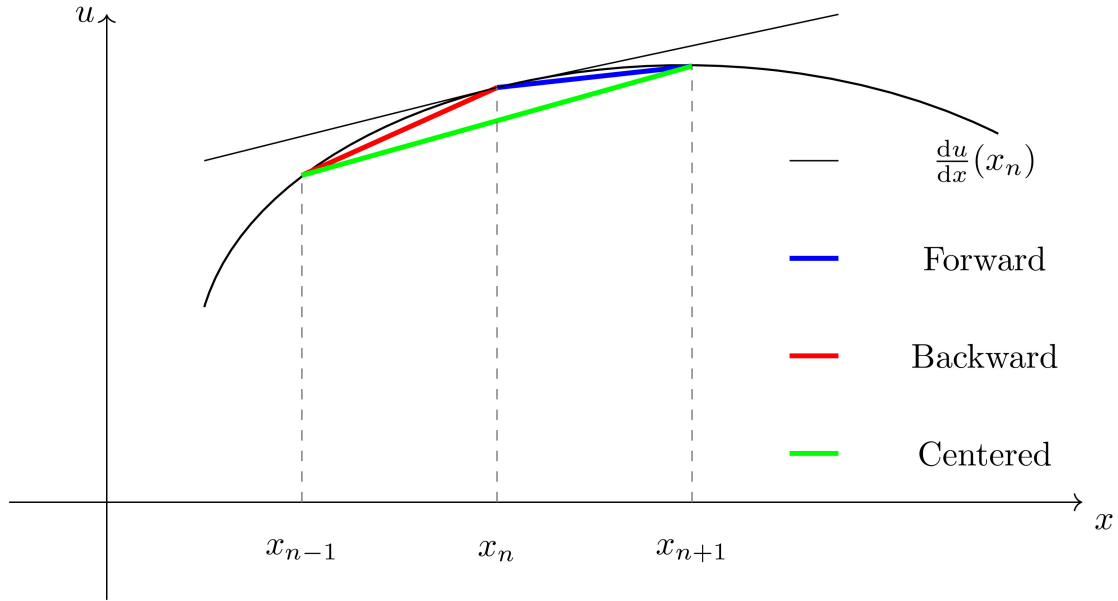
$$\frac{d^2u}{dx^2}(x_n) \approx \frac{u(x_n) - 2u(x_{n-1}) + u(x_{n-2})}{h^2}$$

- **Centred Difference:**

$$\frac{du}{dx}(x_n) \approx \frac{u(x_{n+1}) - u(x_{n-1})}{2h}$$

$$\frac{d^2u}{dx^2}(x_n) \approx \frac{u(x_{n+1}) - 2u(x_n) + u(x_{n-1}))}{h^2}$$

The graphical interpretation of the approximations to the first derivatives are shown below.



i Second Derivative Expression

To show how the second derivative expressions are obtained, consider the centred difference approximation

$$\frac{du}{dx}(x_n) \approx \frac{u(x_{n+1}) - u(x_{n-1}))}{2h}.$$

To derive the expression for the second derivative, introduce two fictitious points $x_{n-0.5}$ (which is half-way between x_{n-1} and x_n) and $x_{n+0.5}$ (which is half-way between x_n and x_{n+1}). Then

$$\begin{aligned} \frac{d^2u}{dx^2}(x_n) &= \frac{d}{dx} \left(\frac{du}{dx}(x_n) \right) \approx \frac{d}{dx} \left(\frac{u(x_{n+0.5}) - u(x_{n-0.5}))}{h} \right) \\ &\approx \frac{u'(x_{n+0.5}) - u'(x_{n-0.5}))}{h} \\ &\approx \frac{\frac{u(x_{n+1}) - u(x_n)}{h} - \frac{u(x_n) - u(x_{n-1}))}{h}}{h} \\ &= \frac{u(x_{n+1}) - 2u(x_n) + u(x_{n-1}))}{h^2}. \end{aligned}$$

The derivation of the second derivative approximations for the forward and backward differences can be done in a very similar way but without the need for half steps.

Any of these three approximations can be used to approximate the derivatives of the function u at the point x_n . Denote the approximation of u at the point x_n by U_n , i.e. $U_n \approx u(x_n)$, then

- **Forward Difference:**

$$\frac{du}{dx}(x_n) \approx \frac{U_{n+1} - U_n}{h} \quad ; \quad \frac{d^2u}{dx^2}(x_n) \approx \frac{U_{n+2} - 2U_{n+1} + U_n}{h^2}$$

- **Backward Difference:**

$$\frac{du}{dx}(x_n) \approx \frac{U_n - U_{n-1}}{h} \quad ; \quad \frac{d^2u}{dx^2}(x_n) \approx \frac{U_n - 2U_{n-1} + U_{n-2}}{h^2}$$

- **Centred Difference:**

$$\frac{du}{dx}(x_n) \approx \frac{U_{n+1} - U_{n-1}}{2h} \quad ; \quad \frac{d^2u}{dx^2}(x_n) \approx \frac{U_{n+1} - 2U_n + U_{n-1}}{h^2}.$$

These approximations will form the basis for solving the BVP.

8.2.1 Discretisation of the Differential Equation

Returning to the differential equation

$$a(x)\frac{d^2u}{dx^2} + b(x)\frac{du}{dx} + c(x)u = f(x).$$

Evaluate this equation at $x = x_n$ for some n , then

$$a(x_n)\frac{d^2u}{dx^2}(x_n) + b(x_n)\frac{du}{dx}(x_n) + c(x_n)u(x_n) = f(x_n).$$

For now, suppose the *centred differencing approximation* is used to approximate the derivatives. Replacing the approximations of the derivatives of u at x_n gives

$$a(x_n)\frac{U_{n+1} - 2U_n + U_{n-1}}{h^2} + b(x_n)\frac{U_{n+1} - U_{n-1}}{2h} + c(x_n)U_n = f(x_n).$$

This can be simplified by collecting the U terms resulting in:

$$\alpha_n U_{n-1} + \beta_n U_n + \gamma_n U_{n+1} = f(x_n)$$

$$\alpha_n = \frac{a(x_n)}{h^2} - \frac{b(x_n)}{2h}, \quad \beta_n = -\frac{2a(x_n)}{h^2} + c(x_n), \quad \gamma_n = \frac{a(x_n)}{h^2} + \frac{b(x_n)}{2h}.$$

This expression will hold for all the values of $n = 1, 2, \dots, N-1$ (otherwise there will be points x_{-1} and x_{N+1} which are outside the domain $[0, L]$). Therefore, this means that there will be $N-1$ equations in $N+1$ unknowns which are $U_0, U_1, U_2, \dots, U_N$.

This system may seem to be undetermined however, there are two boundary conditions that have not been taken into consideration yet, namely $u(x_0) = u_l$ and $u(x_N) = u(L) = u_r$. Since these are known, the approximations U_0 and U_N have defined values, i.e. $U_0 \approx u(x_0) = u_l$ and $U_N = u(x_N) = u_r$. This eliminates two of the unknowns giving $N-1$ equations in $N-1$ unknowns.

At $n = 1$, the approximation to the differential equation is

$$\alpha_1 U_0 + \beta_1 U_1 + \gamma_1 U_2 = f(x_1)$$

and since U_0 is already known, then it can be taken to the right hand side to give

$$\beta_1 U_1 + \gamma_1 U_2 = f(x_1) - \alpha_1 u_0.$$

Similarly, at $n = N - 1$, the approximation is

$$\alpha_{N-1} U_{N-2} + \beta_{N-1} U_{N-1} + \gamma_{N-1} U_N = f(x_{N-1})$$

and since U_N is known, this can be rewritten as

$$\alpha_{N-1} U_{N-2} + \beta_{N-1} U_{N-1} = f(x_{N-1}) - \gamma_{N-1} u_L.$$

For $n = 2, 3, \dots, N - 2$, the approximation is

$$\alpha_n U_{n-1} + \beta_n U_n + \gamma_n U_{n+1} = f(x_n)$$

where U_{n-1}, U_n and U_{n+1} are all unknown. In summary, all of these $N - 1$ equations are:

$$\begin{aligned} n = 1 : & \quad \beta_1 U_1 + \gamma_1 U_2 = f(x_1) - \alpha_1 u_0 \\ n = 2 : & \quad \alpha_2 U_1 + \beta_2 U_2 + \gamma_2 U_3 = f(x_2) \\ n = 3 : & \quad \alpha_3 U_2 + \beta_3 U_3 + \gamma_3 U_4 = f(x_3) \\ & \quad \vdots \\ n = N - 3 : & \quad \alpha_{N-3} U_{N-4} + \beta_{N-3} U_{N-3} + \gamma_{N-3} U_{N-2} = f(x_{N-3}) \\ n = N - 2 : & \quad \alpha_{N-2} U_{N-3} + \beta_{N-2} U_{N-2} + \gamma_{N-2} U_{N-1} = f(x_{N-2}) \\ n = N - 1 : & \quad \alpha_{N-1} U_{N-2} + \beta_{N-1} U_{N-1} = f(x_{N-1}) + \gamma_{N-1} u_L \end{aligned}$$

These can be written in matrix form as $A\mathbf{U} = \mathbf{g}$, namely

$$\underbrace{\begin{pmatrix} \beta_1 & \gamma_1 & 0 & 0 & \dots & 0 & 0 & 0 & 0 \\ \alpha_2 & \beta_2 & \gamma_2 & 0 & \dots & 0 & 0 & 0 & 0 \\ 0 & \alpha_3 & \beta_3 & \gamma_3 & \dots & 0 & 0 & 0 & 0 \\ 0 & 0 & \alpha_4 & \beta_4 & \dots & 0 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \beta_{N-4} & \gamma_{N-4} & 0 & 0 \\ 0 & 0 & 0 & 0 & \dots & \alpha_{N-3} & \beta_{N-3} & \gamma_{N-3} & 0 \\ 0 & 0 & 0 & 0 & \dots & 0 & \alpha_{N-2} & \beta_{N-2} & \gamma_{N-2} \\ 0 & 0 & 0 & 0 & \dots & 0 & 0 & \alpha_{N-1} & \beta_{N-1} \end{pmatrix}}_A \underbrace{\begin{pmatrix} U_1 \\ U_2 \\ U_3 \\ U_4 \\ \vdots \\ U_{N-4} \\ U_{N-3} \\ U_{N-2} \\ U_{N-1} \end{pmatrix}}_U = \underbrace{\begin{pmatrix} f(x_1) - \alpha_1 u_l \\ f(x_2) \\ f(x_3) \\ f(x_4) \\ \vdots \\ f(x_{N-4}) \\ f(x_{N-3}) \\ f(x_{N-2}) \\ f(x_{N-1}) - \gamma_{N-1} u_r \end{pmatrix}}_g.$$

The matrix A is of size $(N-1) \times (N-1)$ all of whose terms are known, the vector $\mathbf{g}$ of size $(N-1) \times 1$ also has terms that are all known. The unknown vector here is $\mathbf{U}$ and it can be found by inverting A to give $\mathbf{U} = A^{-1}\mathbf{g}$.

Carrying out matrix inversions by hand can become increasingly cumbersome if A is larger than 2×2 and therefore this process should be done computationally. This can be done using TDMA as explained in Section 2.2.3 or solved in MATLAB by using either $\mathbf{U} = \text{inv}(\mathbf{A}) * \mathbf{g}$ or $\mathbf{U} = \mathbf{A} \backslash \mathbf{g}$. The *backslash* method is faster than explicit matrix inversion if the matrix is of a large size.

The same process can be done for the forward and backward differencing approximations as well, the only difference will be the expressions for α , β and γ :

- Forward Differencing:

$$\alpha_n = \frac{a(x_n)}{h^2} - \frac{b(x_n)}{h} + c(x_n), \quad \beta_n = -\frac{2a(x_n)}{h^2} + \frac{b(x_n)}{h}, \quad \gamma_n = \frac{a(x_n)}{h^2}.$$

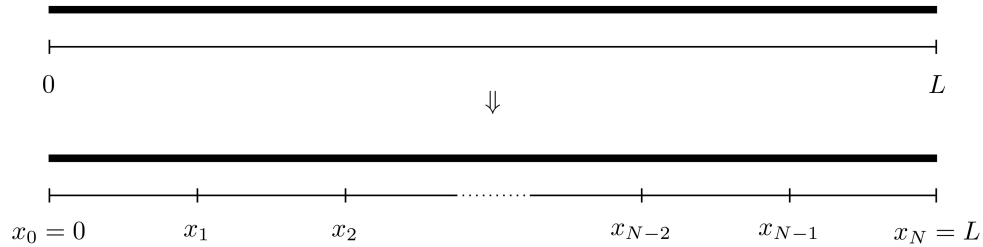
- Backward Differencing:

$$\alpha_n = \frac{a(x_n)}{h^2}, \quad \beta_n = -\frac{2a(x_n)}{h^2} - \frac{b(x_n)}{h}, \quad \gamma_n = \frac{a(x_n)}{h^2} + \frac{b(x_n)}{h} + c(x_n).$$

i Steps of The Finite Difference Method

In summary, these are the steps of the finite difference method:

1. Divide the interval $[0, L]$ into N equally sized sections, each of width $h = \frac{L}{N}$ and label the points as $x_0, x_1, x_2, \dots, x_N$ where $x_n = nh$.



2. The values of the function u are to be found at all the locations x_n . Denote the approximation to the function u at the point x_n by U_n , i.e. $U_n \approx u(x_n)$ for all $n = 0, 1, 2, \dots, N$.
3. Evaluate the differential equation at all the points x_n where the derivatives are replaced by their *finite difference approximations*.
4. This will result in a set of $N - 1$ linear equations in $N + 1$ unknowns, namely, $U_0, U_1, U_2, \dots, U_N$.
5. The values for U_0 and U_N are known from the boundary conditions, since $U_0 = u(0) = u_l$ and $U_N = u(L) = u_r$ and no approximation is needed since the exact values are known.
6. Write the whole system of equations in the matrix form $AU = g$ and solve using TDMA of MATLAB's backlash operator.

BVP Example

Consider the boundary value problem

$$\frac{d^2 u}{dx^2} = x^3, \quad x \in [0, 2] \quad \text{with} \quad u(0) = 0 \quad \text{and} \quad u(2) = 1.$$

The differential equation itself can be solved analytically to give

$$u(x) = \frac{1}{20}x^5 - \frac{3}{10}x.$$

This example will be used for the purposes of demonstration and comparison between the numerically obtained solution and the exact solution.

Suppose the interval $[0, 2]$ is to be divided into 5 equally sized sections, therefore $N = 5$ and $h = \frac{L}{N} = \frac{2}{5} = 0.4$.



The functions $a(x), b(x), c(x)$ and $f(x)$ in this interval are:

$$a(x) = 1, \quad b(x) = 0, \quad c(x) = 0, \quad f(x) = x^3.$$

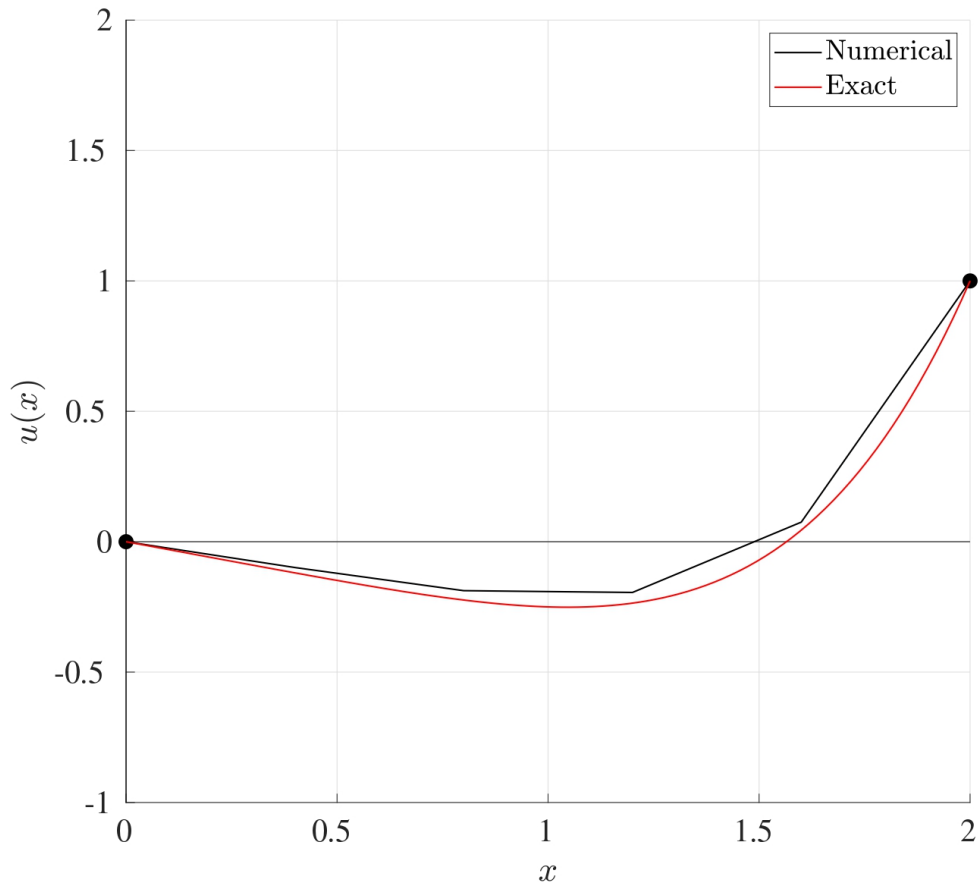
The matrix values are

$$\begin{aligned}\alpha_n &= \frac{a(x_n)}{h^2} - \frac{b(x_n)}{2h} = 6.25 \\ \beta_n &= -\frac{2a(x_n)}{h^2} + c(x_n) = -12.5 \\ \gamma_n &= \frac{a(x_n)}{h^2} + \frac{b(x_n)}{2h} = 6.25.\end{aligned}$$

These can be used to obtain expressions for the matrix A and the vector $\mathbf{g}$ as

$$A = \begin{pmatrix} -12.5 & 6.25 & 0 & 0 \\ 6.25 & -12.5 & 6.25 & 0 \\ 0 & 6.25 & -12.5 & 6.25 \\ 0 & 0 & 6.25 & -12.5 \end{pmatrix}, \quad \mathbf{g} = \begin{pmatrix} 0.064 \\ 0.512 \\ 1.728 \\ 4.096 \end{pmatrix}.$$

This system can be solved using $\mathbf{U} = \text{inv}(\mathbf{A}) * \mathbf{g}$ or $\mathbf{U} = \mathbf{A} \backslash \mathbf{g}$. The numerical solution is compared to exact solution below.



The advantage of using this boundary value solver is that the computations are in no way taxing on MATLAB. The system that results is composed entirely of linear equations and this system is solvable (provided the boundary value problem does indeed have a solution which may not always be possible). MATLAB's backslash operator is very effective in dealing with matrices, especially owing to the fact that the matrix A is a tridiagonal matrix.

8.3 MATLAB Code

Below is the MATLAB code that solves the BVP

$$\frac{d^2u}{dx^2} + 2\frac{du}{dx} + e^{-x}u = \sin(x), \quad x \in [0, 10] \quad \text{with} \quad u(0) = 1 \quad \text{and} \quad u(10) = -1$$

using the centred differencing method with $N = 50$.

```

1  function BVP_CD
2
3  %% Solve BVPs using centered differences
4
5  % The bvp is written in the form
6  % a(x) u'' + b(x) u' + c(x) u = f(x) on x in [x0,L]
7  % with the boundary conditions u(x0)=ul and u(L)=ur.
8  % After the centered difference approximation is
9  % used, the system will be written in the form AU=g.
10
11 %% Lines to change:
12
13 % Line 26 : x0 - Start point
14 % Line 29 : L - End point
15 % Line 32 : N - Number of subdivisions
16 % Line 35 : x1 - Left boundary value
17 % Line 38 : xr - Right boundary value
18 % Line 119 : Expression for the function a(x)
19 % Line 127 : Expression for the function b(x)
20 % Line 135 : Expression for the function c(x)
21 % Line 143 : Expression for the function f(x)
22
23 %% Set up input values
24
25 % Start point
26 x0=0;
27
28 % End point

```

```

29 L=10;
30
31 % Number of subdivisions
32 N=50;
33
34 % Boundary value at x=x0
35 ul=1;
36
37 % Boundary value at x=L
38 ur=-1;
39
40 %% Set up BVP solver parameters
41
42 % Interval width
43 h=(L-x0)/N;
44
45 % X = Vector of locations
46 % (x1, x2, x3, ..., xN) (notice the start is x1 NOT x0)
47 X=x0+h:h:L;
48
49 % Evaluate the functions a(x), b(x), c(x) and f(x) at X
50 aX=a(X);
51 bX=b(X);
52 cX=c(X);
53 fX=f(X);
54
55 % Find the expressions for alpha, beta and gamma at X
56 alpha=aX/(h^2)-bX/(2*h);
57 beta=-2*aX/(h^2)+cX;
58 gamma=aX/(h^2)+bX/(2*h);
59
60 % Set up the vector g on the right hand side
61 g=zeros(N-1,1);
62 g(1)=fX(1)-alpha(1)*ul;
63 g(N-1)=fX(N-1)-gamma(N-1)*ur;
64 for j=2:1:N-2
65     g(j)=fX(j);
66 end
67
68 % Set up the matrix A on the left hand side (LHS_A is
69 % to avoid confusion with the function a(x))
70 A=zeros(N-1,N-1);
71 A(1,1)=beta(1);
72 A(1,2)=gamma(1);
73 A(N-1,N-1)=beta(N-1);

```

```

74 A(N-1,N-2)=alpha(N-1);
75 for j=2:1:N-2
76     A(j,j-1)=alpha(j);
77     A(j,j)=beta(j);
78     A(j,j+1)=gamma(j);
79 end
80
81 % Solve for the unknown vector U (it is then readjusted
82 % from a column vector to a row vector for plotting)
83 U=A\g;
84 U=U';
85
86 % Add the missing term x0 to the start of the vector x
87 X=[x0,X];
88
89 % Add the left and right boundary values to the vector U
90 U=[ul,U,ur];
91
92 %% Setting plot parameters
93
94 % Clear figure
95 clf
96
97 % Hold so more than one line can be drawn
98 hold on
99
100 % Turn on grid
101 grid on
102
103 % Setting font size and style
104 set(gca,'FontSize',20,'FontName','Times')
105
106 % Label the axes
107 xlabel('$t$', 'Interpreter','Latex')
108 ylabel('$u(t)$', 'Interpreter','Latex')
109
110 % Plot solution
111 plot(X,U, '-k', 'LineWidth',2)
112
113 end
114
115 function [A]=a(X)
116 A=zeros(size(X));
117 for i=1:1:length(X)
118     x=X(i);

```

```

119     A(i)=1;
120 end
121 end
122
123 function [B]=b(X)
124 B=zeros(size(X));
125 for i=1:length(X)
126     x=X(i);
127     B(i)=2;
128 end
129 end
130
131 function [C]=c(X)
132 C=zeros(size(X));
133 for i=1:length(X)
134     x=X(i);
135     C(i)=exp(-x);
136 end
137 end
138
139 function [F]=f(X)
140 F=zeros(size(X));
141 for i=1:length(X)
142     x=X(i);
143     F(i)=sin(x);
144 end
145 end

```

8.4 Comparison Between Forward, Backward & Centred Difference Approximations

The main difference between the different differencing schemes is the order of accuracy. Indeed, the error of the forward and backward differencing methods are $\mathcal{O}(h)$ whereas the error for the centred differencing is $\mathcal{O}(h^2)$. This means that if the stepsize h was reduced by a factor of 10, then the error for the forward and backward finite difference approximations would also reduce by a factor of 10 while the centred would reduce by a factor of 100.

Comparison Between CD, FD and BD

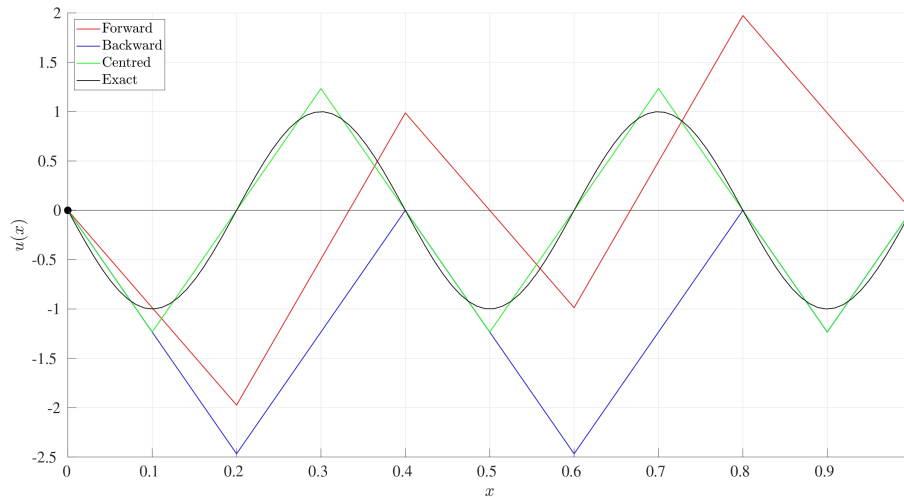
Consider the BVP

$$\frac{d^2 u}{dx^2} = 25\pi^2 \sin(5\pi x), \quad x \in [0, 1] \quad \text{with} \quad u(0) = 0 \quad \text{and} \quad u(1) = 0.$$

This has the exact solution

$$u(x) = -\sin(5\pi x).$$

Below are the plots for the numerical solution to this boundary value problem using the forward (red), backward (blue) and centred (green) difference approximations compared to the exact solution when $N = 10$.



It can be seen that even for this relatively crude interval subdivision of $N = 10$, the centred approximation has yielded a far more favourable result compared to the other two methods. The following table shows the 2-norm error between the exact solution and the approximation for different values of N :

Method	$N = 10$	$N = 20$	$N = 50$	$N = 100$
Forward	4.1444	3.0875	1.9823	1.4048
Backward	4.7243	3.8535	2.0939	1.4251
Centred	0.5226	0.1677	0.0413	0.0146

It can be seen that even when $N = 100$, the 2-norm error has still not reduced below 1 for the forward and backward difference approximations but the centred has already achieved that even at $N = 10$. This is just a demonstration to show that how a simple change in the way in which derivatives are approximated can have such a drastic effect on the final solution.

8.5 MATLAB's In-Built Procedures

MATLAB has an in-built mechanism that can also solve second (or even higher order) BVPs, this is done using the `bvp4c` command.

Below is the MATLAB code that solves the BVP

$$\frac{d^2u}{dx^2} + 2\frac{du}{dt} + e^{-x}u = \sin(x), \quad x \in [0, 10] \quad \text{with} \quad u(0) = 1 \quad \text{and} \quad u(10) = -1$$

using `bvp4c`.

```
1 function BVP_InBuilt
2
3 %% Solve BVPs using bvp4c
4
5 % The bvp is written in the form
6 % a(x) u'' + b(x) u' + c(x) u = f(x) on x in [x0,L]
7 % with the boundary conditions u(x0)=ul and u(L)=ur.
8
9 %% Lines to change:
10
11 % Line 24 : x0 - Start point
12 % Line 27 : L - End point
13 % Line 30 : N - Number of spatial points
14 % Line 33 : xl - Left boundary value
15 % Line 36 : xr - Right boundary value
16 % Line 44 : Expression for the function a(x)
17 % Line 45 : Expression for the function b(x)
18 % Line 46 : Expression for the function c(x)
19 % Line 47 : Expression for the function f(x)
20
21 %% Set up input values
22
23 % Start point
24 x0=0;
25
26 % End point
27 L=10;
28
29 % Number of spatial points
30 N=50;
31
32 % Boundary value at x=x0
33 ul=1;
34
35 % Boundary value at x=L
36 ur=-1;
37
38 %% Set up BVP solver parameters
39
40 % Set up solving space
```

```

41 X=linspace(x0,L,N);
42
43 % Define the functions in the BVP
44 a= @(x) 1;
45 b= @(x) 2;
46 c= @(x) exp(-x);
47 f= @(x) sin(x);
48
49 %% Set up BVP solving parameters
50
51 % First, write the second order ODE as a set of first order
52 % ODEs:
53 % U'=V
54 % V'=(-b(x)*V-c(x)*U+f(x))/a(x)
55
56 % Second order BVPs can have more than one solution
57 % and vector v is the initialising vector of solutions.
58 % It can be kept as a vector of zeros
59 v=[0 0];
60
61 % Initialise vectors for space and v
62 init=bvpinit(X,v);
63
64 % Solve the bvp subject to the boundary values and
65 % initial guesses
66 sol=bvp4c(@(x,u) DUDT(x,u,a,b,c,f),@(x0,L) BCs(x0,L,ul,ur),init);
67
68 % Evaluate the solution at the grid points
69 U=deval(sol,X);
70
71 % Convert U to columns for consistency
72 U=U';
73
74 %% Setting plot parameters
75
76 % Clear figure
77 clf
78
79 % Hold so more than one line can be drawn
80 hold on
81
82 % Turn on grid
83 grid on
84
85 % Setting font size and style

```

```

86 set(gca,'FontSize',20,'FontName','Times')
87
88 % Label the axes
89 xlabel('$t$', 'Interpreter','Latex')
90 ylabel('$u(t)$', 'Interpreter','Latex')
91
92 % Plot solution
93 plot(X,U(:,1),'-k','LineWidth',2)
94
95 end
96
97 function [dudx]=DUDT(x,u,a,b,c,f)
98
99 dudx(1)=u(2);
100
101 dudx(2)=(-b(x)*u(2)-c(x)*u(1)+f(x))/(a(x));
102
103 end
104
105 function res=BCs(x0,L,ul,ur)
106 % The boundary conditions are written as
107 % u(x0)=ul
108 % x(L)=ur
109
110 res=[x0(1)-ul;L(1)-ur];
111
112 end

```

9 Mixed Value Problems

Initial and boundary value problems are not the only two ways in which conditions can be expressed. Sometimes these conditions can be presented in a *mixed form* where the condition on one or both boundaries may depend on the derivative of the solution function. For instance, consider the steady-state convection-diffusion equation on a bar on length 5 with density ρ , convective velocity v , specific heat capacity C_p , thermal conductivity k_f and heat source f :

$$-k_f \frac{d^2 T}{dx^2} + \rho v C_p \frac{dT}{dx} = f(x) \quad \text{on } x \in [0, 5] \quad \text{with } T(0) = 100 \quad \text{and} \quad \frac{dT}{dx}(5) = 0$$

where $T(x)$ is the temperature at x . This set of conditions are known as **Mixed Conditions**: the first $T(0) = 100$ means that the temperature at the location $x = 0$ is 100, the second $\frac{dT}{dx}(5) = 0$ means that at the location $x = 5$, there is no heat *flux*. This can be quite useful if say, a metal rod is being heated to 100°C on one side and insulated on the other.

The method to solving MVPs is the same as boundary value problems subject to a few modifications.

9.1 Finite Difference Method for MVPs

Consider the differential equation

$$a(x) \frac{d^2 u}{dx^2} + b(x) \frac{du}{dx} + c(x)u = f(x) \quad \text{with } 0 < x < L$$

as before. The interval $[0, L]$ will be split into N equally sized sections each of width $h = \frac{L}{N}$ and the grid points are labelled $x_n = nh$ for $n = 0, 1, 2, \dots, N$. This differential equation can be discretised using the centred difference approximation just as before to give

$$\alpha_n U_{n-1} + \beta_n U_n + \gamma_n U_{n+1} = f(x_n) \quad \text{for } n = 1, 2, \dots, N-1$$

$$\text{where } \alpha_n = \frac{a(x_n)}{h^2} - \frac{b(x_n)}{2h}, \quad \beta_n = -\frac{2a(x_n)}{h^2} + c(x_n), \quad \gamma_n = \frac{a(x_n)}{h^2} + \frac{b(x_n)}{2h}.$$

This gives a set of $N-1$ equations in $N+1$ unknowns, namely $U_0, U_1, U_2, \dots, U_N$ (recall that $U_n \approx u(x_n)$ for $n = 0, 1, 2, \dots, N$).

When the differential equation is subjected to two boundary conditions, say

$$u(0) = u_l \quad \text{and} \quad u(L) = u_r,$$

then expressions for U_0 and U_L are provided which gives $N - 1$ equations in $N - 1$ unknowns, hence resulting in a well-defined system which can be solved as before.

However, suppose that a set of mixed conditions is given as

$$\frac{du}{dx}(0) = \tilde{u}_l \quad \text{and} \quad u(L) = u_r.$$

In this case, only $U_N \approx u(L) = u_r$ is explicitly known, meaning that there will be $N - 1$ equations in N unknowns since $U_0 \approx u(x_0)$ is not known giving an under-determined system (a system with more unknowns than equations). So either one more equation is needed or one more unknown needs to be removed. All the unknowns are certainly needed, otherwise the solution will be incomplete, so the alternative is to find another equation to add to the set of equations.

The set of $N - 1$ equations is:

$$\begin{aligned} n = 1 : \quad & \alpha_1 U_0 + \beta_1 U_1 + \gamma_1 U_2 = f(x_1) \\ n = 2 : \quad & \alpha_2 U_1 + \beta_2 U_2 + \gamma_2 U_3 = f(x_2) \\ & \vdots \\ n = N - 1 : \quad & \alpha_{N-1} U_{N-2} + \beta_{N-1} U_{N-1} = f(x_{N-1}) - \gamma_{N-1} u_L. \end{aligned}$$

All these come from the discretisation

$$\alpha_n U_{n-1} + \beta_n U_n + \gamma_n U_{n+1} = f(x_n).$$

Evaluating this at $n = 0$ gives

$$\alpha_0 U_{-1} + \beta_0 U_0 + \gamma_0 U_1 = f(x_0). \quad (9.1)$$

Initially, this may seem to be quite strange since there is a point U_{-1} which is the approximation to the solution u at the point $x = x_{-1} = -h$ which is certainly out of the range of consideration. This point is considered to be *an artificial grid point* that will act as a placeholder in meantime.

Consider the condition at the start point

$$\frac{du}{dx}(0) = \tilde{u}_l.$$

Using the centred finite difference approximation on the derivative gives

$$\tilde{u}_l = \frac{du}{dx}(0) = \frac{du}{dx}(x_0) \approx \frac{u(x_1) - u(x_{-1})}{2h} \approx \frac{U_1 - U_{-1}}{2h} \implies \frac{U_1 - U_{-1}}{2h} \approx \tilde{u}_l$$

This approximation can be manipulated to provide an expression for the artificial point U_{-1} as

$$U_{-1} = U_1 - 2h\tilde{u}_l.$$

Replacing this into the equation Equation 9.1 will eliminate U_{-1} completely giving an equation in terms of U_0 and U_1 only, namely

$$\beta_0 U_0 + (\gamma_0 + \alpha_0) U_1 = f(x_0) + 2h\tilde{u}_l \alpha_0.$$

Therefore, another equation has been found which now completes the system of N equations in N unknowns. Thus the system of equations is:

$$\begin{aligned}
n = 0 : \quad & \beta_0 U_0 + (\gamma_0 + \alpha_0) U_1 = f(x_0) + 2h\tilde{u}_l \alpha_0 \\
n = 1 : \quad & \alpha_1 U_0 + \beta_1 U_1 + \gamma_1 U_2 = f(x_1) \\
n = 2 : \quad & \alpha_2 U_1 + \beta_2 U_2 + \gamma_2 U_3 = f(x_2) \\
& \vdots \\
n = N - 1 : \quad & \alpha_{N-1} U_{N-2} + \beta_{N-1} U_{N-1} = f(x_{N-1}) - \gamma_{N-1} u_L.
\end{aligned}$$

This can be written in matrix form as $A\mathbf{U} = \mathbf{g}$ where

$$\underbrace{\begin{pmatrix} \beta_0 & \gamma_0 + \alpha_0 & 0 & \dots & 0 & 0 & 0 \\ \alpha_1 & \beta_1 & \gamma_1 & \dots & 0 & 0 & 0 \\ 0 & \alpha_2 & \beta_2 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & \beta_{N-3} & \gamma_{N-3} & 0 \\ 0 & 0 & 0 & \dots & \alpha_{N-2} & \beta_{N-2} & \gamma_{N-2} \\ 0 & 0 & 0 & \dots & 0 & \alpha_{N-1} & \beta_{N-1} \end{pmatrix}}_A \underbrace{\begin{pmatrix} U_0 \\ U_1 \\ U_2 \\ \vdots \\ U_{N-3} \\ U_{N-2} \\ U_{N-1} \end{pmatrix}}_U = \underbrace{\begin{pmatrix} f(x_0) + 2h\alpha_0\tilde{u}_l \\ f(x_1) \\ f(x_2) \\ \vdots \\ f(x_{N-3}) \\ f(x_{N-2}) \\ f(x_{N-1}) - \gamma_{N-1}u_r \end{pmatrix}}_g.$$

This can once again be solved on MATLAB using $\mathbf{U}=\text{inv}(\mathbf{A})*\mathbf{g}$ or $\mathbf{U}=\mathbf{A} \backslash \mathbf{g}$.

If, on the other hand, the mixed conditions were instead

$$u(0) = u_l \quad \text{and} \quad \frac{du}{dx}(L) = \tilde{u}_r,$$

then the artificial point will be located at $x = x_{N+1}$ but the same procedure can be done

give the matrix system $AU = g$ where

$$\underbrace{\begin{pmatrix} \beta_1 & \gamma_1 & 0 & \dots & 0 & 0 & 0 \\ \alpha_2 & \beta_2 & \gamma_2 & \dots & 0 & 0 & 0 \\ 0 & \alpha_3 & \beta_3 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & \beta_{N-2} & \gamma_{N-2} & 0 \\ 0 & 0 & 0 & \dots & \alpha_{N-1} & \beta_{N-1} & \gamma_{N-1} \\ 0 & 0 & 0 & \dots & 0 & \alpha_N + \gamma_N & \beta_N \end{pmatrix}}_A \underbrace{\begin{pmatrix} U_1 \\ U_2 \\ U_3 \\ \vdots \\ U_{N-2} \\ U_{N-1} \\ U_N \end{pmatrix}}_U = \underbrace{\begin{pmatrix} f(x_1) - \alpha_1 u_l \\ f(x_2) \\ f(x_3) \\ \vdots \\ f(x_{N-2}) \\ f(x_{N-1}) \\ f(x_N) - 2h\gamma_N \tilde{u}_r \end{pmatrix}}_g.$$

Mixed Value Problem

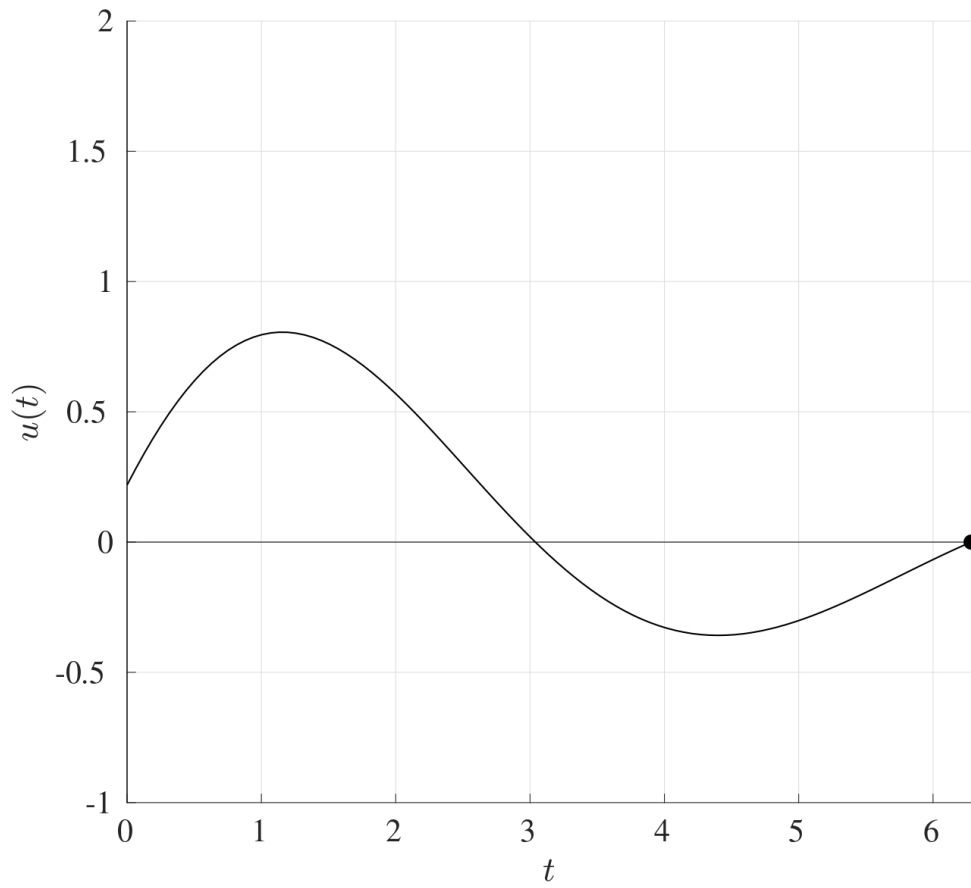
Consider the differential equation for a damped harmonic oscillator

$$\frac{d^2 u}{dt^2} + 0.5 \frac{du}{dt} + u = 0 \quad \text{for } 0 < t < 2\pi$$

with the mixed conditions

$$\frac{du}{dt}(0) = 1 \quad \text{and} \quad u(2\pi) = 0.$$

This MVP is to determine the trajectory of the mass if the launching speed at the start is 1, which is $\frac{du}{dt}(0) = 1$, and after 2π seconds, the mass reaches its equilibrium state, which is $u(2\pi) = 0$. Notice that there is no restriction on the starting location, only the starting speed, so the mass can start anywhere as long as it is launched with a velocity 1.



The starting location here happens to be at 0.2188 but that is not restricted by the mixed conditions as long as the gradient at the start is 1.

10 Symmetric Boundary Conditions

The use of symmetric boundary conditions arises in many cases where conditions at the ends are not known explicitly but they are related. For instance, consider the ODE representing the conduction problem

$$-k_f \frac{d^2 T}{dx^2} = q_{gen}(x) \quad \text{in} \quad -L < x < L$$

where k_f is the material's conductivity and q_{gen} is the heat transfer. Symmetric boundary conditions can be imposed as

$$T(-L) = T(L) \quad \text{and} \quad k_f \frac{dT}{dx}(L) = \alpha(T(L) - T_{air})$$

for some constant α . This problem can be interpreted as an insulated metal rod of length $2L$ that has been heated all the way through and then as it cools, it loses heat equally from both ends (which is the condition $T(-L) = T(L)$), and that this heat loss at L is proportional to the temperature gradient between the rod and the air (which is the second condition $k_f \frac{dT}{dx}(L) = \alpha(T(L) - T_{air})$). The issue with this type of problems is that the temperature at both boundaries are not explicitly known, but it is known that they are the same.

10.1 Finite Difference Method for Symmetric Boundary Value Problems

This problem can be tackled in a very similar way to BVPs and MVPs. Consider the differential equation

$$a(x) \frac{d^2 u}{dx^2} + b(x) \frac{du}{dx} + c(x)u = f(x) \quad \text{with} \quad -L < x < L.$$

The interval $[-L, L]$ will be split into N equally sized sections each of width $h = \frac{2L}{N}$ and the grid points are labelled $x_n = -L + nh$ for $n = 0, 1, 2, \dots, N$. This differential equation can be discretised using the centred difference approximation (just as in Section 8.2) to give

$$\alpha_n U_{n-1} + \beta_n U_n + \gamma_n U_{n+1} = f(x_n) \quad \text{for} \quad n = 1, 2, \dots, N-1$$

$$\text{where} \quad \alpha_n = \frac{a(x_n)}{h^2} - \frac{b(x_n)}{2h}, \quad \beta_n = -\frac{2a(x_n)}{h^2} + c(x_n), \quad \gamma_n = \frac{a(x_n)}{h^2} + \frac{b(x_n)}{2h}.$$

This gives a set of $N-1$ equations in $N+1$ unknowns, namely $U_0, U_1, U_2, \dots, U_N$. In this case, neither U_0 nor U_N are explicitly known, therefore none of the unknowns can be eliminated from the boundary conditions *per se*.

Suppose the given conditions are

$$u(-L) = u(L) \quad \text{and} \quad \frac{du}{dx}(L) = pu(L) + q$$

where p and q are some constants. The first condition is the symmetric boundary condition which represents the fact that the value of the unknown solution u at both ends is the same, then $U_0 = U_N$, even though neither is explicitly known. The term U_0 can be eliminated since determining U_N automatically determines U_0 , this reduces the number of unknowns to N .

Consider the discretisation at $n = 1$, namely

$$\alpha_1 U_0 + \beta_1 U_1 + \gamma_1 U_2 = f(x_1),$$

since $U_0 = U_N$, this can be rewritten in terms of U_N instead as

$$\beta_1 U_1 + \gamma_1 U_2 + \alpha_1 U_N = f(x_1).$$

The discretised form of the differential equation at $n = N$ is

$$\alpha_N U_{N-1} + \beta_N U_N + \gamma_N U_{N+1} = f(x_N). \quad (10.1)$$

Just as in the case with the MVPs, an artificial point U_{N+1} is introduced which is the solution approximated at the point $x = x_{N+1} = L + h$ which is beyond the computational domain.

To find an expression for U_{N+1} , first consider the second condition

$$\frac{du}{dx}(x_N) = \frac{du}{dx}(L)pu(L) + q \approx pU_N + q.$$

The LHS can be rewritten in terms of its centred differencing approximation as

$$\frac{du}{dx}(x_N) \approx \frac{u(x_{N+1}) - u(x_{N-1}))}{2h} \approx \frac{U_{N+1} - U_{N-1}}{2h}.$$

Combining these two can give an expression for U_{N+1} as:

$$\frac{U_{N+1} - U_{N-1}}{2h} \approx pU_N + q \quad \implies \quad U_{N+1} = U_{N-1} + 2hpU_N + 2hq.$$

Replacing this into Equation 10.1 gives

$$(\alpha_N + \gamma_N)U_{N-1} + (\beta_N + 2hp\gamma_N)U_N = f(x_N) - 2hq\gamma_N,$$

thus providing the last equation to complete the set. Finally, this system can be written in matrix form as $AU = g$ where

$$\underbrace{\begin{pmatrix} \beta_1 & \gamma_1 & 0 & \dots & 0 & 0 & \alpha_1 \\ \alpha_2 & \beta_2 & \gamma_2 & \dots & 0 & 0 & 0 \\ 0 & \alpha_3 & \beta_3 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & \beta_{N-2} & \gamma_{N-2} & 0 \\ 0 & 0 & 0 & \dots & \alpha_{N-1} & \beta_{N-1} & \gamma_{N-1} \\ 0 & 0 & 0 & \dots & 0 & \alpha_N + \gamma_N & \beta_N + 2hp\gamma_N \end{pmatrix}}_A \underbrace{\begin{pmatrix} U_1 \\ U_2 \\ U_3 \\ \vdots \\ U_{N-2} \\ U_{N-1} \\ U_N \end{pmatrix}}_U = \underbrace{\begin{pmatrix} f(x_1) \\ f(x_2) \\ f(x_3) \\ \vdots \\ f(x_{N-2}) \\ f(x_{N-1}) \\ f(x_N) - 2hq\gamma_N \end{pmatrix}}_g.$$

This can then be solved in MATLAB but bearing in mind that $U_0 = U_N$ which determines the function U at $-L$ and L .

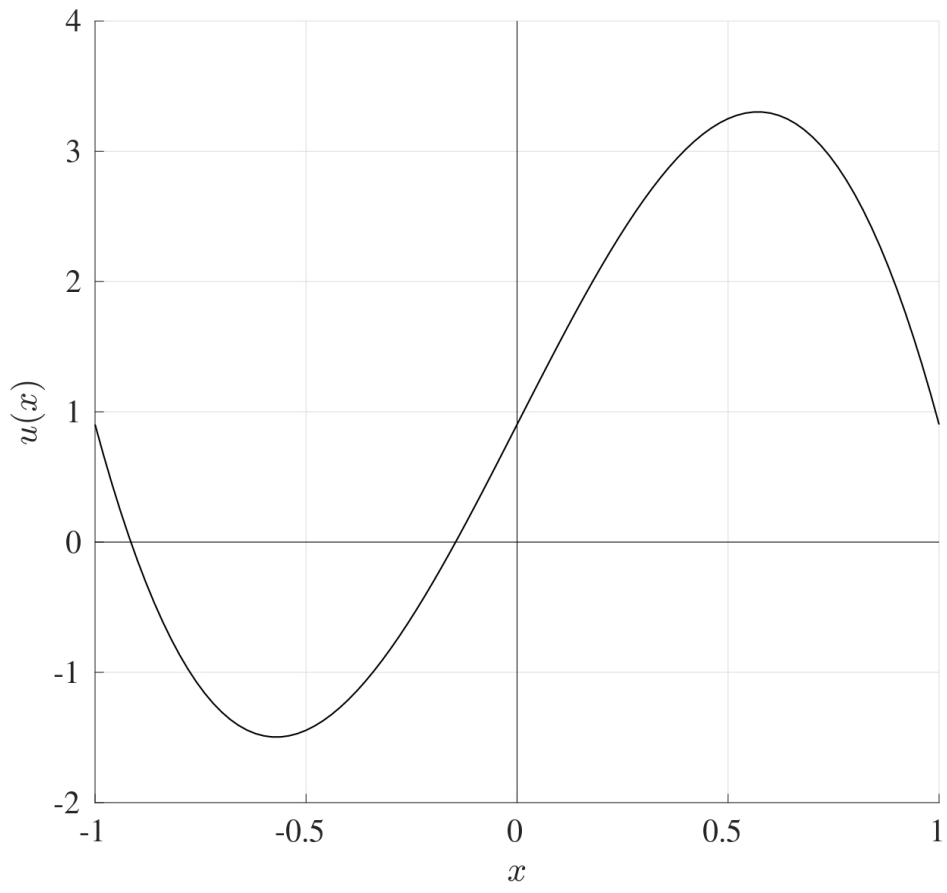
Symmetric Boundary Value Problem

Consider the conduction problem

$$-\frac{d^2T}{dx^2} = 40 \sin(x) \quad \text{in} \quad -1 < x < 1$$

with the conditions

$$T(-1) = T(1) \quad \text{and} \quad \frac{dT}{dx}(1) = \frac{1}{2}(T(1) - 25).$$



Part IV

Solving Partial Differential Equations

Ordinary differential equations have been the main focus of this course so far but this will now be extended to *partial differential equations*. The differential equations that will be studied here are the *1-Dimensional Heat (or Diffusion) Equation*, *1-Dimensional Advection (or Convection) Equation*, *1-Dimensional Wave Equation*.

11 Heat Equation

The 1-dimensional *heat* (or *diffusion*) *equation* is a partial differential equation that represents the heat transfer across a rod and is given by

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2} \quad \text{with} \quad 0 < x < L \quad \text{and} \quad t > 0$$

where $u = u(x, t)$ is the temperature at location x at time t and α is the thermal diffusivity¹. This equation represents the flow of heat along the length of a rod of length L .

This partial differential equation has three derivatives in total, two derivatives in x and one derivative in t , this means that three conditions are needed, two on x and one on t :

- $u(x, 0) = u_{init}(x)$ for $x \in [0, L]$: Initial heat distribution across the rod;
- $u(0, t) = u_l(t)$ for $t > 0$: The temperature at the left end of the rod;
- $u(L, t) = u_r(t)$ for $t > 0$: The temperature at the right end of the rod.

This set of conditions along with the differential equation are known collectively as an **Initial-Boundary Value Problem** and can be solved using the **Method of Lines**.

11.1 The Method of Lines for the Heat Equation

The outline of the method of lines for the heat equation is as follows:

1. Divide the spatial interval $[0, L]$ into N_x equally sized sections and label the points as $x_0, x_1, x_2, \dots, x_{N_x}$ where $x_n = nh_x$ and the spatial interval width is $h_x = \frac{L}{N_x}$.



2. **Left Hand Side:** For each point x_n , define the approximation $U_n(t) \approx u(x_n, t)$. Therefore the left hand side of the heat equation can be written as

$$\frac{\partial u}{\partial t}(x_n, t) \approx \frac{dU_n}{dt}(t)$$

¹The thermal diffusivity will always be regarded as a constant and usually takes the form $\alpha = \frac{k}{\rho C_p}$ where k is the thermal conductivity, ρ is the density of the material and C_p is the specific heat capacity.

and this holds for $n = 1, 2, \dots, N_x - 1$ since $U_0(t) \approx u(0, t) = u_l(t)$ and $U_{N_x}(t) \approx u(L, t) = u_r(t)$ are already known from the boundary conditions. Notice that the derivative of U_n is an ordinary derivative since U_n is a function of t only.

3. **Right Hand Side:** Use the finite difference approximation to approximate the *spatial* derivative in the differential equation. Here, the centred difference approximation for the second derivative will be used, namely

$$\frac{\partial^2 u}{\partial x^2}(x_n, t) \approx \frac{U_{n+1}(t) - 2U_n(t) + U_{n-1}(t)}{h_x^2}.$$

Therefore the right hand side of the heat equation will become

$$\alpha \frac{\partial^2 u}{\partial x^2}(x_n, t) \approx \frac{\alpha}{h_x^2} [U_{n-1}(t) - 2U_n(t) + U_{n+1}(t)].$$

This holds for $n = 1, 2, \dots, N_x - 1$ bearing in mind, once again, that $U_0(t) \approx u(0, t) = u_l(t)$ and $U_{N_x}(t) \approx u(L, t) = u_r(t)$ are known beforehand.

4. These can be combined to give the discretised form of the heat equation

$$\frac{dU_n}{dt} = \frac{\alpha}{h_x^2} [U_{n-1} - 2U_n + U_{n+1}]$$

for all $n = 1, 2, \dots, N_x - 1$ where $U_n = U_n(t)$. This means that the partial differential equation has been split into $N_x - 1$ ordinary differential equations.

5. This entire system of $N_x - 1$ equations can now be written in matrix form as $\frac{d\mathbf{U}}{dt} = \mathbf{A}\mathbf{U} + \mathbf{b}$ where

$$\underbrace{\frac{d}{dt} \begin{pmatrix} U_1(t) \\ U_2(t) \\ U_3(t) \\ \vdots \\ U_{N_x-3}(t) \\ U_{N_x-2}(t) \\ U_{N_x-1}(t) \end{pmatrix}}_{\mathbf{U}} = \frac{\alpha}{h_x^2} \underbrace{\begin{pmatrix} -2 & 1 & 0 & \dots & 0 & 0 & 0 \\ 1 & -2 & 1 & \dots & 0 & 0 & 0 \\ 0 & 1 & -2 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & -2 & 1 & 0 \\ 0 & 0 & 0 & \dots & 1 & -2 & 1 \\ 0 & 0 & 0 & \dots & 0 & 1 & -2 \end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix} U_1(t) \\ U_2(t) \\ U_3(t) \\ \vdots \\ U_{N_x-3}(t) \\ U_{N_x-2}(t) \\ U_{N_x-1}(t) \end{pmatrix}}_{\mathbf{U}} + \underbrace{\frac{\alpha}{h_x^2} \begin{pmatrix} u_l(t) \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \\ u_r(t) \end{pmatrix}}_{\mathbf{b}}$$

subject to the *initial condition*

$$\mathbf{U}_0 = \begin{pmatrix} U_1(0) \\ U_2(0) \\ U_3(0) \\ \vdots \\ U_{N_x-3}(0) \\ U_{N_x-2}(0) \\ U_{N_x-1}(0) \end{pmatrix} \approx \begin{pmatrix} u(x_1, 0) \\ u(x_2, 0) \\ u(x_3, 0) \\ \vdots \\ u(x_{N_x-3}, 0) \\ u(x_{N_x-2}, 0) \\ u(x_{N_x-1}, 0) \end{pmatrix} = \begin{pmatrix} u_{init}(x_1) \\ u_{init}(x_2) \\ u_{init}(x_3) \\ \vdots \\ u_{init}(x_{N_x-3}) \\ u_{init}(x_{N_x-2}) \\ u_{init}(x_{N_x-1}) \end{pmatrix}.$$

This system can now be solved using any of the IVP solvers with a temporal stepsize h_t .

In essence, the *Method of Lines* has converted a PDE into a set of ODEs using the same techniques as BVPs and will be solved in the same way as IVPs.

Heat Equation

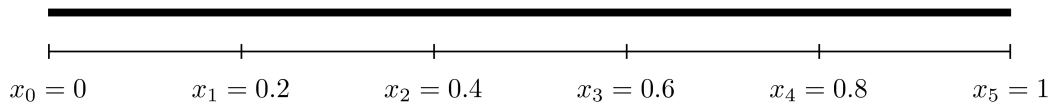
Consider an iron rod (of thermal diffusivity $\alpha = 2.3 \times 10^{-5}$) of length 1 where the middle section of length 0.2 has been heated to a temperature of 1 while the rest is at 0. The ends of the rod have been kept at a constant temperature of 2. This system can be represented by the IBVP

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}, \quad x \in [0, 1], \quad t > 0$$

$$u(x, 0) = u_{init}(x) = \begin{cases} 0 & 0 \leq x < 0.4 \\ 1 & 0.4 \leq x < 0.6 \\ 0 & 0.6 \leq x \leq 1 \end{cases},$$

$$u(0, t) = u_l(t) = 2, \quad u(1, t) = u_r(t) = 2.$$

First, divide the interval $[0, 1]$ into five equal sections (which will be of width $h_x = \frac{1-0}{5} = 0.2$).



This system can be discretised using the centred difference method and written in matrix form as $\frac{d\mathbf{U}}{dt} = \mathbf{A}\mathbf{U} + \mathbf{b}$ where

$$\underbrace{\frac{d}{dt} \begin{pmatrix} U_1(t) \\ U_2(t) \\ U_3(t) \\ U_4(t) \\ U_5(t) \end{pmatrix}}_{\mathbf{U}} = \frac{\alpha}{h^2} \underbrace{\begin{pmatrix} -2 & 1 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 1 & -2 \end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix} U_1(t) \\ U_2(t) \\ U_3(t) \\ U_4(t) \\ U_5(t) \end{pmatrix}}_{\mathbf{U}} + \frac{\alpha}{h^2} \underbrace{\begin{pmatrix} u_l(t) \\ 0 \\ 0 \\ 0 \\ u_r(t) \end{pmatrix}}_{\mathbf{b}}$$

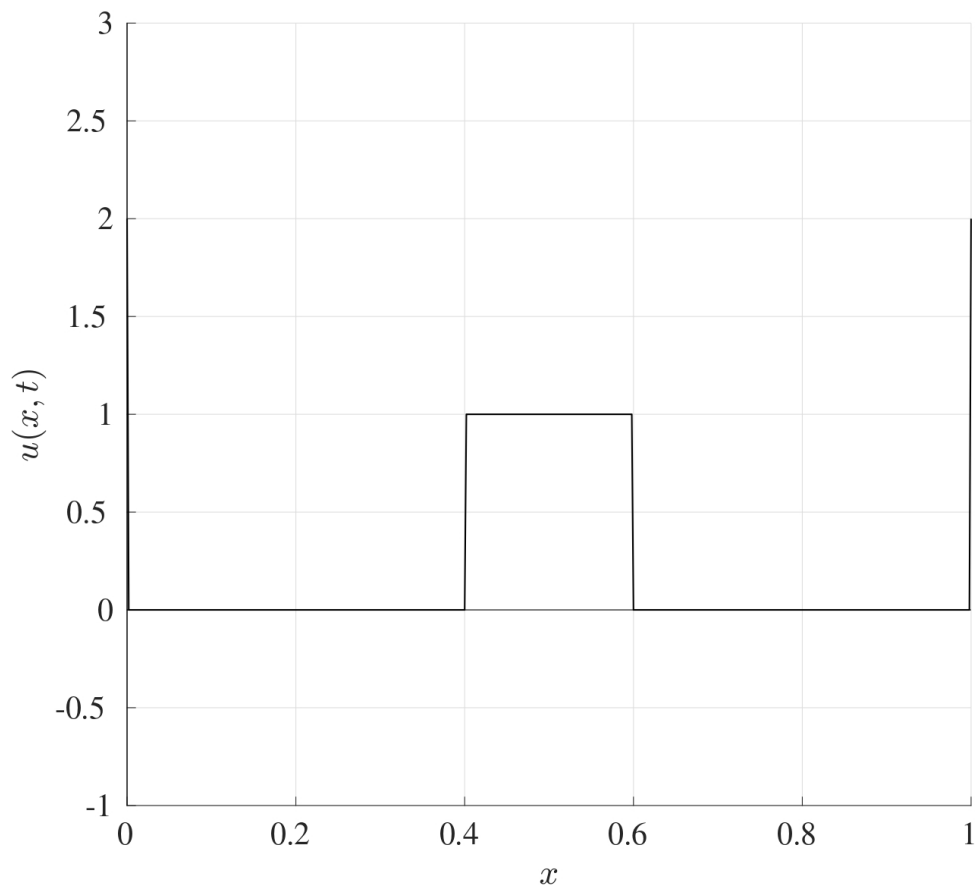
The differential equation

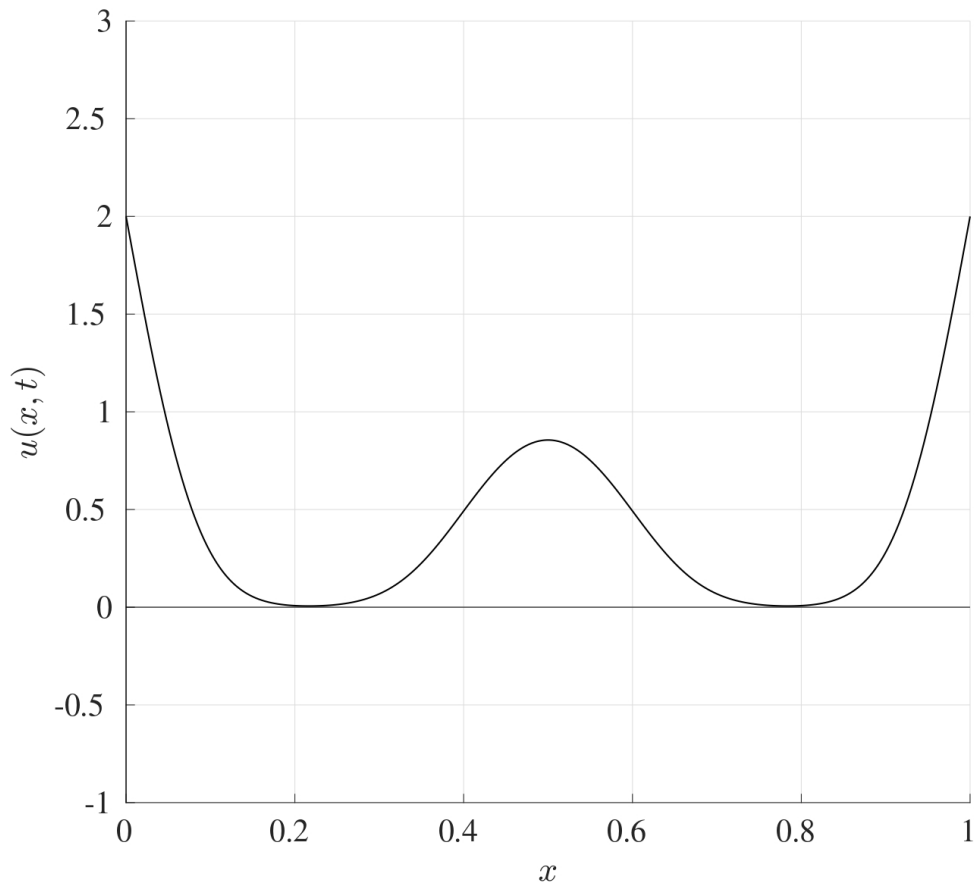
$$\frac{d\mathbf{U}}{dt} = A\mathbf{U} + \mathbf{b}$$

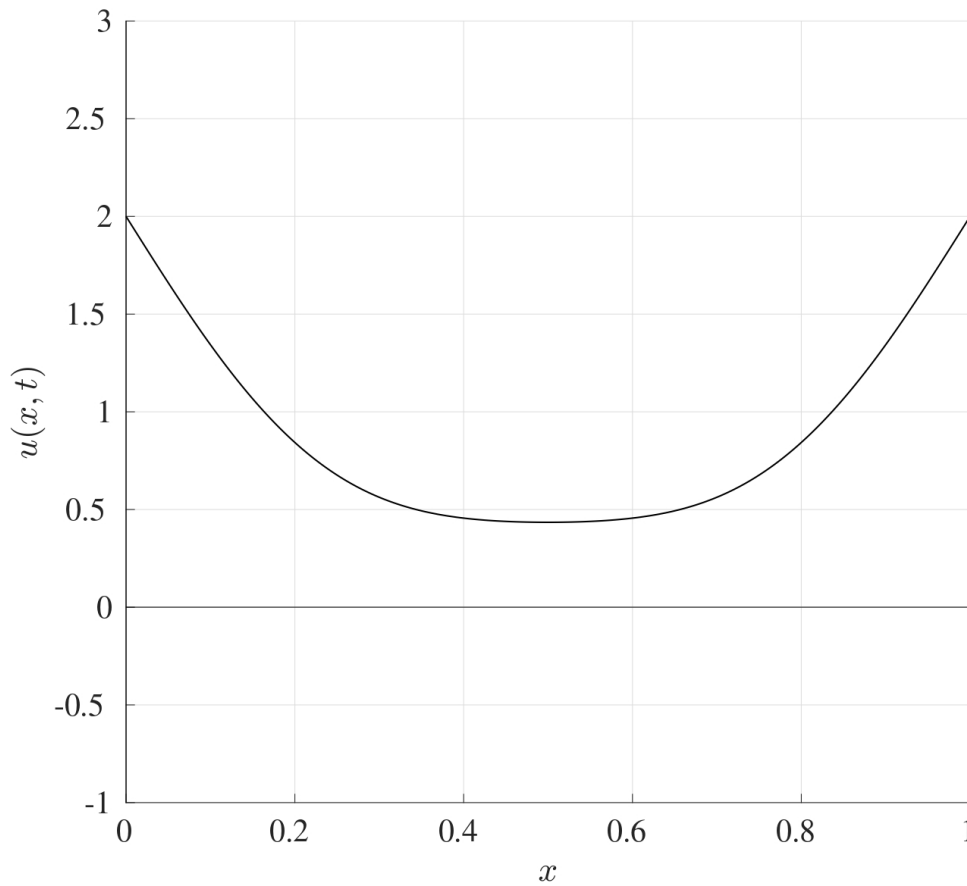
can be solved using the Euler method with the initial condition

$$\mathbf{U}(0) = \begin{pmatrix} u_{init}(x_1) \\ u_{init}(x_2) \\ u_{init}(x_3) \\ u_{init}(x_4) \\ u_{init}(x_5) \end{pmatrix}$$

subject to a time stepsize h_t . Below are the plots of the heat distribution at $t = 0, 100, 1000$ for $N_x = 500$ ($h_x = 0.002$) and $h_t = 0.02$ ($N_t = 50000$).







At the beginning, the temperature at the ends is 2 and the middle section is at a temperature of 1. As time progresses, the heat evens out across the iron bar until eventually, the whole bar will be the same temperature.

11.2 Linear Advection Equation

The heat equation deals with heat transfer through diffusion throughout a material. Another way in which heat transfer can be achieved by advection (or convection) and this is given by

$$\frac{\partial u}{\partial t} = -v \frac{\partial u}{\partial x} \quad \text{with } 0 < x < L \quad \text{and } t > 0$$

where $u = u(x, t)$ is the temperature at location x at time t and v is the flow speed.

This partial differential equation has two derivatives in total, one in x and one in t , this means that two conditions are needed, one spatial and one temporal:

- $u(x, 0) = u_{init}(x)$ for $x \in [0, L]$: Initial heat distribution across the rod;

- $u(0, t) = u_l(t)$ for $t > 0$: The temperature at the left end of the rod.

Consider the PDE along with the initial condition only, namely $u(x, 0) = u_{init}(x)$ for $x \in [0, L]$. The exact solution to this differential equation is given by

$$u(x, t) = u_{init}(x - vt),$$

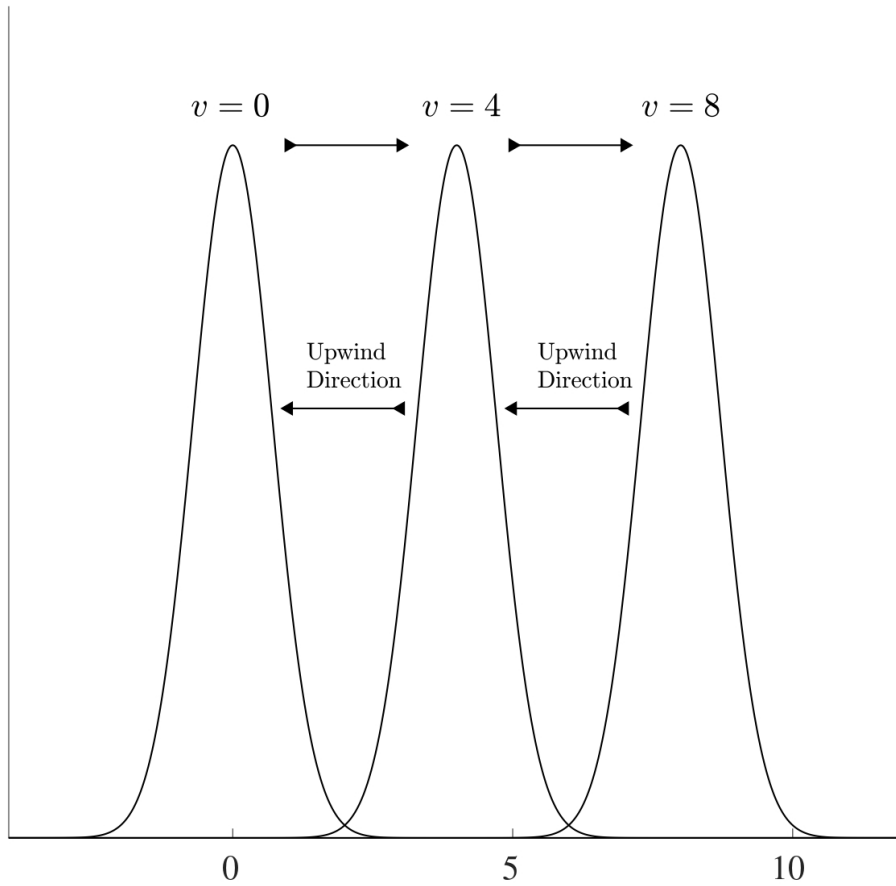
this can be verified from the partial differential equation as follows:

$$\frac{\partial u}{\partial t} = -v \frac{\partial u}{\partial x} \quad \text{at} \quad u(x, t) = u_{init}(x - vt)$$

$$\text{LHS} = \frac{\partial}{\partial t} u(x, t) = \frac{\partial}{\partial t} (u_{init}(x - vt)) = -v u'_{init}(x - vt)$$

$$\text{RHS} = \frac{\partial}{\partial x} u(x, t) = \frac{\partial}{\partial x} (u_{init}(x - vt)) = -v u'_{init}(x - vt).$$

This means that if the initial heat profile takes the form of $u_{init}(x)$, then after time t , the profile will look exactly the same but shifted to the right by a distance vt .



The “information” moves from left to right so if the finite differences are to be used, the centred differencing approach would not be suitable since the information on the right is not known yet. Therefore the backwards differencing approximation will be the most suitable. This is known as an *upwind/upstream scheme* (i.e. against the direction of the wind/stream) if $v > 0$. Therefore using the convention $U_n(t) \approx u(x_n, t)$ where $x = x_n$ is the discretisation of the spatial points for $n = 0, 1, 2, \dots, N_x$, the backward differencing approximation to the spatial derivative is

$$\frac{\partial u}{\partial x}(x_n, t) \approx \frac{\partial U_n}{\partial x} = \frac{U_n - U_{n-1}}{h_x}.$$

Therefore is discretised advection equation is

$$\frac{dU_n}{dt} = \frac{v}{h_x} (U_{n-1} - U_n) \quad \text{for } n = 1, 2, \dots, N_x$$

and this can be solved subject to the initial condition

$$u(x, 0) = u_{init}(x)$$

and boundary condition

$$u(0, t) = u_l(t)$$

to give the discretised set of equations in the form $\frac{d\mathbf{U}}{dt} = \mathbf{A}\mathbf{U} + \mathbf{b}$ where

$$\underbrace{\frac{d}{dt} \begin{pmatrix} U_1(t) \\ U_2(t) \\ U_3(t) \\ \vdots \\ U_{N_x-2}(t) \\ U_{N_x-1}(t) \\ U_{N_x}(t) \end{pmatrix}}_{\mathbf{U}} = \frac{v}{h_x} \underbrace{\begin{pmatrix} -1 & 0 & 0 & \dots & 0 & 0 & 0 \\ 1 & -1 & 0 & \dots & 0 & 0 & 0 \\ 0 & 1 & -1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & -1 & 0 & 0 \\ 0 & 0 & 0 & \dots & 1 & -1 & 0 \\ 0 & 0 & 0 & \dots & 0 & 1 & -1 \end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix} U_1(t) \\ U_2(t) \\ U_3(t) \\ \vdots \\ U_{N_x-2}(t) \\ U_{N_x-1}(t) \\ U_{N_x}(t) \end{pmatrix}}_{\mathbf{U}} + \underbrace{\frac{v}{h_x} \begin{pmatrix} u_l(t) \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \\ 0 \end{pmatrix}}_{\mathbf{b}}$$

and the initial condition is

$$\mathbf{U}_0 = \begin{pmatrix} U_1(0) \\ U_2(0) \\ U_3(0) \\ \vdots \\ U_{N_x-2}(0) \\ U_{N_x-1}(0) \\ U_{N_x}(0) \end{pmatrix} \approx \begin{pmatrix} u(x_1, 0) \\ u(x_2, 0) \\ u(x_3, 0) \\ \vdots \\ u(x_{N_x-2}, 0) \\ u(x_{N_x-1}, 0) \\ u(x_{N_x}, 0) \end{pmatrix} = \begin{pmatrix} u_{init}(x_1) \\ u_{init}(x_2) \\ u_{init}(x_3) \\ \vdots \\ u_{init}(x_{N_x-2}) \\ u_{init}(x_{N_x-1}) \\ u_{init}(x_{N_x}) \end{pmatrix}$$

11.3 Convection-Diffusion Equation

The heat (or diffusion) equation dictates the spread of heat across a length of material while on the other hand, the advection (or convection) equation dictates the flow of heat in a certain direction. The combination of these two effects gives rise to the **Convection-Diffusion Equation** which takes the form

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2} - v \frac{\partial u}{\partial x} \quad \text{with} \quad 0 < x < L, \quad t > 0.$$

Just as in the heat equation, this partial differential equation has three derivatives in total, two derivatives in x and one derivative in t , this means that three conditions are needed, two on x and one on t , these will be as follows:

- $u(x, 0) = u_{init}(x)$ for $x \in [0, L]$: Initial heat distribution across the rod;
- $u(0, t) = u_l(t)$ for $t > 0$: The temperature at the left end of the rod;
- $u(L, t) = u_r(t)$ for $t > 0$: The temperature at the right end of the rod.

In order to discretise this system, a finite difference approximation needs to be chosen first. The centred difference approximation was used for the heat equation and the backwards difference approximation for the advection. Here, the combination of both will be used. Even though this might initially seem like an inconsistency, but in fact, this will allow the system to present a distinct stable advantage as will be seen in the next section.

This system can be discretised in exactly the same way as before, so for $n = 1, 2, \dots, N_x - 1$,

$$\frac{dU_n}{dt}(t) = \frac{\alpha}{h_x^2} [U_{n-1}(t) - 2U_n(t) + U_{n+1}(t)] - \frac{v}{h_x} [U_n(t) + U_{n-1}(t)].$$

This system can be written in the form $\frac{dU}{dt} = AU + \mathbf{b}$ where

$$A = \frac{\alpha}{h_x^2} \begin{pmatrix} -2 & 1 & 0 & \dots & 0 & 0 & 0 \\ 1 & -2 & 1 & \dots & 0 & 0 & 0 \\ 0 & 1 & -2 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & -2 & 1 & 0 \\ 0 & 0 & 0 & \dots & 1 & -2 & 1 \\ 0 & 0 & 0 & \dots & 0 & 1 & -2 \end{pmatrix} + \frac{v}{h_x} \begin{pmatrix} -1 & 0 & 0 & \dots & 0 & 0 & 0 \\ 1 & -1 & 0 & \dots & 0 & 0 & 0 \\ 0 & 1 & -1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & -1 & 0 & 0 \\ 0 & 0 & 0 & \dots & 1 & -1 & 0 \\ 0 & 0 & 0 & \dots & 0 & 1 & -1 \end{pmatrix}, \quad (11.1)$$

$$U = \begin{pmatrix} U_1(t) \\ U_2(t) \\ \vdots \\ U_{N-2}(t) \\ U_{N-1}(t) \end{pmatrix}, \quad b = \frac{\alpha}{h_x^2} \begin{pmatrix} u_l(t) \\ 0 \\ \vdots \\ 0 \\ u_r(t) \end{pmatrix} + \frac{v}{h_x} \begin{pmatrix} u_l(t) \\ 0 \\ \vdots \\ 0 \\ 0 \end{pmatrix}.$$

and this system can be solved using an Euler iteration subject to the initial condition $u(x, 0) = u_{init}(x)$.

11.4 Asymptotic Stability

The method of lines is essentially a hybrid method that makes use of a combination between a finite difference approximation and the Euler method and is very effective at solving partial differential equations, as seen from solving the heat, advection and convection-diffusion equations. The derivation of the method of lines for the different methods builds on the very same principle and the codes can be adapted quite easily. One main issue that arises here is the choice for the stepsizes for both the spatial and temporal discretisations, i.e. the choice of h_t and h_x respectively. When both methods are combined, there needs to be a restriction on both stepsizes.

The first issue that needs to be addressed is the asymptotic stability of the heat equation and the advection equation. For arbitrarily large matrices, it may not be simple to determine if all the eigenvalues are negative since it may be computationally restrictive to do so. However, a result can be used to see if all the eigenvalues are negative without explicitly calculating them.

Theorem 11.1 (Gershgorin Circle Theorem). *Let A be an $N \times N$ given by*

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1N} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2N} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{N1} & a_{N2} & a_{N3} & \dots & a_{NN} \end{pmatrix}.$$

On the complex plane, consider N closed discs, each centred at the locations a_{ii} for $i = 1, 2, \dots, n$ (the diagonal terms) where the disc centred at a_{ii} has a radius R_i where

$$R_i = \sum_{j \neq i} |a_{ij}|.$$

Then all the eigenvalues of the matrix A will have to lie in at least one of these discs. In other words, every eigenvalues of A satisfies

$$|\lambda - a_{ii}| \leq R_i \quad \text{for at least one } i = 1, 2, \dots, n.$$

Gershgorin Circle Theorem Exapmple

Consider the matrix

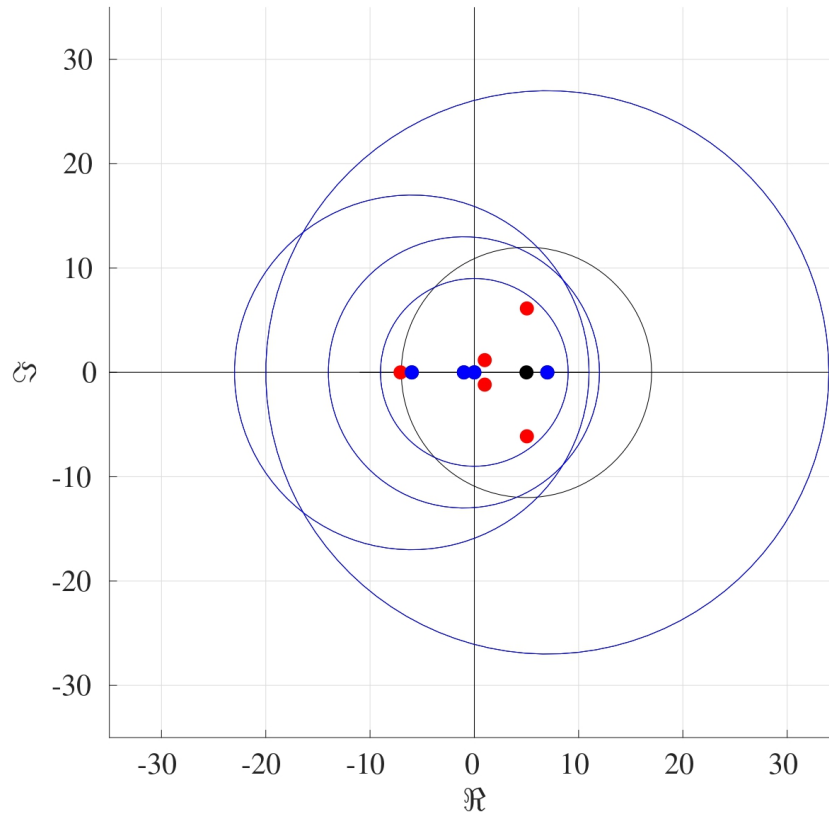
$$A = \begin{pmatrix} -1 & 3 & 4 & 2 & -4 \\ 0 & 5 & 4 & 7 & 1 \\ 4 & -2 & 0 & -3 & 0 \\ 6 & -6 & -4 & -6 & -1 \\ 7 & 4 & 7 & 9 & 7 \end{pmatrix}.$$

Following the steps of the theorem:

1. Indicate the locations of the diagonal terms (namely $-1, 5, 0, -6, 7$) on the complex plane.
2. Find the radii R_i which are equal to the row sum of the absolute terms *without* the diagonal terms, in other words,

$$\text{abs}(A) = \begin{pmatrix} 1 & 3 & 4 & 2 & 4 \\ 0 & 5 & 4 & 7 & 1 \\ 4 & 2 & 0 & 3 & 0 \\ 6 & 6 & 4 & 6 & 1 \\ 7 & 4 & 7 & 9 & 7 \end{pmatrix} \begin{array}{l} \rightarrow 3 + 4 + 2 + 4 = 13 \rightarrow R_1 \\ \rightarrow 0 + 4 + 7 + 1 = 12 \rightarrow R_2 \\ \rightarrow 4 + 2 + 3 + 0 = 9 \rightarrow R_3 \\ \rightarrow 6 + 6 + 4 + 1 = 17 \rightarrow R_4 \\ \rightarrow 7 + 4 + 7 + 9 = 27 \rightarrow R_5 \end{array}$$

3. Draw a circle around $a_{11} = -1$ with radius $R_1 = 13$, a circle around $a_{22} = 5$ with radius $R_2 = 12$ and so on.
4. All the eigenvalues of the matrix A must lie in at least one of the circles indicated. Indeed, the following figure shows the diagonal terms each with circles around them with the appropriate radius. The eigenvalues are given in red and the blue circles are those which contain all said eigenvalues.



i Note 2: Explicit Formula

If a matrix A of size $N \times N$ is tridiagonal with m on the main diagonal, r on the super diagonal and l on the subdiagonal, then the matrix A will have eigenvalues

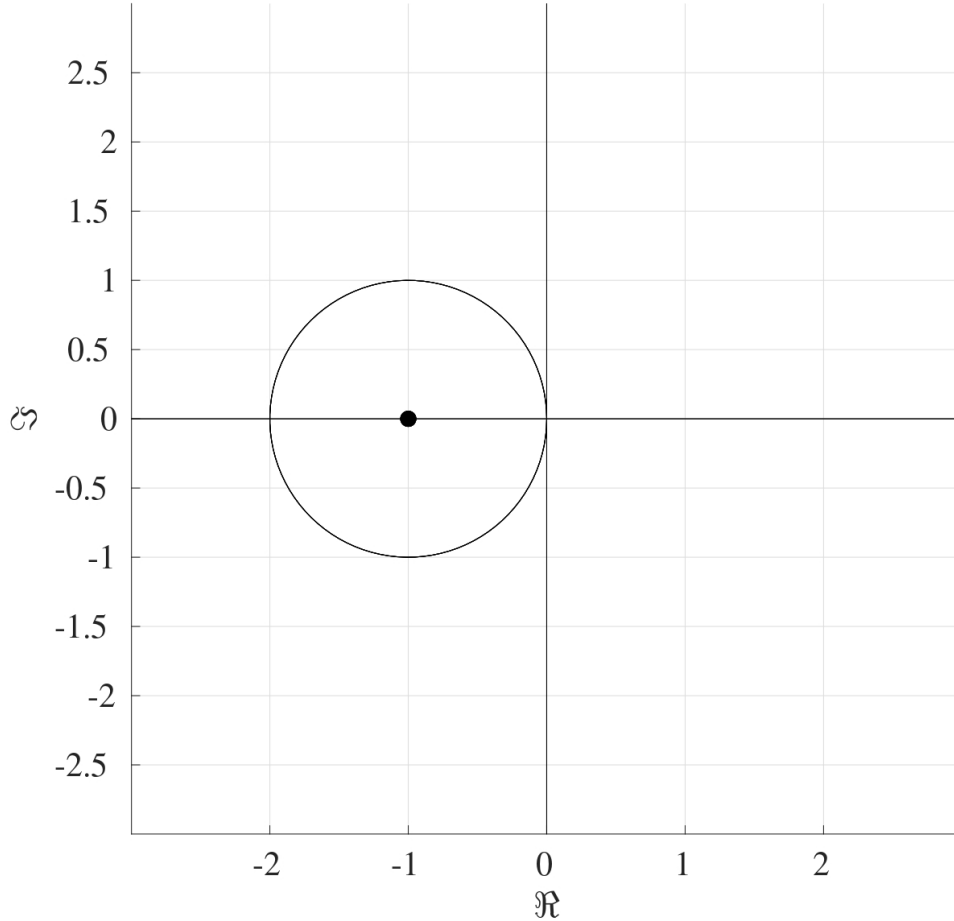
$$\lambda_n = m - 2\sqrt{rl} \cos\left(\frac{n\pi}{N+1}\right) \quad \text{for } n = 1, 2, \dots, N.$$

11.4.1 Stability of the Euler Method for the Advection Equation

Consider the matrix A_2 of size $N \times N$ from the advection equation

$$A_2 = \begin{pmatrix} -1 & 0 & \dots & 0 & 0 \\ 1 & -1 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & -1 & 0 \\ 0 & 0 & \dots & 1 & -1 \end{pmatrix}.$$

Following the steps of the Gershgorin theorem, the centres of all the circles on the complex plane will be located at the diagonal terms, all of which are -1 . The radii of these circles are the row sums of the matrix A_2 without the diagonal terms, which means that all the radii will be 1. The figure below shows the circle that results on the complex plane. Therefore regardless of what the eigenvalues might be, it is known that they will always have negative real parts and therefore the advection matrix forms an asymptotically stable system.



i Note

Using the formula given in Note 2, the eigenvalues of the advection matrix A_2 are all equal to -1 , which does indeed mean that the system is asymptotically stable.

Since the advection equation is asymptotically stable, a bound for the temporal stepsize needs to be found. Consider the advection equation after the discretisation $\frac{dU}{dt} = AU + b$ where $A = \frac{v}{h_x} A_2$. The Euler method is numerically stable if the time step h_t satisfies

$$\|\mathcal{I} + h_t A\|_{\infty} \leq 1.$$

First calculate $\mathcal{I} + h_t A$:

$$\mathcal{I} + h_t A = \mathcal{I} + \frac{vh_t}{h_x} A_2 = \begin{pmatrix} 1 - \tilde{v} & 0 & 0 & \dots & 0 & 0 & 0 \\ \tilde{v} & 1 - \tilde{v} & 0 & \dots & 0 & 0 & 0 \\ 0 & \tilde{v} & 1 - \tilde{v} & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 0 & \tilde{v} & 1 - \tilde{v} \end{pmatrix}.$$

where $\tilde{v} = \frac{vh_t}{h_x}$. Now taking the absolute value of all the terms and taking the row sums gives:

$$\text{abs} \left(\mathcal{I} + \frac{vh_t}{h_x} A_2 \right) = \begin{pmatrix} |1 - \tilde{v}| & 0 & 0 & \dots & 0 & 0 & 0 \\ \tilde{v} & |1 - \tilde{v}| & 0 & \dots & 0 & 0 & 0 \\ 0 & \tilde{v} & |1 - \tilde{v}| & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 0 & \tilde{v} & |1 - \tilde{v}| \end{pmatrix} \begin{matrix} \rightarrow |1 - \tilde{v}| \\ \rightarrow \tilde{v} + |1 - \tilde{v}| \\ \rightarrow \tilde{v} + |1 - \tilde{v}| \\ \vdots \\ \rightarrow \tilde{v} + |1 - \tilde{v}| \end{matrix}$$

The row sums of the absolute terms of this matrix are

$$a = |1 - \tilde{v}| \quad \text{and} \quad b = |1 - \tilde{v}| + \tilde{v}.$$

Since it is assumed that $v > 0$, then $b > a$ therefore, $\|\mathcal{I} + h_t A\|_\infty = b = |1 - \tilde{v}| + \tilde{v}$. Consider the two cases when $1 - \tilde{v} > 0$ and $1 - \tilde{v} < 0$.

1. If $1 - \tilde{v} > 0$, then $0 < \tilde{v} < 1$:

$$\|\mathcal{I} + h_t A\|_\infty = |1 - \tilde{v}| + \tilde{v} = 1 - \tilde{v} + \tilde{v} = 1.$$

Therefore if $1 - \tilde{v} > 0$, then $\|\mathcal{I} + h_t A\|_\infty \leq 1$.

2. If $1 - \tilde{v} < 0$, then $\tilde{v} > 1$:

$$\|\mathcal{I} + h_t A\|_\infty = |1 - \tilde{v}| + \tilde{v} = \tilde{v} - 1 + \tilde{v} = 2\tilde{v} - 1,$$

therefore in this case, if $\|\mathcal{I} + h_t A\|_\infty$ needs to be less than or equal to 1, then

$$\|\mathcal{I} + h_t A\|_\infty \leq 1 \quad \implies \quad 2\tilde{v} - 1 \leq 1 \quad \implies \quad \tilde{v} \leq 1$$

which contradicts with the assumption that $\tilde{v} > 1$.

Therefore, the Euler method will produce a convergent solution if

$$\tilde{v} < 1 \quad \implies \quad v \frac{h_t}{h_x} < 1.$$

In terms of number of spatial and temporal points N_x and N_t respectively, this restriction would be

$$v \frac{t_f - t_0}{L - x_0} \frac{N_x}{N_t} < 1$$

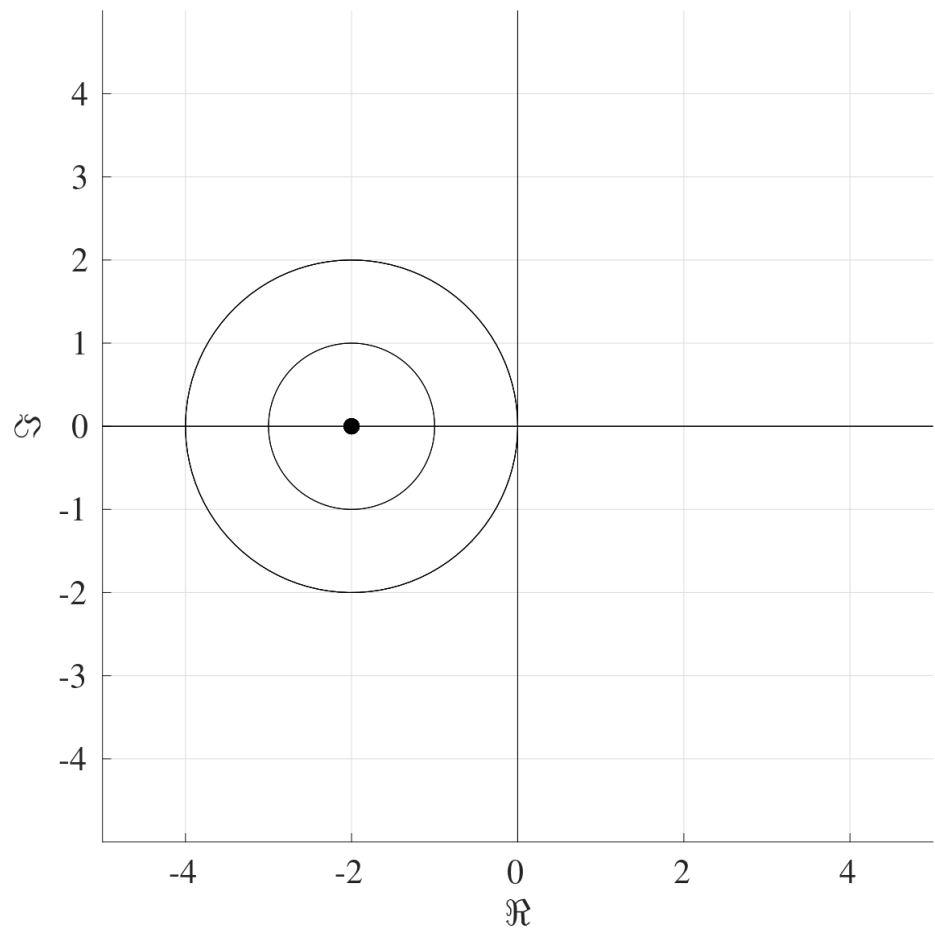
So for a fixed velocity v , if the time step h_t is to be halved, then the spatial step would also need to be halved as well.

11.4.2 Stability of the Euler Method for the Heat Equation

Consider the matrix A_1 of size $N \times N$ from the heat equation

$$A_1 = \begin{pmatrix} -2 & 1 & \dots & 0 & 0 \\ 1 & -2 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & -2 & 1 \\ 0 & 0 & \dots & 1 & -2 \end{pmatrix}.$$

The steps of the Gershgorin theorem can be followed to produce the following figure on the



complex plane.

Once again, this shows that all the eigenvalues will have negative real parts even though their explicit values are not known.

i Note

Using the formula given in Note 2, the eigenvalues of the heat matrix A_1 are equal given by

$$\lambda_n = -2 - 2 \cos \left(\frac{n\pi}{N_x + 1} \right) \quad \text{for } n = 1, 2, \dots, N_x,$$

all of which are negative real numbers, meaning that the system is asymptotically stable.

To determine the bound on the stepsize, consider the heat equation after the discretisation, which is $\frac{dU}{dt} = AU + \mathbf{b}$ where $A = \frac{\alpha}{h_x^2} A_1$. The Euler method is numerically stable if the time step h_t satisfies

$$\|\mathcal{I} + h_t A\|_{\infty} \leq 1.$$

First calculate $\mathcal{I} + h_t A$:

$$\mathcal{I} + h_t A = \mathcal{I} + \frac{\alpha h_t}{h_x^2} A_1 = \begin{pmatrix} 1 - 2\tilde{\alpha} & \tilde{\alpha} & 0 & \dots & 0 & 0 & 0 \\ \tilde{\alpha} & 1 - 2\tilde{\alpha} & \tilde{\alpha} & \dots & 0 & 0 & 0 \\ 0 & \tilde{\alpha} & 1 - 2\tilde{\alpha} & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 0 & \tilde{\alpha} & 1 - 2\tilde{\alpha} \end{pmatrix}$$

where $\tilde{\alpha} = \frac{\alpha h_t}{h_x^2}$. Now taking the absolute value of all the terms and taking the row sums gives:

$$\text{abs} \left(\mathcal{I} + \frac{\alpha h_t}{h_x^2} A_1 \right) = \begin{pmatrix} |1 - 2\tilde{\alpha}| & \tilde{\alpha} & 0 & \dots & 0 & 0 & 0 \\ \tilde{\alpha} & |1 - 2\tilde{\alpha}| & \tilde{\alpha} & \dots & 0 & 0 & 0 \\ 0 & \tilde{\alpha} & |1 - 2\tilde{\alpha}| & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 0 & \tilde{\alpha} & |1 - 2\tilde{\alpha}| \end{pmatrix} \begin{matrix} \rightarrow \tilde{\alpha} + |1 - 2\tilde{\alpha}| \\ \rightarrow 2\tilde{\alpha} + |1 - 2\tilde{\alpha}| \\ \rightarrow 2\tilde{\alpha} + |1 - 2\tilde{\alpha}| \\ \vdots \\ \rightarrow \tilde{\alpha} + |1 - 2\tilde{\alpha}| \end{matrix}$$

The row sums of the absolute terms of this matrix are

$$a = \tilde{\alpha} + |1 - 2\tilde{\alpha}| \quad \text{and} \quad b = 2\tilde{\alpha} + |1 - 2\tilde{\alpha}|.$$

Since $\tilde{\alpha} > 0$, then $b > a$ and therefore, $\|\mathcal{I} + h_t A\|_{\infty} = b = 2\tilde{\alpha} + |1 - 2\tilde{\alpha}|$. Consider the two cases $1 - 2\tilde{\alpha} > 0$ and $1 - 2\tilde{\alpha} < 0$.

1. If $1 - 2\tilde{\alpha} > 0$, then $0 < \tilde{\alpha} < \frac{1}{2}$:

$$\|\mathcal{I} + h_t A\|_{\infty} = |1 - 2\tilde{\alpha}| + 2\tilde{\alpha} = 1 - 2\tilde{\alpha} + 2\tilde{\alpha} = 1,$$

therefore $\|\mathcal{I} + h_t A\|_{\infty} \leq 1$.

2. If $1 - 2\tilde{\alpha} < 0$, then $\tilde{\alpha} > \frac{1}{2}$:

$$\|\mathcal{I} + h_t A\|_{\infty} = |1 - 2\tilde{\alpha}| + 2\tilde{\alpha} = 2\tilde{\alpha} - 1 + 2\tilde{\alpha} = 4\tilde{\alpha} - 1,$$

therefore in this case, if $\|\mathcal{I} + h_t A\|_\infty$ needs to be less than or equal to 1, then

$$\|\mathcal{I} + h_t A\|_\infty \leq 1 \implies 4\tilde{\alpha} - 1 \leq 1 \implies \tilde{\alpha} \leq \frac{1}{2}$$

which contradicts with the assumption that $\tilde{\alpha} > \frac{1}{2}$.

This means that the Euler method produces a stable convergent solution if

$$\tilde{\alpha} < \frac{1}{2} \implies \alpha \frac{h_t}{h_x^2} < \frac{1}{2}.$$

In terms of number of spatial and temporal points N_x and N_t respectively, this restriction would be

$$2\alpha \frac{t_f - t_0}{(L - x_0)^2} \frac{N_x^2}{N_t} < 1$$

So for a fixed diffusivity α , if the time step h_t is to be halved, then the spatial step would should be quartered.

11.4.3 Stability of the Convection-Diffusion Equation

Now that it has been established that both the heat and advection equations are asymptotically stable and the stepsize bounds have been found, it is time to combine both cases to tackle the convection-diffusion equation.

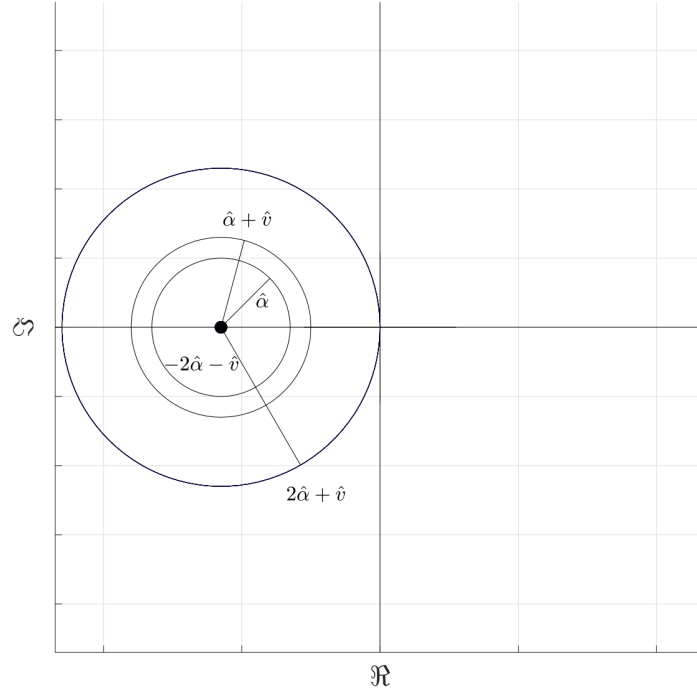
When discretised, the convection-diffusion equation can be written as $\frac{d\mathbf{U}}{dt} = \mathbf{A}\mathbf{U} + \mathbf{b}$ where the matrix A is given by

$$A = \frac{\alpha}{h_x^2} \begin{pmatrix} -2 & 1 & \dots & 0 & 0 \\ 1 & -2 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & -2 & 1 \\ 0 & 0 & \dots & 1 & -2 \end{pmatrix} + \frac{v}{h_x} \begin{pmatrix} -1 & 0 & \dots & 0 & 0 \\ 1 & -1 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & -1 & 0 \\ 0 & 0 & \dots & 1 & -1 \end{pmatrix}.$$

The Gershgorin theorem can be applied to the matrix A to show that all the eigenvectors have negative real parts. Indeed,

$$A = \begin{pmatrix} -2\hat{\alpha} - \hat{v} & \hat{\alpha} & 0 & \dots & 0 & 0 & 0 \\ \hat{\alpha} + \hat{v} & -2\hat{\alpha} - \hat{v} & \hat{\alpha} & \dots & 0 & 0 & 0 \\ 0 & \hat{\alpha} + \hat{v} & -2\hat{\alpha} - \hat{v} & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 0 & \hat{\alpha} + \hat{v} & -2\hat{\alpha} - \hat{v} \end{pmatrix}.$$

where $\hat{\alpha} = \frac{\alpha}{h_x^2}$ and $\hat{v} = \frac{v}{h_x}$. By the Gershgorin theorem, the centres of the circles will be located at the diagonal terms, namely at $-2\hat{\alpha} - \hat{v}$ with the radii $\hat{\alpha}$, $\hat{\alpha} + \hat{v}$ and $2\hat{\alpha} + \hat{v}$. The largest radius is $2\hat{\alpha} + \hat{v}$ which means that all the eigenvalues will be negative as shown below. Therefore the convection-diffusion equation is asymptotically stable.



i Note

For the convection-diffusion equation, the matrix $A = \hat{\alpha}A_1 + \hat{v}A_2$, using the formula given in Note 2 will give the eigenvalues

$$\lambda_n = -2\hat{\alpha} - \hat{v} - 2\sqrt{\hat{\alpha}(\hat{\alpha} + \hat{v})} \cos\left(\frac{n\pi}{N+1}\right) \quad \text{for } n = 1, 2, \dots, N$$

all of which are negative, meaning that A is asymptotically stable.

To find the bound for the stepsizes, consider the convection-diffusion equation after the discretisation $\frac{dU}{dt} = AU + \mathbf{b}$ where $A = \frac{\alpha}{h_x^2}A_1 + \frac{v}{h_x}A_2$. The Euler method is numerically stable if the time step h_t satisfies

$$\|\mathcal{I} + h_t A\|_{\infty} \leq 1.$$

Calculating $\mathcal{I} + h_t A$:

$$\mathcal{I} + h_t A = \begin{pmatrix} 1 - 2\tilde{\alpha} - \tilde{v} & \tilde{\alpha} & 0 & \dots & 0 & 0 & 0 \\ \tilde{\alpha} + \tilde{v} & 1 - 2\tilde{\alpha} - \tilde{v} & \tilde{\alpha} & \dots & 0 & 0 & 0 \\ 0 & \tilde{\alpha} + \tilde{v} & 1 - 2\tilde{\alpha} - \tilde{v} & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 1 - 2\tilde{\alpha} - \tilde{v} & \tilde{\alpha} & 0 \\ 0 & 0 & 0 & \dots & \tilde{\alpha} + \tilde{v} & 1 - 2\tilde{\alpha} - \tilde{v} & \tilde{\alpha} \\ 0 & 0 & 0 & \dots & 0 & \tilde{\alpha} + \tilde{v} & 1 - 2\tilde{\alpha} - \tilde{v} \end{pmatrix}$$

where $\tilde{\alpha} = \frac{\alpha h_t}{h_x^2}$ and $\tilde{v} = \frac{v h_t}{h_x}$. Taking the absolute value of all the terms and adding the rows gives

$$\text{abs}(\mathcal{I} + h_t A) = \begin{pmatrix} |1 - 2\tilde{\alpha} - \tilde{v}| & \tilde{\alpha} & 0 & \dots & 0 & 0 & 0 \\ \tilde{\alpha} + \tilde{v} & |1 - 2\tilde{\alpha} - \tilde{v}| & \tilde{\alpha} & \dots & 0 & 0 & 0 \\ 0 & \tilde{\alpha} + \tilde{v} & |1 - 2\tilde{\alpha} - \tilde{v}| & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 0 & \tilde{\alpha} + \tilde{v} & |1 - 2\tilde{\alpha} - \tilde{v}| \end{pmatrix}$$

$$\begin{aligned} &\rightarrow \tilde{\alpha} + |1 - 2\tilde{\alpha} - \tilde{v}| \\ &\rightarrow 2\tilde{\alpha} + \tilde{v} + |1 - 2\tilde{\alpha} - \tilde{v}| \\ &\rightarrow 2\tilde{\alpha} + \tilde{v} + |1 - 2\tilde{\alpha} - \tilde{v}| \\ &\vdots \\ &\rightarrow \tilde{\alpha} + \tilde{v} + |1 - 2\tilde{\alpha} - \tilde{v}| \end{aligned}$$

The row sums of the absolute terms of this matrix are

$$a = \tilde{\alpha} + |1 - 2\tilde{\alpha} - \tilde{v}|, \quad b = 2\tilde{\alpha} + \tilde{v} + |1 - 2\tilde{\alpha} - \tilde{v}| \quad \text{and} \quad c = \tilde{\alpha} + \tilde{v} + |1 - 2\tilde{\alpha} - \tilde{v}|.$$

Since $\tilde{\alpha} > 0$ and $\tilde{v} > 0$, then $b > c > a$, therefore, $\|\mathcal{I} + h_t A\|_\infty = b = 2\tilde{\alpha} + \tilde{v} + |1 - 2\tilde{\alpha} - \tilde{v}|$. Consider the two cases $1 - 2\tilde{\alpha} - \tilde{v} > 0$ and $1 - 2\tilde{\alpha} - \tilde{v} < 0$.

1. If $1 - 2\tilde{\alpha} - \tilde{v} > 0$, then $2\tilde{\alpha} + \tilde{v} < 1$:

$$\|\mathcal{I} + h_t A\|_\infty = |1 - 2\tilde{\alpha} - \tilde{v}| + 2\tilde{\alpha} + \tilde{v} = 1 - 2\tilde{\alpha} - \tilde{v} + 2\tilde{\alpha} + \tilde{v} = 1,$$

therefore $\|\mathcal{I} + h_t A\|_\infty \leq 1$.

2. If $1 - 2\tilde{\alpha} - \tilde{v} < 0$, then $2\tilde{\alpha} + \tilde{v} > 1$:

$$\|\mathcal{I} + h_t A\|_\infty = |1 - 2\tilde{\alpha} - \tilde{v}| + 2\tilde{\alpha} + \tilde{v} = 2\tilde{\alpha} + \tilde{v} - 1 + 2\tilde{\alpha} + \tilde{v} = 4\tilde{\alpha} + 2\tilde{v} - 1,$$

therefore in this case, if $\|\mathcal{I} + h_t A\|_\infty$ needs to be less than or equal to 1, then

$$\|\mathcal{I} + h_t A\|_\infty \leq 1 \quad \implies \quad 4\tilde{\alpha} + 2\tilde{v} - 1 \leq 1 \quad \implies \quad 2\tilde{\alpha} + \tilde{v} \leq 1$$

which contradicts with the assumption that $2\tilde{\alpha} + \tilde{v} > 1$.

This means that the Euler method will produce a stable convergent solution if

$$2\tilde{\alpha} + \tilde{v} < 1 \quad \implies \quad 2\alpha \frac{h_t}{h_x^2} + v \frac{h_t}{h_x} < 1.$$

This means that a choice can be made with regards to the bounds of the different components, for instance, the values of h_x and h_t can be chosen such that

$$\tilde{\alpha} < \frac{1}{4} \quad \text{and} \quad \tilde{v} < \frac{1}{2} \quad \text{or} \quad \tilde{\alpha} < \frac{1}{3} \quad \text{and} \quad \tilde{v} < \frac{1}{3}$$

or any combination thereof provided that the choices satisfy the inequality $2\tilde{\alpha} + \tilde{v} < 1$.

Bound for Convection-Diffusion

Consider the convection-diffusion equation

$$\frac{\partial u}{\partial t} = 0.1 \frac{\partial^2 u}{\partial x^2} - 0.5 \frac{\partial u}{\partial x} \quad \begin{array}{l} t \in [0, 10] \\ x \in [-2, 2] \end{array}$$

$$u(x, 0) = u_{init}(x) = 10, \quad u(-2, t) = u_l(t) = 1, \quad u(2, t) = u_r(t) = 0.$$

This can be discretised to give $\frac{dU}{dt} = AU$ where

$$\begin{aligned} & \frac{d}{dt} \underbrace{\begin{pmatrix} U_1(t) \\ U_2(t) \\ \vdots \\ U_{N-1}(t) \\ U_N(t) \end{pmatrix}}_U \\ &= \underbrace{\left[\frac{0.1}{h_x^2} \begin{pmatrix} -2 & 1 & \dots & 0 & 0 \\ 1 & -2 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & -2 & 1 \\ 0 & 0 & \dots & 1 & -2 \end{pmatrix} + \frac{0.5}{h_x} \begin{pmatrix} -1 & 0 & \dots & 0 & 0 \\ 1 & -1 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & -1 & 0 \\ 0 & 0 & \dots & 1 & -1 \end{pmatrix} \right]}_A \underbrace{\begin{pmatrix} U_1(t) \\ U_2(t) \\ \vdots \\ U_{N-1}(t) \\ U_N(t) \end{pmatrix}}_U \end{aligned}$$

subject to

$$U(0) = \begin{pmatrix} u_{init}(x_1) \\ u_{init}(x_2) \\ \vdots \\ u_{init}(x_{N-1}) \\ u_{init}(x_N) \end{pmatrix} \quad \text{where } u_{init}(x) = 10.$$

As yet, the value of N has not been put forward since the stepsizes need to be established first. For a stable Euler method, the stepsizes h_t and h_x need to satisfy

$$2\alpha \frac{h_t}{h_x^2} + v \frac{h_t}{h_x} < 1 \quad \implies \quad 2 \frac{h_t}{h_x} + 5 \frac{h_t}{h_x^2} < 10.$$

If $h_t = 2.5 \times 10^{-5}$ and $h_x = 0.02$ (which corresponds to $N_t = 40000$ and $N_x = 100$), then the Euler method will be stable.

11.5 Convection-Diffusion Equation with Centred Differencing

The discretisation of the Convection-Diffusion equation was done using a hybrid approach; implementing the centred differencing for the second derivative and a backward differencing for the first derivative. This converted the convection-diffusion equation into the form

$\frac{d\mathbf{U}}{dt} = \mathbf{A}\mathbf{U} + \mathbf{b}$ where the matrix $\mathbf{A}$ has $-2\hat{\alpha} - \hat{v}$ in the main diagonal, $\hat{\alpha}$ in the superdiagonal and $\hat{\alpha} + \hat{v}$ in the subdiagonal (where $\hat{\alpha} = \frac{\alpha}{h_x^2}$ and $\hat{v} = \frac{v}{h_x}$). This approach guaranteed asymptotic stability.

Suppose that the centred differencing scheme was used for both the first and second derivatives, this will convert the convection-diffusion equation into the matrix system $\frac{d\mathbf{U}}{dt} = \mathbf{A}\mathbf{U} + \mathbf{b}$ where the matrix $\mathbf{A}$ has $-2\hat{\alpha}$ in the main diagonal, $\hat{\alpha} - \hat{v}$ in the superdiagonal and $\hat{\alpha} + \hat{v}$ in the subdiagonal (where $\hat{\alpha} = \frac{\alpha}{h_x^2}$ and $\hat{v} = \frac{v}{2h_x}$). Using the Gershgorin Circle Theorem, the matrix $\mathbf{A}$ is asymptotically stable if $\hat{v} < \hat{\alpha}$, or $h_x v < 2\alpha$, which results in a stability condition on the physical parameters of the system. If this condition holds, then the system is asymptotically stable, otherwise it will not be stable meaning that conditions on the stability with the space and time discretisations are not guaranteed.

12 Wave Equation

The same method of lines can be extended to obtain a numerical solution to the wave equation, with some caveats.

Consider a string of L , the 1-dimensional wave equation that describes the movement of waves in the string is given by

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2} \quad \text{with} \quad 0 < x < L \quad \text{and} \quad t > 0$$

where $u = u(x, t)$ is the vertical displacement of the string at location x at time t and c is the wave speed.

This partial differential equation has four derivatives in total, two derivatives in x and two derivative in t , this means that four conditions are needed, two on x and two on t :

- $u(x, 0) = u_{init}(x)$ for $x \in [0, L]$: Initial wave profile of the string;
- $\frac{\partial u}{\partial t}(x, 0) = v_{init}(x)$ for $x \in [0, L]$: Initial wave speed of the string;
- $u(0, t) = u_l(t)$ for $t > 0$: Vertical displacement of the string at the left end;
- $u(L, t) = u_r(t)$ for $t > 0$: Vertical displacement of the string at the right end.

12.1 The Method of Lines for the Wave Equation

The outline of the method of lines for the wave equation is as follows:

1. Divide the spatial interval $[0, L]$ into N_x equally sized sections and label the points as $x_0, x_1, x_2, \dots, x_{N_x}$ where $x_n = nh_x$ and the spatial interval width is $h_x = \frac{L}{N_x}$.
2. **Left Hand Side:** For each point x_n , define the approximation $U_n(t) \approx u(x_n, t)$. Therefore the left hand side of the wave equation can be written as

$$\frac{\partial^2 u}{\partial t^2}(x_n, t) \approx \frac{d^2 U_n}{dt^2}(t)$$

and this holds for $n = 1, 2, \dots, N_x - 1$ since $U_0(t) \approx u(0, t) = u_l(t)$ and $U_{N_x}(t) \approx u(L, t) = u_r(t)$ are already known from the boundary conditions. Notice that the derivative of U_n is an ordinary derivative since U_n is a function of t only.

3. **Right Hand Side:** Use the finite difference approximation to approximate the *spatial* derivative in the differential equation. Here, the centred difference approximation for the second derivative will be used, namely

$$\frac{\partial^2 u}{\partial x^2}(x_n, t) \approx \frac{U_{n+1}(t) - 2U_n(t) + U_{n-1}(t)}{h_x^2}.$$

Therefore the right hand side of the wave equation will become

$$c^2 \frac{\partial^2 u}{\partial x^2}(x_n, t) \approx \frac{\alpha}{h_x^2} [U_{n-1}(t) - 2U_n(t) + U_{n+1}(t)].$$

This holds for $n = 1, 2, \dots, N_x - 1$ bearing in mind, once again, that $U_0(t) \approx u(0, t) = u_l(t)$ and $U_{N_x}(t) \approx u(L, t) = u_r(t)$ are known beforehand.

4. These can be combined to give the discretised form of the heat equation

$$\frac{d^2 U_n}{dt^2} = \frac{c^2}{h_x^2} [U_{n-1} - 2U_n + U_{n+1}]$$

for all $n = 1, 2, \dots, N_x - 1$ where $U_n = U_n(t)$. This means that the partial differential equation has been split into $N_x - 1$ ordinary differential equations.

5. This system has a second order derivative in t . To reduce it to a single derivative in t , let

$$\frac{dU_n}{dt} = V_n,$$

this means that

$$\frac{dV_n}{dt} = \hat{c} [U_{n-1} - 2U_n + U_{n+1}] \quad \text{where} \quad \hat{c} = \frac{c^2}{h_x^2}.$$

This substitution can convert the the wave equation into $2(N_x - 1)$ equations that can now be written in matrix form as $\frac{d\mathbf{U}}{dt} = \mathbf{A}\mathbf{U} + \mathbf{b}$ where

$$\mathbf{U} = \begin{pmatrix} V_1 \\ V_2 \\ \vdots \\ V_{N_x-2} \\ V_{N_x-1} \\ U_1 \\ U_2 \\ \vdots \\ U_{N_x-2} \\ U_{N_x-1} \end{pmatrix} \quad \text{and} \quad \mathbf{b}(t) = \begin{pmatrix} \hat{c}u_l(t) \\ 0 \\ \vdots \\ 0 \\ \hat{c}u_r(t) \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{pmatrix}$$

and $\mathbf{A}$ is a block matrix of the form

$$\mathbf{A} = \begin{bmatrix} \mathcal{O} & \hat{c}\mathbf{A}_1 \\ \mathcal{I} & \mathcal{O} \end{bmatrix}$$

where $\mathcal{O}$ is the zero matrix of size $N_x - 1$, $\mathcal{I}$ is the identity matrix of size $N_x - 1$ and A_1 is the same matrix as in the heat equation of size $N_x - 1$ (namely the matrix with -2 on the diagonal and 1 on the super- and sub diagonals). This differential equation can be solved subject to the initial condition

$$\mathbf{U}_0 = \begin{pmatrix} V_1(0) \\ V_2(0) \\ \vdots \\ V_{N_x-2}(0) \\ V_{N_x-1}(0) \\ U_1(0) \\ U_2(0) \\ \vdots \\ U_{N_x-2}(0) \\ U_{N_x-1}(0) \end{pmatrix} \approx \begin{pmatrix} \frac{\partial u}{\partial t}(x_1, 0) \\ \frac{\partial u}{\partial t}(x_2, 0) \\ \vdots \\ \frac{\partial u}{\partial t}(x_{N_x-2}, 0) \\ \frac{\partial u}{\partial t}(x_{N_x-1}, 0) \\ u(x_1, 0) \\ u(x_2, 0) \\ \vdots \\ u(x_{N_x-2}, 0) \\ u(x_{N_x-1}, 0) \end{pmatrix} = \begin{pmatrix} v_{init}(x_1) \\ v_{init}(x_2) \\ \vdots \\ v_{init}(x_{N_x-2}) \\ v_{init}(x_{N_x-1}) \\ u_{init}(x_1) \\ u_{init}(x_2) \\ \vdots \\ u_{init}(x_{N_x-2}) \\ u_{init}(x_{N_x-1}) \end{pmatrix}.$$

This system can now be solved using any of the IVP solvers with a temporal stepsize h_t .

12.2 Stability

All the diagonal entries of the matrix A are zero, this means that using the Gershgorin Circle Theorem will produce circles all centred at the origin which would not guarantee that all the eigenvalues have negative real parts. Another approach is needed.

All the eigenvalues λ of the matrix A must satisfy

$$\det(A - \lambda \tilde{\mathcal{I}}) = 0$$

where $\tilde{\mathcal{I}}$ is the identity matrix of size $2(N_x - 1) \times 2(N_x - 1)$. The matrices A and $\tilde{\mathcal{I}}$ can be written in block form as

$$A = \begin{bmatrix} \mathcal{O} & \hat{c}A_1 \\ \mathcal{I} & \mathcal{O} \end{bmatrix} \quad \text{and} \quad \tilde{\mathcal{I}} = \begin{bmatrix} \mathcal{I} & \mathcal{O} \\ \mathcal{O} & \mathcal{I} \end{bmatrix}$$

where $\mathcal{O}$ is the zero matrix, $\mathcal{I}$ is the identity matrix and A_1 is the same as the heat matrix, all of which have size $(N_x - 1) \times (N_x - 1)$. This means that the eigenvalues must satisfy

$$\det(A - \lambda \tilde{\mathcal{I}}) = 0 \quad \implies \quad \det \begin{bmatrix} -\lambda \mathcal{I} & \hat{c}A_1 \\ \mathcal{I} & -\lambda \mathcal{I} \end{bmatrix} = 0.$$

Since all the submatrices are commutative, then

$$\det \begin{bmatrix} -\lambda \mathcal{I} & \hat{c}A_1 \\ \mathcal{I} & -\lambda \mathcal{I} \end{bmatrix} = \det(\lambda^2 \mathcal{I} - \hat{c}A).$$

i Detereminant of Block Matrices

Consider the block matrix M given by

$$M = \begin{bmatrix} A & B \\ C & D \end{bmatrix}$$

where A and D are square submatrices. There are multiple formulas for obtaining the determinant of such a matrix:

- If the matrix A is invertible, then

$$\det \begin{bmatrix} A & B \\ C & D \end{bmatrix} = \det(A) \det(D - CA^{-1}B)$$

- If the matrix D is invertible, then

$$\det \begin{bmatrix} A & B \\ C & D \end{bmatrix} = \det(D) \det(A - BD^{-1}C)$$

- If all the submatrices are square and of the same size:

- if $CD = DC$

$$\det \begin{bmatrix} A & B \\ C & D \end{bmatrix} = \det(AD - BC)$$

- if $CA = AC$

$$\det \begin{bmatrix} A & B \\ C & D \end{bmatrix} = \det(AD - CB)$$

- if $BD = DB$

$$\det \begin{bmatrix} A & B \\ C & D \end{bmatrix} = \det(DA - BC)$$

- if $AB = BA$

$$\det \begin{bmatrix} A & B \\ C & D \end{bmatrix} = \det(DA - CB)$$

A MATLAB Basics

This Appendix will cover some of the basic procedures in MATLAB.

A.1 Command Window

When MATLAB is opened, you will be faced with a window containing several parts.

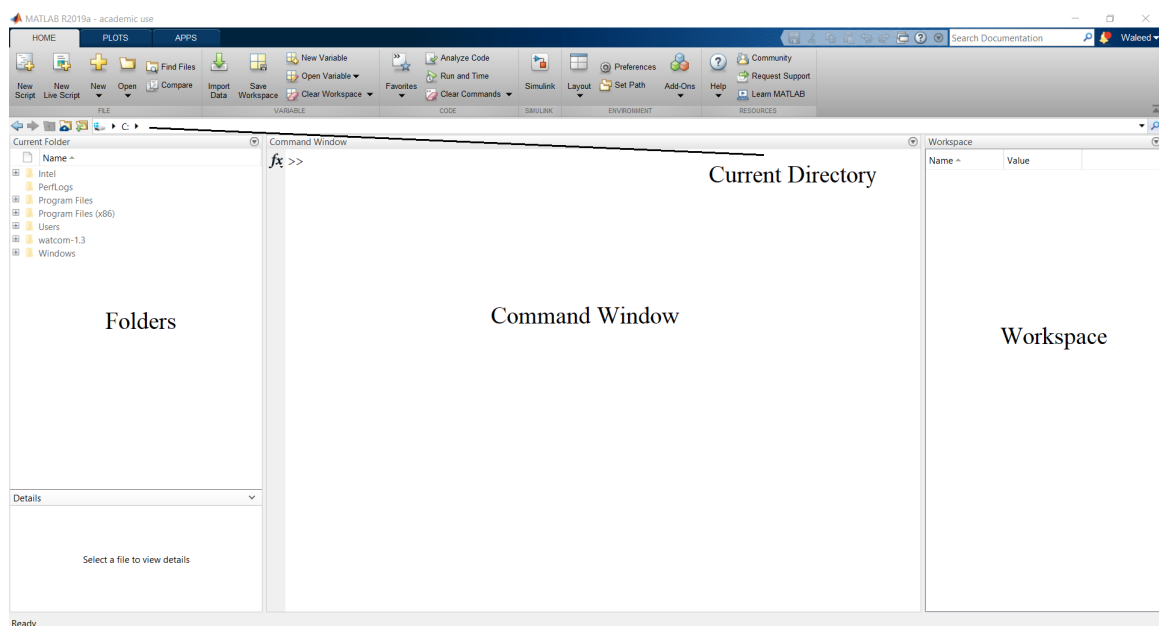


Figure A.1: Default MATLAB layout.

These different areas serve the following purpose:

- **Command Window:** This is the main window where the first line starts with `>>`. This is where commands are executed, note that once a command has been run (i.e. you pressed Enter), then what has been written cannot be edited or undone and therefore, this is a suitable space for running or executing codes only, not for writing extensive codes.
- **Directory:** This is the destination folder that MATLAB is going to refer to in either opening or saving codes. Note that all MATLAB files are saved as `.m` files.

- **Current Folder:** This displays the functions, figures, subfolders, scripts, codes, etc. that are in the current directory.
- **Workspace:** This displays all the the variables that have been used, along with their types (number, matrix, etc.) and their values.

A.2 Executing Commands in the Command Window

The command window will be where all the codes and functions are executed. It can also be used to perform quick calculations. Some examples of MATLAB syntax and built-in functions are shown below:

Mathematical Symbol	MATLAB Syntax
$+$	<code>+</code>
$-$	<code>-</code>
$\times$	<code>*</code>
$\div$	<code>/</code>
3^5	<code>3^5</code>
π	<code>pi</code>
e^2	<code>exp(2)</code>
$\sin(\pi)$	<code>sin(pi)</code>
$\sin^{-1}(\pi)$	<code>asin(pi)</code>
$\lfloor 3.6 \rfloor$	<code>floor(3.6)</code>
$\lceil 4.7 \rceil$	<code>ceil(4.7)</code>
$ -4 $	<code>abs(-4)</code>
$1 + 2i$	<code>1+2i</code>
i	<code>0+i</code>
$\Re(1 + 2i)$	<code>real(1+2i)</code>
$\Im(3 - 4i)$	<code>imag(3-4i)</code>
2×10^7	<code>2e7</code>
$147 \pmod{5}$	<code>mod(147,5)</code>

All trigonometric functions follow the same syntax as `sin`, but bear in mind that by default, all the angles should be in *radians* and not in degrees. To use degrees, just put a `d` at the end of the trigonometric function, i.e. use `sind`, `cosd`, `asind`, etc.

The functions `ceil` and `floor` are the ceiling and floor functions respectively. Their purpose is to round up to the nearest integer (ceiling) or round down to the nearest integer (floor). Standard rounding can be done using `round`.

Another important function is `mod` which find the remainder when dividing one number by another. For example, $147 \pmod{5}$ is the remainder after dividing 147 by 5 which is 2.

```

1  >> 2+2
2  ans =
3      4
4  >> sin(0)
5  ans =
6      0
7  >> sin(pi/2)
8  ans =
9      1
10 >> sin(30)
11 ans =
12    -0.9880
13 >> sind(30)
14 ans =
15    0.5000
16 >> pi
17 ans =
18    3.1416
19 >> exp(1)
20 ans =
21    2.7813
22 >> ceil(2.1)
23 ans =
24      3
25 >> floor(6.9)
26 ans =
27      6
28 >> round(2.3)
29 ans =
30      2
31 >> mod(147,5)
32 ans =
33      2

```

If the outcome of a calculation is an integer, then MATLAB will usually display it as an integer, if not, then by default, it will display the solution as a number to 4 decimal places. The number of decimal places can be increased by using `format long` and reversed by using `format short`.

i Note

Note that any command executed in the Command Window will be applied globally, so if `format long` is used, it will apply to everything executed in the Command Window until it is reversed or MATLAB is restarted.

```

1  >> pi/2
2  ans =
3      1.5708
4  >> format long
5  >>pi/2
6  ans =
7      1.570796326794897
8  >> format short
9  >> pi/2
10 ans =
11      1.5708

```

A.3 Defining Variables

MATLAB is a numerical programming language that relies on a “box” feature. This means that standard algebraic practices cannot be used, for instance, writing $2x = x + 1$ makes perfect sense mathematically and yields a solution of $x = 1$, however writing $2*x=x+1$ makes no sense in MATLAB.

Note

A very important note to bear in mind here is that in MATLAB syntax, $2x$ has no meaning. In order to multiply terms, the multiplication sign $*$ needs to be used.

A “box” with a given name, which is always on the left hand side of the $=$ sign, is assigned a value, which is on the right hand side, and the value can then be manipulated or changed, so there are no variables in MATLAB *per se*. In the following example, a “box” is given the name x and the number 3 is assigned to it, calculations can then be done by referring to the number that is in said box. The values within the boxes can be redefined by using the $=$ sign again.

```

1  >> x=3
2  x =
3      3
4  >> x+1
5  ans =
6      4
7  >> x+x
8  ans =
9      6
10 >> 3*x
11 ans =
12      9
13 >> y=(2*x)^x

```

```

14 y =
15     216
16 >> y+10
17 ans =
18     226

```

On the other hand, $x = x + 2$ makes no sense mathematically but within MATLAB syntax (as is the case with most other programming languages), this simply means calculate $x+2$ (which is on the right hand side of the $=$ sign) using the value already in the box labelled x (which is 3), then redefine the value in that same box to take this new value, so the box labelled x is now assigned the value 5.

```

1 >> x=3
2 x =
3     3
4 >> x=x+2
5 x =
6     5
7 >> x=3*x
8 x =
9    15

```

A.4 Naming Variables

There are certain rules with regards to what names can be used for the variables:

- Names can be of **any length** (within the bounds of reason of course to avoid confusion).
- Names are **case sensitive**, so **a** and **A** are two different variable names.
- Names must contain **no spaces**, underscores can be used instead. For example, **Bad Name** is not a viable variable name but **GoodName** and **Also_A_Good_Name** are both valid.
- Names must contain **no operators or symbols**, with the exception of the underscore, so **do not use** **! ? . , ; + - * / & # % \$**.
- Names **can contain numbers** as long as they are not the first character. For example **1Forrest1** is not a viable variable name but **OneForrest1** or **Obi1Kenobi** are both viable.
- Names cannot be the same as already **existing functions**, for instance, a variable cannot be given the name **sin** since there is already a built-in function with that same name, however, one could use **Sin** since variable names are case sensitive (although this particular example is not recommended since it may cause confusion).

```

1  >> P_1=1
2  P_1 =
3      1
4  >> P_2=P_1+2
5  P_2 =
6      3
7  >> PP_3=P_1+p_2
8  Undefined function or variable 'p_2'.
9  >> PP_3=P_1+P_2
10 PP_3 =
11      4

```

Typing `whos x` in the command window will give the properties of `x`, namely its size (in a matrix sense), storage allocation, class and attributes, but *not* its value. Typing `whos` on its own will give a list of all the variables that have been used along with their properties, alternatively, these can also be found in the Workspace.

A.5 Scripts & Functions

Within the command window, nothing can be edited once it has been executed which is inconvenient if the code is longer than a single line. In that case, it is best to use the *Editor*. By default, the Editor can be opened by clicking on *New Script*, this is a window in which any length of code can be written, saved and then executed with the *Run* button. If any changes need to be made then the editor window will allow that with ease, once changes are made, the code can be run again.

A **function** is very similar to a script but the difference between them is that a function can take in several inputs and produce several outputs and must always have the format:

```

1  function [output1,output2,...]=Function_Name(input1,input2,...)
2      Body of the code
3  end

```

The **function** cannot always be executed with the *Run* button but will often need to be called in the Command Window to allow for the inputs to be placed.

The name of the function follows the same rules as the variable names mentioned before. One of the most important technicalities that has to be addressed is that the *functions and scripts that are used must be in the same as folder as is stated in the directory*.

When writing functions, or scripts of any kind, there are two important characteristics that need to be considered:

- **Commentary:** When writing codes, it is important to provide some comments on what is being done to give context and to allow for accessibility and reproducibility. This can be done by using % at the beginning of the line. This makes MATLAB ignore everything that comes after it, allowing for commentary of bits of code that need context. This is generally good practice in writing codes since the user can make comments about inputs, outputs, procedures, etc. without affecting the execution of the code.
- **Suppression:** On MATLAB, any line of code that is written will produce an output (many other coding languages do not unless prompted to do so). So in functions, performing an action will always produce an output whether it is needed or not. This is where semicolon ; can be used. The semi-colon suppresses the output, this means that if there are several calculations to be made, sometimes the intermediate stages do not need to be seen, only the final answer, in this case the semicolon allows the calculation to be done but not printed out in the command window.

Example of function

Consider a cube with side length L (in m) and mass M (in kg), then the object will have density

$$\rho = \frac{M}{L^3}.$$

The following code calculates this density with the inputs being the mass M and length L with the output being the density ρ :

```

1 function [rho]=Calculate_Density(M,L)
2
3 % M:  Mass of cube in kg
4 % L:  Side length of cube in m
5
6 rho = M/(L^3);
7
8 end

```

This function, which is called `Calculate_Density`, has two inputs, namely M and L , and one output, namely ρ . Notice that the list of inputs must always be in round brackets (...) while the outputs should be in square brackets [...].

To use this function, just type the name of the function in the command window with the inputs and outputs in *exactly the same order in which they appear* in the function and using the same set of brackets as well, i.e. (...) for inputs and [...] for outputs.

```

1 >> [rho] = Calculate_Density(100,20)
2 rho =
3     0.0125

```

Expanding on this, suppose that a new function is desired where the user will input

the mass in pounds and the length in inches but the desired density should still be in kg m^{-2} . A few more lines can be added in that case.

```
1 function [rho]=Calculate_Density_Imperial(M,L)
2
3 % M: Mass of cube in lbs
4 % L: Side length of cube in inches
5
6 M = M/2.20462;      % Converts lbs to kg
7 L = L/39.3701;      % Converts inches to m
8
9 rho = M/(L^3);
10
11 end
```

Note that here, the same variable name has been used and then redefined. So initially, M will be input in pounds, say $M=50$, then at line 6, the same variable name is redefined, so the new mass will be $M = \frac{50}{2.20462} = 22.6796$, but the same name is used for both. Similarly for L when it is converted from inches to meters.

In this case, the function can be executed with a mass of 50lbs and a side length of 10in to give:

```
1 >> [rho] = Calculate_Density_Imperial(50,10)
2 rho =
3      1.3840e+03
```

One of the major differences in using scripts and functions is the assignment of variables and their declaration. In a script, if a variable C was given the value 3 (so $C=3$ was in the script) then this value of C will be declared *globally*, meaning that it can be used in the command window and it will still take the same value. However in functions, the variables are declared *locally*, so if in a function the variable C was given the value 3, this will only hold within the function itself and nowhere outside it.

A.6 Exercises

! Exercise 1: Metric Cone

Write a MATLAB function that takes inputs h and r and outputs the volume of a cone (in cubic meters) with height h in meters and radius r in meters.

Test the code on a cone with height 5m, radius 3m (which should give a volume of 47.1238898m^3).

Solution 1

```
1 function [V]=Cone_Vol1(h,r)
2
3 % This function calculates the volume of a cone in m^3
4
5 % Inputs:
6 % h: Height of the cone in m
7 % r: Radius of the cone in m
8
9 % Output:
10 % V: Volume of the cone in m^3
11
12 V=pi*(r^2)*h/3;
13
14 end
```

Code test with $h = 5$ and $r = 3$:

```
1 >> [V]=Cone_Vol1(5,3)
2 V =
3     47.129
```

Excercise 2: Imperial Cone

Write a MATLAB function that takes inputs h and r and outputs the volume of a cone (in cubic meters) with height h in inches and diameter d in yards.

Test the code on a cone with height 10in, diameter 1yd (which should give a volume of 0.0556m^3).

Solution 2

```
1 function [V]=Cone_Vol2(h,d)
2
3 % This function calculates the volume of a cone in m^3
4
5 % Inputs:
6 % h: Height of the cone in inches
7 % d: Diameter of the cone in yards
8
9 % Convert h from inches to metres
10 h = h*0.0254;
11
12 % Convert d from yards to metres
13 d = d*0.9144;
14
15 % Radius of cone base is half the diameter
16 r = d/2;
17
18 % Output:
19 % V: Volume of the cone in m^3
20
21 V=pi*(r^2)*h/3;
22
23 end
```

Code test with $h = 10$ and $d = 1$:

```
1 >> [V]=Cone_Vol2(10,1)
2 V =
3     0.0556
```

B Arrays in MATLAB

MATLAB is one of the most versatile programming languages when it comes to working with vectors and matrices, hence the name MATLAB, particularly MATrix LABoratory. In MATLAB, vectors essentially represent lists and matrices represent tables.

B.1 Vectors

To form a vector, use square brackets and separate the terms using commas to form a row vector or semicolons to form a column vector.

```
1 >> v=[1,2,3,4]
2 v =
3     1     2     3     4
4 >> u=[1;2;3;4]
5 u =
6     1
7     2
8     3
9     4
```

An algebraic sequence (a sequence where the consecutive terms differ by a fixed value) can be formed into a vector by using colons as $v=a:n:b$. This forms a vector v where the first term is a , then next term is $a+n$, then $a+2*n$, etc. until b is reached. If the sequence goes beyond b , then b is ignored and the last term before b will be the last term of the sequence. Note that $v=a:b$ will produce a row vector from a to b in steps of 1.

```
1 >> u=[1:1:10]
2 u =
3     1     2     3     4     5     6     7     8     9    10
4 >> v=[20:3:30]
5 v =
6    20    23    26    29
7 >> w=[100:-20:-40]
8 w =
9   100    80    60    40    20     0   -20   -40
```

Some useful operations that can be applied to vectors are: For a vector v :

- `abs(v)` takes the absolute value of all the terms of the vector $\mathbf{v}$.
- `v'` takes the transpose of the vector $\mathbf{v}$, namely $\mathbf{v}^T$, so it changes $\mathbf{v}$ from a row vector to a column vector and vice versa.
- `length(v)` finds how many terms there are in the vector $\mathbf{v}$.
- `max(v)` finds the maximum value in the vector $\mathbf{v}$ while `min(v)` finds the minimum value.
- `[a,b]=max(v)` produces two outputs, `a` which is the maximum value in the vector $\mathbf{v}$ and `b` which is its location in $\mathbf{v}$, similarly with `[a,b]=min(v)`. (Note that in MATLAB, array positions start from 1, unlike Python which starts from 0.)
- `sum(v)` takes the sum of all the terms in the vector $\mathbf{v}$.
- `mean(v)` takes the mean of all the terms in the vector $\mathbf{v}$.
- `median(v)` takes the median of all the terms in the vector $\mathbf{v}$.
- `sort(v)` orders the terms of $\mathbf{v}$ in ascending order.
- `sort(v,'descend')` orders the terms of $\mathbf{v}$ in descending order.
- `norm(v)` finds the magnitude of the vector $\mathbf{v}$. Recall that for a vector $\mathbf{v} = (v_1, v_2, \dots, v_N)$, the magnitude of the vector $\mathbf{v}$ is given by:

$$|\mathbf{v}| = \sqrt{\sum_{n=1}^N |v_n|^2} = \sqrt{v_1^2 + v_2^2 + \dots + v_N^2}.$$

- `norm(v,p)` finds the p -norm of the vector $\mathbf{v}$. Recall that for a vector $\mathbf{v} = (v_1, v_2, \dots, v_N)$ and a positive integer p , the p -norm of $\mathbf{v}$, denoted $\|\mathbf{v}\|_p$ is given by

$$\|\mathbf{v}\|_p = \sqrt[p]{\sum_{n=1}^N |v_n|^p} = \sqrt[p]{v_1^p + v_2^p + \dots + v_N^p}.$$

Note that `norm(v)` is the default 2-norm whereas `norm(V,inf)` is the *sup-norm*¹ (also known as the *Chebyshev norm* or *infinity norm*).

¹Recall that for a vector $\mathbf{v}$, the *sup-norm*, denoted $\|\mathbf{v}\|_\infty$ is the maximum absolute term in the vector, i.e. for a vector $\mathbf{v} = (v_1, v_2, \dots, v_N)$,

$$\|\mathbf{v}\|_\infty = \max_{n=1,2,\dots,N} |v_n|.$$

```

1  >> v=[2,-8,6,-2,-9,4]
2  v =
3      2   -8   6   -2   -9   4
4  >> abs(v)
5  ans =
6      2   8   6   2   9   4
7  >> v'
8  ans =
9      2
10     -8
11      6
12     -2
13     -9
14      4
15 >> (v')'
16 ans =
17      2   -8   6   -2   -9   4
18 >> length(v)
19 ans =
20      6
21 >> max(v)
22 ans =
23      6
24 >> [a,b]=max(v)
25 a =
26      6
27 b =
28      3
29 >> min(v)
30 ans =
31     -9
32 >> [a,b]=min(v)
33 a =
34     -9
35 b =
36      5
37 >> sum(v)
38 ans =
39     -7
40 >> mean(v)
41 ans =
42    -1.1667
43 >> median(v)
44 ans =
45      0

```

```

46 >> sort(v)
47 ans =
48     -9    -8    -2     2     4     6
49 >> sort(v,'descend')
50 ans =
51     6     4     2    -2    -8    -9
52 >> norm(v)
53 ans =
54    14.3175
55 >> norm(v,1)
56 ans =
57     31
58 >> norm(v,inf)
59 ans =
60     9

```

B.2 Matrices

To form matrices, the same theme follows as with vectors where a comma indicates the next term on the same row and semicolons move to the next row. Be careful to ensure that *all the rows have the same number of terms*, similarly with the columns.

```

1 >> M=[1,2,3;4,5,6;7,8,9]
2 M =
3     1     2     3
4     4     5     6
5     7     8     9
6 >> N=[1,2,3,4,5;6,7,8,9,10]
7 N =
8     1     2     3     4     5
9     6     7     8     9    10
10 >> P=[1,2,3;4,5,6;7,8]
11 Error using vertcat
12 Dimensions of arrays being concatenated are not consistent.

```

There are some operations that translate from vectors to matrices, for example, for a matrix M:

- `abs(M)` takes the absolute value of all the terms of the matrix M.
- `M'` takes the transpose of the matrix M.

Other functions are not as intuitive, for example, `length(M)` gives *only one* output which is *either* the number of rows *or* the number of columns, whichever is bigger. Whereas `size(M)`

gives two outputs with the first being the number of rows of M and the second is the number of columns of M .

Some matrix functions are done column-wise, for example, `max(M)` *does not* give the maximum value that appears in the matrix, instead it produces a row vector of maxima where the first term is maximum value of all the terms in the first column, the second is the maximum of the second column and so on. This same column-wise approach holds for other functions like `min(M)`, `sum(M)`, `mean(M)` and `sort(M)`; MATLAB works with the matrix as a collection of column vectors and applies these functions to each column separately. To find the maximum/minimum/sum of all the terms in the entire matrix, then the function will need to be used twice, so the maximum element in the whole matrix can be found by using `max(max(M))`.

Note that `[a,b]=max(M)` will give two outputs, the first output `a` is the vector `max(M)` as described above and the second output `b` is the vector of their locations. Similarly for `[a,b]=min(M)`.

Matrix norms are slightly more involved, in terms of their mathematical definition, than vector norms. For a matrix M of size $m \times n$ and a positive integer p , the matrix p -norm imposed by the vector p -norm is given by

$$\|M\|_p = \sup_{\mathbf{x} \in \mathbb{C}^n} \frac{\|M\mathbf{x}\|_p}{\|\mathbf{x}\|_p}$$

Calculating these explicitly can be very difficult since it requires using *all* possible vectors $\mathbf{x} \in \mathbb{C}^n$, however, the most useful norms have some closed forms:

- $\|M\|_1$ is the *maximum absolute column sum*;
- $\|M\|_\infty$ is the *maximum absolute row sum*;
- $\|M\|_2$ is the **Spectral Radius** of M (more specifically, it is the square root of the largest eigenvalue of the matrix $M^H M$ where M^H is the Hermitian of M , or the complex conjugate transpose).

There are other norms that are not imposed by vector norms, like the **Frobenius Norm** which is the square root of the sum of the squares of the absolute value of all the terms, i.e.

$$\|M\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |m_{ij}|^2}.$$

All these norms still use the same syntax as vector norms, i.e. using `norm(M,1)`, `norm(M,2)`, `norm(M,inf)` and `norm(M,'Fro')` (with `norm(M)` being the default 2-norm). This is why it is imperative to be mindful of the context since the same operation can have different meanings depending on whether the input was a vector or a matrix.

```

1 >> M=[-4,5;2,9;-6,10]
2 M =
3     -4     5
4      2     9

```

```

5      -6    10
6  >> abs(M)
7  M =
8      4    5
9      2    9
10     6    10
11 >> M'
12 ans =
13     -4    2   -6
14      5    9    10
15 >> size(M)
16 ans =
17      3    2
18 >> length(M)
19 ans =
20      3
21 >> max(M)
22 ans =
23      2    10
24 >> max(max(M))
25 ans =
26     10
27 >> [a,b]=max(M)
28 a =
29      2    10
30 b =
31      2    3
32 >> min(M)
33 ans =
34     -6    5
35 >> min(min(M))
36 ans =
37     -6
38 >> [a,b]=min(M)
39 a =
40     -6    5
41 b =
42      3    1
43 >> sum(M)
44 ans =
45     -8    24
46 >> sum(sum(M))
47 ans =
48     16
49 >> mean(M)

```

```

50 ans =
51     -2.6667    8.0000
52 >> median(M)
53 ans =
54     -4     9
55 >> sort(M)
56 ans =
57     -6     5
58     -4     9
59     2    10
60 >> sort(M,'descend')
61 ans =
62     2    10
63     -4     9
64     -6     5
65 >> norm(M)
66 ans =
67    15.1099
68 >> norm(M,1)
69 ans =
70     24
71 >> norm(M,inf)
72 ans =
73     16
74 >> norm(M,'Fro')
75 ans =
76    16.1864

```

B.3 Referencing Terms in Arrays

Elements of a vector (row or column) can be referred to by putting the index of the desired element in brackets after the vector's name. For example, $v(4)$ is the 4th element in the vector v .

i MATLAB Indexing

Note that in MATLAB, indexing starts from 1, not from 0 like Python.

If the last element of a vector is desired where its size may not be known, then the index **end** can be used.

```

1 >> u=[9;7;0;1]
2 u =
3     9

```

```

4      7
5      0
6      1
7  >> u(1)
8  ans =
9      9
10 >> u(4)
11 ans =
12      1
13 >> u(end)
14 ans =
15      1
16 >> u(6)
17 Index exceeds array bounds.

```

For matrices, there are two indices, the first denotes the row number and the second the column number:

$$\begin{pmatrix} (1,1) & (1,2) & (1,3) & \dots \\ (2,1) & (2,2) & (2,3) & \dots \\ (3,1) & (3,2) & (3,3) & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

So $M(2,3)$ will output the element of M that is in row 2 and column 3. MATLAB also has the ability to refer to terms in matrices by using one index only. For instance, if a matrix M is of size 3×4 , then $M(10)$ would refer to the “10th element”. Under usual circumstances, this is meaningless unless M is a vector, however, in this case, MATLAB can refer to the 10th element where the elements start from 1 and work their way down columns as such:

$$\begin{pmatrix} (1) & (4) & (7) & (10) \\ (2) & (5) & (8) & (11) \\ (3) & (6) & (9) & (12) \end{pmatrix}$$

Therefore, the 10th element of M would be the element in the 1st row and 4th column for the 3×4 matrix. Using this referencing system is certainly not recommended since it can cause issues with different sized matrices.

MATLAB can also refer to whole rows or whole columns, this is done by using `:`, for example $M(:,3)$ will produce the 3rd column whereas $M(1,:)$ will produce the 1st row.

```

1  >> M=[2,3,1,4;1,6,3,1;4,1,2,8]
2  M =
3      2      3      1      4
4      1      6      3      1
5      4      1      2      8
6  >> M(2,3)

```

```

7  ans =
8      3
9  >> M(3,1)
10 ans =
11     4
12 >> M(end,3)
13 ans =
14     2
15 >> M(end,end)
16 ans =
17     8
18 >> M(:,2)
19 ans =
20     3
21     6
22     1
23 >> M(3,:)
24 ans =
25     4     1     2     8
26 >> M(:,end)
27 ans =
28     4
29     1
30     8
31 >> M(2)
32 ans =
33     1
34 >> M(4)
35 ans =
36     3
37 >> M(12)
38 ans =
39     8

```

B.4 Matrix Operations

Addition and subtraction of matrices (and vectors) follows the usual mathematical rules, namely, both matrices need to be of the same size and all the terms are added elementwise, i.e. the first term is added to the first term, the second to the second, etc.

```

1  >> A=[1,3,7;5,2,6;2,3,2]
2  A =
3      1     3     7
4      5     2     6

```

```

5      2   3   2
6  >> B=[2,3,1;1,6,3;4,1,2]
7  B =
8      2   3   1
9      1   6   3
10     4   1   2
11 >> A+B
12 ans =
13     3   6   8
14     6   8   9
15     6   4   4

```

Matrices and vectors can be multiplied or divided by *a scalar value* using the `*` and `/` operations.

```

1  >> 2*A
2  ans =
3      2   6  14
4     10   4  12
5      4   6   4
6  >> B/2
7  ans =
8     1.00   1.50   0.50
9     0.50   3.00   1.50
10    2.00   0.50   1.00

```

Matrix multiplication is carried out using the `*` operator. Recall that for two matrices A , of size $m \times n$, and B , of size $p \times q$, the matrix product AB is only possible if $n = p$ (i.e. the number of columns of A is equal to the number of rows of B) and the resulting matrix AB will then be of size $m \times q$.

```

1  >> A*B
2  ans =
3     33   28   24
4     36   33   23
5     15   26   15

```

Elementwise multiplication and division of matrices (also known as the **Hadamard Operations**) is also a possibility in MATLAB. So for matrices A and B of the same size, the elementwise product (denoted mathematically as $A \circ B$) produces a matrix that is of the same size as A and B where the first element is the product of the first element of A and the first element of B , the second element is the product of the second element of A and the second element of B and so on. This is done using a dot `.` before the operations, in other words, the elementwise product $A \circ B$ is written as `A.*B`, similarly for elementwise division using `./` and elementwise exponentiation using `.^`. Bear in mind this is *only possible if* the matrices/vectors are of the same size, just as in addition and multiplication.

```

1  >> A.*B
2  ans =
3      2    9    7
4      5   12   18
5      8    3    4
6  >> A./B
7  ans =
8      0.50    1.00    7.00
9      5.00    0.33    2.00
10     0.50    3.00    1.00
11 >> A.^2
12 ans =
13     1    9   49
14    25    4   36
15     4    9    4
16 >> A^2
17 ans =
18     30     30     39
19     27     37     59
20     21     18     36

```

There are some special matrices and matrix forms built into MATLAB such as:

- `[]`: empty vector/matrix which contains no terms, therefore has size 0×0 and is usually used as a placeholder.
- `zeros(a,b)`: forms a matrix of zeros with size $a \times b$.
- `ones(a,b)`: forms a matrix of ones with size $a \times b$.
- `eye(a,b)`: forms an identity matrix (ones on the main diagonal, zeros otherwise) of size $a \times b$.
- `rand(a,b)`: forms a matrix of size $a \times b$ where all the elements are randomly chosen from a normal distribution whose entries lie between 0 and 1.
- `randi([M,N],a,b)`: forms a matrix of size $a \times b$ where all the elements are randomly chosen integers from a normal distribution whose entries lie between M and N.
- `diag(v)`: forms a square matrix whose diagonal entries are the elements of the vector `v`.

There are also some matrix operations that are very useful such as:

- `inv(A)`: find the inverse of the matrix `A`.
- `det(A)`: find the determinant of the matrix `A`.
- `trace(A)`: find the trace of the matrix `A` (which is the sum of the diagonal entries).

B.5 Substitution & Concatenation

Sometimes, vectors and matrices need to be augmented, either by adding, removing or changing some terms.

For both vectors and matrices, individual values can be substituted and redefined by referring to its index. For example, consider the vector v and suppose that its second element is to be changed, this can be done by using $v(2)=$ to assign a new value that will overwrite the original value.

```
1 >> v=[1,3,7,5]
2 v =
3     1     3     7     5
4 >> v(2)
5 ans =
6     3
7 >> v(2)=8
8 v =
9     1     8     7     5
10 >> v(4)=0
11 v =
12     1     8     7     0
```

The same syntax can be used to redefine an element in terms of itself or in terms of others, like defining the second element as twice its original value or setting an element to be the sum of some other elements.

```
1 >> v(2)=10*v(2)
2 v =
3     1    80     7     0
4 >> v(1)=v(3)
5 v =
6     7    80     7     0
7 >> v(4)=v(1)+v(2)+v(3)
8 v =
9     7    80     7    94
```

The same can be done with matrices as well where this replacement can either be done by elements, rows or columns.

```
1 >> M=[2,1;3,6]
2 M =
3     2     1
4     3     6
5 >> M(1,2)
```

```

6  ans =
7      1
8  >> M(1,2)=4
9  M =
10     2  4
11     3  6
12  >> M(2,2)=0
13  M =
14     2  4
15     3  0
16  >> M(1,:)
17  ans =
18     2  4
19  >> M(1,:)= [9,1]
20  M =
21     9  1
22     3  6
23  >> M(:,2)
24  ans =
25     1
26     6
27  >> M(:,2)= [4;0]
28  M =
29     9  4
30     3  0

```

Matrices and vectors can also be concatenated or cut, that simply means that terms can be added or removed, this is done by using the comma or semi-colon depending on the situation. Not only can terms be added, but whole rows and columns can be added as well but it is *critical* that the terms are added in a consistent fashion, meaning that if a new row is to be added, then it must be of the same size as all the other rows otherwise it will not make sense. To remove rows or columns, then simply assign an empty vector, namely `[]`, to the desired location.

```

1  >> A=[1,7]
2  A =
3      1  7
4  >> A=[A,4]           % Add 4 to the end
5  A =
6      1  7  4
7  >> A=[8,A]           % Add 8 to the start
8  A =
9      8  1  7  4
10 >> A=[A; [0,5,7,9]] % Add a new row
11 A =

```

```

12      8   1   7   4
13      0   5   7   9
14 >> A=[A,[0;1]]      % Add a new column
15 A =
16      8   1   7   4   0
17      0   5   7   9   1
18 >> A(:,3)=[]        % Remove third column
19 A =
20      8   1   4   0
21      0   5   9   1
22 >> A(1,:)=[]        % Remove first row
23 A =
24      0   5   9   1
25 >> A(end)=[]        % Remove last term
26 A =
27      0   5   9

```

B.6 Finding Terms

Sometimes, finding some terms is desired, say if the user needs to find all the values in a list that are greater than 5, or less than -1 , or equal to 2. In this case, the comparative operators should be used which are:

Operation	MATLAB Syntax
Less than	<
Less than or equal to	<=
Equal to	==
Greater than	>
Greater than or equal to	>=
Not equal to	~=

These operators need to be used in conjunction with the `find` function. So for a given vector v , if the terms greater than 5 need to be found, then use `find(v>5)`, this will produce a vector of *indices* that denote the locations of the values that greater than 5. If there are no such values that satisfy the condition, then an empty vector will be produced, namely `[]`. This can be very useful if, say, all the values greater than 5 need to be multiplied by 10, or all the values that are less than -1 need to be changed to 0, or all the values that are equal to 2 need to be removed.

```

1 >> v=[1,2,-5,12,-3,2]
2 v =
3      1      2     -5     12     -3      2

```

```

4  >> i=find(v>5)
5  ans =
6      4
7  >> v(i)
8  ans =
9      12
10 >> v(i)=10*v(i)
11 v =
12      1      2     -5    120     -3      2
13 >> j=find(v<-1)
14 ans =
15      3      5
16 >> v(j)
17 ans =
18     -5     -3
19 >> v(j)=0
20 v =
21      1      2      0    120      0      2
22 >> k=find(v==2)
23 ans =
24      2      6
25 >> v(k)=[]
26 v =
27      1      0    120      0

```

When finding terms in matrices, MATLAB tends to provide the location in the single index form rather than in the dual form. In other words, if a matrix is of size 3×3 and MATLAB needs to refer to the (2,3) element (second row, third column), it would display the index as the 7th element. This is an important distinction that needs to be made.

```

1  >> M=[2,0,5;-1,2,9;-6,1,-8]
2  M =
3      2      0      5
4     -1      2      9
5     -6      1     -8
6  >> m=find(M>5)
7  m =
8      8
9  >> M(m)
10 ans =
11      9
12 >> M(m)=M(m)*10
13 M =
14      2      0      5
15     -1      2     90

```

```

16      -6    1   -8
17 >> n=find(M<0)
18 n =
19      2
20      3
21      9
22 >> M(n)
23 ans =
24      -1
25      -6
26      -9
27 >> M(n)=0
28 M =
29      2    0    5
30      0    2   90
31      0    1    0

```

An alternative way of finding terms would be to dispense with the `find` command altogether. This will produce a binary matrix showing the locations of the terms that satisfy the condition (with 1 being true and 0 being false).

```

1 >> A=[1,4,6,9,2;7,3,1,6,0]
2 A =
3      1    4    6    9    2
4      7    3    1    6    0
5 >> find(A>5)
6 ans =
7      2
8      5
9      7
10     8
11 >> A>5
12 ans =
13     2×5 logical array
14      0    0    1    1    0
15      1    0    0    1    0

```

B.7 Exercises

! Exercise 1: Matrix Calculations

$$A = \begin{pmatrix} 1 & 2 \\ 5 & 8 \end{pmatrix} \quad ; \quad B = \begin{pmatrix} 4 & 0 & -4 \\ -1 & 0 & 1 \\ 2 & 1 & 3 \end{pmatrix} \quad ; \quad C = \begin{pmatrix} 1 & 0 & 4 \\ 2 & -2 & 6 \end{pmatrix}$$
$$\mathbf{u} = \begin{pmatrix} 1 \\ 8 \end{pmatrix} \quad ; \quad \mathbf{v} = \begin{pmatrix} 0 \\ 3 \\ 4 \end{pmatrix}$$

Using MATLAB, write a command/script to produce:

- The matrix AC .
- Element (2,3) of the matrix CB .
- Third element of the matrix $\mathbf{u}^T C$.
- Element (1,2) of the matrix $\mathbf{u}\mathbf{v}^T$.
- Trace of B^2 .
- Maximum and minimum terms in $B\mathbf{v}$.
- 2-norm of $\mathbf{v}$.
- Frobenius norm of B .
- The determinant of B .
- The inverse of $134(C^T C + \mathcal{I})$ where $\mathcal{I}$ is the identity matrix.
- The eigenvalues and eigenvectors of $\mathbf{v}\mathbf{u}^T C$.

 Solution 1

```

1  >> A=[1,2;5,8];
2  >> B=[4,0,-4;-1,0,1;2,1,3];
3  >> C=[1,0,4;2,-2,6];
4  >> u=[1;8];
5  >> v=[0;3;4];
6  >> A*C
7  ans =
8      5      -4      16
9      21     -16      68
10 >> D=C*B
11 D =
12     12     4     8
13     22     6     8
14 >> D(2,3)
15 ans =
16     8
17 >> E=u'*C
18 E =
19     17    -16    52
20 >> E(3)
21 ans =
22     52
23 >> F=u*v'
24 F =
25     0     3     4
26     0    24    32
27 >> F(1,2)
28 ans =
29     3
30 >> trace(B*B)
31 ans =
32    11
33 >> G=B*v
34 G =
35    -16
36     4
37    15
38 >> max(G)
39 ans =
40    15
41 >> min(G)
42 ans =
43    -16
44 >> norm(v,2)
45 ans =
46     5
47 >> norm(B,'Fro')
48 ans =
49    6.9282
50 >> det(B)
51 ans =
52     0
53 >> H=134*(C'*C+eye(3))
54 H =
55     804    -536    2144
56    -536     670   -1608

```

C Loops

Loops are some of the most important features in any programming language and they fall under three types: `if`, `while` and `for` loops.

C.1 if Loops

An `if` command executes a loop if a certain condition is satisfied. This requires the use of comparative operators which are:

Operation	MATLAB Syntax
Less than	<code><</code>
Less than or equal to	<code><=</code>
Equal to	<code>==</code>
Greater than	<code>></code>
Greater than or equal to	<code>>=</code>
Not equal to	<code>~=</code>

An `if` loops must have the following structure:

```
1  if compare <=> compare with
2
3      do something
4
5  elseif compare <=> compare with
6
7      do something else
8
9  else
10
11      do something if none of the above conditions have been met
12
13  end
```

if Loop Example

Suppose a function is to be written which takes a number N as an input then in the command window, displays “The Good” if it is positive, “The Bad” if it is negative and “The Ugly” if it is zero¹.

```
1 function Good_Bad_Ugly(N)
2
3 if N>0                % First check if the input N is positive
4
5     disp('The Good')   % If N is positive, display 'The Good'
6
7 elseif N<0            % If N is not positive, check if it is negative
8
9     disp('The Bad')    % If N is negative, display 'The Bad'
10
11 elseif N==0           % If N is neither positive nor negative, check
12                        % if it zero
13
14     disp('The Ugly')   % If N is zero, display 'The Ugly'
15
16 end
17
18 end
```

The `disp` command outputs the variables stated within the brackets, if the argument is single quotation marks, namely `'...'`, then it will be displayed verbatim. Note that here, the line will not start with `ans =` since it is was only asked to display and not specify variables. This function can be run within the command window as follows:

```
1 >> Good_Bad_Ugly(3)
2 The Good
3 >> Good_Bad_Ugly(-5)
4 The Bad
5 >> Good_Bad_Ugly(0)
6 The Ugly
```

In `if` loops, it is always a good idea to have a few `elseif` commands in order to have all the cases covered, this is because sometimes, MATLAB can misunderstand some inputs. For instance, suppose that the input is the complex number $1 - 2i$:

```
1 >> Good_Bad_Ugly(1-2i)
2 The Good
```

This does not make sense since the number $1 - 2i$ is neither positive nor negative, nor zero for that matter. In this case, MATLAB takes the real part only without being

prompted to do so, and prints the output and since the real part is 1, the output will be **The Good**. In order to accommodate for this, an extra condition can be added in the form of another `if` loop that considers this and displays “The Complex” if the number is complex.

```
1 function Good_Bad_Ugly(N)
2
3 if imag(N)~=0           % First, check if N has a non-zero imaginary
4                         % part
5
6     disp('The Complex') % If N does have a non-zero imaginary part,
7                         % display 'The Complex'
8
9 else                   % Otherwise, run the code as before
10
11     if N>0
12
13         disp('The Good')
14
15     elseif N<0
16
17         disp('The Bad')
18
19     elseif N==0
20
21         disp('The Ugly')
22
23     end
24
25 end
26
27 end
```

In this case, if the input as $1 - 2i$, then the output will be **The Imaginary**.

It is important to note that in `if` loops, the code will quit the loop after the first time the `if` condition is satisfied and will not check the other conditions.

if Loop Ordering

Suppose a function is to be written which takes an input N and displays “Multiple of 2” if it is a multiple of 2, “Multiple of 3” if it is a multiple of 3 and “Too high to count” otherwise. This function will require the use of the `mod` syntax; for numbers N and b ,

¹In reference to the 1966 film “The Good, the Bad and the Ugly.”

`mod(N,b)` will produce 0 if N is a multiple of b.

```
1 function Mult(N)
2
3 if mod(N,2)==0      % Check if N is a multiple of 2
4
5     disp('Multiple of 2')
6
7 elseif mod(N,3)==0 % Check if N is a multiple of 3
8
9     disp('Multiple of 3')
10
11 else
12
13     disp('Too high to count')
14
15 end
16
17 end
```

Run this code with the inputs 10, 15, 19 and 24:

```
1 >> Mult(10)
2 Multiple of 2
3 >> Mult(15)
4 Multiple of 3
5 >> Mult(19)
6 Too high to count
7 >> Mult(24)
8 Multiple of 2
```

For the inputs 10, 15 and 19, the results are as expected however with 24, only one output is produced, suggesting that 24 is a multiple of 2 only. The reason this is produced is because the `if` loop checked the first condition and since it was satisfied, it executed the code block underneath and quit the whole loop, not running through the others. That is why it is very important to be aware of the ordering of the `if` and `elseif` commands.

C.2 while Loops

The `while` loop is somewhat similar to the `if` loop in the sense that values of two terms are being compared but here, the loop will keep repeating until the condition is no longer satisfied.

while Loop Example

Suppose a function is to be written that takes two inputs, N and d and keeps subtracting d from N until it can no longer do so without becoming negative, the function should then output the last positive integer after this repeating operation. This code is the equivalent of finding the remainder of dividing a number N by d (or taking $N \pmod{d}$). For example, if $N = 9$ and $d = 4$, then $N - d = 5$, $N - 2d = 1$, $N - 3d = -3$, then the function would take the inputs $(N, d) = (9, 4)$ and outputs 1.

```
1 function [r]=Remainder(N,d)
2
3 M=N;           % Start with the number M being equal to N
4
5 while M-d>=0    % As long as M-d is non-negative, run the loop
6
7     M=M-d;      % Since M-d is non-negative, find M-d
8                 % and let M be equal to this new value,
9                 % this keeps repeating until M-d<0
10
11 end
12
13 r=M;           % Set the remainder r to be this final value M
14
15 end
```

This can be used in the command window as follows (note that here, because there is only one output, then it does not need to be explicitly stated in square brackets):

```
1 >> [r]=Remainder(9,4)
2 r =
3     1
4 >> [r]=Remainder(10,2)
5 r =
6     0
7 >> Remainder(14515,135)
8 ans =
9     70
10 >> Remainder(1e12,42578)
11 ans =
12    20554
```

Suppose now that this code is to be modified so that it can also output the number of times d can be subtracted from N . For example, as before, if $(N, d) = (9, 4)$, the remainder is 1 and the number of times d must be subtracted from N to obtain this remainder is 2, this is the equivalent of finding the number of times the **while** loop

actually ran. This is a very common procedure and the way to tackle this is by use of a “counter”. This is a variable that starts with the value 0 and every time the while loop is run, 1 is added to it. This modification can be done as follows.

```
1 function [r,counter]=Remainder(N,d)
2
3 M=N;                % Start with the number M being equal to N
4
5 counter=0;          % Start with the counter being 0
6
7 while M-d>=0        % As long as M-d is non-negative, run the loop
8
9     M=M-d;          % Since M-d is non-negative, find M-d
10                    % and let M be equal to this new value
11
12     counter=counter+1; % Add 1 to the counter every time
13                    % the while loop is run
14 end
15
16 r=M;                % Set the remainder r to be this final value M
17
18 end
```

This can be used in the command window as follows (in this case, since there are two outputs, they both have to be stated, but they don't need to be of the same name, only the same order):

```

1 >> [r,counter]=Remainder(9,4)
2 r =
3     1
4 counter =
5     2
6 >> [r,c]=Remainder(10,2)
7 r =
8     0
9 c =
10    5
11 >> [R,C]=Remainder(14515,135)
12 R =
13    70
14 C =
15   107
16 >> [r,c]=Remainder(1e12,42578)
17 r =
18  20554
19 c =
20 23486307

```

Caution 1: Collatz Conjecture

In mathematics, there is a famous algorithm known as the *Collatz Conjecture*, the steps of the algorithm are as follows:

1. Pick any positive integer.
2. i. If the number is even, divide by 2.
- ii. If the number is odd, multiply by 3 and add 1.
3. Repeat Step 2.

For instance, if the input is the number 10, the sequence of numbers will be as follows:

$$10 \xrightarrow{\div 2} 5 \xrightarrow{\times 3 + 1} 16 \xrightarrow{\div 2} 8 \xrightarrow{\div 2} 4 \xrightarrow{\div 2} 2 \xrightarrow{\div 2} 1$$

Similarly, if the input is 21:

$$21 \xrightarrow{\times 3 + 1} 64 \xrightarrow{\div 2} 32 \xrightarrow{\div 2} 16 \xrightarrow{\div 2} 8 \xrightarrow{\div 2} 4 \xrightarrow{\div 2} 2 \xrightarrow{\div 2} 1$$

Both number sequences end up at 1 from two different starting numbers of 10 and 21. (The algorithm is stopped at 1 since if the algorithm is carried on after reaching 1, then a loop will be formed going 4, 2, 1, 4, 2, 1,) The *Collatz Conjecture* states that regardless of the starting value, this sequence will *always* reach a 4-2-1 loop. This statement has been put forward in 1937 and has not yet been proven or disproven but has been computed for numbers larger than 10^{17} , all the numbers end at the 4-2-1 loop.

The **while** loop can be used in conjunction with the **if** loop in order to make a function that outputs the number of steps it takes to get to 1. This code can be checked by having an input of 10 and the output should be 6 since the algorithm required 6 steps before reaching 1, similarly, if the input is 21, then the output should be 7 and these can be used as test cases.

In writing codes, it is helpful to start with a pseudocode:

1. Read the input number.
2. As long as the number is greater than 1, do the following:
 - i. If the number is even, divide by 2.
 - ii. If the number is odd, multiply by 3 and add 1.
3. Repeat Step 2 until 1 is reached.

From this pseudocode, it is clear that Step 2 can be represented by an **if** loop. Steps 2 and 3 require the number to be greater than 1, since it is unknown when that will happen, the **while** loop can be used. Now, the pseudocode can be translated into MATLAB syntax with an input value of **a** and an output value **N** which is the number of steps it takes to get to 1.

```
1 function [N]=Collatz(a)
2
3 N=0; % Start with N=0
4
5 while a>1 % Perform the code block as long as the number
6           % is bigger than 1
7
8     if mod(a,2)==0 % Check if the number is even
9
10        a=a/2; % If it is, redefine a as a/2
11
12    else % Otherwise, if a is odd
13
14        a=3*a+1; % Redefine a as 3a+1
15
16    end
17
18    N=N+1; % Every time the code block is run, add 1 to N
19
20 end
21
22 end
```

This code can be checked using the test cases:

```

1 >> Collatz(10)
2 ans =
3     6
4 >> Collatz(21)
5 ans =
6     7
7 >> Collatz(1000)
8 ans =
9    111

```

The function `Collatz` should only be able to take integer inputs. A custom error message can be made to ensure that; the following can be added in Line 2:

```

1 if mod(a,1)~=0
2     error('a must be an integer')
3 end

```

C.3 Multiple Conditions for if & while Loops

Occasionally, multiple conditions may need to be satisfied when running `if` or `while` loops, this can be done with the `&&` for conjunctive conditions (equivalent to *and*) and `||` for disjunctive conditions (equivalent to *or*).

Collatz Isolation

For the function `Collatz` in Caution 1, the code should only be able to take any positive integer. An exclusion was introduced to produce an error message if the input was not an integer. Suppose that another condition is to be added that would produce the same error message if the input value is non-positive or not real. This can be done using the *or* syntax, which is `||`.

```

1 if imag(a)~=0 || mod(a,1)~=0 || a<=0 || imag(a)~=0
2     error('a must be an integer')
3 end

```

C.4 for Loops

A `for` loop is different compared to the `while` and `if` loops since it does not require comparison, instead, it runs through a series of terms that have been predefined.

for Loop Example 1

Suppose a simple **for** loop is needed that takes an input value N and adds all the positive integers from 1 to N . So if $N = 10$, then the function would output the sum of the numbers from 1 to 10, namely 55. This can be written as follows:

```
1 function [Sum]=Summation(N)
2
3 Sum=0;
4
5 for i=1:1:N
6
7     Sum=Sum+i;
8
9 end
10
11 end
```

This simple code starts with a `Sum=0`, then the variable `i` runs from 1 to N and adds itself onto `Sum`, the final result would be the sum of all the positive integers from 1 to N^2 .

for Loop Example 2

Suppose a **for** loop is desired that takes a vector $\mathbf{v}$ as an input and outputs the vector $\mathbf{u}$ whose elements are the squares of $\mathbf{v}$ ³.

The vector $\mathbf{v}$ will be a part of the input but the vector $\mathbf{u}$ needs to be *initialised*, meaning that $\mathbf{u}$ has to be predefined in some way. Since the size of $\mathbf{u}$ will be the same as $\mathbf{v}$, then the vector $\mathbf{u}$ can be initialised as a vector of zeros that is the same size as $\mathbf{v}$, this can be done using `u=zeros(size(v))`. The code can then be written by replacing the appropriate term in the list.

```
1 function [u]=Square(v)
2
3 u=zeros(size(v));
4
5 for i=1:1:N
6
7     u(i)=v(i)^2;
8
9 end
10
11 end
```

³Bear in mind that this is a contrived example for the sake of demonstration. This exact procedure can be done in one single command `sum(1:1:10)`.

Alternatively, if the size of $\mathbf{u}$ is not known, then it can be initialised as an empty array `[]` and terms can be concatenated to it.

```
1 function [u]=Square2(v)
2
3 u=[];
4
5 for i=1:1:N
6
7     u=[u,v(i)^2];
8
9 end
10
11 end
```

C.5 Exercises

! Exercise 1

Write a MATLAB function called `Fib` that takes an input N and produces a value F that is the N^{th} term of the Fibonacci sequence starting from 1,3 (recall that a Fibonacci sequence is a sequence where any term is the sum of the previous two terms). For example, if $N = 5$, then the first 5 terms of this Fibonacci sequence are (1, 3, 4, 7, 11), meaning that the output should be $F = 11$. Use the following test cases to verify that the code produces the correct results:

- $N = 10$: $F = 123$;
- $N = 20$: $F = 15127$;
- $N = 50$: $F = 28143753123$.

³Just as before, this is intended to be a contrived example to show the working of a `for` loop. This procedure can be done in a single command as `u=v.^2` for elementwise exponentiation.

💡 Solution 1

```
1 function [F]=Fib(N)
2
3 S=zeros(1,N);           % Initialise the sequence S as a list of N zeros
4
5 S(1)=1;                 % Redefine the first term of S to be equal to 1
6
7 S(2)=3;                 % Redefine the second term of S to be equal to 3
8
9 for n=3:1:N             % Starting from the third term onwards
10
11     S(n)=S(n-1)+S(n-2); % Let the nth term of S be the sum of the
12                        % previous two terms
13
14 end
15
16 F=S(end);              % Let F be the last term in the sequence S,
17                        % alternatively, F=S(N) can be used since it is known that
18                        % N is the last term
19
20 end
```

❗ Exercise 2

Write a MATLAB function called **Fib2** that takes an input M and produces values c and G where G is the largest term of the Fibonacci sequence starting from 2,5 such that $G < M$ and the number of terms in the sequence up to that point is c . For example, if $M = 60$, start a Fibonacci sequence with the 2,5 until a number above M is reached and count the number terms. So if $M = 60$, then the sequence is (2, 5, 7, 12, 19, 31, 50, 81), meaning that $G = 50$ (since it is the largest term in the sequence that is less than M) and $c = 6$ (since it takes 6 steps to get to 50). Use the following test cases to verify that the code produces the correct results:

- $M = 100$: $G = 81$, $c = 9$;
- $M = 1000$: $G = 898$, $c = 14$;
- $M = 10^9$: $G = 638162747$, $c = 42$.

Solution 2

```
1 function [c,G]=Fib2(M)
2
3 S=[2,5];           % Since, in principle, the number of terms is not known,
4                     % then define S as the sequence starting with 2 and 5
5
6 while S(end)<M % Run the while loop as long as the last term of the
7               % sequence is less than M
8
9     S=[S S(end)+S(end-1)]; % Redefine S in terms of itself; start
10                                % with the sequence S and append an extra
11                                % term at the end that is the sum of the
12                                % last term and the one before it
13
14 end
15
16 G=S(end-1); % G will be the second to last term (since the last one
17             % is bigger than M)
18
19 c=length(S); % c is simply the length of S
20
21 end
```

D Plotting in MATLAB

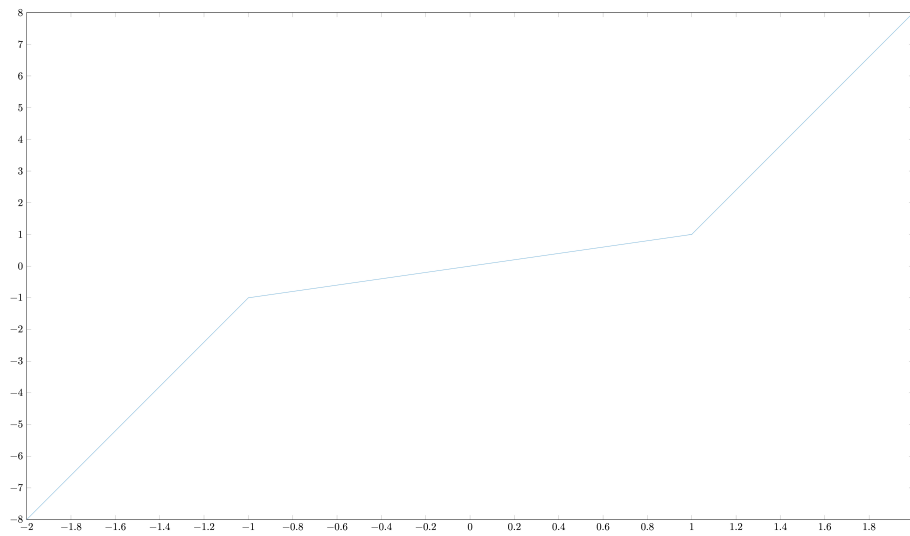
D.1 Forming Lists for Plotting

Suppose the function x^3 is to be plotted. First of all, a range of x values is needed, so if the function needs to be plotted in the interval $[-2, 2]$, then a vector needs to be formed that spans this particular domain, the more points there are, the smoother the function will be. This can be done by using, say, `x=-2:1:2` which produces a vector `x` with 5 points, namely `x=[-2 -1 0 1 2]`.

Secondly, the values on the y -axis need to be formed. For every x value, the value on the y axis will be at x^3 , this can be done using elementwise exponentiation as `y=x.^3`. In this case, the `x` and `y` vectors will be `x=[-2 -1 0 1 2]` and `y=[-8 -1 0 1 8]`.

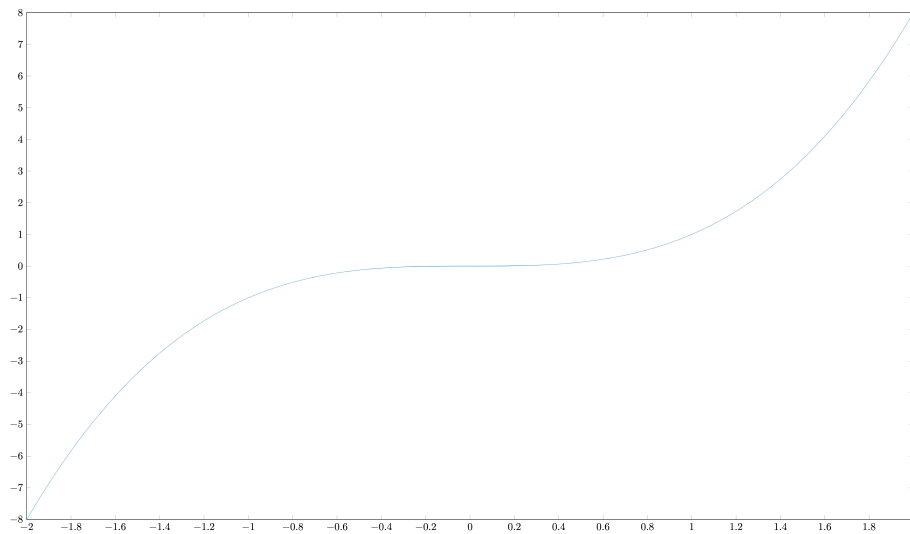
Now the plotting can commence. The `plot` function takes two arguments, the first is the set of coordinates on the horizontal axis and the second is the corresponding set of coordinates on the vertical axis. The `plot` function then plots the first against the second to form a set of points and connects them with lines. In other words, `plot(x,y)` draws points at the coordinates $(x(1), y(1)) = (-2, -8)$, $(x(2), y(2)) = (-1, -1)$, $(x(3), y(3)) = (0, 0)$, etc. and draws a line that connects all these points in the order they appear in.

```
1 >> x=-2:1:2;  
2 >> y=x.^3;  
3 >> plot(x,y)
```



Clearly, 5 points is not enough to plot a function accurately, so the domain vector x must be made finer by choosing smaller increments by saying something like `x=-2:0.1:2` (in this case, `x=[-5 -4.9 -4.8 -4.7 ... 4.7 4.8 4.9 5]`). A very convenient way of achieving this is by using the `linspace` function where `linspace(a,b)` forms a vector between `a` and `b` with 100 equally spaced points. If a different mesh is required, then add an extra argument `n` as `linspace(a,b,n)`, this forms a vector between `a` and `b` consisting of `n` equally spaced points. Therefore, the range of x values can be refined as `x=linspace(-2,2)`.

```
1 >> x=linspace(-2,2);
2 >> y=x.^3;
3 >> plot(x,y)
```



Notice that the semicolons are placed since the output does not need to be seen and it is therefore suppressed, otherwise MATLAB will output all 100 terms of `x` and `y` which not necessary.

D.2 Line Properties

The `plot` function has many additional options that can change the plotting colour, shape, style, line widths and many more (these can be referred to by simply typing `help plot` into the command window). Some of these options can be incorporated into a plot by adding them into the `plot` function itself as additional inputs as `plot(x,y,'Color','r','LineStyle','-','LineWidth',2)`.

Some of the available colours are:

Colour	'Color' Syntax
red	'r'
blue	'b'
green	'g'
cyan	'c'
magenta	'm'
yellow	'y'
black	'k'
white	'w'

Some of the available line styles are:

Line Style	'LineStyle' Syntax
Solid	'-'
Dashed	'--'
Dotted	'.'
Chain	'-.'

The colours and line styles can be combined into one, so if a blue solid line is needed, then it can simply be done by using `'-b'` and the plotting command will be `plot(x,y,'-b')`.

D.3 Multiple Plots

It would stand to reason that if two different functions are to be plotted on the same figure space, say $y = x^2$ as a red solid line and $z = x^3$ as a blue dashed line for $x \in [-5, 5]$, then the following commands can be executed:

```
1 >> x=linspace(-5,5);
2 >> y=x.^2;
3 >> z=x.^3;
4 >> plot(x,y,'-r')
5 >> plot(x,z,'--b')
```

Unfortunately, MATLAB has a habit of overwriting plots every time the `plot` command is used, so in this case MATLAB would plot the graph of `y` then remove it and plot the graph of `z`. In order to avoid that, typing `hold on` before any `plot` command allows plotting more than one plot in the same figure space as well allowing some augmentation. This can be reverted by `hold off`.

```
1 >> hold on
2 >> x=linspace(-5,5);
3 >> y=x.^2;
4 >> z=x.^3;
5 >> plot(x,y,'-r')
6 >> plot(x,z,'--b')
7 >> hold off
```

D.3.1 Legends

When there is more than one line plotted in the same figure space, it is useful to have a legend to distinguish between the different plots. So if the functions `y` and `z` are plotted as above, then a legend can be added that labels them by simply using `legend('Function y','Function z')`. This labels the first plot with `Function y` and the second with `Function z`. Remember,

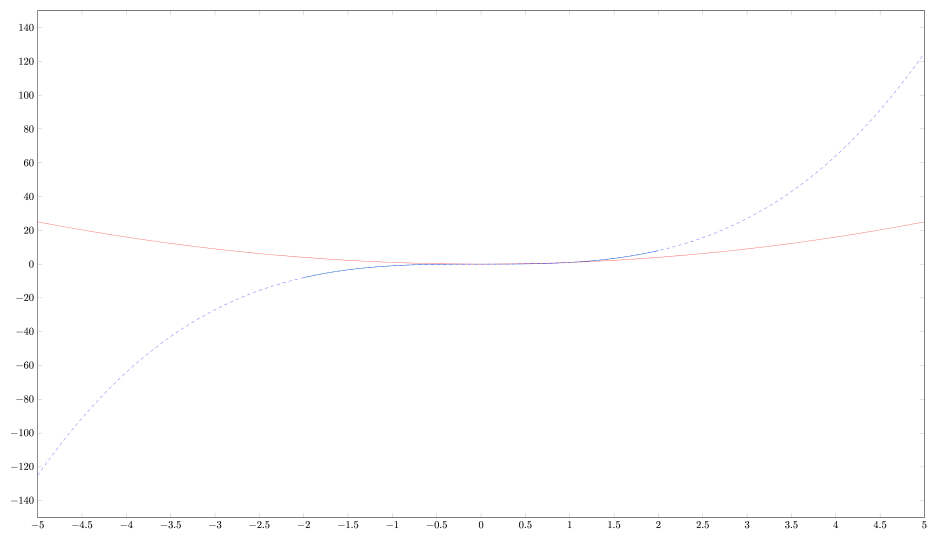
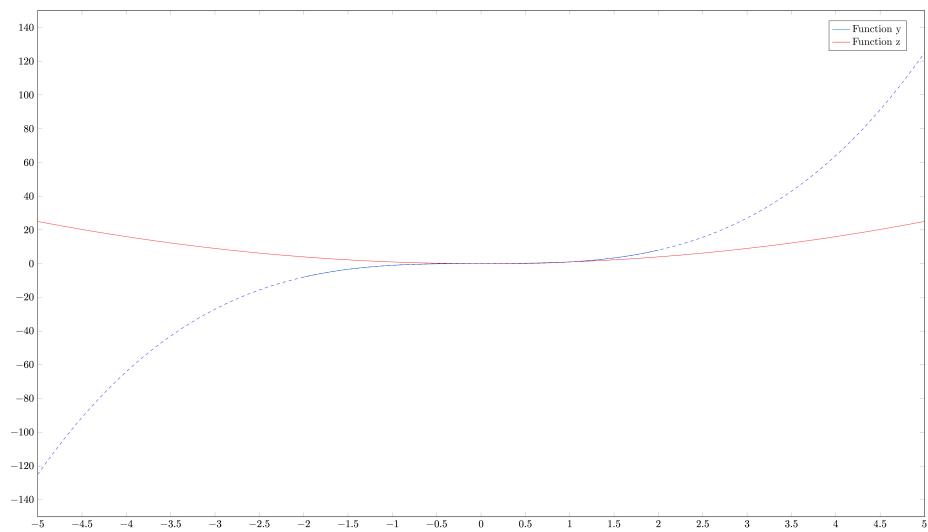


Figure D.1

quotation marks need to be inserted so they are displayed verbatim, otherwise MATLAB will produce an error since there are no variable with the names `Function y` or `Function z`.



D.4 Figure Properties

Some useful figure functions are:

- `clf`: Clears the figure space.
- `figure`: Opens a new figure window.
- `figure(n)`: Goes to figure window number n (and creates one if it is not open to begin with) and plots within that window.

The figures themselves can be augmented by introducing titles, grid lines and labelling the x - and y -axes, all these can be achieved as long as the `hold on` command is active:

- Title: `title('Put title here')`, the title must be in quotation marks.
- Grid: `grid on` and `grid off`.
- x -axis: `xlabel('Label for x axis')`.
- y -axis: `ylabel('Label for y axis')`.

MATLAB usually adjusts the axes so that the graphs fit but sometimes, the axes need to be readjusted according to the user's preference, this can be done by using `axis([left right down up])` where `left` is the leftmost point, `right` is the rightmost point, etc.

D.5 Subplots

Plotting multiple functions is very useful only if the axes can be maintained but if they are different, then the information can be quite distorted when interpreted graphically. In this case, subplots can be used to display more than one plot on the same figure space but on different sections. The command `subplot(a,b,n)` generates a grid of size $a \times b$ (a rows and b columns) and starts plotting in the n^{th} location where the top left is 1 and continues across the rows.

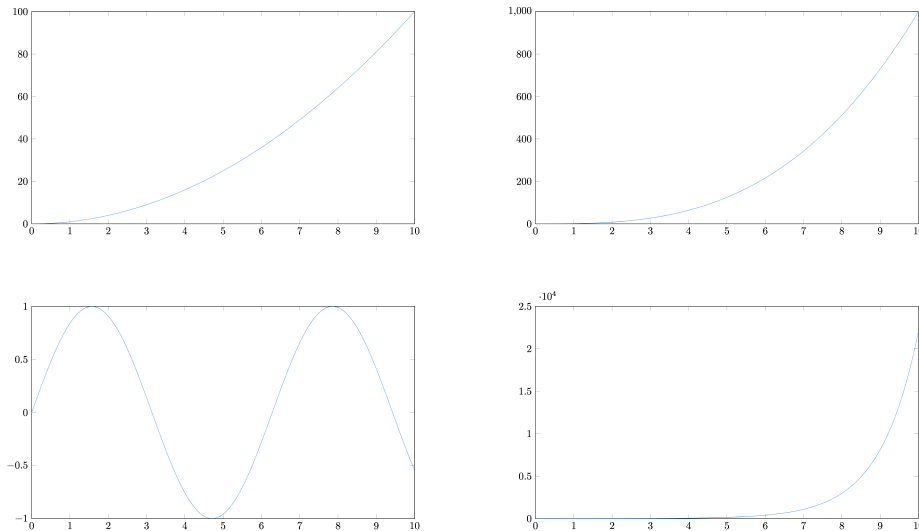
Suppose that for $x \in [0, 10]$, four functions are to be plotted: $y = x^2$ on the top left, $z = x^3$ on the top right, $w = \sin(x)$ on the bottom left and $u = e^x$ on the bottom right. This means that a 2×2 grid is needed so the first two terms in `subplot` are 2. The function y has to be plotted after `subplot(2,2,1)` while z is to be plotted after `subplot(2,2,2)` and so on.

```
1 >> x=linspace(0,10);
2 >> y=x.^2;
3 >> z=x.^3;
4 >> w=sin(x);
5 >> u=exp(x);
6 >> subplot(2,2,1)
7 >> plot(x,y)
8 >> subplot(2,2,2)
9 >> plot(x,z)
10 >> subplot(2,2,3)
```

```

11 >> plot(x,w)
12 >> subplot(2,2,4)
13 >> plot(x,u)

```



One issue in this case is that all the subplots will behave independently, so turning on the grid in one subplot will not do the same for all the rest. Therefore, operations such as `grid on` and `hold on` need to be done for each of the subplots individually.

D.6 Aesthetics

Fonts in figures can usually be an issue since the default setting may not be to the user's liking. As seen in the figures above, the font on the axes is quite small which could make it difficult to read especially if the plots are to be in a report or dissertation. In that case, a special command needs to be run after `hold on` and before any plotting can commence. The command `set(gca, 'FontSize', 20, 'FontName', 'Times')` sets the fontsize to 20 and the font to Times New Roman globally on all axes, legends and titles.

On MATLAB, the mathematical symbols will be displayed as regular text instead of mathematical symbols (like “x” instead of “ x ”). This can be adjusted by using LaTeX syntax by using dollar signs around the mathematical symbols. For example, the x - and y -axes can be labelled with “ x ” and “ y ” by using `xlabel('$x$', 'Interpreter', 'Latex')` and `ylabel('$y$', 'Interpreter', 'Latex')`. The same can be done in the title as `title('Plot of  $x$  Against  $y$ ', 'Interpreter', 'Latex')`.

The legend entries need slightly more work; if two functions y and z are plotted, then they can be labelled in maths typesetting by first defining legend in terms of a placeholder

variable as `Leg=legend('Function $y$', 'Function $z$')` then prescribing the interpreter as `set(Legend, 'Interpreter')`. MATLAB usually places the legend on the top right corner by default but this can be modified by the 'Location' argument and change it to `East`, `West`, `NorthEast`, `SouthWest` and so on, meaning that the new prescription for the legend would be `set(Legend, 'Interpreter', 'Location', 'SouthWest')`.

Remember, this modification of font shapes, sizes and the different styles is only for aesthetic reasons and serves no purpose otherwise.

Lots of Plots

Suppose that the following need to be plotted:

1. The function $x(t) = \cos(t)$ for $t \in [0, 10]$ as a blue solid line of thickness 1.
2. The function $y(t) = e^{0.2t}$ for $t \in [0, 10]$ as a red chain of thickness 2.
3. The function $z(t) = e^{\sin(t)}$ for $t \in [0, 10]$ as a black dashed line of thickness 3.
4. The legend appears in the bottom right corner and labels $x(t)$ as “cos(t)”, $y(t)$ as “Function $y(t)$ ” and $z(t)$ as “Last”.
5. The title of the figure should be “Some Random Functions”.
6. The horizontal axis labelled as “ t ”.
7. The vertical axis labelled as “Functions”.
8. The horizontal axis ranges from 0 to 10 and the vertical axis ranges from -2 to 8.
9. Axis lines are drawn to represent the horizontal and vertical axes.

Each of these can be executed separately by the following commands:

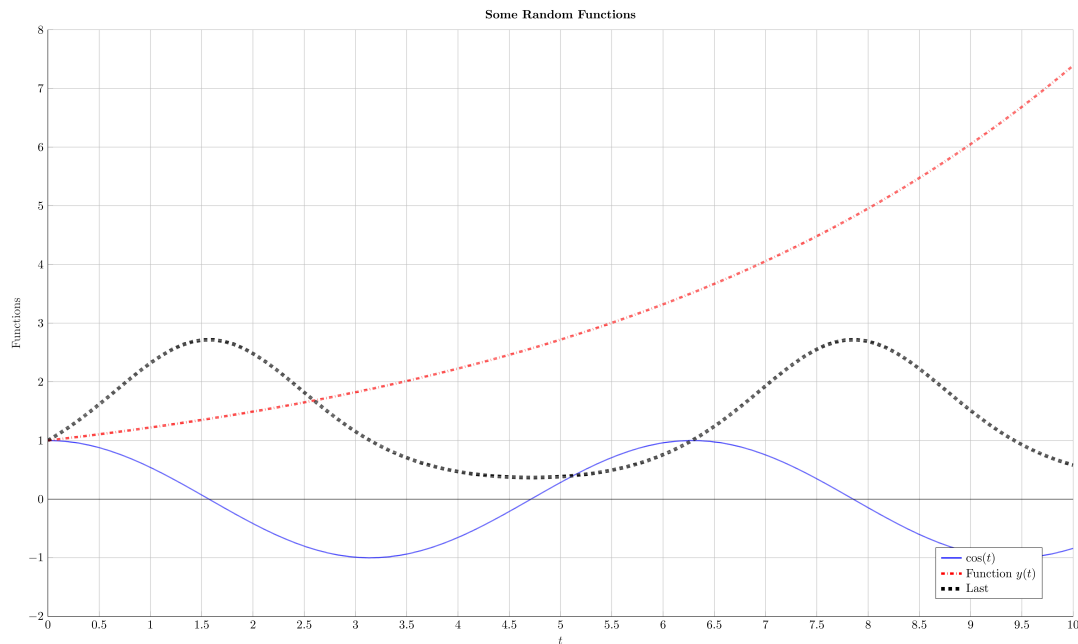
```
0. t=linspace(0,10);
1. x=cos(t); plot(t,x,'-b','LineWidth',1)
2. y=exp(0.2*t); plot(t,y,'-.r','LineWidth',2)
3. z=exp(sin(t)); plot(t,z,'--k','LineWidth',3)
4. Leg=legend('$\cos(t)$','Function $y(t)$','Last');
   set(Leg,'Interpreter','Latex','Location','SouthEast')
5. title('Some Random Functions','Interpreter','Latex')
6. xlabel('$t$','Interpreter','Latex')
7. ylabel('Functions','Interpreter','Latex')
8. axis([0 10 -2 8])
9. plot([0 10],[0 0],'-k'); plot([0 0],[-2 8],'-k')
```

A MATLAB script can be written to execute all these in order:

```

1  clf                                % Clears the figure before plotting
2
3  hold on                            % Allows more than one plot in the same figure
4
5  grid on                            % Produces a grid
6
7  set(gca,'FontSize',20,'FontName','Times') % Sets the font globally
8
9  t=linspace(0,10); % Horizontal axis values
10
11 x=cos(t); % Vector of values for the x function
12 y=exp(0.2*t); % Vector of values for the y function
13 z=exp(sin(t)); % Vector of values for the z function
14
15 plot(t,x,'-b','LineWidth',1) % Plots t against x
16 plot(t,y,'-.r','LineWidth',2) % Plots t against y
17 plot(t,z,'--k','LineWidth',3) % Plots t against z
18
19 title('Some Random Functions','Interpreter','Latex') % Title
20
21 xlabel('$t$','Interpreter','Latex') % Horizontal axis label
22 ylabel('Functions','Interpreter','Latex') % Vertical axis label
23
24 axis([0 10 -2 8]) % Sets the axes
25 plot([0 10],[0 0],'-k') % Plots the horizontal axis
26 plot([0 0],[-2 8],'-k') % Plots the vertical axis
27
28 Leg=legend('$\cos(t)$','Function $y(t)$','Last'); % Sets the legend
29
30 set(Leg,'Interpreter','Latex','Location','SouthEast'); % Sets the font, interpreter and

```



All these commands can be executed in the command window rather than writing them in a script but if a mistake is made, then it cannot be undone and the entire stream of commands needs to be redone once again. Using a script on the other hand will allow for easy alteration.

D.7 Discrete Plots

The `plot` function does not just plot functions, all it needs are two vectors of the same length and it can plot them against one another. So if the graph is to be plotted as a series of points (discrete plot) rather than coordinates connected with a line, then the change in the `plot` function is quite straight forward, simply replace `'LineStyle'` with `'MarkerStyle'` and `'LineWidth'` with `'MarkerSize'`. This will use discrete points rather than connecting them with lines. The different marker styles are:

Marker Style	'MarkerStyle' Syntax
Dot ·	'.'
Cross ×	'x'
Asterisk *	'*'
Circle ○	'o'
Crosshair +	'+'
Square □	's'
Diamond ◇	'd'
Pentagram ★	'p'
Upward Triangle △	'^'

Marker Style	'MarkerStyle' Syntax
Downward Triangle ∇	'v'
Rightward Triangle $\triangleright$	'>'
Leftward Triangle $\triangleleft$	'<'

The colours work in the same way. These discrete plots can be combined with the line plot all in one command, for example, to plot a function with a red dashed line connecting circles, the plot command will be `plot(x,y,'--or')`.

Collatz Conjecture Plot

Consider to the Collatz conjecture from Section C.2, suppose that the number of steps it takes to reach 1 is to be plotted against the starting values, say from 1 to N where N will be the input. This will require the use of many of the tools developed so far. First of all, a function that takes in a starting value and outputs the number of steps is needed, which that has already been done in the code `Collatz`. Since the inputs will be all the numbers from 1 to N , a `for` loop will be suitable for the job. Finally, the `plot` function with markers will be employed since connecting the points with lines will not make sense in this particular context.

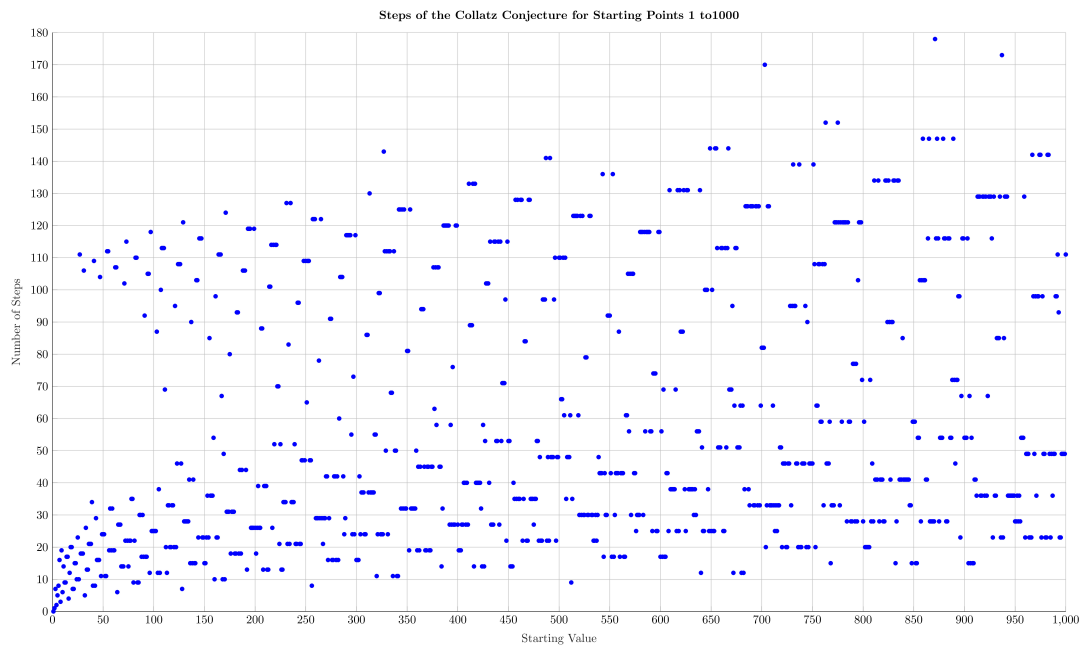
In order for the `plot` function to work, it needs two vectors of the same length. For this particular example, the first vector is the list of numbers from 1 to N , which will be labelled `X` and will be on the x -axis, and the second is the vector of the number of steps for a starting value to decrease to 1 and this is labelled `Y`. The terms in the vector `Y` will have to be calculated individually by using the `Collatz` function. Of course, since the size of `Y` is the same size as `X`, it can be initialised by using `Y=zeros(size(X))`, the terms can then be substituted after they have been calculated. The code to execute this plotting procedure is as follows:

```

1 function Plot_Collatz(N)
2
3 X=1:1:N;           % List of starting values from 1 to N
4
5 Y=zeros(size(X));  % Initialise the vector Y
6
7 for i=X
8
9     [y]=Collatz(i);    % Run the Collatz algorithm for the starting
10                        % value i
11
12     Y(i)=y;           % Record the the number of steps in the i-th
13                        % element of the vector Y
14
15 end
16
17 clf
18 hold on
19 grid on
20 set(gca,'FontSize',20,'FontName','Times')
21
22 plot(X,Y,'.b','MarkerSize',10)
23
24 title(strcat('Steps of the Collatz Conjecture for Starting Points 1 to',' ',num2str(N)),
25
26 xlabel('Starting Value','Interpreter','Latex')
27 ylabel('Number of Steps','Interpreter','Latex')
28
29 end

```

The code can now be run in the command window using `Plot_Collatz(1000)` will give the following plot:



There are a few things that need to be observed in the above code:

- In Line 4, the `for` loop starts with `i=X`, this means that the values of `i` would run through all the values of the vector `X` in order. So the `for` loop does not need to take terms from a uniform set but it can be from any set of values and those will be taken in the order they appear.
- Line 6 runs the `Collatz` function for the input value `i` to produce a value `y` and this is then recorded in the vector `Y` in the i^{th} location in Line 8, hence `Y(i)=y`. Of course there will be no issues there since the size of `Y` is known and has already been initialised in Line 3 as a vector of zeros of the same size as `X`, the values are then replaced by the desired terms.
- Notice that here, the main function `Plot_Collatz` (also known as the *top level function*) refers to another function, namely `Collatz`. This code should be saved as a separate `.m` file and *has to be in the same directory* as `Plot_Collatz`, otherwise the code will not work. An alternative would be to put the `Collatz` function after the end of `Plot_collatz`.

```

1 function Plot_Collatz(N)
2
3     Body of Plot_Collatz
4
5 end
6
7 function [n]=Collatz(a)
8
9     Body of Collatz
10
11 end

```

- The Collatz function requires a single input, but in some cases, there could be many inputs and many outputs, in that case when calling the function, the sequence of inputs and outputs *must be in exactly the same order* as it appears in the function itself.

D.8 Plot Cheat Sheet

MATLAB Command	Purpose
<code>clf</code>	Clear figure space
<code>figure</code>	Opens a new figure space
<code>figure(n)</code>	Plots in figure space <code>n</code>
<code>hold on</code>	Allows more than one plot to be drawn on the same figure
<code>hold off</code>	Cancels <code>hold on</code>
<code>grid on</code>	Turns on the plot grid
<code>grid off</code>	Turns off the plot grid
<code>plot([a,b],[c,d])</code>	Plots a straight line from point (a,c) to (b,d)
<code>set(gca,'FontSize',20)</code>	Sets the global font size to 20
<code>set(gca,'FontName','Times')</code>	Sets the global font to Times
<code>axis([left right down up])</code>	Sets the axes where the <i>x</i> -axis goes from left to right and the <i>y</i> -axis from down to up
<code>title('Plot')</code>	Adds the title “Plot” to the figure
<code>xlabel('x')</code>	Labels the <i>x</i> -axis with “x”
<code>xlabel('\$x\$', 'Interpreter', 'Latex')</code>	Labels the <i>x</i> -axis with “ <i>x</i> ”
<code>Leg=legend('Plot 1','Plot 2',...)</code>	Gives the legend a handle “Leg” for further modification and labels the first plotted line as “Plot 1”, the second as “Plot 2”, etc.
<code>set(Leg,'Interpreter','Latex')</code>	Renders the legend in LaTeX, just like the labels

MATLAB Command	Purpose
<code>x=linspace(a,b)</code>	Generates a vector x with 100 points from a to b
<code>x=linspace(a,b,n)</code>	Generates a vector x with n points from a to b
<code>plot(x,y)</code>	Plots the vector x against the vector y as long as they are of the same size
<code>plot(x,y,'-b')</code>	Plots x against y with a blue line (continuous)
<code>plot(x,y,'-b','LineWidth',2)</code>	Plots x against y with a blue line of thickness 2
<code>plot(x,y,'xk')</code>	Plots x against y with black crosses (discrete)
<code>plot(x,y,'xk','MarkerSize',10)</code>	Plots x against y with black crosses of size 10

E Reading & Writing Data

Reading and writing data files can be important for importing data for analysis on MATLAB and exporting data for further processing elsewhere.

E.1 Writing Into Data Files

Data can be exported from MATLAB into a .dat or .txt file, both of which can be opened with *Notepad*.

Writing Data

Suppose that a list of values of x from 0 to 100 need to be exported along with a corresponding list of x^2 , $\sin(x)$ and e^{-x} as seen here:

x	x^2	$\sin(x)$	e^x
1	1	0.84147	0.36788
2	4	0.90930	0.13534
$\vdots$	$\vdots$	$\vdots$	$\vdots$
99	9801	-0.9992	1.0112×10^{-43}
100	10000	-0.5063	3.7200×10^{-44}

First, define each of these columns.

```
1 >> x=[1:1:100]'; % Column vector of values from 1 to 100
2
3 >> c1=x.^2; % Column of x^2 terms
4
5 >> c2=sin(x); % Column of sin(x) terms
6
7 >> c3=exp(x); % Column of e^x terms
8
9 >> M=[x,c1,c2,c3]; % Form a matrix out of the columns
```

Now that the matrix is ready to be exported, a file needs to be opened with the desired name, say “Data_Write.dat” (.txt would also work). First, the file itself needs to be created in order to write the data into, this can be done by using `file_name=fopen('Data_Write.dat','w')`. The 'w' indicates that MATLAB needs

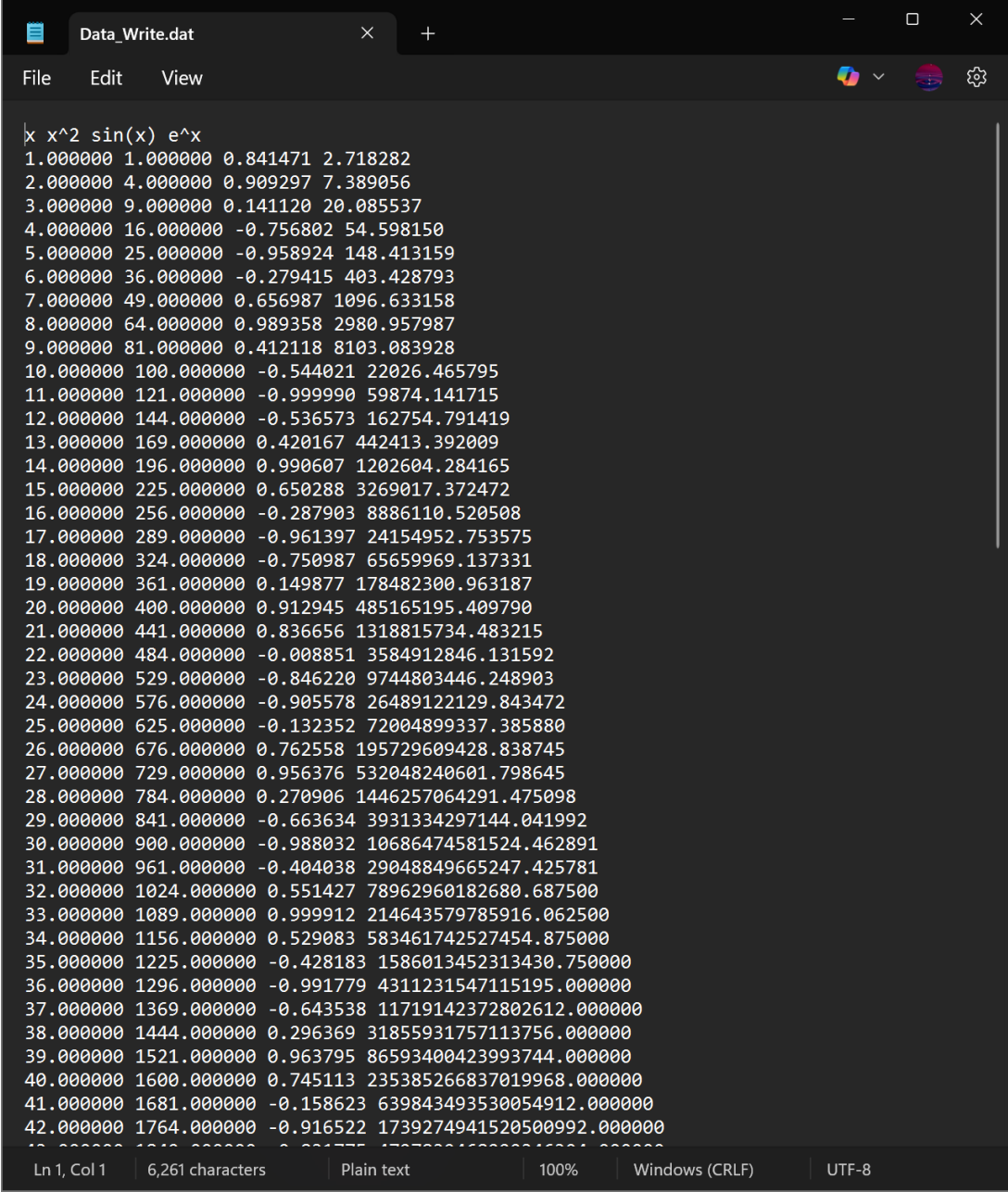
to *write* the data into this file. The data can then be written into the file using the `fprintf` command as `fprintf(file_name,'%f %f %f %f \r\n',M')`

The `%` sign determines the specification of the output and here, `%f` indicates that the output should be in the form of a floating point number. There are four columns so four specifiers need to be declared (hence `%f` appearing four times). The `\r\n` syntax indicates that MATLAB needs to move to the next line, otherwise, all the values will be printed on a single line (`\r\n` needs to be used when opening using Microsoft Notepad, otherwise `\n` would suffice). The matrix is printed as `M'` instead of `M` since Notepad works on the reverse dimensions, so the rows on MATLAB are columns on Notepad and vice versa (for some obscure reason).

After writing all the data, the file needs to be closed so the data is not removed or overwritten using `fclose(file_name)`.

Without context, this data is meaningless so an additional row can be added before writing the data as a title for every column as `fprintf(file_name,'x x^2 sin(x) exp(x) \r\n')`. All these can be combined into the following executable section:

```
1 x=[1:1:100]';          % Column vector of values from 1 to 100
2
3 c1=x.^2;               % Column of x^2 terms
4 c2=sin(x);             % Column of sin(x) terms
5 c3=exp(x);             % Column of e^x terms
6
7 M=[x,c1,c2,c3];        % Form a matrix out of the columns
8
9 my_file=fopen('Data_Write.dat','w'); % Open the file 'Data_Write.dat',
10                                     % also works with 'Data_Write.txt'
11
12 fprintf(my_file,'x x^2 sin(x) e^x \r\n');
13 fprintf(my_file,'%f %f %f %f \r\n',M');
14
15 fclose(my_file);
```



```
x x^2 sin(x) e^x
1.000000 1.000000 0.841471 2.718282
2.000000 4.000000 0.909297 7.389056
3.000000 9.000000 0.141120 20.085537
4.000000 16.000000 -0.756802 54.598150
5.000000 25.000000 -0.958924 148.413159
6.000000 36.000000 -0.279415 403.428793
7.000000 49.000000 0.656987 1096.633158
8.000000 64.000000 0.989358 2980.957987
9.000000 81.000000 0.412118 8103.083928
10.000000 100.000000 -0.544021 22026.465795
11.000000 121.000000 -0.999990 59874.141715
12.000000 144.000000 -0.536573 162754.791419
13.000000 169.000000 0.420167 442413.392009
14.000000 196.000000 0.990607 1202604.284165
15.000000 225.000000 0.650288 3269017.372472
16.000000 256.000000 -0.287903 8886110.520508
17.000000 289.000000 -0.961397 24154952.753575
18.000000 324.000000 -0.750987 65659969.137331
19.000000 361.000000 0.149877 178482300.963187
20.000000 400.000000 0.912945 485165195.409790
21.000000 441.000000 0.836656 1318815734.483215
22.000000 484.000000 -0.008851 3584912846.131592
23.000000 529.000000 -0.846220 9744803446.248903
24.000000 576.000000 -0.905578 26489122129.843472
25.000000 625.000000 -0.132352 72004899337.385880
26.000000 676.000000 0.762558 195729609428.838745
27.000000 729.000000 0.956376 532048240601.798645
28.000000 784.000000 0.270906 1446257064291.475098
29.000000 841.000000 -0.663634 3931334297144.041992
30.000000 900.000000 -0.988032 10686474581524.462891
31.000000 961.000000 -0.404038 29048849665247.425781
32.000000 1024.000000 0.551427 78962960182680.687500
33.000000 1089.000000 0.999912 214643579785916.062500
34.000000 1156.000000 0.529083 583461742527454.875000
35.000000 1225.000000 -0.428183 1586013452313430.750000
36.000000 1296.000000 -0.991779 4311231547115195.000000
37.000000 1369.000000 -0.643538 11719142372802612.000000
38.000000 1444.000000 0.296369 31855931757113756.000000
39.000000 1521.000000 0.963795 86593400423993744.000000
40.000000 1600.000000 0.745113 235385266837019968.000000
41.000000 1681.000000 -0.158623 639843493530054912.000000
42.000000 1764.000000 -0.916522 1739274941520500992.000000
```

E.1.1 Output Formats

When writing data, it is often times important to present the data in a certain form or with certain spacings. For example, $\sin(x)$ is better presented as a floating point and e^x is better presented in scientific notation. These can be done by changing the format after the % sign as follows:

Syntax	Display	Example
<code>%f</code>	Floating point	$0.5 \rightarrow 0.50000$
<code>%e</code>	Scientific notation	$\pi \rightarrow 3.1415e+00$
<code>%g</code>	Floating Point with no trailing 0's	$0.5000 \rightarrow 0.5$
<code>%i</code>	Integer	$\pi \rightarrow 3$

Note

There are many others that print numbers as strings (`%s`) or in hexadecimal notation (`%x`).

E.1.2 Alignment

The way in which the data is spaced out is important since it allows the data to be read more easily. By default, using `%f` will print the data as a floating point with six decimal places, one space will be added before the next item is printed. This can be changed to `%15.10f` which will print the data as a floating point but will dedicate 15 spaces to write the value to 10 decimal places.

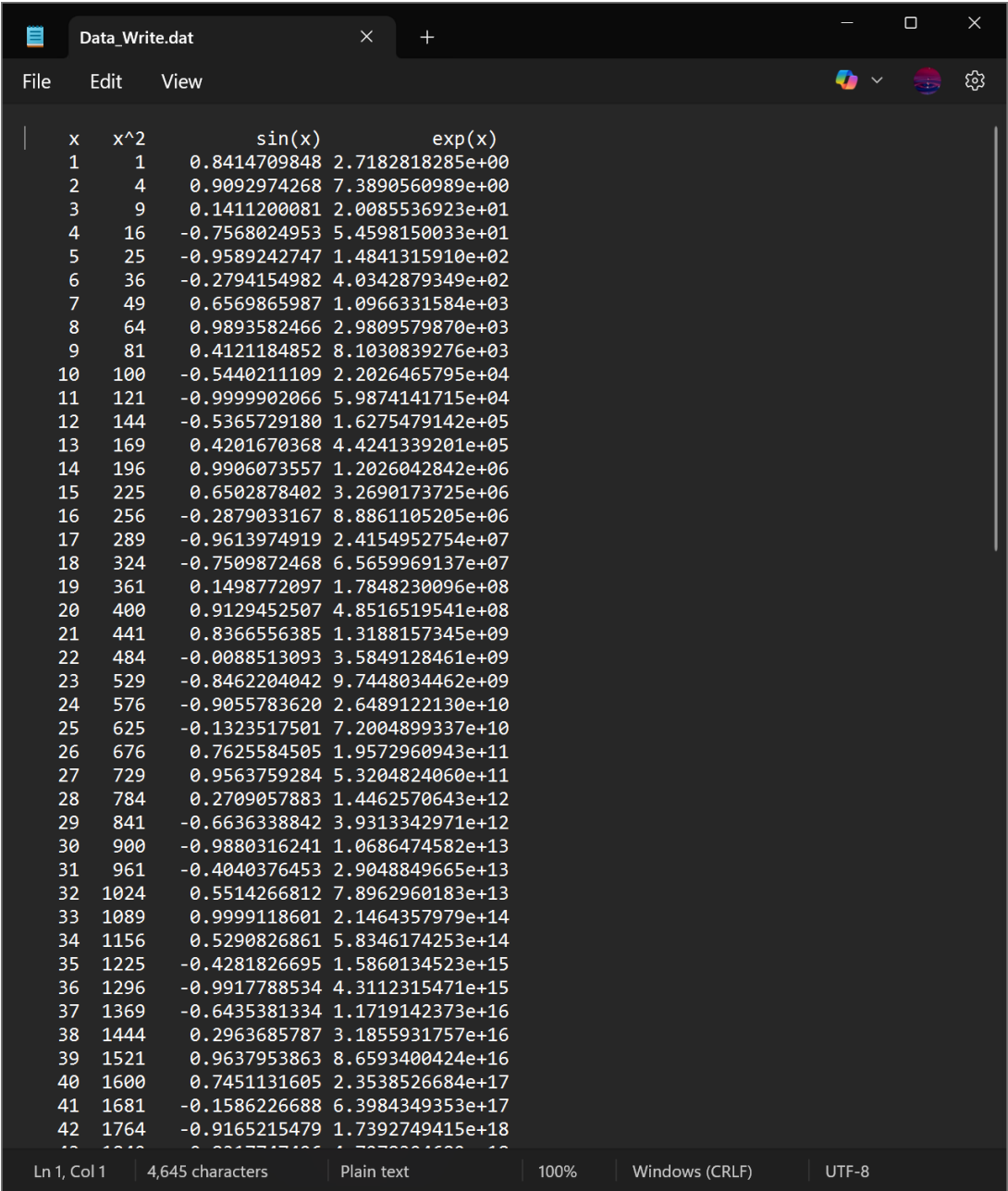
Writing Better Data

The same code can be used as before with the alignment and decimal modifications.

```

1  >> x=[1:1:100]';
2
3  >> c1=x.^2;
4  >> c2=sin(x);
5  >> c3=exp(x);
6
7  >> M=[x,c1,c2,c3];
8
9  >> my_file=fopen('Data_Write.dat','w');
10
11 >> fprintf(my_file,'%5s %5s %15s %15s \r\n','x','x^2','sin(x)','exp(x)');
12
13 >> fprintf(my_file,'%5i %5i %15.10f %15.10e \r\n',M);
14
15 >> fclose(my_file);

```



```
File Edit View
x x^2 sin(x) exp(x)
1 1 0.8414709848 2.7182818285e+00
2 4 0.9092974268 7.3890560989e+00
3 9 0.1411200081 2.0085536923e+01
4 16 -0.7568024953 5.4598150033e+01
5 25 -0.9589242747 1.4841315910e+02
6 36 -0.2794154982 4.0342879349e+02
7 49 0.6569865987 1.0966331584e+03
8 64 0.9893582466 2.9809579870e+03
9 81 0.4121184852 8.1030839276e+03
10 100 -0.5440211109 2.2026465795e+04
11 121 -0.9999902066 5.9874141715e+04
12 144 -0.5365729180 1.6275479142e+05
13 169 0.4201670368 4.4241339201e+05
14 196 0.9906073557 1.2026042842e+06
15 225 0.6502878402 3.2690173725e+06
16 256 -0.2879033167 8.8861105205e+06
17 289 -0.9613974919 2.4154952754e+07
18 324 -0.7509872468 6.5659969137e+07
19 361 0.1498772097 1.7848230096e+08
20 400 0.9129452507 4.8516519541e+08
21 441 0.8366556385 1.3188157345e+09
22 484 -0.0088513093 3.5849128461e+09
23 529 -0.8462204042 9.7448034462e+09
24 576 -0.9055783620 2.6489122130e+10
25 625 -0.1323517501 7.2004899337e+10
26 676 0.7625584505 1.9572960943e+11
27 729 0.9563759284 5.3204824060e+11
28 784 0.2709057883 1.4462570643e+12
29 841 -0.6636338842 3.9313342971e+12
30 900 -0.9880316241 1.0686474582e+13
31 961 -0.4040376453 2.9048849665e+13
32 1024 0.5514266812 7.8962960183e+13
33 1089 0.9999118601 2.1464357979e+14
34 1156 0.5290826861 5.8346174253e+14
35 1225 -0.4281826695 1.5860134523e+15
36 1296 -0.9917788534 4.3112315471e+15
37 1369 -0.6435381334 1.1719142373e+16
38 1444 0.2963685787 3.1855931757e+16
39 1521 0.9637953863 8.6593400424e+16
40 1600 0.7451131605 2.3538526684e+17
41 1681 -0.1586226688 6.3984349353e+17
42 1764 -0.9165215479 1.7392749415e+18
```

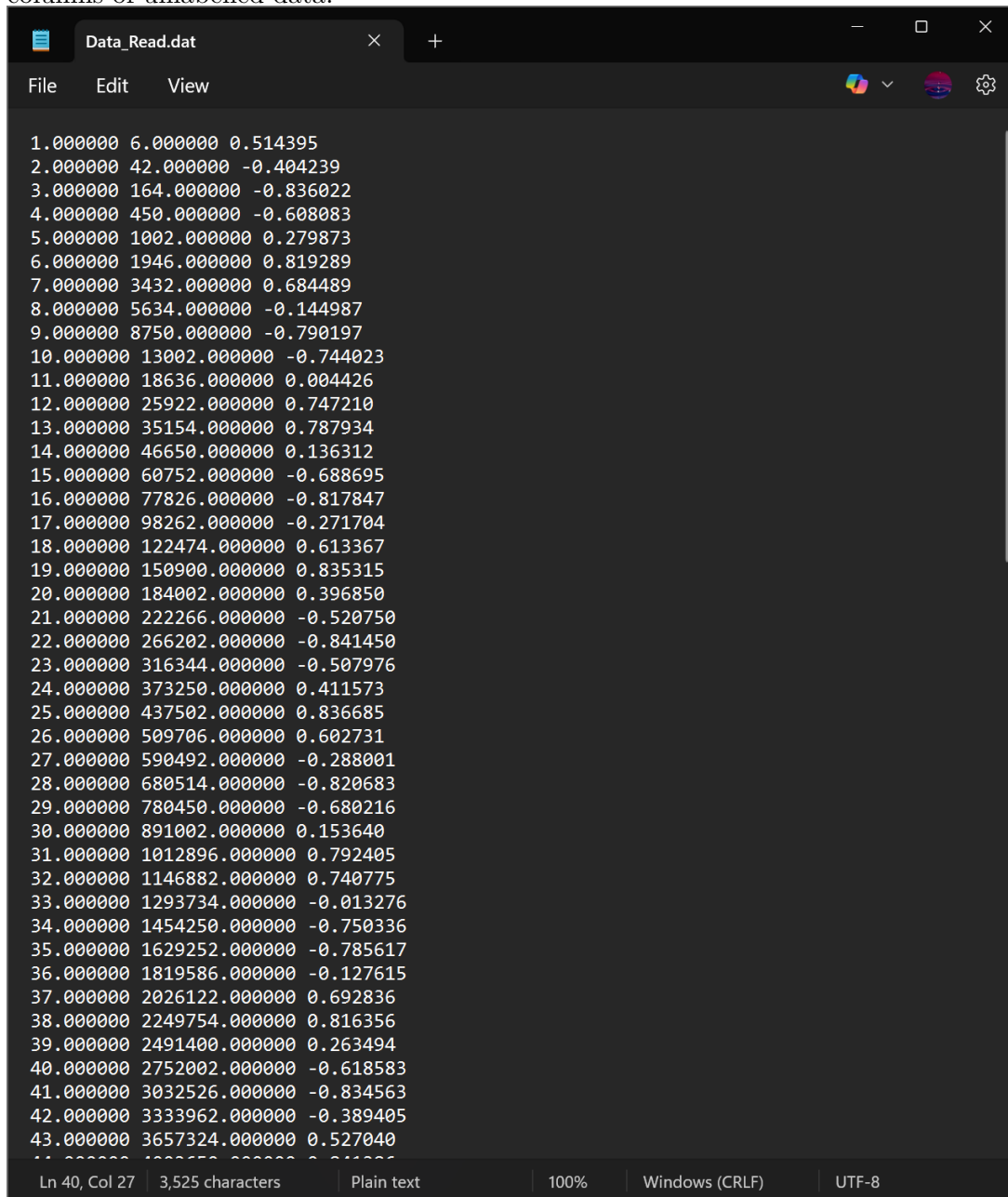
Ln 1, Col 1 | 4,645 characters | Plain text | 100% | Windows (CRLF) | UTF-8

E.2 Reading From Data Files

Reading data from a .dat or .txt files is similar to writing.

Reading Data

Suppose that there is a data file called “Data_Read.dat” (or .txt) that has three columns of unlabelled data.



```
Data_Read.dat
File Edit View
1.000000 6.000000 0.514395
2.000000 42.000000 -0.404239
3.000000 164.000000 -0.836022
4.000000 450.000000 -0.608083
5.000000 1002.000000 0.279873
6.000000 1946.000000 0.819289
7.000000 3432.000000 0.684489
8.000000 5634.000000 -0.144987
9.000000 8750.000000 -0.790197
10.000000 13002.000000 -0.744023
11.000000 18636.000000 0.004426
12.000000 25922.000000 0.747210
13.000000 35154.000000 0.787934
14.000000 46650.000000 0.136312
15.000000 60752.000000 -0.688695
16.000000 77826.000000 -0.817847
17.000000 98262.000000 -0.271704
18.000000 122474.000000 0.613367
19.000000 150900.000000 0.835315
20.000000 184002.000000 0.396850
21.000000 222266.000000 -0.520750
22.000000 266202.000000 -0.841450
23.000000 316344.000000 -0.507976
24.000000 373250.000000 0.411573
25.000000 437502.000000 0.836685
26.000000 509706.000000 0.602731
27.000000 590492.000000 -0.288001
28.000000 680514.000000 -0.820683
29.000000 780450.000000 -0.680216
30.000000 891002.000000 0.153640
31.000000 1012896.000000 0.792405
32.000000 1146882.000000 0.740775
33.000000 1293734.000000 -0.013276
34.000000 1454250.000000 -0.750336
35.000000 1629252.000000 -0.785617
36.000000 1819586.000000 -0.127615
37.000000 2026122.000000 0.692836
38.000000 2249754.000000 0.816356
39.000000 2491400.000000 0.263494
40.000000 2752002.000000 -0.618583
41.000000 3032526.000000 -0.834563
42.000000 3333962.000000 -0.389405
43.000000 3657324.000000 0.527040
Ln 40, Col 27 | 3,525 characters | Plain text | 100% | Windows (CRLF) | UTF-8
```

First, the file needs to be opened with `fopen` but in order to prepare it for reading, use the augmentation `'r'` (instead of `'w'` for writing). The format has to be specified, in this case, it would be `'%f %f %f'` since there are three terms that need to be read which are all placed into a row and separated by a space. The size of the data itself also needs to be specified as well, and since there are three columns, that could be

defined as `[3 Inf]` if the number of rows is unknown. The commands to read the data can be written as follows:

```
1 my_file=fopen('Data_Read.dat','r');
2
3 formatSpec = '%f %f %f';
4
5 Size_Data= [3 Inf];
6
7 M=fscanf(my_file,formatSpec,Size_Data);
8
9 M=M';
10
11 fclose(my_file);
```

This will produce an array `M` that contains all the data.

E.3 Reading & Writing Data with Excel

Writing data into Microsoft Excel is much simpler than `.dat` or `.txt` since spacing and formatting are built into excel. The difference is using `writematrix` and `readmatrix` instead of `fprintf` and `fscanf` and the file extension should be `.xlsx` and does not have to be opened and closed.

Reading & Writing with Excel

Suppose the data as before needs to be written into Excel, this can be done as follows:

```
1 x=[1:1:100]';
2
3 c1=x.^2;
4
5 c2=sin(x);
6
7 c3=exp(x);
8
9 M=[x,c1,c2,c3];
10
11 writematrix(M,'Data_Excel.xlsx');
```

	A	B	C	D	E	F	G	H	I	J
1	1	1	0.841470985	2.718281828						
2	2	4	0.909297427	7.389056099						
3	3	9	0.141120008	20.08553692						
4	4	16	-0.756802495	54.59815003						
5	5	25	-0.958924275	148.4131591						
6	6	36	-0.279415498	403.4287935						
7	7	49	0.656986599	1096.633158						
8	8	64	0.989358247	2980.957987						
9	9	81	0.412118485	8103.083928						
10	10	100	-0.544021111	22026.46579						
11	11	121	-0.999990207	59874.14172						
12	12	144	-0.536572918	162754.7914						
13	13	169	0.420167037	442413.392						
14	14	196	0.990607356	1202604.284						
15	15	225	0.65028784	3269017.372						
16	16	256	-0.287903317	8886110.521						
17	17	289	-0.961397492	24154952.75						
18	18	324	-0.750987247	65659969.14						
19	19	361	0.14987721	178482301						
20	20	400	0.912945251	485165195.4						
21	21	441	0.836655639	1318815734						
22	22	484	-0.008851309	3584912846						
23	23	529	-0.846220404	9744803446						
24	24	576	-0.905578362	26489122130						
25	25	625	-0.13235175	72004899337						
26	26	676	0.76255845	1.9573E+11						
27	27	729	0.956375928	5.32048E+11						
28	28	784	0.270905788	1.44626E+12						
29	29	841	-0.663633884	3.93133E+12						
30	30	900	-0.888031624	1.06865E+13						

The same data can be read using `Data=readmatrix('Data_Excel.xlsx')`.

F Gaussian Elimination Method

The **Gaussian Elimination Method** is an algorithm that transforms the linear system $A\mathbf{x} = \mathbf{b}$ where $A \in \mathbb{C}^{N \times N}$ and $\mathbf{b} \in \mathbb{C}^N$ into an equivalent upper triangular system $U\mathbf{x} = \mathbf{g}$ after $N - 1$ steps, where $U \in \mathbb{C}^{N \times N}$ is an upper triangular matrix and $\mathbf{g} \in \mathbb{C}^N$. This uses **Elementary Row Operations** (swapping rows, multiplying a row by a constant, adding two rows), after which point, the system $U\mathbf{x} = \mathbf{g}$ can be solved by the backward substitution. Note that this method is possible when the elementary row operations are performed on both A and $\mathbf{b}$ simultaneously, so if rows i and j are swapped in A , the rows i and j must also be swapped in $\mathbf{b}$, similarly for the other operations.

The Gaussian elimination method can be performed as follows (the superscripts in brackets will be the step number):

Parallel Example

The algorithm will be explained and an example will be done in parallel to explain the steps with the matrix system $A\mathbf{x} = \mathbf{b}$ where

$$A = \begin{pmatrix} 2 & -1 & 1 \\ -1 & 1 & 2 \\ 1 & 2 & -1 \end{pmatrix} \quad \text{and} \quad \mathbf{b} = \begin{pmatrix} 1 \\ 1 \\ 2 \end{pmatrix}.$$

1. **Establish the starting matrix:** If $a_{11} \neq 0$, then set $A^{(1)} = A$ and $\mathbf{b}^{(1)} = \mathbf{b}$ as

$$A^{(1)} = \begin{pmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1j}^{(1)} & \cdots & a_{1N}^{(1)} \\ a_{21}^{(1)} & a_{22}^{(1)} & \cdots & a_{2j}^{(1)} & \cdots & a_{2N}^{(1)} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{j1}^{(1)} & a_{j2}^{(1)} & \cdots & a_{jj}^{(1)} & \cdots & a_{jN}^{(1)} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{N1}^{(1)} & a_{N2}^{(1)} & \cdots & a_{Nj}^{(1)} & \cdots & a_{NN}^{(1)} \end{pmatrix} \in \mathbb{R}^{N \times N} \quad \text{where} \quad a_{11}^{(1)} \neq 0$$

$$\text{and} \quad \mathbf{b}^{(1)} = \begin{pmatrix} b_1^{(1)} \\ b_2^{(1)} \\ \vdots \\ b_j^{(1)} \\ \vdots \\ b_N^{(1)} \end{pmatrix}.$$

If $a_{11} = 0$, then swap the first row with any other row whose first term is not zero and the result will be the starting matrix $A^{(1)}$.

 $A^{(1)}$

$$A^{(1)} = A = \begin{pmatrix} 2 & -1 & 1 \\ -1 & 1 & 2 \\ 1 & 2 & -1 \end{pmatrix} \quad \text{and} \quad \mathbf{b}^{(1)} = \mathbf{b} = \begin{pmatrix} 1 \\ 1 \\ 2 \end{pmatrix}.$$

2. **Form the multiplier vector:** The desired outcome is to have the matrix A be upper triangular, i.e. all the terms below the diagonal should be 0. To achieve this, introduce a vector $\mathbf{m}_1$ of multipliers, whose i^{th} entry is given by

$$m_{i1} = \frac{a_{i1}^{(1)}}{a_{11}^{(1)}} \quad \text{for all } i = 1, 2, \dots, N,$$

hence the reason why the assumption $a_{11}^{(1)} \neq 0$ must be imposed. Essentially, the vector $\mathbf{m}_1$ is the first column of A divided by the first element of A .

 $\mathbf{m}_1$

$$\mathbf{m}_1 = \frac{1}{a_{11}^{(1)}} \begin{pmatrix} a_{11}^{(1)} \\ a_{21}^{(1)} \\ a_{31}^{(1)} \end{pmatrix} = \begin{pmatrix} 1 \\ -\frac{1}{2} \\ \frac{1}{2} \end{pmatrix}$$

3. **Elimination terms in the first column:** For $j = 2, 3, \dots, N$, multiply row 1 by $-m_{j1}$ and add it to row j to give the new row j :

$$\begin{pmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \dots & a_{1j}^{(1)} & \dots & a_{1N}^{(1)} \\ a_{21}^{(1)} - m_{21}a_{11}^{(1)} & a_{22}^{(1)} - m_{21}a_{12}^{(1)} & \dots & a_{2j}^{(1)} - m_{21}a_{1j}^{(1)} & \dots & a_{2N}^{(1)} - m_{21}a_{1N}^{(1)} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{j1}^{(1)} - m_{j1}a_{11}^{(1)} & a_{j2}^{(1)} - m_{j1}a_{12}^{(1)} & \dots & a_{jj}^{(1)} - m_{j1}a_{1j}^{(1)} & \dots & a_{jN}^{(1)} - m_{j1}a_{1N}^{(1)} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{N1}^{(1)} - m_{N1}a_{11}^{(1)} & a_{N2}^{(1)} - m_{N1}a_{12}^{(1)} & \dots & a_{Nj}^{(1)} - m_{N1}a_{1j}^{(1)} & \dots & a_{NN}^{(1)} - m_{N1}a_{1N}^{(1)} \end{pmatrix}.$$

 Row j Operations

$$\begin{pmatrix} 2 & -1 & 1 \\ -1 & 1 & 2 \\ 1 & 2 & -1 \end{pmatrix}$$

$$\begin{aligned}
& \xrightarrow{r_2 \rightarrow -\left(-\frac{1}{2}\right)r_1 + r_2} \begin{pmatrix} 2 & -1 & 1 \\ -\left(-\frac{1}{2}\right)(2) - 1 & -\left(-\frac{1}{2}\right)(-1) + 1 & -\left(-\frac{1}{2}\right)(1) + 2 \\ 1 & 2 & -1 \end{pmatrix} \\
& = \begin{pmatrix} 2 & -1 & 1 \\ 0 & \frac{1}{2} & \frac{5}{2} \\ 1 & 2 & -1 \end{pmatrix} \\
& \xrightarrow{r_3 \rightarrow -\left(\frac{1}{2}\right)r_1 + r_3} \begin{pmatrix} 2 & -1 & 1 \\ 0 & \frac{1}{2} & \frac{5}{2} \\ -\left(\frac{1}{2}\right)(2) + 1 & -\left(\frac{1}{2}\right)(-1) + 2 & -\left(\frac{1}{2}\right)(1) - 1 \end{pmatrix} \\
& = \begin{pmatrix} 2 & -1 & 1 \\ 0 & \frac{1}{2} & \frac{5}{2} \\ 0 & \frac{3}{2} & -\frac{3}{2} \end{pmatrix}
\end{aligned}$$

Notice that by the definition of m_{j1} , the first element in every row must be equal to 0, therefore, this set of operation makes all the terms in the first column equal to 0 except the first. Define this new matrix as the second term in the iteration:

$$\begin{pmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \dots & a_{1j}^{(1)} & \dots & a_{1n}^{(1)} \\ 0 & a_{22}^{(1)} - m_{21}a_{12}^{(1)} & \dots & a_{2j}^{(1)} - m_{21}a_{1j}^{(1)} & \dots & a_{2n}^{(1)} - m_{21}a_{1n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & a_{j2}^{(1)} - m_{j1}a_{12}^{(1)} & \dots & a_{jj}^{(1)} - m_{j1}a_{1j}^{(1)} & \dots & a_{jn}^{(1)} - m_{j1}a_{1n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & a_{n2}^{(1)} - m_{n1}a_{12}^{(1)} & \dots & a_{nj}^{(1)} - m_{n1}a_{1j}^{(1)} & \dots & a_{nn}^{(1)} - m_{n1}a_{1n}^{(1)} \end{pmatrix} \Rightarrow \begin{pmatrix} a_{11}^{(2)} & a_{12}^{(2)} & \dots & a_{1j}^{(2)} & \dots & a_{1n}^{(2)} \\ a_{21}^{(2)} & a_{22}^{(2)} & \dots & a_{2j}^{(2)} & \dots & a_{2n}^{(2)} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{j1}^{(2)} & a_{j2}^{(2)} & \dots & a_{jj}^{(2)} & \dots & a_{jn}^{(2)} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{n1}^{(2)} & a_{n2}^{(2)} & \dots & a_{nj}^{(2)} & \dots & a_{nn}^{(2)} \end{pmatrix} = A^{(2)}$$

where for all $i, j = 2, 3, \dots, N$

$$a_{11}^{(2)} = a_{11}^{(1)} \quad ; \quad a_{1i}^{(2)} = a_{1i}^{(1)} \quad ; \quad a_{i1}^{(2)} = 0 \quad ; \quad a_{ij}^{(2)} = a_{ij}^{(1)} - m_{i1}a_{1j}^{(1)}$$

🔥 $A^{(2)}$

$$A^{(2)} = \begin{pmatrix} 2 & -1 & 1 \\ 0 & \frac{1}{2} & \frac{5}{2} \\ 0 & \frac{1}{2} & -\frac{3}{2} \end{pmatrix}$$

4. **Modification of the right hand side:** The vector $\mathbf{b}$ has to also undergo the same operations as A , i.e. for $j = 2, \dots, N$, let row j of $\mathbf{b}^{(1)}$ be row 1 multiplied by $-m_{j1}$ plus row j and the final vector is the vector $\mathbf{b}^{(2)}$.

🔥 $\mathbf{b}^{(1)}$

$$\mathbf{b}^{(1)} = \begin{pmatrix} 1 \\ 1 \\ 2 \end{pmatrix} \xrightarrow[r_3 \rightarrow -\left(\frac{1}{2}\right)r_1 + r_3]{r_2 \rightarrow -\left(-\frac{1}{2}\right)r_1 + r_2} \begin{pmatrix} 1 \\ -\left(-\frac{1}{2}\right)(1) + 1 \\ -\left(\frac{1}{2}\right)(1) + 2 \end{pmatrix} = \begin{pmatrix} 1 \\ \frac{3}{2} \\ \frac{3}{2} \end{pmatrix} = \mathbf{b}^{(2)}.$$

5. **Matrix representation of elimination:** This whole procedure can be written as $A^{(2)} = M^{(1)}A^{(1)}$ and $\mathbf{b}^{(2)} = M^{(1)}\mathbf{b}^{(1)}$ where

$$M^{(1)} = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ -m_{21} & 1 & 0 & \dots & 0 \\ -m_{31} & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -m_{n1} & 0 & 0 & \dots & 1 \end{pmatrix}.$$

🔥 $M^{(1)}$

$$M^{(1)} = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ -\frac{1}{2} & 0 & 1 \end{pmatrix}$$

To check:

$$M^{(1)}A^{(1)} = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ -\frac{1}{2} & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & -1 & 1 \\ -1 & 1 & 2 \\ 1 & 2 & -1 \end{pmatrix} = \begin{pmatrix} 2 & -1 & 1 \\ 0 & \frac{1}{2} & \frac{5}{2} \\ 0 & \frac{1}{2} & -\frac{3}{2} \end{pmatrix} = A^{(2)}$$

$$M^{(1)}\mathbf{b}^{(1)} = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ -\frac{1}{2} & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 1 \\ \frac{3}{2} \\ \frac{3}{2} \end{pmatrix} = \mathbf{b}^{(2)}$$

6. **Repeat for other columns:** The process must now be repeated for the rest of the rows, specifically, those that have non-zero pivot points, i.e. the first point in a row

that is non-zero. This process can be done more simply by generating the M matrices in the same way as before without going through the starting steps. This process should be repeated until the last row is reached.

Multiplier Matrices

The matrix $M^{(2)}$ can be generated in the same way as $M^{(1)}$, so

$$M^{(2)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -5 & 1 \end{pmatrix}.$$

To check:

$$M^{(2)}A^{(2)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -5 & 1 \end{pmatrix} \begin{pmatrix} 2 & -1 & 1 \\ 0 & \frac{1}{2} & \frac{5}{2} \\ 0 & \frac{5}{2} & -\frac{3}{2} \end{pmatrix} = \begin{pmatrix} 2 & -1 & 1 \\ 0 & \frac{1}{2} & \frac{5}{2} \\ 0 & 0 & -14 \end{pmatrix} = A^{(3)}$$

$$M^{(2)}\mathbf{b}^{(2)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -5 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ \frac{3}{2} \\ \frac{3}{2} \end{pmatrix} = \begin{pmatrix} 1 \\ \frac{3}{2} \\ -6 \end{pmatrix} = \mathbf{b}^{(3)}$$

7. **Solve using backwards substitution:** After repeating for all other columns (a total of $N - 1$ times), the final matrix $A^{(N)}$ will be an upper triangular matrix with non-zero terms on the diagonal and the system can then be solved by backwards substitution.

Backwards Substitution

$$\begin{aligned} A^{(1)}\mathbf{x} = \mathbf{b}^{(1)} &\implies A^{(2)}\mathbf{x} = \mathbf{b}^{(2)} \implies A^{(3)}\mathbf{x} = \mathbf{b}^{(3)} \\ \implies \begin{pmatrix} 2 & -1 & 1 \\ 0 & \frac{1}{2} & \frac{5}{2} \\ 0 & 0 & -14 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} &= \begin{pmatrix} 1 \\ \frac{3}{2} \\ -6 \end{pmatrix} \implies \begin{aligned} 2x_1 - x_2 + x_3 &= 1 \\ \frac{1}{2}x_2 + \frac{5}{2}x_3 &= \frac{3}{2} \\ -14x_3 &= -6 \end{aligned} \\ &\implies \mathbf{x} = \frac{1}{7} \begin{pmatrix} 5 \\ 6 \\ 3 \end{pmatrix}. \end{aligned}$$

The total number of operations in every step is given in the table below (the “steps” here refer to the matrix manipulation step and not exactly to the step numbers of the algorithm):

Step	Multiplications	Additions	Divisions
1	$(N - 1)^2$	$(N - 1)^2$	$N - 1$
2	$(N - 2)^2$	$(N - 2)^2$	$N - 2$
3	$(N - 3)^2$	$(N - 3)^2$	$N - 3$
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Step	Multiplications	Additions	Divisions
$N - 2$	4	4	2
$N - 1$	1	1	1

This means that the total number of multiplications is

$$1 + 4 + \cdots + (N - 3)^2 + (N - 2)^2 + (N - 1)^2 = \sum_{n=1}^{N-1} n^2 = \frac{N(N - 1)(2N - 1)}{6},$$

similarly for the additions. Whereas the total number of divisions is

$$1 + 2 + \cdots + (N - 3) + (N - 2) + (N - 1) = \sum_{n=1}^{N-1} n = \frac{N(N - 1)}{2}.$$

Therefore the total number of operations is

$$\frac{N(N - 1)(2N - 1)}{6} + \frac{N(N - 1)(2N - 1)}{6} + \frac{N(N - 1)}{2} = \frac{2}{3}N^3 - \frac{1}{2}N^2 - \frac{1}{6}N.$$

This means that for large N , the Gaussian elimination algorithm requires $\mathcal{O}\left(\frac{2}{3}N^3\right)$ operations when A is a non-sparse matrix. This procedure is computationally expensive even for moderate sized matrices, this also assumes that the pivot points are non-zero, or more specifically, that the matrix has non-zero determinant. As an illustration of this computational complexity, if $N = 10^6$ (which not atypical), then for a computer with the computing power of 1 Gigafllops per second, an $N \times N$ system will need 21 years to find a solution. A lot of more modern computational techniques are based on attempting to reduce this computational complexity, either by eliminating terms in some suitable way or chnaging the matrix in a more palatable form.

Overall, every step of this process can be represented by a matrix transformation $M^{(n)}$. This means that in order to convert the matrix A into an upper triangular matrix U , the matrix transformations $M^{(1)}, M^{(2)}, \dots, M^{(N-1)}$ have to be applied reverse order as

$$U = M^{(N-1)}M^{(N-2)} \dots M^{(1)}A.$$

This can be written as

$$U = MA \quad \text{where} \quad M = M^{(N-1)}M^{(N-2)} \dots M^{(1)}. \quad (\text{F.1})$$

Notice that every matrix $M^{(n)}$ is lower triangular and this fact will be used later on in **?@sec-LU**.

G Matrix Decompositions

G.1 Orthogonality & QR Factorisation

Intuitively, the concept of orthogonality is crucial for defining the “amount of information” in a set of vectors; although this is also associated with the concept of linear independence, the “most informative” linearly independent vectors are those that are also orthogonal.

Recall that for a set of vectors $\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_M \in \mathbb{R}^N$ where $M \leq N$, the vectors are **Orthogonal** if $\langle \mathbf{q}_m, \mathbf{q}_n \rangle = 0$ for all $m \neq n$. The set of vectors is called **Orthonormal** if

$$\langle \mathbf{q}_m, \mathbf{q}_n \rangle = \delta_{mn} = \begin{cases} 0 & \text{if } m \neq n \\ 1 & \text{if } m = n. \end{cases}$$

If $N = M$, then the vectors form a linearly independent basis of $\mathbb{R}^N$.

A square matrix Q is called **Orthogonal** if all its columns are orthonormal to one another. Some of the properties of orthogonal matrices are:

- An orthogonal matrix Q satisfies $Q^{-1} = Q^T$, therefore $Q^T Q = Q Q^T = \mathcal{I}$;
- The determinant of an orthogonal matrix is 1 or -1 ;
- The product of two orthogonal matrices is orthogonal.
- Given a matrix $Q_1 \in \mathbb{R}^{M \times K}$ with $K < M$ and with orthonormal columns, there exists a matrix $Q_2 \in \mathbb{R}^{M \times (M-K)}$ such that $Q = [Q_1, Q_2]$ is orthogonal. In other words, for a “tall” rectangular matrix with orthonormal columns, there exist a set of vectors that can be concatenated with the matrix to form an orthogonal square matrix.
- Orthogonal matrices preserve the 2-norm of vectors and matrices. In other words, if $Q \in \mathbb{R}^{N \times N}$ is an orthogonal matrix, then for every $\mathbf{x} \in \mathbb{R}^N$ and $A \in \mathbb{R}^{N \times M}$:

$$\|Q\mathbf{x}\|_2 = \|\mathbf{x}\|_2 \quad ; \quad \|QA\|_2 = \|A\|_2.$$

There are two particularly relevant classes of orthogonal matrices:

- The **Householder Reflection Matrix** (named after Alston Scott Householder) is a reflection matrix on a plane that contains the origin. The reflection matrix is given by

$$P = \mathcal{I} - 2\mathbf{v}\mathbf{v}^T$$

where $\mathbf{v}$ is the unit vector that is normal to the hyperplane in which the reflection has been performed. The matrix P is in fact symmetric and orthogonal (i.e. $P^{-1} = P^T = P$). Reflection transformations appear in many numerical linear algebra algorithms and their main use is to transform a vector $\mathbf{x} \in \mathbb{R}^N$ to another vector $\mathbf{y} \in \mathbb{R}^N$ with the same magnitude (meaning that for given vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^N$ with $\|\mathbf{x}\|_2 = \|\mathbf{y}\|_2$, there exists a reflection matrix P such that $P\mathbf{x} = \mathbf{y}$).

- The ***Givens Rotation Matrix*** (named after James Wallace Givens) represents a rotation in the plane that can be spanned by two vectors. The matrix of transformation is denoted $G(i, j; \theta)$ where the vector $G(i, j; \theta)\mathbf{x}$ is simply the vector $\mathbf{x}$ rotated θ radians anti-clockwise on a plane that is parallel to the (i, j) -plane. The matrix $G(i, j; \theta)$ is essentially an identity matrix with the (i, i) and (j, j) terms replaced by $\cos(\theta)$, the (i, j) term replaced by $\sin(\theta)$ and the (j, i) term replaced by $-\sin(\theta)$. For example, in $\mathbb{R}^5$, the matrix $G(2, 4; \theta)$ is

$$G(2, 4; \theta) = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & \cos(\theta) & 0 & -\sin(\theta) & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & \sin(\theta) & 0 & \cos(\theta) & 0 \\ 1 & 0 & 0 & 0 & 1 \end{pmatrix} \end{matrix}$$



Figure G.1: Photo of (from the left): Jim Wilkinson, Wallace Givens, George Forsythe, Alston Householder, Peter Henrici, Fritz Bauer

Since both reflection and rotation matrices are orthogonal matrix transformations, a sequence of reflections and rotations can be represented by the matrix Q^T (which would also be orthogonal). To this end, any matrix $A \in \mathbb{R}^{M \times N}$ with $M \geq N$ can be transformed by $Q^T \in \mathbb{R}^{M \times M}$ to give a block matrix with an upper triangular matrix occupying the first N rows with $M - N$ zero rows below it, i.e.

$$Q^T A = \begin{bmatrix} R_1 \\ 0 \end{bmatrix}$$

where $R_1 \in \mathbb{R}^{N \times N}$ is an upper triangular square matrix. Equivalently, A can be written as $A = QR$ where Q is the orthogonal transformation matrix and R is a block rectangular matrix consisting of a square lower triangular matrix and a block zero matrix. This type of decomposition is called the **QR Factorisation**. The full QR factorisation can be visually represented as follows:

There is a much more concise form of the QR factorisation where only the first several columns of Q are considered since the rest will be multiplied by 0 anyway, this gives an “economy version” of the QR factorisation written as $A = Q_1 R_1$ which be visually represented as follows:

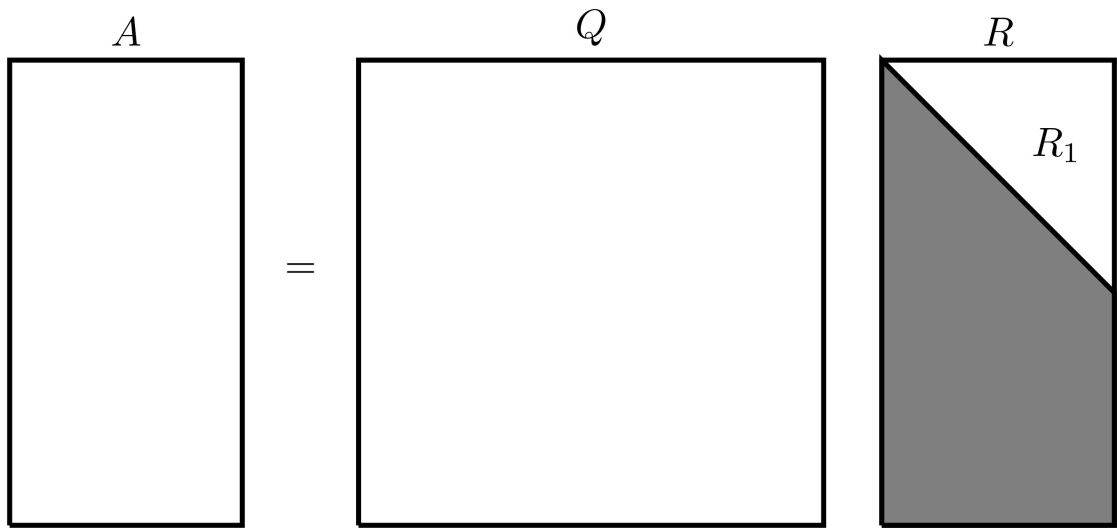


Figure G.2

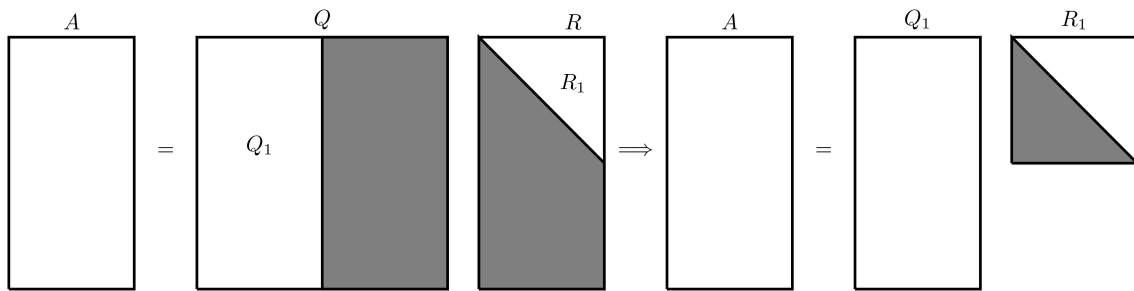


Figure G.3

The QR decomposition of a matrix can be performed on any matrix (square or rectangular). The following sections will show how this can be done using reflections and rotations.

G.1.1 QR Decomposition Using Reflections

The following will explain how the QR decomposition can be performed using reflection matrices on a square matrix $A \in \mathbb{R}^{N \times N}$. Denote the n^{th} column of the matrix A by $\mathbf{a}_n$, this means A can be written as

$$A = \begin{pmatrix} \vdots & \vdots & \vdots \\ \mathbf{a}_1 & \mathbf{a}_2 & \dots & \mathbf{a}_N \\ \vdots & \vdots & \vdots \end{pmatrix}.$$

The vector $\mathbf{e}_n$ will denote the n^{th} canonical basis vector, i.e. the vector with all its entries being equal to 0 except the element in location n which is equal to 1.

Paralell Example

This process will also be applied in parallel to the following matrix

$$A = \begin{pmatrix} -1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & 1 & 1 \end{pmatrix}.$$

In this case,

$$\mathbf{a}_1 = \begin{pmatrix} -1 \\ 1 \\ 1 \end{pmatrix}, \quad \mathbf{a}_2 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \quad \text{and} \quad \mathbf{a}_3 = \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix}.$$

First, find a reflection matrix that transforms the first column of A into $(\alpha, 0, \dots, 0)^T$ where $\alpha = \|\mathbf{a}_1\|_2$. Let $\mathbf{u} = \mathbf{a}_1 - \alpha \mathbf{e}_1$ and $\mathbf{v} = \frac{\mathbf{u}}{\|\mathbf{u}\|_2}$, then the first reflection matrix is

$$P_1 = \mathcal{I} - 2\mathbf{v}\mathbf{v}^T.$$

This can be verified by checking that all the terms in the first column of the matrix $A_2 = P_1 A$ are zero except for the first term.

First Reflection Matrix

The 2-norm of the first column of A is $\alpha = \sqrt{3}$, then

$$\mathbf{u} = \mathbf{a}_1 - \alpha \mathbf{e}_1 = \begin{pmatrix} -1 \\ 1 \\ 1 \end{pmatrix} - \sqrt{3} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} -1 - \sqrt{3} \\ 1 \\ 1 \end{pmatrix}$$

$$\mathbf{v} = \frac{\mathbf{u}}{\|\mathbf{u}\|} = \frac{1}{\sqrt{6 + 2\sqrt{3}}} \begin{pmatrix} -1 - \sqrt{3} \\ 1 \\ 1 \end{pmatrix}.$$

$$\begin{aligned}
P_1 &= \mathcal{I} - 2\mathbf{v}\mathbf{v}^T = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - \frac{2}{6+2\sqrt{3}} \begin{pmatrix} -1-\sqrt{3} \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} -1-\sqrt{3} & 1 & 1 \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - \frac{1}{3+\sqrt{3}} \begin{pmatrix} 4+2\sqrt{3} & -1-\sqrt{3} & -1-\sqrt{3} \\ -1-\sqrt{3} & 1 & 1 \\ -1-\sqrt{3} & 1 & 1 \end{pmatrix} \\
&= \frac{1}{3+\sqrt{3}} \begin{pmatrix} -1-\sqrt{3} & 1+\sqrt{3} & 1+\sqrt{3} \\ 1+\sqrt{3} & 2+\sqrt{3} & -1 \\ 1+\sqrt{3} & -1 & 2+\sqrt{3} \end{pmatrix}
\end{aligned}$$

The matrix P_1 can be simplified to give

$$P_1 = \frac{1}{6} \begin{pmatrix} -2\sqrt{3} & 2\sqrt{3} & 2\sqrt{3} \\ 2\sqrt{3} & 3+\sqrt{3} & -3+\sqrt{3} \\ 2\sqrt{3} & -3+\sqrt{3} & 3+\sqrt{3} \end{pmatrix}$$

To verify that this matrix is valid, consider the product $A_2 = P_1 A$:

$$\begin{aligned}
A_2 = P_1 A &= \frac{1}{6} \begin{pmatrix} -2\sqrt{3} & 2\sqrt{3} & 2\sqrt{3} \\ 2\sqrt{3} & 3+\sqrt{3} & -3+\sqrt{3} \\ 2\sqrt{3} & -3+\sqrt{3} & 3+\sqrt{3} \end{pmatrix} = \begin{pmatrix} -1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & 1 & 1 \end{pmatrix} \\
&= \frac{1}{3} \begin{pmatrix} 3\sqrt{3} & \sqrt{3} & -\sqrt{3} \\ 0 & 2\sqrt{3} & -3+\sqrt{3} \\ 0 & 2\sqrt{3} & 3+\sqrt{3} \end{pmatrix},
\end{aligned}$$

indeed, all the terms in the first column are 0 except for the first.

Repeat the same process for the $(N-1) \times (N-1)$ bottom right submatrix of A_2 then once the new matrix P_2 is obtained (of size $(N-1) \times (N-1)$), place it at the bottom right of the $N \times N$ identity. When this process is repeated a total of $N-1$ times, the result will be an upper triangular matrix.

Second Reflection Matrix

Consider the matrix

$$A_2 = \frac{1}{3} \begin{pmatrix} 3\sqrt{3} & \sqrt{3} & -\sqrt{3} \\ 0 & 2\sqrt{3} & -3+\sqrt{3} \\ 0 & 2\sqrt{3} & 3+\sqrt{3} \end{pmatrix}.$$

Let B be the bottom right 2×2 submatrix of A_2 ,

$$A_2 = \frac{1}{3} \begin{pmatrix} 3\sqrt{3} & \sqrt{3} & -\sqrt{3} \\ 0 & \boxed{\begin{matrix} 2\sqrt{3} & -3 + \sqrt{3} \\ 2\sqrt{3} & 3 + \sqrt{3} \end{matrix}} \\ 0 & & \end{pmatrix} \implies B = \begin{pmatrix} \frac{2\sqrt{3}}{3} & \frac{-3+\sqrt{3}}{3} \\ \frac{2\sqrt{3}}{3} & \frac{3+\sqrt{3}}{3} \end{pmatrix}.$$

Figure G.4

Repeat the same process as before with the matrix B : The 2-norm of the first column of B is $\beta = \frac{2\sqrt{6}}{3}$. Then

$$\begin{aligned} \mathbf{u} &= \mathbf{b}_1 - \beta \mathbf{e}_1 = \begin{pmatrix} \frac{2\sqrt{3}}{3} \\ \frac{2\sqrt{3}}{3} \end{pmatrix} - \frac{2\sqrt{6}}{3} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{2\sqrt{3}}{3} \begin{pmatrix} 1 - \sqrt{2} \\ 1 \end{pmatrix} \\ \mathbf{v} &= \frac{\mathbf{u}}{\|\mathbf{u}\|} = \frac{1}{\sqrt{4 - 2\sqrt{2}}} \begin{pmatrix} 1 - \sqrt{2} \\ 1 \end{pmatrix} \\ \tilde{P}_2 &= \mathcal{I} - 2\mathbf{v}\mathbf{v}^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{2}{4 - 2\sqrt{2}} \begin{pmatrix} 1 - \sqrt{2} \\ 1 \end{pmatrix} \begin{pmatrix} 1 - \sqrt{2} & 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{1}{2 - \sqrt{2}} \begin{pmatrix} 3 - 2\sqrt{2} & 1 - \sqrt{2} \\ 1 - \sqrt{2} & 1 \end{pmatrix} = \begin{pmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \end{pmatrix}. \end{aligned}$$

Consider the product $\tilde{P}_2 B$:

$$\tilde{P}_2 B = \begin{pmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \end{pmatrix} \begin{pmatrix} \frac{2\sqrt{3}}{3} & \frac{-3+\sqrt{3}}{3} \\ \frac{2\sqrt{3}}{3} & \frac{3+\sqrt{3}}{3} \end{pmatrix} = \begin{pmatrix} \frac{2\sqrt{6}}{3} & \frac{\sqrt{6}}{3} \\ 0 & 1 \end{pmatrix}$$

which does change the matrix B into upper triangular form.

Let the matrix P_2 be the identity matrix with the bottom 2×2 submatrix replaced with $\tilde{P}_2$, i.e.

$$P_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0 & \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 & 0 & 0 \\ 0 & \sqrt{2} & \sqrt{2} \\ 0 & \sqrt{2} & -\sqrt{2} \end{pmatrix}.$$

The product $P_2 A_2$ should be lower triangular, indeed

$$P_2 A_2 = \frac{1}{6} \begin{pmatrix} 2 & 0 & 0 \\ 0 & \sqrt{2} & \sqrt{2} \\ 0 & \sqrt{2} & -\sqrt{2} \end{pmatrix} \begin{pmatrix} 3\sqrt{3} & \sqrt{3} & -\sqrt{3} \\ 0 & 2\sqrt{3} & -3 + \sqrt{3} \\ 0 & 2\sqrt{3} & 3 + \sqrt{3} \end{pmatrix} = \frac{1}{6} \begin{pmatrix} 6\sqrt{3} & 2\sqrt{3} & -2\sqrt{3} \\ 0 & 4\sqrt{6} & 2\sqrt{6} \\ 0 & 0 & -6\sqrt{2} \end{pmatrix}.$$

This sequence of steps will generate $N - 1$ reflection matrices denoted $P_1, P_2, \dots, P_{N-1}$ which when applied to A in reverse order (i.e. the product is $P_{N-1} \dots P_2 P_1 A$), must give an upper triangular matrix R . Since P_n are orthogonal for all $n = 1, 2, \dots, N - 1$, then their product will also be orthogonal.

Let $P = P_{N-1} \dots P_2 P_1$, then $R = PA$ meaning that $A = P^{-1}R$. Since P is orthogonal, then $P^{-1} = P^T$ which will be equal to Q in the QR factorisation.

Final QR Decomposition

The matrices in question are

$$P_1 = \frac{1}{6} \begin{pmatrix} -2\sqrt{3} & 2\sqrt{3} & 2\sqrt{3} \\ 2\sqrt{3} & 3 + \sqrt{3} & -3 + \sqrt{3} \\ 2\sqrt{3} & -3 + \sqrt{3} & 3 + \sqrt{3} \end{pmatrix}, \quad P_2 = \frac{1}{2} \begin{pmatrix} 2 & 0 & 0 \\ 0 & \sqrt{2} & \sqrt{2} \\ 0 & \sqrt{2} & -\sqrt{2} \end{pmatrix}$$

The matrix product $P_2 P_1 A$ should give the matrix R which is upper triangular, indeed

$$R = P_2 P_1 A = \frac{1}{6} \begin{pmatrix} 6\sqrt{3} & 2\sqrt{3} & -2\sqrt{3} \\ 0 & 4\sqrt{6} & 2\sqrt{6} \\ 0 & 0 & -6\sqrt{2} \end{pmatrix}.$$

Let

$$\begin{aligned} P = P_2 P_1 &= \frac{1}{12} \begin{pmatrix} 2 & 0 & 0 \\ 0 & \sqrt{2} & \sqrt{2} \\ 0 & \sqrt{2} & -\sqrt{2} \end{pmatrix} \begin{pmatrix} -2\sqrt{3} & 2\sqrt{3} & 2\sqrt{3} \\ 2\sqrt{3} & 3 + \sqrt{3} & -3 + \sqrt{3} \\ 2\sqrt{3} & -3 + \sqrt{3} & 3 + \sqrt{3} \end{pmatrix} \\ &= \begin{pmatrix} -\frac{\sqrt{3}}{3} & \frac{\sqrt{3}}{3} & \frac{\sqrt{3}}{3} \\ \frac{\sqrt{6}}{3} & \frac{\sqrt{6}}{6} & \frac{\sqrt{6}}{6} \\ 0 & \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \end{pmatrix}. \end{aligned}$$

Therefore

$$Q = P^{-1} = P^T = \begin{pmatrix} -\frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{3} & 0 \\ \frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{6} & \frac{\sqrt{2}}{2} \\ \frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{6} & -\frac{\sqrt{2}}{2} \end{pmatrix},$$

hence giving the QR decomposition of A as

$$\underbrace{\begin{pmatrix} -1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & 1 & 1 \end{pmatrix}}_A = \underbrace{\begin{pmatrix} -\frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{3} & 0 \\ \frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{6} & \frac{\sqrt{2}}{2} \\ \frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{6} & -\frac{\sqrt{2}}{2} \end{pmatrix}}_Q \underbrace{\begin{pmatrix} \sqrt{3} & \frac{\sqrt{3}}{3} & -\frac{\sqrt{3}}{3} \\ 0 & \frac{2\sqrt{6}}{3} & \frac{\sqrt{6}}{3} \\ 0 & 0 & -\sqrt{2} \end{pmatrix}}_R$$

G.1.2 QR Decomposition Using Rotations

The following will explain how the QR decomposition can be performed using rotation matrices on a square matrix $A \in \mathbb{R}^{N \times N}$.

Parallel Example

This process will also be applied in parallel to the following matrix

$$A = \begin{pmatrix} -1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & 1 & 1 \end{pmatrix}.$$

The rotation matrices should make all the terms in the lower triangular part of the matrix equal to zero. Starting with the lower left most element a_{N1} , this element can be eliminated by using the rotation matrix $G(1, N; \theta)$ where $\theta = \arctan\left(-\frac{a_{N1}}{a_{11}}\right)$. When applied to A , this should eliminate the term a_{N1} .

First Rotation Matrix

For the matrix

$$A = \begin{pmatrix} -1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & 1 & 1 \end{pmatrix}.$$

The angle θ will be $\theta = \arctan\left(-\frac{a_{31}}{a_{11}}\right) = \arctan(1) = \frac{\pi}{4}$. Therefore the rotation matrix will be

$$G_1 = G\left(1, 3; \frac{\pi}{4}\right) = \begin{pmatrix} \cos\left(\frac{\pi}{4}\right) & 0 & -\sin\left(\frac{\pi}{4}\right) \\ 0 & 1 & 0 \\ \sin\left(\frac{\pi}{4}\right) & 0 & \cos\left(\frac{\pi}{4}\right) \end{pmatrix} = \begin{pmatrix} \frac{\sqrt{2}}{2} & 0 & -\frac{\sqrt{2}}{2} \\ 0 & 1 & 0 \\ \frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \end{pmatrix}.$$

This can be verified by considering the product $A_2 = G_1 A$:

$$A_2 = G_1 A = \begin{pmatrix} \frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \\ 0 & 1 & 0 \\ -\frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \end{pmatrix} \begin{pmatrix} -1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} -\sqrt{2} & 0 & 0 \\ 1 & 1 & -1 \\ 0 & \sqrt{2} & \sqrt{2} \end{pmatrix}$$

which does eliminate a_{31} .

This process can be repeated for all other terms in the lower triangular section to reduce A into an upper triangular matrix. In these cases, to eliminate the element in position (m, n) , the angle $\theta = \arctan\left(-\frac{a_{mn}}{a_{nn}}\right)$ and the rotation matrix is $G(n, m; \theta)$.

Second & Third Rotation Matrices

Repeat the same process as above to the matrix A_2 to eliminate the term in position

(2,1): $\theta_2 = \arctan\left(-\frac{a_{21}}{a_{11}}\right) = \arctan\left(\frac{1}{\sqrt{2}}\right)$ and $G_2 = G(1, 2; \theta_2)$ is

$$G_2 = G(1, 2; \theta_2) = \begin{pmatrix} \cos(\theta_2) & -\sin(\theta_2) & 0 \\ \sin(\theta_2) & \cos(\theta_2) & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} \frac{\sqrt{6}}{3} & -\frac{\sqrt{3}}{3} & 0 \\ \frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{3} & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Applying G_2 to A_2 should eliminate the (2,1) element, indeed

$$A_3 = G_2 A_2 = \begin{pmatrix} -\sqrt{3} & -\frac{\sqrt{3}}{3} & \frac{\sqrt{3}}{3} \\ 0 & \frac{\sqrt{6}}{3} & -\frac{\sqrt{6}}{3} \\ 0 & \sqrt{2} & \sqrt{2} \end{pmatrix}.$$

Finally, the term in position (2,3) needs to be eliminated: $\theta_3 = \arctan\left(-\frac{a_{32}}{a_{22}}\right) = \arctan(\sqrt{3})$ and $G_3 = G(2, 3; \theta_3)$ is

$$G_3 = G(2, 3; \theta_3) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta_3) & -\sin(\theta_3) \\ 0 & \sin(\theta_3) & \cos(\theta_3) \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & \sqrt{3} \\ 0 & -\sqrt{3} & 1 \end{pmatrix}.$$

Applying G_3 to A_3 should eliminate the (3,2) element, indeed

$$G_3 A_3 = \frac{1}{2} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & \sqrt{3} \\ 0 & -\sqrt{3} & 1 \end{pmatrix} \begin{pmatrix} -\sqrt{3} & -\frac{\sqrt{3}}{3} & \frac{\sqrt{3}}{3} \\ 0 & \frac{\sqrt{6}}{3} & -\frac{\sqrt{6}}{3} \\ 0 & \sqrt{2} & \sqrt{2} \end{pmatrix} = \frac{1}{3} \begin{pmatrix} -3\sqrt{3} & -\sqrt{3} & \sqrt{3} \\ 0 & 2\sqrt{6} & \sqrt{6} \\ 0 & 0 & 3\sqrt{2} \end{pmatrix}.$$

This process will generate a sequence of *at most* $\frac{1}{2}N(N-1)$ rotation matrices (since this is the number of terms that need to be eliminated). Suppose that M rotation matrices are needed where $M \in \left\{1, 2, \dots, \frac{1}{2}N(N-1)\right\}$, then when these are applied to A in reverse order (the product $G_M G_{M-1} \dots G_2 G_1 A$), then the result should be the upper triangular matrix R . Let $G = G_M G_{M-1} \dots G_2 G_1$, then $R = GA$. Since all the rotation matrices are orthogonal, then their product must also be orthogonal, therefore if $Q = G^{-1} = G^T$, then $A = QR$, hence giving the QR decomposition of A .

Final QR Decomposition

The matrices in question are

$$G_1 = \begin{pmatrix} \frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \\ 0 & 1 & 0 \\ -\frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \end{pmatrix}, \quad G_2 = \begin{pmatrix} \frac{\sqrt{6}}{3} & -\frac{\sqrt{3}}{3} & 0 \\ \frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{3} & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad \text{and} \quad G_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{2} & \frac{\sqrt{3}}{2} \\ 0 & -\frac{\sqrt{3}}{2} & \frac{1}{2} \end{pmatrix}.$$

The product of the rotation matrices is

$$G = G_3 G_2 G_1 = \begin{pmatrix} \frac{\sqrt{3}}{3} & -\frac{\sqrt{3}}{3} & -\frac{\sqrt{3}}{3} \\ \frac{\sqrt{6}}{3} & \frac{\sqrt{6}}{6} & \frac{\sqrt{6}}{6} \\ 0 & \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \end{pmatrix}.$$

Therefore

$$Q = G^{-1} = G^T = \begin{pmatrix} \frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{3} & 0 \\ -\frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{6} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{6} & -\frac{\sqrt{2}}{2} \end{pmatrix}$$

hence giving the QR decomposition of A as

$$\underbrace{\begin{pmatrix} -1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & 1 & 1 \end{pmatrix}}_A = \underbrace{\begin{pmatrix} \frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{3} & 0 \\ -\frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{6} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{3}}{3} & \frac{\sqrt{6}}{6} & -\frac{\sqrt{2}}{2} \end{pmatrix}}_Q \underbrace{\begin{pmatrix} -\sqrt{3} & -\frac{\sqrt{3}}{3} & \frac{\sqrt{3}}{3} \\ 0 & \frac{2\sqrt{6}}{3} & \frac{\sqrt{6}}{3} \\ 0 & 0 & \sqrt{2} \end{pmatrix}}_R.$$

Generally, the QR decomposition of a matrix is unique up to sign differences (as seen from the examples above where some of the rows and columns have different signs but in the end, the result will be the same).

G.1.3 QR Decomposition in MATLAB

In MATLAB, the QR decomposition can be done with the `qr` function.

```

1 >> A=[4,6,1;0,1,-1;0,1,2]
2 A =
3     4     6     1
4     0     1    -1
5     0     1     2
6 >> [Q,R]=qr(A)
7 Q =
8     1.0000         0         0
9         0    -0.7071   -0.7071
10        0    -0.7071    0.7071
11 R =
12     4.0000     6.0000     1.0000
13         0    -1.4142   -0.7071
14         0         0     2.1213

```

If the matrix is rectangular, then the economy version of the QR decomposition can be found using `qr(A,"econ")`.

G.2 Eigenvalue Decomposition

For a matrix $A \in \mathbb{C}^{N \times N}$, the value $\lambda \in \mathbb{C}$ and non-zero vector $\mathbf{v} \in \mathbb{C}^N$ are known as the **Eigenvalue** and **Eigenvector**, respectively, if they satisfy the relationship $A\mathbf{v} = \lambda\mathbf{v}$. These can be written in eigenpair notation as $\{\lambda; \mathbf{v}\}$.

In MATLAB, to find the eigenvalues and eigenvectors of a matrix A , use `[V,E]=eig(A)`. This will produce a matrix V whose columns are the eigenvectors of A and a diagonal matrix E whose entries are the corresponding eigenvalues where the (n,n) element of E is the eigenvalue that corresponds to the eigenvector in column n of V . However, if only `eig(A)` is run without specifying the outputs, MATLAB will produce a column vector of eigenvalues only.

```
1 >> A=[-2,-4,2;-2,1,2;4,2,5]
2 A =
3     -2    -4     2
4     -2     1     2
5      4     2     5
6 >> eig(A)
7 ans =
8     -5
9      3
10     6
11 >> [V,E]=eig(A)
12 V =
13     0.8165    0.5345    0.0584
14     0.4082   -0.8018    0.3505
15    -0.4082   -0.2672    0.9347
16
17 E =
18    -5     0     0
19     0     3     0
20     0     0     6
```

Therefore, the matrix A has the following eigenpairs

$$\left\{ -5 ; \begin{pmatrix} 0.8165 \\ 0.4082 \\ -0.4082 \end{pmatrix} \right\}, \quad \left\{ 3 ; \begin{pmatrix} 0.5345 \\ -0.8018 \\ -0.2672 \end{pmatrix} \right\}, \quad \left\{ 6 ; \begin{pmatrix} 0.0584 \\ 0.3505 \\ 0.9347 \end{pmatrix} \right\}.$$

Notice that the eigenvectors are not represented in the most pleasant form, the reason is that MATLAB normalises eigenvectors by default, meaning that the magnitude of every eigenvector is 1. In order to convert this to a more palatable form, the columns should be individually multiplied or divided by any scalar value¹. The easiest way to do this is to, first

¹Remember that any scalar multiple of an eigenvector is still an eigenvector.

of all, divide every individual column by its minimum value, then any other manipulations can be carried out afterwards.

```

1  >> v1=V(:,1)/min(V(:,1))
2  ans =
3      -2
4      -1
5       1
6  >> v2=V(:,2)/min(V(:,2))
7  ans =
8      -0.6667
9       1.0000
10     0.3333
11 >> v2=3*v2
12 ans =
13     -2
14      3
15      1
16 >> v3=V(:,3)/min(V(:,3))
17 ans =
18      1
19      6
20     16

```

This produces a far more appealing set of eigenpairs:

$$\left\{ -5; \begin{pmatrix} -2 \\ -1 \\ 1 \end{pmatrix} \right\}, \quad \left\{ 3; \begin{pmatrix} -2 \\ 3 \\ 1 \end{pmatrix} \right\}, \quad \left\{ 6; \begin{pmatrix} 1 \\ 6 \\ 16 \end{pmatrix} \right\}.$$

G.2.1 Eigendecomposition

Suppose that the matrix $A \in \mathbb{C}^{N \times N}$ has N linearly independent eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N$ with their associated eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_N$. Let V be the matrix whose columns are the eigenvectors of A and let Λ be the diagonal matrix whose entries are the corresponding eigenvalues (in the same way that MATLAB produces the matrices $\mathbf{E}$ and $\mathbf{V}$). In other words, if the matrix A has the eigenpairs

$$\{\lambda_1; \mathbf{v}_1\}, \quad \{\lambda_2; \mathbf{v}_2\}, \quad \dots \quad \{\lambda_N; \mathbf{v}_N\},$$

then the matrices V and Λ are

$$V = \begin{pmatrix} \vdots & \vdots & \dots & \vdots \\ \mathbf{v}_1 & \mathbf{v}_2 & \dots & \mathbf{v}_N \\ \vdots & \vdots & \dots & \vdots \end{pmatrix} \quad \text{and} \quad \Lambda = \begin{pmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_N \end{pmatrix}.$$

The matrix A can then be written as $A = V\Lambda V^{-1}$ and this is called the **Eigendecomposition** of A . If V is an orthogonal matrix (as MATLAB produces it), then the eigendecomposition of A is $A = V\Lambda V^T$.

This particular decomposition of matrices is useful when the matrix A acts as a repeated transformation in a vector space. For example, suppose that the vector $\mathbf{y}$ can be found by applying the matrix transformation A on the vector $\mathbf{x}$ 100 times, this means that $\mathbf{y} = A^{100}\mathbf{x}$. Under usual circumstances, calculating A^{100} is incredibly cumbersome but if the eigendecomposition of A is used, then the problem can be reduced into taking the power of a diagonal matrix instead. Indeed,

$$\begin{aligned}\mathbf{y} &= A^{100}\mathbf{x} \\ \mathbf{y} &= \underbrace{AA \dots A}_{100 \text{ times}} \mathbf{x} \\ \mathbf{y} &= \underbrace{(V\Lambda V^{-1})(V\Lambda V^{-1}) \dots (V\Lambda V^{-1})}_{100 \text{ times}} \mathbf{x} \\ \mathbf{y} &= V\Lambda V^{-1}V\Lambda V^{-1}V\Lambda V^{-1}\mathbf{x} \\ \mathbf{y} &= V\Lambda^{100}V^{-1}\mathbf{x}.\end{aligned}$$

Therefore, instead of calculating A^{100} , the matrix Λ^{100} can be calculated instead which will be much easier since Λ is a diagonal matrix (remember that the power of a diagonal matrix is just the power of its individual terms). If V is orthogonal, then the calculation will be simpler since the matrix V does not need to be inverted, only its transpose taken.

Luckily, MATLAB can perform this decomposition as seen with the `eig` command.

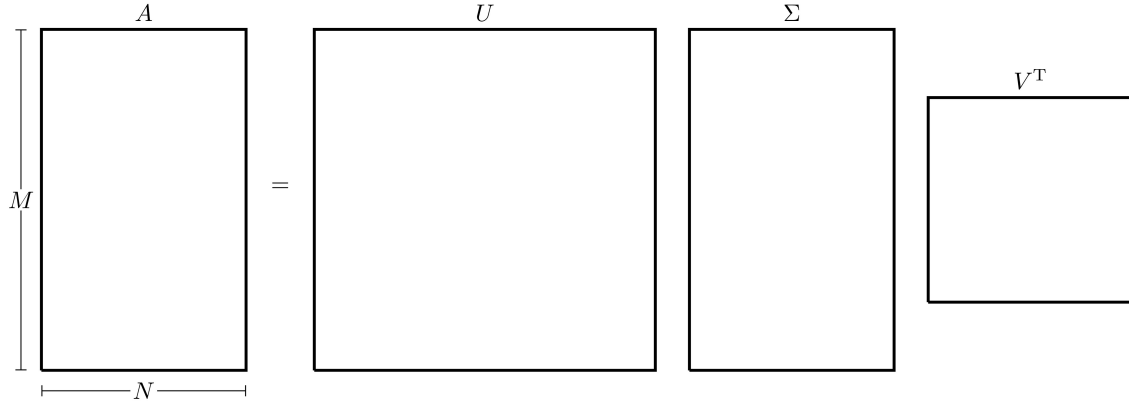
G.3 Singular Value Decomposition (SVD)

What happens if a square matrix A does not have a full system of eigenvectors? What happens if A is a rectangular matrix? In cases like this, some of the previous decompositions can fail, however there is one more way in which these issues can be resolved and it is by using the **Singular Value Decomposition**.

For $A \in \mathbb{R}^{M \times N}$, orthogonal matrices $U \in \mathbb{R}^{M \times M}$ and $V \in \mathbb{R}^{N \times N}$ can always be found such that $AV = U\Sigma$ where $\Sigma \in \mathbb{R}^{M \times N}$ is a diagonal matrix that can be written as $\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$ where $p = \min\{M, N\}$ whose entries are positive and arranged in descending order, i.e.

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0.$$

Since V is an orthogonal matrix, then A can be written as $A = U\Sigma V^T$, this form is called the **Singular Value Decomposition** (SVD) of A . If $M > N$, this can be illustrated as follows:



The scalar values σ_i are called the **Singular Values of A** , the columns of U are called **Left Singular Vectors** and the columns of V are called **Right Singular Vectors**. In a vector sense, the SVD of A given by $A = U \Sigma V^T$ can be written as $A \mathbf{v}_i = \sigma_i \mathbf{u}_i$ for all $i = 1, 2, \dots, p$ (where $\mathbf{u}_i$ and $\mathbf{v}_i$ are the columns of U and V respectively).

Properties of the SVD

- The SVD of a matrix $A \in \mathbb{C}^{M \times N}$ requires $\mathcal{O}(MNp)$ computations (where $p = \min\{M, N\}$).
- The singular values are also useful when calculating the 2-norm of a matrix. Recall that for a matrix $A \in \mathbb{C}^{M \times N}$, the 2-norm of A can be written in terms of the spectral radius of $A^H A$ as

$$\|A\|_2 = \sqrt{\rho(A^H A)}$$

where the spectral radius is the largest eigenvalue in absolute value. This can also be written in terms of the singular values as

$$\|A\|_2 = \sqrt{\sigma_{\max}(A)}$$

where $\sigma_{\max}(A)$ represents the largest singular value of matrix A , which (as per the way in which the singular values have been arranged) is going to be σ_1 .

- If $A \in \mathbb{R}^{N \times N}$, then the eigenvalues of AA^T and $A^T A$ are equal to the squares of the singular values of A , indeed, if $A = U \Sigma V^T$, then

$$AA^T = (U \Sigma V^T) (U \Sigma V^T)^T = U \Sigma V^T V \Sigma^T U^T = U \Sigma^2 U^T$$

$$A^T A = (U \Sigma V^T)^T (U \Sigma V^T) = V \Sigma^T U^T U \Sigma V^T = V \Sigma^2 V^T$$

since Σ is a diagonal square matrix.

- Let $r, s \in \mathbb{N}$ and $\tau \in \mathbb{R}$, suppose that the singular values $\sigma_1, \sigma_2, \dots, \sigma_p$ of A satisfy

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_s > \tau \geq \sigma_{s+1} \geq \dots \geq \sigma_r > \sigma_{r+1} = \sigma_{r+2} = \dots = \sigma_p = 0.$$

Then r is the **Rank** of A and s is the **τ -rank** of A . In fact, if $\tau = \varepsilon_M$ (the machine precision), then s is called the **Numerical Rank** of A .

- Specific singular vectors span specific subspaces defined in connection to A . For instance, if the rank of A is r , then $A\mathbf{v}_i = \mathbf{0}$ for all $i = r + 1, \dots, N$. As a consequence, the vectors $\mathbf{v}_{r+1}, \mathbf{v}_{r+2}, \dots, \mathbf{v}_N$ span the null-space of A , denoted by

$$\text{null}(A) = \left\{ \mathbf{x} \in \mathbb{R}^N : A\mathbf{x} = \mathbf{0} \right\}.$$

- If $A = U\Sigma V^T$, then A can be rewritten as

$$A = \sum_{i=1}^r E_i$$

where $E_i = \sigma_i \mathbf{u}_i \mathbf{v}_i^T$ is a rank-1 matrix. It can be seen that

$$\|E_i\| = \|\sigma_i \mathbf{u}_i \mathbf{v}_i^T\|_2 = \sigma_i.$$

Since the norm of a matrix is a measure of the “magnitude” of a matrix, it can be said that A is made up of very specific elementary rank-1 matrices, in such a way that E_1 is the most “influential” one.

The singular value decomposition of the matrix $A \in \mathbb{R}^{M \times N}$ can be done by following these steps:

Parallel Example

These steps will be applied in parallel to the matrix

$$A = \begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix}.$$

1. Calculate the eigenpairs of AA^T and $A^T A$.

Eigenpairs

The eigenpairs of AA^T are

$$\left\{ 25; \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right\} \quad \text{and} \quad \left\{ 9; \begin{pmatrix} -1 \\ 1 \end{pmatrix} \right\}.$$

Similarly, the eigenpairs of $A^T A$ are

$$\left\{ 25; \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \right\}, \quad \left\{ 9; \begin{pmatrix} 1 \\ -1 \\ 4 \end{pmatrix} \right\} \quad \text{and} \quad \left\{ 0; \begin{pmatrix} -2 \\ 2 \\ 1 \end{pmatrix} \right\}.$$

2. Normalise the eigenvectors by dividing by their 2-norm (this will in fact be the default output from MATLAB's `eig` function).

🔥 Normalise Eigenvectors

The *normalised* eigenpairs of AA^T are

$$\left\{ 25; \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right\} \quad \text{and} \quad \left\{ 9; \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \end{pmatrix} \right\}.$$

Similarly, the *normalised* eigenpairs of $A^T A$ are

$$\left\{ 25; \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \right\}, \quad \left\{ 9; \frac{1}{\sqrt{18}} \begin{pmatrix} 1 \\ -1 \\ 4 \end{pmatrix} \right\} \quad \text{and} \quad \left\{ 0; \frac{1}{3} \begin{pmatrix} -2 \\ 2 \\ 1 \end{pmatrix} \right\}.$$

3. The matrix of singular values Σ must be of the same size as A , i.e. $\Sigma \in \mathbb{R}^{M \times N}$, where the diagonal terms are the square roots of the eigenvalues of AA^T and $A^T A$ (only the ones that are shared by the two matrix products) arranged in descending order. There will only be p diagonal terms where $p = \min\{M, N\}$.

🔥 Terms of Σ

The matrix Σ must be of size 2×3 . The eigenvalues of AA^T and $A^T A$ are 25 and 9. Therefore the matrix Σ and is given by

$$\Sigma = \begin{pmatrix} \sqrt{25} & 0 & 0 \\ 0 & \sqrt{9} & 0 \end{pmatrix} = \begin{pmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{pmatrix}.$$

4. The matrix $U \in \mathbb{R}^{M \times M}$ will be the matrix whose columns are the normalised eigenvectors of $A^T A$ arranged in the same order as the values appear in Σ . Note that if $\mathbf{v}$ is a normalised eigenvector, then $-\mathbf{v}$ will also be a normalised eigenvector, therefore this will give rise to 2^M possible cases for U (which will be narrowed down later).

🔥 Matrix U

The normalised eigenpairs of $A^T A$ are

$$\left\{ 25; \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right\} \quad \text{and} \quad \left\{ 9; \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \end{pmatrix} \right\}.$$

If $\mathbf{u}_1$ is the first normalised eigenvector and $\mathbf{u}_2$ is the second normalised eigenvector (i.e. $\mathbf{u}_1 = (1, 1)^T$ and $\mathbf{u}_2 = (-1, 1)^T$), then the matrix $U \in \mathbb{R}^{2 \times 2}$ can take one of four

possible forms

$$U_1 = \begin{pmatrix} \mathbf{u}_1 & \mathbf{u}_2 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}, \quad U_2 = \begin{pmatrix} \mathbf{u}_1 & -\mathbf{u}_2 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$$U_3 = \begin{pmatrix} -\mathbf{u}_1 & \mathbf{u}_2 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 & -1 \\ -1 & 1 \end{pmatrix}, \quad U_4 = \begin{pmatrix} -\mathbf{u}_1 & -\mathbf{u}_2 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 & 1 \\ -1 & -1 \end{pmatrix}.$$

5. The matrix $V \in \mathbb{R}^{N \times N}$ will be the matrix whose columns are the normalised eigenvectors of AA^T arranged in the same order as the values appear in Σ . Just as before, there will technically be 2^N choices of V . In this case, one choice of U or V should be fixed.

Matrix V

The normalised eigenpairs of $A^T A$ are

$$\left\{ 25; \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \right\}, \quad \left\{ 9; \frac{1}{\sqrt{18}} \begin{pmatrix} 1 \\ -1 \\ 4 \end{pmatrix} \right\} \quad \text{and} \quad \left\{ 0; \frac{1}{3} \begin{pmatrix} -2 \\ 2 \\ 1 \end{pmatrix} \right\}.$$

Since V has a larger size than U , fix V as the matrix whose columns are the normalised eigenvectors of AA^T with no sign changes. This can be accommodated for later on by picking an appropriate choice for U . Then

$$V = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{18}} & -\frac{2}{3} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{18}} & \frac{2}{3} \\ 0 & \frac{4}{\sqrt{18}} & \frac{1}{3} \end{pmatrix}.$$

6. The correct choice for the matrix U can be found in one of two ways:

- **Trial & Error:** Perform the multiplication $U\Sigma V^T$ for the different choices of U until the correct one is found that gives A . Alternatively, U can be fixed and the different choices for V can be investigated.

Trial & Error

Consider the product $U\Sigma V^T$ for the different choices of U and see which one gives the

matrix A :

$$\begin{aligned} U_1 \Sigma V^T &= \begin{pmatrix} 2 & 3 & -2 \\ 3 & 2 & 2 \end{pmatrix} \neq A \\ U_2 \Sigma V^T &= \begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix} = A \\ U_3 \Sigma V^T &= \begin{pmatrix} -3 & -2 & -2 \\ -2 & -3 & 2 \end{pmatrix} \neq A \\ U_4 \Sigma V^T &= \begin{pmatrix} -2 & -3 & 2 \\ -3 & -2 & -2 \end{pmatrix} \neq A \end{aligned}$$

Therefore the correct choice for U is U_2 .

- **Pseudo-Inversion:** First, consider the expression $A = U \Sigma V^T$, multiplying both sides by V on the right gives $AV = U \Sigma$ (since V is orthogonal meaning that $V^T V = \mathcal{I}$). Since Σ is rectangular in general, it does not have an inverse but it does have a **Pseudo-Inverse**². Since Σ is a diagonal matrix, then the pseudo-inverse will also be a diagonal matrix with the diagonal entries being the reciprocals of the singular values. For example, if

$$\Sigma = \begin{pmatrix} \sigma_1 & 0 & 0 & 0 \\ 0 & \sigma_2 & 0 & 0 \\ 0 & 0 & \sigma_3 & 0 \end{pmatrix},$$

then the pseudo-inverse of Σ is

$$\Sigma^+ = \begin{pmatrix} \frac{1}{\sigma_1} & 0 & 0 \\ 0 & \frac{1}{\sigma_2} & 0 \\ 0 & 0 & \frac{1}{\sigma_3} \\ 0 & 0 & 0 \end{pmatrix}.$$

Similarly if

$$\Sigma = \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \\ 0 & 0 & 0 \end{pmatrix},$$

then the pseudo-inverse of Σ is

$$\Sigma^- = \begin{pmatrix} \frac{1}{\sigma_1} & 0 & 0 & 0 \\ 0 & \frac{1}{\sigma_2} & 0 & 0 \\ 0 & 0 & \frac{1}{\sigma_3} & 0 \end{pmatrix}.$$

Therefore multiplying both sides of $AV = U \Sigma$ by Σ^+ on the right will give the desired expression for U which is $U = AV \Sigma^+$.

²For a matrix $B \in \mathbb{C}^{M \times N}$ with $M < N$, then the pseudo-inverse is the matrix $B^+ \in \mathbb{C}^{N \times M}$ such that $BB^+ = \mathcal{I} \in \mathbb{R}^{M \times M}$. Similarly, if $B \in \mathbb{C}^{M \times N}$ with $M > N$, the pseudo-inverse is the matrix $B^- \in \mathbb{C}^{N \times M}$ such that $B^-B = \mathcal{I} \in \mathbb{R}^{N \times N}$. Note that if a matrix is square and invertible, then the pseudo-inverse is the inverse.

🔥 Pseudo-Inverse

The pseudo-inverse of $\Sigma \in \mathbb{R}^{2 \times 3}$ is $\Sigma^+ \in \mathbb{R}^{3 \times 2}$ where its diagonal terms are the reciprocals of those in Σ , i.e.

$$\Sigma = \begin{pmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{pmatrix} \implies \Sigma^+ = \begin{pmatrix} \frac{1}{5} & 0 \\ 0 & \frac{1}{3} \\ 0 & 0 \end{pmatrix}.$$

This can be verified by showing that $\Sigma\Sigma^+ = \mathcal{I}$. To find U , calculate

$$U = A\Sigma^+ = \begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{18}} & -\frac{2}{3} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{18}} & \frac{2}{3} \\ 0 & \frac{4}{\sqrt{18}} & \frac{1}{3} \end{pmatrix} \begin{pmatrix} \frac{1}{5} & 0 \\ 0 & \frac{1}{3} \\ 0 & 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

7. This finally gives all the matrices required for the SVD of A .

🔥 SVD of A

$$\underbrace{\begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix}}_A = \underbrace{\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}}_U \underbrace{\begin{pmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{pmatrix}}_\Sigma \underbrace{\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{18}} & -\frac{1}{\sqrt{18}} & \frac{4}{\sqrt{18}} \\ -\frac{2}{3} & \frac{2}{3} & \frac{1}{3} \end{pmatrix}}_{V^T}.$$

Note that if the SVD of a matrix A is known, it can also be useful in finding pseudo inverse of A :

$$\begin{aligned} A &= U\Sigma V^T \\ \xRightarrow{\times V} AV &= U\Sigma V^T V \\ \xRightarrow{V^{-1}=V^T} AV &= U\Sigma \\ \xRightarrow{\times \Sigma^+} AV\Sigma^+ &= U\Sigma\Sigma^+ \\ \xRightarrow{\Sigma\Sigma^+=\mathcal{I}} AV\Sigma^+ &= U \\ \xRightarrow{\times U^T} AV\Sigma^+U^T &= U^T \\ \xRightarrow{UU^T=\mathcal{I}} AV\Sigma^+U^T &= \mathcal{I}. \end{aligned}$$

Therefore, the matrix $A^+ = V\Sigma^+U^T$ is the pseudo-inverse of A .

Pseudo-Inverse of A

Find the pseudo-inverse of A where

$$A = \begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix}.$$

The SVD of A is

$$A = \underbrace{\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}}_U \underbrace{\begin{pmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{pmatrix}}_{\Sigma} \underbrace{\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{18}} & -\frac{1}{\sqrt{18}} & \frac{4}{\sqrt{18}} \\ -\frac{2}{3} & \frac{2}{3} & \frac{1}{3} \end{pmatrix}}_{V^T}.$$

The pseudo-inverse of A is

$$A^+ = V\Sigma^+U^T = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{18}} & -\frac{2}{3} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{18}} & \frac{2}{3} \\ 0 & \frac{4}{\sqrt{18}} & \frac{1}{3} \end{pmatrix} \begin{pmatrix} \frac{1}{5} & 0 \\ 0 & \frac{1}{3} \\ 0 & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} = \frac{1}{45} \begin{pmatrix} 7 & 2 \\ 2 & 7 \\ 10 & -10 \end{pmatrix}.$$

G.3.0.1 SVD in MATLAB

In MATLAB, the SVD of a matrix can be found with the **svd** command.

```
1 >> A=[3, 2, 2; 2, 3, -2]
2 A =
3     3     2     2
4     2     3    -2
5 >> [U,S,V]=svd(A)
6 U =
7    -0.7071    0.7071
8    -0.7071   -0.7071
9 S =
10    5.0000         0         0
11         0    3.0000         0
12 V =
13    -0.7071    0.2357   -0.6667
14    -0.7071   -0.2357    0.6667
15    -0.0000    0.9428    0.3333
16 >> U*S*V'-A    % Check is A=USV'
17 ans =
18    1.0e-14 *
19         0         0   -0.0222
20   -0.0222   -0.1332    0.0666
```

Notice that sometimes, due to round-off error, $U*S*V'-A$ may not exactly be equal to the zero matrix, but it is still close enough to it.

H Data Fitting

H.1 Linear Regression

Linear Regression, or *Linear Least Squares* (LS), problems originally arose from the need to fit a mathematical model to given observations; typically, to reduce the influence of errors in the observations. It is desirable to use a greater number of measurements than the number of unknown parameters in the model (more equations than unknowns), hence leading to an overdetermined system of equations. In other words, given $\mathbf{b} \in \mathbb{R}^M$ and $A \in \mathbb{R}^{M \times N}$ with $M > N$, a solution $\mathbf{x} \in \mathbb{R}^N$ needs to be found such that $A\mathbf{x}$ is the “best” approximation to $\mathbf{b}$.

For instance, consider a set of M data points (or measurements) (t_i, y_i) for $i = 1, 2, \dots, M$. The idea behind linear regression is to find a parameter vector $\mathbf{x} \in \mathbb{R}^N$ such that the linear function y given by

$$y(t) = f(x, t) = \sum_{j=1}^N x_j \varphi_j(t)$$

can approximate the data in the best possible way, by reducing the error between the measurement (t_i, y_i) and the approximation $(t_i, y(t_i))$.

There are M equations represented by the M measurements and N unknowns, which are the terms of $\mathbf{x}$. Replacing the measurements into the equation for y gives an overdetermined system

$$y_i = \sum_{j=1}^N x_j \varphi_j(t_i) \quad \text{for } i = 1, \dots, M.$$

This system can be written in matrix form as $A\mathbf{x} = \mathbf{b}$ where the elements of A are $a_{ij} = \varphi_j(t_i)$ and the elements of $\mathbf{b}$ are $b_i = y_i$. The “best” way to fit the data can be different depending upon the discipline, but the one of the simplest and most statistically motivated choice is to find a vector $\mathbf{x}$ where the square of the distance between the points is reduced as much as possible, i.e. reduce the value of $(y(t_i) - y_i)^2$. More formally, this can be written as a minimisation problem to find

$$\min_{\mathbf{x}} \|\mathbf{r}\|_2 \quad \text{where } \mathbf{r} = \mathbf{b} - A\mathbf{x} \quad \text{is the residual}$$

and the linear least squares solution is

$$\tilde{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \|A\mathbf{x} - \mathbf{b}\|_2.$$

Sometimes the solution $\mathbf{x}$ may not be unique (if the rank of A is less than N), in that case, the solution will be the one with the smallest 2-norm.

🔥 Hooke's Law

Hooke's law states that the length l of an extension of a spring is directly proportional to the force F applied, specifically the extension can be written in terms of the force as $l = e + kF$ where e is the equilibrium position and k is the spring stiffness, both of which are constants to be determined. Assume that an experiment was conducted and the following data was obtained

F	1	2	3	4	5
l	7.97	10.2	14.2	16.0	21.2

Therefore, a system of 5 equations in 2 unknowns is

$$\begin{aligned} 7.97 &= e + k \\ 10.2 &= e + 2k \\ 14.2 &= e + 3k \\ 16.0 &= e + 4k \\ 21.2 &= e + 5k. \end{aligned}$$

This system can be written in matrix form as

$$\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{pmatrix} \begin{pmatrix} e \\ k \end{pmatrix} = \begin{pmatrix} 7.97 \\ 10.2 \\ 14.2 \\ 16.0 \\ 21.2 \end{pmatrix}.$$

This is an example of *****Inverse Problem** in which the parameters need to be found from the given data.

This minimisation problem can also be solved using the QR decomposition of the matrix A . Suppose that the matrix A can be written as $A = QR$ where Q is an orthogonal matrix and R is upper triangular, then

$$A\mathbf{x} - \mathbf{b} = QR\mathbf{x} - \mathbf{b} \quad \text{since } A = QR \quad = Q(R\mathbf{x} - Q^T\mathbf{b}) \quad \text{since } Q^{-1} = Q^T.$$

Thus the 2-norm of the residual $\mathbf{r} = A\mathbf{x} - \mathbf{b}$ is

$$\begin{aligned} \|\mathbf{r}\|_2 &= \|A\mathbf{x} - \mathbf{b}\|_2 \\ &= \|Q(R\mathbf{x} - Q^T\mathbf{b})\|_2 \\ &= \|R\mathbf{x} - Q^T\mathbf{b}\|_2 \quad \text{since } Q \text{ is orthogonal, then } \|Q\mathbf{v}\|_2 = \|\mathbf{v}\|_2. \end{aligned}$$

As already noted, in many problems of estimating N parameters in a process with M experimental data points, the number of observations is usually larger than the number of

parameters, i.e. $M \geq N$. The problem of minimising $\|R\mathbf{x} - Q^T\mathbf{b}\|_2$ may be solved directly as follows: let $\mathbf{c} = Q^T\mathbf{b}$, so that

$$R\mathbf{x} - Q^T\mathbf{b} = R\mathbf{x} - \mathbf{c} = \begin{pmatrix} r_{11} & r_{12} & \dots & r_{1N} \\ 0 & r_{22} & \dots & r_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & r_{NN} \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix} - \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_N \\ c_{N+1} \\ \vdots \\ c_M \end{pmatrix} = \begin{pmatrix} r_{11}x_1 + r_{12}x_2 + \dots + r_{1N}x_N - c_1 \\ r_{22}x_2 + \dots + r_{2N}x_N - c_2 \\ \vdots \\ r_{NN}x_N - c_N \\ -c_{N+1} \\ \vdots \\ -c_M \end{pmatrix}.$$

This vector can be written as $\mathbf{d} + \mathbf{f}$ where

$$\mathbf{d} = \begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_N \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad \text{with} \quad d_i = -c_i + \sum_{j=i}^N r_{ij}x_j \quad \text{and} \quad \mathbf{f} = - \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ c_{N+1} \\ \vdots \\ c_M \end{pmatrix}.$$

Also note that the vector $\mathbf{d}$ can be written as $R\mathbf{x} - \tilde{\mathbf{c}}$ where $\tilde{\mathbf{c}}$ is the first N rows of $\mathbf{c}$.

It can be seen that the vectors $\mathbf{d}$ and $\mathbf{f}$ are orthogonal (since $\mathbf{d} \cdot \mathbf{f} = 0$), therefore

$$\|\mathbf{r}\|_2 = \|\mathbf{d}\|_2 + \|\mathbf{f}\|_2.$$

i Note

Recall that for two vectors $\mathbf{x}$ and $\mathbf{y}$, the *Triangle Inequality* states that $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ and the equality holds when $\mathbf{x}$ and $\mathbf{y}$ are orthogonal.

Since only the vector $\mathbf{d}$ depends on $\mathbf{x}$, then in order to minimise $\|\mathbf{r}\|_2$, a choice for $\mathbf{x}$ is needed such that $\|\mathbf{d}\|_2 = 0$, meaning that $\mathbf{d}$ must be the zero vector (by the rules of norms). Therefore, if $\mathbf{d} = \mathbf{0}$, then

$$R\mathbf{x} = \tilde{\mathbf{c}} \quad \implies \quad \mathbf{x} = R^{-1}\tilde{\mathbf{c}} = \begin{pmatrix} r_{11} & r_{12} & \dots & r_{1N} \\ 0 & r_{22} & \dots & r_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & r_{NN} \end{pmatrix}^{-1} \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_N \end{pmatrix}$$

which will be the best least squares fit. The residual $\|\mathbf{r}\|_2$ will then be equal to $\|\mathbf{f}\|_2$ which will be an estimate for how good the best is.

H.2 Lines of Best Fit Using polyfit

Sometimes when experimental data is given, a lines of best fit is needed to see which lines would best fit the data.

Suppose there is data stored in an Excel file called `Data.xlsx` consisting of two columns that will be labelled `x` and `y` and the line of best fit needs to be found. The `polyfit` function can fit a polynomial function to this data, so if a linear function $y = ax + b$ needs to be fitted, then `p=polyfit(x,y,1)` will produce two outputs which are the coefficients a and b respectively. The fitted data can then be plotted using the `polyval` command. All in all, the function below will read data, plot the raw data and the line of best fit:

```
1 function Line_Best_Fit
2
3 Data = xlsread('Data.xlsx');
4
5 x = Data(:,1);
6 y = Data(:,2);
7
8 clf
9 hold on
10 grid on
11 plot(x,y, '.k')
12
13 p = polyfit(x,y,1);
14
15 X = linspace(min(x),max(x));
16
17 Y = polyval(p,X);
18
19 plot(X,Y, '-r')
20
21 end
```

The degree of the polynomial can be changed until the appropriate fitting is found. For this data, it seems that a degree three polynomial would be most appropriate

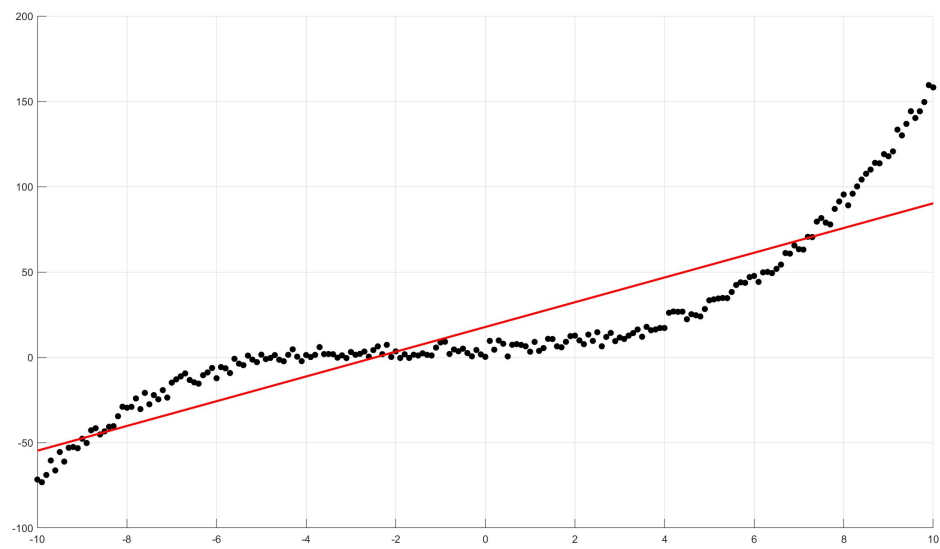


Figure H.1: `polyfit(x,y,1)` gives $y = 7.2484x + 17.7404$

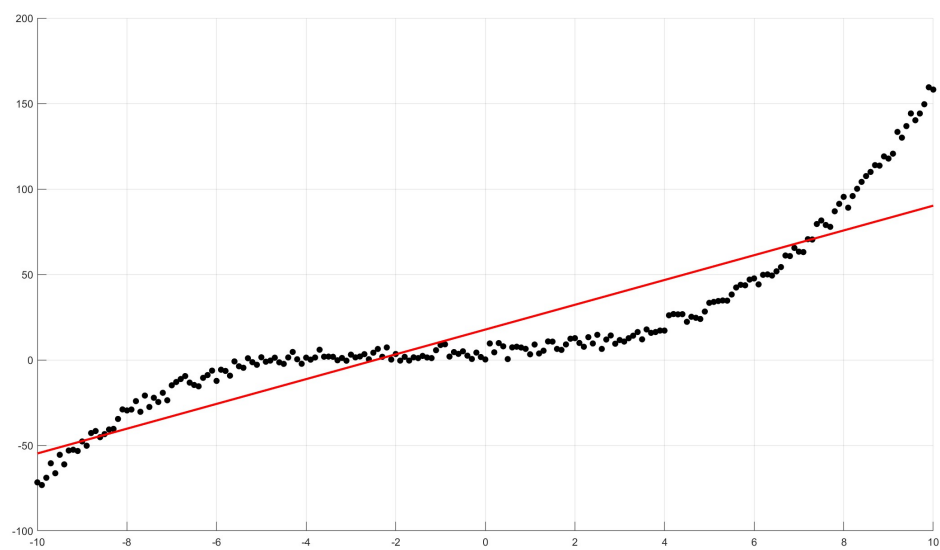


Figure H.2: `polyfit(x,y,3)` gives $y = 0.1100x^3 + 0.3920x^2 + 0.5854x + 4.5430$

I Eigenvalue Problems

Given a square matrix $A \in \mathbb{C}^{N \times N}$, the **Eigenvalue Problem** consists of finding a scalar $\lambda \in \mathbb{C}$ and a vector $\mathbf{v} \neq \mathbf{0}$ such that $A\mathbf{v} = \lambda\mathbf{v}$. Any such λ is called an **Eigenvalue** of A , while $\mathbf{v}$ is the associated **Eigenvector**. For any matrix A and its eigenvalue λ , the associated eigenvector is not unique; in fact, any multiple of an eigenvector is still an eigenvector. The eigenvalue/eigenvector pair will be written in **Eigenpair** notation as $\{\lambda; \mathbf{v}\}$.

In order to calculate the eigenvalues of a matrix $A \in \mathbb{C}^{N \times N}$, consider the polynomial

$$p(\lambda) = \det(A - \lambda I).$$

This will be a polynomial of degree N , in fact, any root of the polynomial $p(\lambda)$ is an eigenvalue of A and vice versa. Note that if the highest order coefficient of p is equal to 1, then the polynomial is known as the **Characteristic Polynomial of A** . More generally, for any matrix $A \in \mathbb{C}^{N \times N}$, the characteristic polynomial is given by $P(\lambda) = (-1)^N \det(A - \lambda I)$. This means that the matrix A of size $N \times N$ must have N eigenvalues (not necessarily unique). Also, if A is a real matrix, the polynomial $p(\lambda)$ will have real coefficients and therefore (by the Fundamental Theorem of Algebra), any complex eigenvalues will appear in complex conjugate pairs. If A is a diagonal or triangular matrix, then the eigenvalues are simply the diagonal terms. After the eigenvalues have been found, the eigenvectors can be calculated by finding a general form of the vector $\mathbf{v}$ that satisfies $(A - \lambda I)\mathbf{v} = \mathbf{0}$.

If the eigenvector $\mathbf{v}$ is known, the eigenvalue λ can be recovered by using the **Rayleigh Quotient**

$$\lambda = \frac{\mathbf{v}^H A \mathbf{v}}{\|\mathbf{v}\|_2^2}$$

where $\mathbf{v}^H = \mathbf{v}^T$ is the Hermitian of $\mathbf{v}$ (the complex conjugate transpose).

Caution

Let

$$A = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

To find the eigenvalues, first consider the polynomial

$$p(\lambda) = \det(A - \lambda I) = \det \left(\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right) = \det \begin{pmatrix} -\lambda & -1 \\ 1 & -\lambda \end{pmatrix} = \lambda^2 + 1.$$

This polynomial has two roots, $\lambda_1 = i$ and $\lambda_2 = -i$, hence giving the two eigenvalues of A .

To calculate the eigenvectors, consider the eigenvalues separately, then for each eigenvalue, find the vector $\mathbf{v} = (V_1 \ V_2)^T$ that satisfies $(A - \lambda I)\mathbf{v} = \mathbf{0}$:

- $\lambda_1 = i$:

$$(A - \lambda_1 I)\mathbf{v} = \mathbf{0} \implies \begin{pmatrix} -i & -1 \\ 1 & -i \end{pmatrix} \begin{pmatrix} V_1 \\ V_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies \begin{aligned} -iV_1 - V_2 &= 0 \\ V_1 - iV_2 &= 0 \end{aligned}$$

which gives two equations in two unknowns. However, notice that if the first equation is multiplied by i , the second equation will be obtained and therefore, the problem is underdetermined (i.e. one equation in two unknowns). This must *always* be the case, finding an eigenvector must always result in an underdetermined system. In this case, solving one equation would suffice. Solving the second equation will give V_1 in terms of V_2 as $V_1 = iV_2$. Therefore the eigenvector $\mathbf{v}$ will be

$$\mathbf{v} = \begin{pmatrix} V_1 \\ V_2 \end{pmatrix} = \begin{pmatrix} iV_2 \\ V_2 \end{pmatrix} = \begin{pmatrix} i \\ 1 \end{pmatrix} V_2.$$

Now any value of V_2 can be chosen (except 0), and the result will be the eigenvector (this also shows why any multiple of an eigenvector is also an eigenvector), in this case, choose $V_2 = 1$. This gives the first eigenpair

$$\left\{ i; \begin{pmatrix} i \\ 1 \end{pmatrix} \right\}.$$

- $\lambda_2 = -i$:

$$(A - \lambda_2 I)\mathbf{v} = \mathbf{0} \implies \begin{pmatrix} i & -1 \\ 1 & i \end{pmatrix} \begin{pmatrix} V_1 \\ V_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies \begin{aligned} iV_1 - V_2 &= 0 \\ V_1 + iV_2 &= 0 \end{aligned}$$

which gives one equations in two unknowns (since the first equation multiplied by $-i$ gives the second). Solving the second equation will give V_1 in terms of V_2 as $V_1 = -iV_2$. Therefore the eigenvector $\mathbf{v}$ will be

$$\mathbf{v} = \begin{pmatrix} V_1 \\ V_2 \end{pmatrix} = \begin{pmatrix} -iV_2 \\ V_2 \end{pmatrix} = \begin{pmatrix} -i \\ 1 \end{pmatrix} V_2.$$

For the sake of simplicity, choose $V_2 = i$ (once again, any non-zero value of V_2 can be chosen). This gives the second eigenpair

$$\left\{ -i; \begin{pmatrix} 1 \\ i \end{pmatrix} \right\}.$$

Therefore, the matrix has the eigenpairs

$$\left\{ i; \begin{pmatrix} i \\ 1 \end{pmatrix} \right\}, \quad \left\{ -i; \begin{pmatrix} 1 \\ i \end{pmatrix} \right\}.$$

This can be verified by showing that $A\mathbf{v} = \lambda\mathbf{v}$ for each eigenpair:

$$A\mathbf{v}_1 = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} i \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ i \end{pmatrix} = i \begin{pmatrix} i \\ 1 \end{pmatrix} = \lambda_1 \mathbf{v}_1$$

$$A\mathbf{v}_2 = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ i \end{pmatrix} = \begin{pmatrix} -i \\ 1 \end{pmatrix} = -i \begin{pmatrix} 1 \\ i \end{pmatrix} = \lambda_2 \mathbf{v}_2.$$

For a matrix $A \in \mathbb{C}^{N \times N}$, there will always be N eigenvalues (not necessarily distinct). If an eigenvalue is repeated, then the same eigenvalue will have multiple eigenvectors however, it is possible that there might not necessarily be a total of N eigenvectors.

If the matrix A has a complete set of eigenvectors (meaning it has N distinct eigenvectors), then A is said to be **Diagonalisable**, i.e. there exists a non-singular matrix $V \in \mathbb{C}^{N \times N}$ whose columns are the eigenvectors of A and a diagonal matrix $\Lambda \in \mathbb{C}^{N \times N}$ whose entries are the eigenvalues of A , such that $A = V\Lambda V^{-1}$. Note that the order in which the eigenvalues and eigenvectors are placed in columns should be the same in both matrices, in other words, if the matrix A has N eigenpairs given by $\{\lambda_1; \mathbf{v}_1\}, \{\lambda_2; \mathbf{v}_2\}, \dots, \{\lambda_N; \mathbf{v}_N\}$, then

$$\Lambda = \begin{pmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_N \end{pmatrix}, \quad V = \begin{pmatrix} \vdots & \vdots & \cdots & \vdots \\ \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_N \\ \vdots & \vdots & \cdots & \vdots \end{pmatrix}.$$

Caution

From the example above, the matrices Λ and V are

$$\Lambda = \begin{pmatrix} i & 0 \\ 0 & -i \end{pmatrix}, \quad V = \begin{pmatrix} i & 1 \\ 1 & i \end{pmatrix}.$$

The matrix A is diagonalisable since the product $V\Lambda V^{-1}$ should give A , indeed

$$V\Lambda V^{-1} = \begin{pmatrix} i & 1 \\ 1 & i \end{pmatrix} \begin{pmatrix} i & 0 \\ 0 & -i \end{pmatrix} \begin{pmatrix} i & 1 \\ 1 & i \end{pmatrix}^{-1} = \frac{1}{i^2 - 1} \begin{pmatrix} i & 1 \\ 1 & i \end{pmatrix} \begin{pmatrix} i & 0 \\ 0 & -i \end{pmatrix} \begin{pmatrix} i & -1 \\ -1 & i \end{pmatrix}$$

$$= -\frac{1}{2} \begin{pmatrix} i & 1 \\ 1 & i \end{pmatrix} \begin{pmatrix} -1 & -i \\ i & 1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 0 & 2 \\ -2 & 0 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = A.$$

Note that the existence of a complete system of eigenvectors is helpful in representing a linear transformation (or equivalently a square matrix) of a Euclidean space, such as $\mathbb{R}^N$, as a simple dilation or scaling (i.e. a multiplication by a suitable factor along each coordinate axis) in a suitable system of coordinates, obtained from the original one by a volume-preserving linear map.

If the matrix A is Hermitian, meaning that $A^H = A$, (this happens to be the case in many important applications, then the eigenvalue problem is much simpler since the following properties hold:

- All eigenvalues of A are real (since A is real and all eigenvalues are also real, then all eigenvectors can also be chosen to be real as well, usually by multiplying by an appropriate factor);
- The eigenvectors corresponding to distinct eigenvalues are orthogonal (in fact, the eigenvectors of A can be chosen to be orthonormal);
- The matrix A is always diagonalisable.

i Note

For example, in the numerical approximation of solutions of boundary value problems for second-order differential equations describing “conservative” physical processes, i.e. those where there is no dissipation of energy or it is very weak and can be neglected in the first instance.

!!! Let $\{\lambda; \mathbf{v}\}$ be an eigenpair of A , i.e. $A\mathbf{v} = \lambda\mathbf{v}$, then

$$\begin{aligned}
 & A\mathbf{v} = \lambda\mathbf{v} \\
 & \xRightarrow{\mathbf{v}^H \times} \mathbf{v}^H A\mathbf{v} = \lambda \mathbf{v}^H \mathbf{v} \\
 & \xRightarrow{A^H = A} \mathbf{v}^H A^H \mathbf{v} = \lambda \mathbf{v}^H \mathbf{v} \\
 & \xRightarrow{(A\mathbf{v})^H = \mathbf{v}^H A^H} (A\mathbf{v})^H \mathbf{v} = \lambda \mathbf{v}^H \mathbf{v} \\
 & \xRightarrow{A\mathbf{v} = \lambda\mathbf{v}} (\lambda\mathbf{v})^H \mathbf{v} = \lambda \mathbf{v}^H \mathbf{v} \\
 & \xRightarrow{(\alpha\mathbf{u})^H = \bar{\alpha}\mathbf{u}^H} \bar{\lambda} \mathbf{v}^H \mathbf{v} = \lambda \mathbf{v}^H \mathbf{v} \\
 & \xRightarrow{\mathbf{u}^H \mathbf{u} = \|\mathbf{u}\|_2^2} \bar{\lambda} \|\mathbf{v}\|_2^2 = \lambda \|\mathbf{v}\|_2^2 \\
 & \xRightarrow{\div \|\mathbf{v}\|_2^2 \text{ since } \mathbf{v} \neq \mathbf{0}} \bar{\lambda} = \lambda \implies \lambda \in \mathbb{R}.
 \end{aligned}$$

Let $\{\lambda; \mathbf{v}\}$ and $\{\mu; \mathbf{u}\}$ be real eigenpairs of A where $\lambda \neq \mu$, i.e. $A\mathbf{v} = \lambda\mathbf{v}$ and $A\mathbf{u} = \mu\mathbf{u}$. Then

$$\begin{aligned}
 & A\mathbf{u} = \mu\mathbf{u} \\
 & \xRightarrow{\mathbf{v}^T \times} \mathbf{v}^T A\mathbf{u} = \mu \mathbf{v}^T \mathbf{u} \\
 & \xRightarrow{(A\mathbf{v})^T = \mathbf{v}^T A^T} (A\mathbf{v})^T \mathbf{u} = \mu \mathbf{v}^T \mathbf{u} \\
 & \xRightarrow{A\mathbf{v} = \lambda\mathbf{v}} \lambda \mathbf{v}^T \mathbf{u} = \mu \mathbf{v}^T \mathbf{u} \\
 & \implies (\lambda - \mu) \mathbf{v}^T \mathbf{u} = 0 \\
 & \xRightarrow{\lambda \neq \mu} \mathbf{v}^T \mathbf{u} = 0 \implies \mathbf{u} \text{ and } \mathbf{v} \text{ are orthogonal.}
 \end{aligned}$$

Let V be the matrix whose columns are $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N$ which are the distinct eigenvectors of A . Since all the eigenvectors of A are orthogonal, then $\langle \mathbf{v}_i, \mathbf{v}_j \rangle = \delta_{ij}$ for all $i, j = 1, 2, \dots, N$. This means that V must be an orthogonal matrix, i.e. $V^T V = \mathcal{I}$. Moreover, since $A = V \Lambda V^{-1}$ and $V^{-1} = V^T$, then $A = V \Lambda V^T$. Therefore the diagonal matrix of eigenvalues Λ is equal to $\Lambda = V^T A V$, more specifically $\mathbf{v}_i^T A \mathbf{v}_j = \delta_{ij} \lambda_i$. !!!

Therefore, if a matrix A is real and symmetric, then the eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N$ must satisfy

$$\mathbf{v}_i^T \mathbf{v}_j = \delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} \quad \text{and} \quad \mathbf{v}_i^T A \mathbf{v}_j = \delta_{ij} \lambda_i = \begin{cases} \lambda_i & i = j \\ 0 & i \neq j \end{cases} \quad \text{for all } i, j = 1, 2, \dots, N.$$

Since $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N$ is a set of N linearly independent vectors in $\mathbb{R}^N$, then they must span $\mathbb{R}^N$. Therefore, any vector $\mathbf{x} \in \mathbb{R}^N$ with $\|\mathbf{x}\|_2 = 1$ can be written as a linear combination of $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N$, specifically

$$\mathbf{x} = \sum_{j=1}^N a_j \mathbf{v}_j \quad \text{where} \quad a_j \in \mathbb{R} \quad \text{for all } j = 1, 2, \dots, N.$$

Therefore

$$1 = \|\mathbf{x}\| = \mathbf{x}^T \mathbf{x} = \sum_{j=1}^N a_j^2 \quad \text{and} \quad \mathbf{x}^T A \mathbf{x} = \sum_{j=1}^N \lambda_j a_j^2.$$

Fibonacci Sequence

There are many applications of the eigenvalue decomposition. A simple one involves the analysis of the Fibonacci numbers. Consider the sequence $\{F_n\}_{n \in \mathbb{N}}$ which satisfies

$$F_0 = 0 \quad ; \quad F_1 = 1 \quad ; \quad F_{n+1} = F_n + F_{n-1} \quad , \quad n \geq 1.$$

It is known that the ratio $\frac{F_{n+1}}{F_n} \rightarrow \varphi = \frac{1+\sqrt{5}}{2}$ as $n \rightarrow \infty$. To show that in a different way using eigenvalue decomposition, consider the vector

$$\mathbf{u}_n = \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}.$$

This vector can form the recurrence relation

$$\mathbf{u}_n = A \mathbf{u}_{n-1} \quad \text{where} \quad A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \quad \text{and} \quad \mathbf{u}_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

The vector $\mathbf{u}_n$ can then be written in terms of $\mathbf{u}_0$ by repeated substitution:

$$\mathbf{u}_n = A \mathbf{u}_{n-1} = A^2 \mathbf{u}_{n-2} = \dots = A^{n-2} \mathbf{u}_2 = A^{n-1} \mathbf{u}_1 = A^n \mathbf{u}_0.$$

Therefore $\mathbf{u}_n = A^n \mathbf{u}_0$ but doing this requires calculating the n^{th} power of the matrix A which may be difficult to do.

In order to circumvent calculating A^n explicitly, consider the eigenpairs of A which are (noting that the eigenvectors of A are orthogonal since A is symmetric)

$$\left\{ \frac{1+\sqrt{5}}{2}; \begin{pmatrix} 1+\sqrt{5} \\ 2 \end{pmatrix} \right\} \quad ; \quad \left\{ \frac{1-\sqrt{5}}{2}; \begin{pmatrix} 1-\sqrt{5} \\ 2 \end{pmatrix} \right\}.$$

For the sake of convenience, define $\lambda_{\pm} = \frac{1 \pm \sqrt{5}}{2}$, then the eigenpairs can be rewritten as

$$\left\{ \lambda_+; \begin{pmatrix} 2\lambda_+ \\ 2 \end{pmatrix} \right\} \quad ; \quad \left\{ \lambda_-; \begin{pmatrix} 2\lambda_- \\ 2 \end{pmatrix} \right\}.$$

Since any multiple of an eigenvector is still an eigenvector, then both eigenvectors can be divided by 2 to give

$$\left\{ \lambda_+; \begin{pmatrix} \lambda_+ \\ 1 \end{pmatrix} \right\} \quad ; \quad \left\{ \lambda_-; \begin{pmatrix} \lambda_- \\ 1 \end{pmatrix} \right\}.$$

Let Λ be the diagonal matrix whose entries are the eigenvalues of A and let V be the matrix whose columns are the eigenvectors, i.e.

$$\Lambda = \begin{pmatrix} \lambda_+ & 0 \\ 0 & \lambda_- \end{pmatrix} \quad \text{and} \quad V = \begin{pmatrix} \lambda_+ & \lambda_- \\ 1 & 1 \end{pmatrix}.$$

Since the eigenvectors are distinct, then A is diagonalisable and can be written as $A = V\Lambda V^{-1}$.

This can be verified as follows:

$$\begin{aligned} V\Lambda V^{-1} &= \begin{pmatrix} \lambda_+ & \lambda_- \\ 1 & 1 \end{pmatrix} \begin{pmatrix} \lambda_+ & 0 \\ 0 & \lambda_- \end{pmatrix} \begin{pmatrix} \lambda_+ & \lambda_- \\ 1 & 1 \end{pmatrix}^{-1} \\ &= \frac{1}{\lambda_+ - \lambda_-} \begin{pmatrix} \lambda_+ & \lambda_- \\ 1 & 1 \end{pmatrix} \begin{pmatrix} \lambda_+ & 0 \\ 0 & \lambda_- \end{pmatrix} \begin{pmatrix} 1 & -\lambda_- \\ -1 & \lambda_+ \end{pmatrix} \\ &= \frac{1}{\lambda_+ - \lambda_-} \begin{pmatrix} \lambda_+^2 & \lambda_-^2 \\ \lambda_+ & \lambda_- \end{pmatrix} \begin{pmatrix} 1 & -\lambda_- \\ -1 & \lambda_+ \end{pmatrix} \\ &= \frac{1}{\lambda_+ - \lambda_-} \begin{pmatrix} \lambda_+^2 - \lambda_-^2 & -\lambda_+^2 \lambda_- + \lambda_-^2 \lambda_+ \\ \lambda_+ - \lambda_- & -\lambda_+ \lambda_- + \lambda_- \lambda_+ \end{pmatrix} \\ &= \frac{1}{\lambda_+ - \lambda_-} \begin{pmatrix} (\lambda_+ + \lambda_-)(\lambda_+ - \lambda_-) & \lambda_+ \lambda_- (\lambda_- - \lambda_+) \\ \lambda_+ - \lambda_- & 0 \end{pmatrix} \\ &= \begin{pmatrix} \lambda_+ + \lambda_- & -\lambda_+ \lambda_- \\ 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} \frac{1+\sqrt{5}}{2} + \frac{1-\sqrt{5}}{2} & -\left(\frac{1+\sqrt{5}}{2}\right)\left(\frac{1-\sqrt{5}}{2}\right) \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = A. \end{aligned}$$

Now consider matrix A^2 :

$$A^2 = AA = (V\Lambda V^{-1})(V\Lambda V^{-1}) = V\Lambda^2 V^{-1}.$$

Since Λ is a diagonal matrix, then Λ^2 is also a diagonal matrix whose terms are the squares of Λ , i.e.

$$\Lambda^2 = \begin{pmatrix} \lambda_+ & 0 \\ 0 & \lambda_- \end{pmatrix}^2 = \begin{pmatrix} \lambda_+^2 & 0 \\ 0 & \lambda_-^2 \end{pmatrix}.$$

Similarly, the higher powers can be done in the same way, therefore

$$A^n = V \begin{pmatrix} \lambda_+^n & 0 \\ 0 & \lambda_-^n \end{pmatrix} V^{-1} \quad \text{for all } n \geq 1.$$

This shows a way in which the matrix powers can be calculated easily. Returning to $\mathbf{u}_n = A^n \mathbf{u}_0$:

$$\begin{aligned} \mathbf{u}_n = A^n \mathbf{u}_0 &= V \begin{pmatrix} \lambda_+^n & 0 \\ 0 & \lambda_-^n \end{pmatrix} V^{-1} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \lambda_+ & \lambda_- \\ 1 & 1 \end{pmatrix} \begin{pmatrix} \lambda_+^n & 0 \\ 0 & \lambda_-^n \end{pmatrix} \begin{pmatrix} \lambda_+ & \lambda_- \\ 1 & 1 \end{pmatrix}^{-1} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ &\Rightarrow \mathbf{u}_n = \begin{pmatrix} \frac{\lambda_+^{n+1} - \lambda_-^{n+1}}{\lambda_+ - \lambda_-} \\ \frac{\lambda_+^n - \lambda_-^n}{\lambda_+ - \lambda_-} \end{pmatrix}. \end{aligned}$$

Therefore

$$\frac{F_{n+1}}{F_n} = \frac{\frac{\lambda_+^{n+1} - \lambda_-^{n+1}}{\lambda_+ - \lambda_-}}{\frac{\lambda_+^n - \lambda_-^n}{\lambda_+ - \lambda_-}} = \frac{\lambda_+^{n+1} - \lambda_-^{n+1}}{\lambda_+^n - \lambda_-^n}.$$

Since $0 < |\lambda_-| < 1$, then λ_-^n tends to 0 as n tends to infinity. Therefore, passing the limit as n tends to infinity gives

$$\lim_{n \rightarrow \infty} \frac{F_{n+1}}{F_n} = \lim_{n \rightarrow \infty} \frac{\lambda_+^{n+1} - \lambda_-^{n+1}}{\lambda_+^n - \lambda_-^n} = \lim_{n \rightarrow \infty} \frac{\lambda_+^{n+1}}{\lambda_+^n} = \lim_{n \rightarrow \infty} \lambda_+ = \lambda_+ = \frac{1 + \sqrt{5}}{2}$$

which is indeed the Golden Ratio.

In performing this procedure, there is one important caveat. The matrix V must be inverted which is simple in the 2×2 case but can be computationally expensive for much larger sizes. This, again, can be circumvented by ensuring that V is an orthogonal matrix. Recall that since A is Hermitian, all its eigenvectors, and hence all the columns of V , must be orthogonal. In order to make V an orthogonal matrix, the 2-norm of each of its columns must be equal to 1, this can be done by dividing each column by its norm (which is feasible since any multiple of an eigenvector is still an eigenvector). To normalise the vectors, divide them by their 2-norm:

$$\left\| \begin{pmatrix} 1 \pm \sqrt{5} \\ 2 \end{pmatrix} \right\|_2^2 = (1 \pm \sqrt{5})^2 + 2^2 = 10 \pm 2\sqrt{5}$$

Therefore, after normalisation, eigenpairs will be

$$\left\{ \lambda_+; c_+ \begin{pmatrix} \lambda_+ \\ 1 \end{pmatrix} \right\} \quad ; \quad \left\{ \lambda_-; c_- \begin{pmatrix} \lambda_- \\ 1 \end{pmatrix} \right\} \quad \text{where} \quad c_{\pm} = \frac{2}{\sqrt{10 \pm 2\sqrt{5}}}.$$

This means that the eigenvalue decomposition of A is $A = \tilde{V}\Lambda\tilde{V}^{-1}$ where

$$\tilde{V} = \begin{pmatrix} c_+\lambda_+ & c_-\lambda_- \\ c_+ & c_- \end{pmatrix} \quad \text{and} \quad \Lambda = \begin{pmatrix} \lambda_+ & 0 \\ 0 & \lambda_- \end{pmatrix}.$$

The most important fact about the matrix $\tilde{V}$ is that it is an orthogonal matrix (meaning all its columns are orthonormal). Therefore

$$\tilde{V}^{-1} = \tilde{V}^T = \begin{pmatrix} c_+\lambda_+ & c_+ \\ c_-\lambda_- & c_- \end{pmatrix}.$$

The normalisation procedure is computationally cheap and so is matrix transposition, much more so than matrix inversion. The same matrix power can be used as before:

$$\begin{aligned} \mathbf{u}_n = A^n \mathbf{u}_0 &= \tilde{V} \begin{pmatrix} \lambda_+^n & 0 \\ 0 & \lambda_-^n \end{pmatrix} \tilde{V}^{-1} \mathbf{u}_0 = \begin{pmatrix} c_+\lambda_+ & c_-\lambda_- \\ c_+ & c_- \end{pmatrix} \begin{pmatrix} \lambda_+^n & 0 \\ 0 & \lambda_-^n \end{pmatrix} \begin{pmatrix} c_+\lambda_+ & c_+ \\ c_-\lambda_- & c_- \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ \Rightarrow \quad \mathbf{u}_n &= \sqrt{5} \begin{pmatrix} c_+c_-\lambda_+^{n+1} - c_+c_-\lambda_-^{n+1} \\ c_+c_-\lambda_+^n - c_+c_-\lambda_-^n \end{pmatrix} = c_+c_-\sqrt{5} \begin{pmatrix} \lambda_+^{n+1} - \lambda_-^{n+1} \\ \lambda_+^n - \lambda_-^n \end{pmatrix}. \end{aligned}$$

Therefore

$$\frac{F_{n+1}}{F_n} = \frac{c_+c_-\sqrt{5}(\lambda_+^{n+1} - \lambda_-^{n+1})}{c_+c_-\sqrt{5}(\lambda_+^n - \lambda_-^n)} = \frac{\lambda_+^{n+1} - \lambda_-^{n+1}}{\lambda_+^n - \lambda_-^n}$$

which is the same result as before. The eigenvalue decomposition is useful in this case but for larger matrices, normalisation needs to be done on the eigenvectors in order to avoid inverting matrices.

I.1 Calculating Eigenvalues Using the Power Method

For a diagonalisable matrix A , the **Power Method** is a process used to calculate the smallest and largest eigenvalues (in absolute value) as well as their associated eigenvectors.

Let $A \in \mathbb{R}^{N \times N}$ be a real diagonalisable matrix, then $A = V\Lambda V^{-1}$ where $\Lambda \in \mathbb{R}^{N \times N}$ is the diagonal matrix whose terms are the eigenvalues of A and $V \in \mathbb{R}^{N \times N}$ is the matrix whose columns are the eigenvectors of A corresponding to Λ .

For now, suppose, suppose that A is real and symmetric, then all the eigenvalues are real and all the eigenvectors are orthogonal, furthermore, the eigenvectors can be chosen to be orthonormal in order to make V an orthogonal matrix. Suppose that the eigenvalues of A are ordered in such a way that

$$|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_N|, \quad (\text{I.1})$$

in this case, the largest eigenvalue in magnitude is called the **Dominant Eigenvalue**, which is λ_1 in this case.

The **Power Method** can be summarised as follows: Start with an arbitrary unit vector $\mathbf{q}^{(0)}$ that has a non-zero component in the direction of $\mathbf{v}_1$ (i.e. $\|\mathbf{q}^{(0)}\|_2 = 1$ and $(\mathbf{q}^{(0)})^T \mathbf{v}_1 \neq 0$), then starting from $k = 1$:

- Calculate $\mathbf{z}^{(k)} = A\mathbf{q}^{(k-1)}$;
- Update $\mathbf{q}^{(k)} = \frac{\mathbf{z}^{(k)}}{\|\mathbf{z}^{(k)}\|_2}$ (meaning that $\mathbf{q}^{(k)}$ is still a unit vector);
- Update $\alpha^{(k)} = (\mathbf{q}^{(k)})^T A\mathbf{q}^{(k)}$;
- Update $k \rightarrow k + 1$ and repeat until

$$|\alpha^{(k)} - \alpha^{(k-1)}| \leq \tau |\alpha^{(k)}|$$

where τ is the desired tolerance.

The final value of α will be the eigenvalue of A which has the largest magnitude. This result can be stated more formally as follows:

Theorem I.1 (Power Method). *Let $A \in \mathbb{R}^{N \times N}$ be symmetric with the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_N$ and their corresponding eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N$ such that*

$$|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_N|.$$

Consider a unit vector $\mathbf{q}^{(0)}$ such that $(\mathbf{q}^{(0)})^T \mathbf{v}_1 \neq 0$ (i.e. $\mathbf{q}^{(0)}$ has a component in the direction of $\mathbf{v}_1$). Then the sequence of vectors

$$\mathbf{q}^{(k)} = \frac{A\mathbf{q}^{(k-1)}}{\|A\mathbf{q}^{(k-1)}\|_2}$$

converges to $\mathbf{v}_1$ and

$$\alpha^{(k)} = (\mathbf{q}^{(k)})^T A\mathbf{q}^{(k)}$$

converges to λ_1 as k tends to ∞ .

Proof. Since A is real and symmetric, then the eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N$ can be chosen in such a way that they form an orthonormal basis of $\mathbb{R}^N$, therefore the unit vector $\mathbf{q}^{(0)}$ can be written as a linear combination of $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N$ as

$$\mathbf{q}^{(0)} = \frac{1}{\gamma^{(0)}} \sum_{i=1}^N \beta_i \mathbf{v}_i \quad \text{where} \quad \gamma^{(0)} = \sqrt{\sum_{i=1}^N \beta_i^2}.$$

(The division by $\gamma^{(0)}$ is to ensure that the vector $\mathbf{q}^{(0)}$ is a unit vector.)

It can be proven, by induction (as detailed in Appendix ??), that

$$\mathbf{q}^{(k)} = \frac{A\mathbf{q}^{(k-1)}}{\|A\mathbf{q}^{(k-1)}\|_2} = \frac{1}{\gamma^{(k)}} \sum_{i=1}^N \beta_i \lambda_i^k \mathbf{v}_i \quad \text{where} \quad \gamma^{(k)} = \sqrt{\sum_{i=1}^N \beta_i^2 \lambda_i^{2k}}.$$

This can be rewritten by isolating the first term in the sum as

$$\mathbf{q}^{(k)} = \frac{\beta_1 \lambda_1^k}{\gamma^{(k)}} \left(\mathbf{v}_1 + \sum_{i=2}^N \frac{\beta_i}{\beta_1} \frac{\lambda_i^k}{\lambda_1^k} \mathbf{v}_i \right) \quad \text{with} \quad \gamma^{(k)} = \sqrt{\beta_1^2 \lambda_1^{2k} \left(1 + \sum_{i=2}^N \frac{\beta_i^2}{\beta_1^2} \frac{\lambda_i^{2k}}{\lambda_1^{2k}} \right)}.$$

Since, by the way the eigenvalues have been arranged, it was assumed that λ_1 is the largest eigenvalue in absolute value, then $\left| \frac{\lambda_i}{\lambda_1} \right| < 1$ for all i , and therefore $\left| \frac{\lambda_i^k}{\lambda_1^k} \right|$ tends to 0 as k tends to ∞ . Meaning that as k tends to ∞ , then $\gamma^{(k)} \rightarrow \beta_1 \lambda_1^k$ and hence $\mathbf{q}^{(k)} \rightarrow \mathbf{v}_1$. Now consider the expression for $\alpha^{(k)} = (\mathbf{q}^{(k)})^T A \mathbf{q}^{(k)}$, passing the limit as k tends to ∞ gives

$$\lim_{k \rightarrow \infty} \alpha^{(k)} = \lim_{k \rightarrow \infty} \left[(\mathbf{q}^{(k)})^T A \mathbf{q}^{(k)} \right] = \mathbf{v}_1^T A \mathbf{v}_1 = \lambda_1$$

since A is diagonalisable and $\|\mathbf{v}_1\|_2 = 1$. □

The power method can be generalised in several ways:

- **Inverse Power Method:** A possible generalization involves applying the method to the inverse of the matrix A (provided A is non-singular). Since the eigenvalues of A^{-1} are the reciprocals of those of A , the power method in that case gives an approximation to the eigenvalue of A of minimum modulus. This is called the *Inverse Power Method* which can be formally stated as follows: Given an initial unit vector $\mathbf{x}^{(0)}$, let $\mathbf{y}^{(0)} = \frac{\mathbf{x}^{(0)}}{\|\mathbf{x}^{(0)}\|_2}$. Then, for $k \geq 1$, compute

$$\mathbf{x}^{(k)} = A^{-1} \mathbf{y}^{(k-1)} \quad ; \quad \mathbf{y}^{(k)} = \frac{\mathbf{x}^{(k)}}{\|\mathbf{x}^{(k)}\|_2} \quad ; \quad \mu^{(k)} = (\mathbf{y}^{(k)})^T A^{-1} \mathbf{y}^{(k)}.$$

If A has N linearly independent eigenvectors and the minimum eigenvalue is distinct from all the others, then

$$\lim_{k \rightarrow \infty} \mu^{(k)} = \frac{1}{\lambda_N}$$

if the eigenvalues are arranged by size as before. This means that $(\mu^{(k)})^{-1}$ tends to λ_N as k tends to ∞ . Effectively, at every step k , a linear system of the form $A \mathbf{x}^{(k)} = \mathbf{y}^{(k-1)}$ needs to be solved. It is therefore convenient to find the LU decomposition of A then solving the system since this would require solving two triangular systems at each iteration.

- **Power Method with Shift:** Another generalization of the power method involves approximating the (unknown) eigenvalue of A nearest to a given number σ (either real or complex). Let λ_σ denote such eigenvalue and define the shifted matrix $A_\sigma = A - \sigma \mathcal{I}$ whose eigenvalues are $\lambda(A_\sigma) = \lambda(A) - \sigma$. In order to approximate σ , we can first approximate the eigenvalue of minimum length of A_σ , say $\lambda_n(A_\sigma)$, by applying the inverse power method to $A - \sigma \mathcal{I}$, and then compute $\lambda_\sigma = \lambda_n(A_\sigma) + \sigma$. This technique is known as the *Power Method with Shift* and the number σ is called the *Shift*. Obviously, the inverse power method (without shift) is recovered by simply setting $\sigma = 0$.

- **QR Method:** All the eigenvalues of A can be calculate at once by using the *QR Method* which is based on the QR decomposition of A . Initialise the iteration with $A^{(0)} = A$, then for $k \geq 1$, calculate the QR decomposition of $A^{(k-1)}$ as $A^{(k-1)} = Q^{(k-1)}R^{(k-1)}$ and the next iteration of A will be $A^{(k)} = R^{(k-1)}Q^{(k-1)}$. It can be proven that $A^{(k)}$ converges to an upper triangular matrix as k tends to ∞ . Also

$$A^{(k)} = Q^T A^{(k-1)} Q = Q^{-1} A^{(k-1)} Q$$

which means that, for all k , $A^{(k-1)}$ has same eigenvalues as $A^{(0)} = A$, meaning that the diagonal entries of $A^{(k)}$ get closer and closer to the required eigenvalues of A as k tends to ∞ .

J Numerical Solutions of Non-Linear Equations

An important task in numerical analysis is that of finding the root x of a function $f : \mathbb{R} \rightarrow \mathbb{R}$, i.e. finding the point(s) $x \in \mathbb{R}$ such that $f(x) = 0$ (equivalently, in higher-dimensions, the root of a function $\mathbf{f} : \mathbb{R}^M \rightarrow \mathbb{R}^N$ is a vector/point $\mathbf{x} \in \mathbb{R}^M$ such that $\mathbf{f}(\mathbf{x}) = \mathbf{0}$). It is important to realise that for many real-life industrial problems (such as the discretisations of domains for partial differential equations), the system be very large, having a system of 10000 parameters or even higher is not uncommon.

Throughout this section, the exact roots of non-linear functions will be denoted x^* or $\mathbf{x}^*$. Numerical algorithms for the approximation of x^* or $\mathbf{x}^*$ are usually iterative and the aim is to generate a sequence of values x_k or $\mathbf{x}_k$ such that

$$\lim_{k \rightarrow \infty} x_k = x^* \quad \text{or} \quad \lim_{k \rightarrow \infty} \mathbf{x}_k = \mathbf{x}^*.$$

J.1 One-Dimensional Root-Finding Algorithm

In general, a non-linear function may have several roots and to find a root, an algorithm would require an *initial guess* x_0 which guides the solution procedure. Finding such a guess is usually difficult and requires some *a priori* knowledge.

Any method for solving a problem of the form $f(x) = 0$ (or indeed $\mathbf{f}(\mathbf{x}) = \mathbf{0}$) should have the following properties:

1. It should be “easy” to use, preferably using only information on f , not on its derivatives;
2. It should be fast and be able to find a root to a specified tolerance. More specifically, a sequence $\{x_k\}_{k \in \mathbb{N}}$ generated by a numerical method is said to converge to x^* with *order* p if there exists a constant $C > 0$ such that for a large enough $k_0 \in \mathbb{N}$,

$$\frac{|x_{k+1} - x^*|}{|x_k - x^*|^p} \leq C \quad \text{for all } k \geq k_0.$$

3. It should be reliable, i.e. it should converge to a root close to an initial guess and not diverge or become chaotic. The convergence of iterative methods for root-finding of a non-linear equation depends, in general, on the initial guess x_0 . The method is called:
 - *Locally Convergent* if the convergence holds for any starting guess x_0 that belongs to a suitable neighbourhood of the root x^* ;
 - *Globally Convergent* if the convergence holds for any choice of x_0 .

There is no ideal method, so more practical algorithms use a combination of methods to find the roots.

J.2 Bisection Method

For $k \geq 0$:

1. Find an interval $[a_k, b_k]$ over which f changes sign (i.e. $f(a_k) < 0 < f(b_k)$ or $f(a_k) > 0 > f(b_k)$) and set $c_k = \frac{a_k + b_k}{2}$ and define $x_{k+1} = c_k$;
2. The function f must change sign over one of the two intervals $[a_k, c_k]$ or $[c_k, b_k]$;
3.
 - i. If f changes sign in the interval $[a_k, c_k]$, then let $a_{k+1} = a_k$ and $b_{k+1} = c_k$;
 - ii. If f changes sign in the interval $[c_k, b_k]$, then let $a_{k+1} = c_k$ and $b_{k+1} = b_k$;
4. Update $k \rightarrow k + 1$ and repeat steps 1-3 until $|b_k - a_k| < \tau$ for some tolerance τ ;
5. The sequence of values $\{x_k\}$ will converge to the exact root x^* .

Advantages of the Bisection method:

- No information about the derivative of f is needed.
- For the right choices of a and b , convergence is guaranteed, making it very reliable.
- The more iterations there are, the more accurate the solution will be (not susceptible to numerical errors).
- Iterations are easy to do since they require finding the average only.

Disadvantages of the Bisection method:

- The convergence is very slow, linear at best. This means that if x_k is an estimate for the exact root x^* of f and $e_k = x^* - x_k$ is the error, then if e_k is small, the error at the next iteration will be $|e_{k+1}| \approx K|e_k|$ where $K \in (0, 1)$ is a constant (usually for the bisection method $K = 0.5$).
- Two initial guesses are needed (the values of a and b) in order to specify the bracketing interval, additionally, the function must change sign over this interval.
- The function has to be real and continuous.
- Relies on sign changes, meaning it cannot find repeated roots (like the root of $f(x) = x^2$).
- The method does not work for systems of equations.
- The roots have to be reasonably far away from another another in order to ensure convergence to one root or the other.

J.3 Secant Method

For $k \geq 0$:

1. Consider the value of f at the two points x_k and x_{k+1} ;
2. Draw a straight line through the two points $(x_k, f(x_k))$ and $(x_{k+1}, f(x_{k+1}))$;

3. This line has a root at

$$x_{k+2} = \frac{x_k f(x_{k+1}) - x_{k+1} f(x_k)}{f(x_{k+1}) - f(x_k)};$$

4. Update $k \rightarrow k + 1$ and repeat steps 1-3 for the points x_{k+1} and x_{k+2} ;
5. Continue to produce a set of approximations x_k to the root x^* until either

$$|f(x_k)| < \tau \quad \text{or} \quad |x_{k+1} - x_k| < \tau$$

where τ is some specified tolerance.

Advantages of the Secant method:

- No information about the derivative of f is needed.
- Converges *super-linearly* fashion, i.e. if $e_k = x^* - x_k$, then $|e_{k+1}| \approx K|e_k|^\varphi$ where $K \in (0, 1)$ and φ is the golden ratio.
- Requires only one function evaluation per iteration, making it computationally inexpensive.

Disadvantages of the Secant Method:

- It may not always converge if the initial values are not close enough to the root.
- The method may not converge if the root is near a turning point (i.e. if the function is differentiable and there is a point in $\xi \in [x_0, x_1]$ such that $f(\xi) = 0$, then the method may not converge).
- There is no guaranteed error bound.

J.4 Newton-Raphson Method (NR)

For $k \geq 0$:

1. Evaluate $f(x)$ and $f'(x)$ at x_k ;
2. Approximate f by a line of slope $f'(x_k)$ through the point $(x_k, f(x_k))$;
3. This line has a root at

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)};$$

4. Update $k \rightarrow k + 1$ and repeat steps 1-3 until either

$$|f(x_k)| < \tau \quad \text{or} \quad |x_{k+1} - x_k| < \tau.$$

Advantages of the NR:

- The method is quadratically convergent, i.e. if $e_k = x^* - x_k$, then $|e_{k+1}| \approx K|e_k|^2$.

i Note

To show this rate of convergence, consider the expression for the iteration:

$$\begin{aligned}
 x_{k+1} &= x_k - \frac{f(x_k)}{f'(x_k)} \\
 \Rightarrow_{-x^*} x_{k+1} - x^* &= x_k - x^* - \frac{f(x_k)}{f'(x_k)} \\
 \Rightarrow_{\times -1} x^* - x_{k+1} &= x^* - x_k + \frac{f(x_k)}{f'(x_k)} \\
 \Rightarrow_{f(x^*)=0} x^* - x_{k+1} &= x^* - x_k + \frac{f(x_k) - f(x^*)}{f'(x_k)} \\
 \Rightarrow x^* - x_{k+1} &= \frac{1}{f'(x_k)} [f(x_k) + f'(x_k)(x^* - x_k) - f(x^*)].
 \end{aligned}$$

Recall that by Taylor's Theorem¹, the term in the square brackets can be written as

$$[f(x_k) + f'(x_k)(x^* - x_k) - f(x^*)] = -\frac{1}{2}f''(\xi)(x^* - x_k)^2$$

where ξ is a point between x_k and x^* . Therefore

$$\begin{aligned}
 |e_{k+1}| = |x^* - x_{k+1}| &= \left| \frac{1}{f'(x_k)} [f(x_k) + f'(x_k)(x^* - x_k) - f(x^*)] \right| \\
 &\leq \left| \frac{1}{f'(x_k)} \left[-\frac{1}{2}f''(\xi)(x^* - x_k)^2 \right] \right| \\
 &= \frac{1}{2} \left| \frac{f''(\xi)}{f'(x_k)} \right| |e_n|^2.
 \end{aligned}$$

Therefore the NR converges quadratically. Obviously, some analysis is needed to make this statement precise, but roughly speaking it shows that provided f, f' and f'' are continuous near x^* , $f'(x^*) \neq 0$ and x_0 is close enough to x^* , then Newton's method converges quadratically.

- The method converges locally very quickly.
- Can be generalised to higher dimensions and to sets of equations.

Disadvantages of the NR:

- The function has to be differentiable, meaning it might be difficult to implement if the function was obtained from a set of measurements.

¹For a function $f : \mathbb{R} \rightarrow \mathbb{R}$, Taylor's theorem states that for two points $x, y \in \mathbb{R}$ which are close to one another:

$$f(y) \sim f(x) + f'(x)(y - x) + \frac{1}{2!}f''(\xi)(y - x)^2 \quad \text{for some } \xi \text{ between } x \text{ and } y.$$

- The initial value has to be reasonably close to the root, otherwise the method will not converge.
- If the gradient at the initial point is 0 or close to 0, then the method will not converge.

Note that the NR is a generalisation of the Secant method. Indeed, the general iteration step for the secant method is

$$x_{k+1} = \frac{x_{k-1}f(x_k) - x_k f(x_{k-1})}{f(x_k) - f(x_{k-1})}.$$

The right hand side can be rearranged to give

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} f(x_k) \quad (\text{J.1})$$

which is a simple approximation to the iteration

$$x_{k+1} = x_k - \frac{1}{f'(x_k)} f(x_k) \quad (\text{J.2})$$

which is well-known as NR. The Secant method Equation J.1 is therefore an approximate version of NR which makes use of evaluations of the function f and does not require evaluations of the derivative of f . The disadvantage of the Secant method is that it converges more slowly than NR (although both methods are faster than linear).

Python has an in-built root-finding algorithm called *Brent's Method* (from Brent, R. P., Algorithms for Minimization Without Derivatives. Englewood Cliffs, NJ: Prentice-Hall, 1973. Ch. 3-4.), also sometimes referred to as the *van Wijngaarden-Dekker-Brent Method*. This is a more reliable version of the Secant method since it uses a combination of inverse quadratic extrapolation, root bracketing, interval bisection and inverse quadratic interpolation.

J.5 Solving Systems of Non-Linear Equations Numerically

Methods for solving non-linear systems of equations can be derived as generalisations of the scalar case. Consider the system of equations given by $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ where $\mathbf{f} : \mathbb{R}^N \rightarrow \mathbb{R}^N$ is a given vector-valued function of the N variables $x_1, x_2, \dots, x_N$.

For example, if the function $\mathbf{f}$ is given by

$$\mathbf{f}(\mathbf{x}) = \begin{pmatrix} x_1^2 + x_2^2 - 1 \\ x_1 - x_2 \end{pmatrix},$$

then the root-finding algorithm would need to find x_1 and x_2 such that

$$x_1^2 + x_2^2 - 1 = 0 \quad \text{and} \quad x_1 - x_2 = 0.$$

In this case, the first equation is a unit circle and the second is a straight line. Therefore the solution is where the circle and the line intersect, and it can easily be seen that the solutions are $\pm \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right)$.

To write down NR for a system $\mathbf{f}(\mathbf{x}) = \mathbf{0}$, first write down the “obvious” generalisation of the scalar case Equation J.2, i.e.

$$\mathbf{x}_{n+1} = \mathbf{x}_n - J(\mathbf{x}_n)^{-1} \mathbf{f}(\mathbf{x}_n) \quad (\text{J.3})$$

where the role of the reciprocal of the derivative of $\mathbf{f}$ is replaced by the inverse of the Jacobian matrix $J(\mathbf{x})$ which is given by

$$J(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{pmatrix}$$

(where f_i is the i^{th} component of $\mathbf{f}$ for $i = 1, 2$). More generally for a function $\mathbf{f} : \mathbb{R}^N \rightarrow \mathbb{R}^N$, the Jacobian is given by

$$J(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_N} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_N}{\partial x_1} & \frac{\partial f_N}{\partial x_2} & \cdots & \frac{\partial f_N}{\partial x_N} \end{pmatrix} (\mathbf{x})$$

where f_i is the i^{th} component of $\mathbf{f}$ for $i = 1, 2, \dots, N$. This can be written in element form as

$$J(\mathbf{x})_{ij} = \frac{\partial f_i}{\partial x_j}(\mathbf{x}) \quad \text{for all } i, j = 1, 2, \dots, N.$$

More realistically, Equation Equation J.3 should be written as $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{d}_k$ where the Newton correction $\mathbf{d}_k$ is a vector that can be computed by solving the system of N linear equations $J(\mathbf{x}_k)\mathbf{d}_k = -\mathbf{f}(\mathbf{x}_k)$. This means that each step of NR requires the solution of an N -dimensional linear system where the matrix $J(\mathbf{x})$ and right hand side $\mathbf{f}(\mathbf{x}_k)$ have to be recomputed at every step (note that the inverse of the Jacobian is not normally computed since it is not needed, all that is needed is the solution of a single linear system with coefficient matrix $J(\mathbf{x}_k)$, which can be done without actually computing the inverse of $J(\mathbf{x}_k)$).

J.6 Minimisation Problems

Closely related to the idea of root-finding is the question of minimising a function $g : \mathbb{R}^N \rightarrow \mathbb{R}$. Such a problem can take one of two forms:

1. *Unconstrained* optimisation which minimises $g(\mathbf{x})$;
2. *Constrained* optimisation minimises $g(\mathbf{x})$ with an additional condition. For example, the value of $\mathbf{x}$ needs to be found such that the function g attains its minimum provided that $h(\mathbf{x}) = 0$ or $h(\mathbf{x}) \geq 0$.

An example of a constrained minimisation problem could be to minimise the cost of producing a product in a factory subject to keeping the pollution caused in this production as low as possible.

There are two kinds of minimum points, *global* and *local*: Given a function $g : \mathbb{R}^N \rightarrow \mathbb{R}$

- A *global* minimum is a point $\mathbf{x}^*$ such that $g(\mathbf{x}^*) \leq g(\mathbf{x})$ for all $\mathbf{x} \in \mathbb{R}^N$, i.e.

$$g(\mathbf{x}^*) = \min_{\mathbf{x} \in \mathbb{R}^N} g(\mathbf{x}).$$

- A *local* minimum is a point $\mathbf{x}^*$ such that $g(\mathbf{x}^*) \leq g(\mathbf{x})$ for all $\mathbf{x}$ in a small neighbourhood of $\mathbf{x}^*$. A necessary condition for a local minimum (for a sufficiently smooth function) is that $\nabla g(\mathbf{x}^*) = \mathbf{0}$ where ∇ is the gradient operator given by

$$\nabla \equiv \begin{pmatrix} \frac{\partial}{\partial x_1} \\ \frac{\partial}{\partial x_2} \\ \vdots \\ \frac{\partial}{\partial x_N} \end{pmatrix}.$$

Many algorithms are available for finding local minima but the global minimum is much more difficult since $g(\mathbf{x}^*)$ must be smaller than *all* $\mathbf{x}$ in the entire domain of g . Finding the global minimum of a general function g is not a simple task. Only recently have effective algorithms developed, these include *Simulated Annealing* and *Genetic Algorithms*. These algorithms are used mostly in bioinformatic industries for tasks such as protein design, and by the power generating industry to schedule the on-off times of its power stations.

J.7 Method of Steepest Descent

The simplest way to find a local minimum is the *Method of Steepest Decent*. This method starts from the realisation that for a function $g : \mathbb{R}^N \rightarrow \mathbb{R}$ and a point $\mathbf{x}_0 \in \mathbb{R}^N$, the function g decreases most rapidly in the direction $-\nabla g(\mathbf{x}_0)$.

i Note

Indeed, consider the unit direction $\hat{\mathbf{d}}$ where

$$\left. \frac{d}{dt} \{g(\mathbf{x}_0 + t\hat{\mathbf{d}})\} \right|_{t=0} \text{ is minimised.}$$

By using the chain rule, this implies that

$$\left. \nabla g(\mathbf{x}_0 + t\hat{\mathbf{d}}) \cdot \hat{\mathbf{d}} \right|_{t=0} \text{ is minimised.}$$

Therefore, this implies that $\nabla g(\mathbf{x}_0) \cdot \hat{\mathbf{d}}$ should be “as negative as possible”. By the Cauchy-Schwarz inequality²,

$$\nabla g(\mathbf{x}_0) \cdot \hat{\mathbf{d}} \leq \|\nabla g(\mathbf{x}_0)\|_2 \|\hat{\mathbf{d}}\|_2 = \|\nabla g(\mathbf{x}_0)\|_2 \quad \text{since } \hat{\mathbf{d}} \text{ is a unit vector.}$$

In order for the equality to hold, $\hat{\mathbf{d}}$ should be a scalar multiple of $\nabla g(\mathbf{x}_0)$, i.e. $\hat{\mathbf{d}} = \lambda \nabla g(\mathbf{x}_0)$ for some $\lambda \in \mathbb{R}$. In this case, since $\hat{\mathbf{d}}$ is a unit vector and it is intended to minimise, then

$$\hat{\mathbf{d}} = -\frac{\nabla g(\mathbf{x}_0)}{\|\nabla g(\mathbf{x}_0)\|}$$

meaning that the direction of steepest descent is $-\nabla g(\mathbf{x}_0)$. Note that in order to maximise the function g , the direction of steepest *ascen* is

$$\hat{\mathbf{d}} = \frac{\nabla g(\mathbf{x}_0)}{\|\nabla g(\mathbf{x}_0)\|}.$$

The method of steepest descent can be described as follows: For a starting point $\mathbf{x}_0 \in \mathbb{R}^N$ and $k \geq 0$:

- Let $\mathbf{x}_{k+1} = \mathbf{x}_k - t\nabla g(\mathbf{x}_k)$;
- Find the expression for $g(\mathbf{x}_{k+1})$ in terms of t ;
- Find the value of t which minimises $g(\mathbf{x}_{k+1})$;
- Update $k \rightarrow k + 1$ and repeat Steps 1-3 until $g(\mathbf{x}_k)$ cannot be reduced further. One possible stopping criterion would be

$$|g(\mathbf{x}_{k+1}) - g(\mathbf{x}_k)| < \tau$$

for some tolerance $\tau > 0$.

Note that Step 3 is a one-dimensional minimisation problem. It involves minimising a function of a *single* variable t . This is conceptually an easy thing to do; just go downhill in one direction until it is not possible to go any further. There are many methods of doing this including the *Bisection* and the (faster) *Golden Search Method*.

The method of steepest descent is conceptually easy to understand and implement, however, the algorithm needs to calculate ∇g at every step. The method can also be slow since the sequence of search directions are always orthogonal to one another, meaning that the algorithm can often times make repeated searches in every direction since it will follow a perpendicular zigzag pattern.

Caution

Consider the function $g : \mathbb{R}^2 \rightarrow \mathbb{R}$ where

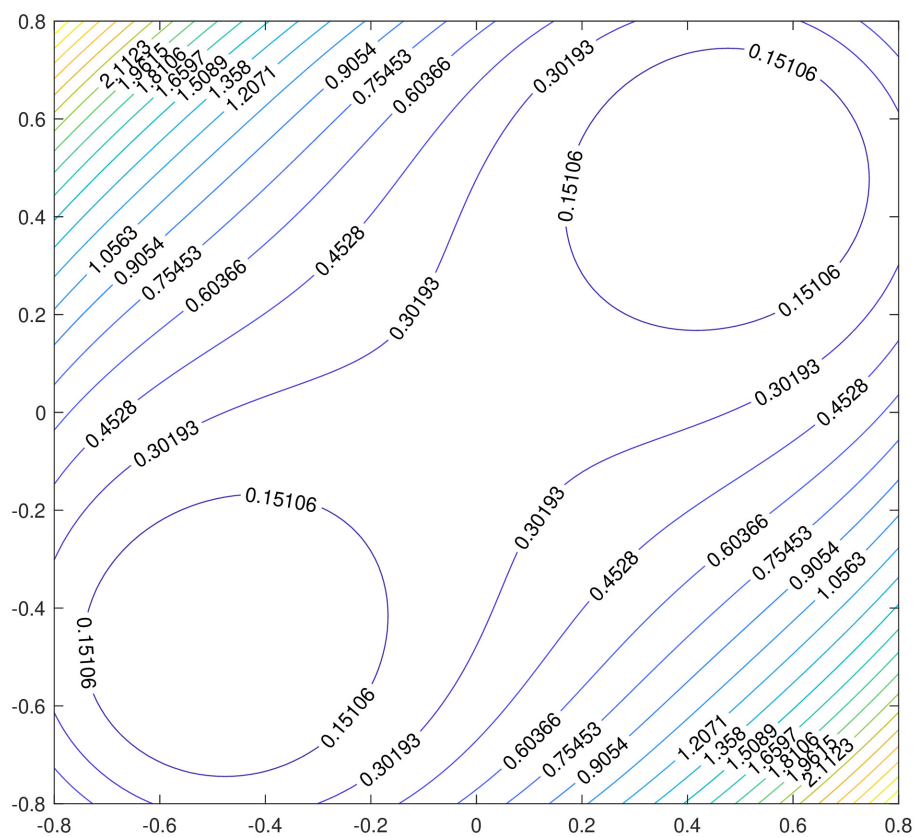
$$g(x, y) = (x - y)^2 + (x^2 + y^2 - 1)^2.$$

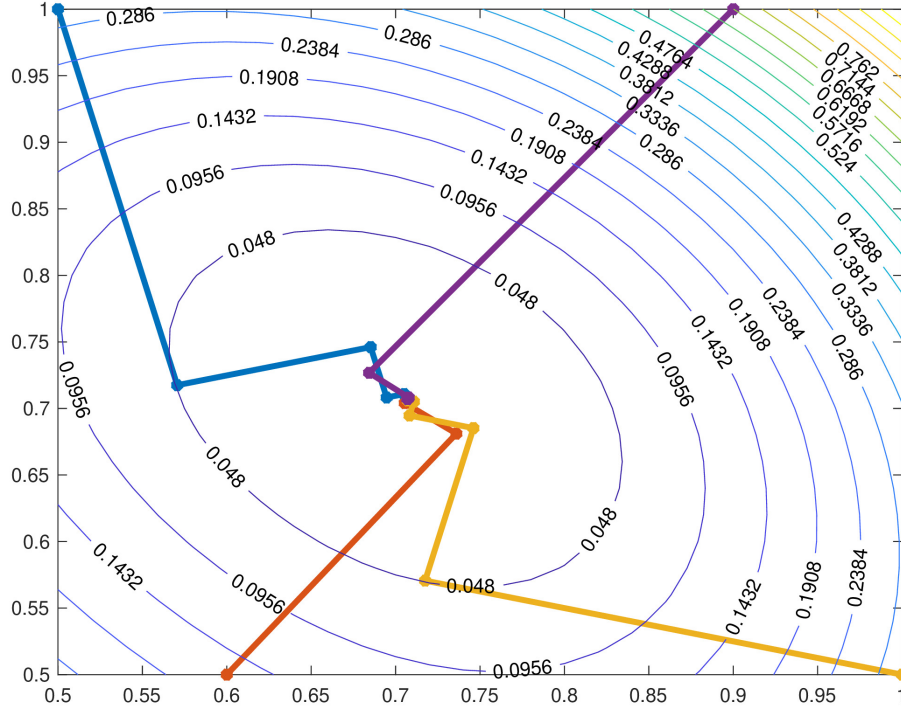
The method of steepest descent is shown below with four different calculations from different starting points.

²Recall that for vectors $\mathbf{u}$ and $\mathbf{v}$, the Cauchy-Schwarz inequality states that

$$|\langle \mathbf{u}, \mathbf{v} \rangle| \leq \|\mathbf{u}\|_2 \|\mathbf{v}\|_2$$

where in this case, the inner product is simply the dot product. Note that equality hold only when $\mathbf{u}$ and $\mathbf{v}$ are linearly *dependent*.





J.8 Variants of the Newton-Raphson Method

A special case of the NR for a system of equations would be to take the vector-valued function $\mathbf{f}$ to be equal to gradient of a function g , i.e. $\mathbf{f} = \nabla g$. This means that the NR can be used in order to implement the steepest descent method. In this case, if $\mathbf{f} = \nabla g$, then the Jacobian will in fact become the *Hessian* matrix. For instance, if $\mathbf{f}(x, y, z) = \nabla g(x, y, z)$, then

$$\mathbf{f}(\mathbf{x}) = \begin{pmatrix} \frac{\partial g}{\partial x} \\ \frac{\partial g}{\partial y} \\ \frac{\partial g}{\partial z} \end{pmatrix}(\mathbf{x}) \quad \text{and} \quad H = J(\mathbf{x}) = \begin{pmatrix} \frac{\partial^2 g}{\partial x^2} & \frac{\partial^2 g}{\partial x \partial y} & \frac{\partial^2 g}{\partial x \partial z} \\ \frac{\partial^2 g}{\partial y \partial x} & \frac{\partial^2 g}{\partial y^2} & \frac{\partial^2 g}{\partial y \partial z} \\ \frac{\partial^2 g}{\partial z \partial x} & \frac{\partial^2 g}{\partial z \partial y} & \frac{\partial^2 g}{\partial z^2} \end{pmatrix}(\mathbf{x}).$$

In general, the elements of the Hessian matrix/Jacobian (which is symmetric) are given by

$$H = J(\mathbf{x})_{ij} = \frac{\partial^2 g_i}{\partial x_i \partial x_j} \quad \text{for all } i, j = 1, 2, \dots, N.$$

Therefore, NR for $\mathbf{f} = \nabla g$ is

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{d}_k \quad \text{with } \mathbf{d}_k \text{ satisfying } H_k \mathbf{d}_k = -\nabla g(\mathbf{x}_k)$$

where $H_k = J(\mathbf{x}_k)$ is the Hessian matrix. Many methods attempt to approximate this by using the iteration

$$\mathbf{x}_{k+1} = \mathbf{x}_k + t_k H_k^{-1} \mathbf{d}_k$$

where t_n is a stepsize, $\mathbf{d}_n$ is the approximate search direction (usually $\mathbf{d}_n \approx -\nabla g(\mathbf{x}_n)$) and H_k^{-1} is the inverse of the Hessian matrix, or at least an approximation to the Hessian. Note that the steepest descent method is one example of this general form where t_k is the result of a line search, $\mathbf{d}_k = -\nabla g(\mathbf{x}_k)$ and H_k is the identity.

J.9 Applications of Minimisation Methods

- **The minimisation of large systems:** An interesting example of this arises in elliptic partial differential equations (for example problems in elasticity or electrostatics), where the solution minimises a function related to the energy of the system. Complicated engineering structures are designed by finding the local minima of a possible configuration as this represents a stable operating structure.
- **Solving symmetric, positive definite linear systems:** For the linear system given by $A\mathbf{x} = \mathbf{b}$ where A is a symmetric positive definite matrix, an approach to do this is by *minimising* the function

$$g(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}.$$

This is the basis of the celebrated *Conjugate Gradient* method. There are also variants for non-symmetric matrices.

- **Solving non-linear systems:** For a non-linear system of the form

$$\mathbf{f}(\mathbf{x}) = \mathbf{0} \quad \implies \quad \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_N \end{pmatrix}(\mathbf{x}) = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix},$$

a solution $\mathbf{x}^*$ would need to be found by *minimising* the function

$$g(\mathbf{x}) = \|\mathbf{f}(\mathbf{x})\|_2^2 = \sum_{n=1}^N |f_n(\mathbf{x})|^2$$

or more generally

$$g(\mathbf{x}) = \sum_{n=1}^N \alpha_n |f_n(\mathbf{x})|^2$$

where $\alpha_n > 0$ are suitably chosen weights. However, in order to solving the system $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ requires finding the global minimum of g and unconstrained minimisation algorithms will only find a local minimum. If the initial guess for the solution is good enough, then the “local” minimum of g near the initial guess will also be a “global” minimum.